

Lecture 1. Word embeddings

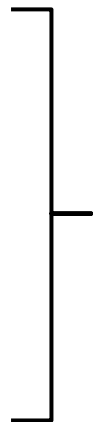
Radoslav Neychev

- NLP: introduction
- Text Preprocessing
- Feature Extraction: classical approach
 - Bag-of-Words
 - Bag-of-Ngramms
 - TF-IDF
- Word Embeddings: Word2Vec, GloVe

Natural Language Processing: Introduction

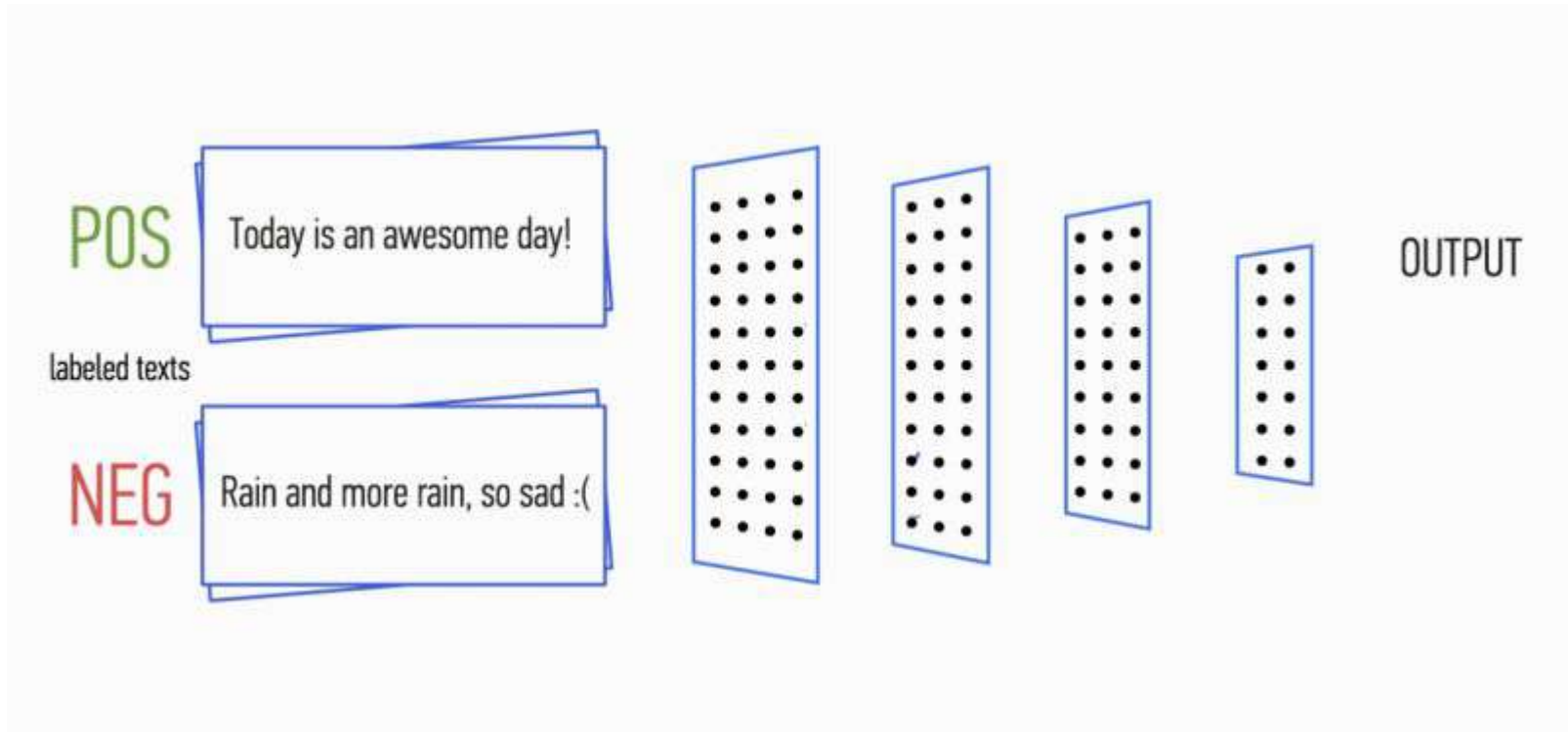
Popular NLP tasks

- Sentiment analysis
- Spam filtering
- Fake news detection
- Topic prediction
- #hashtag prediction



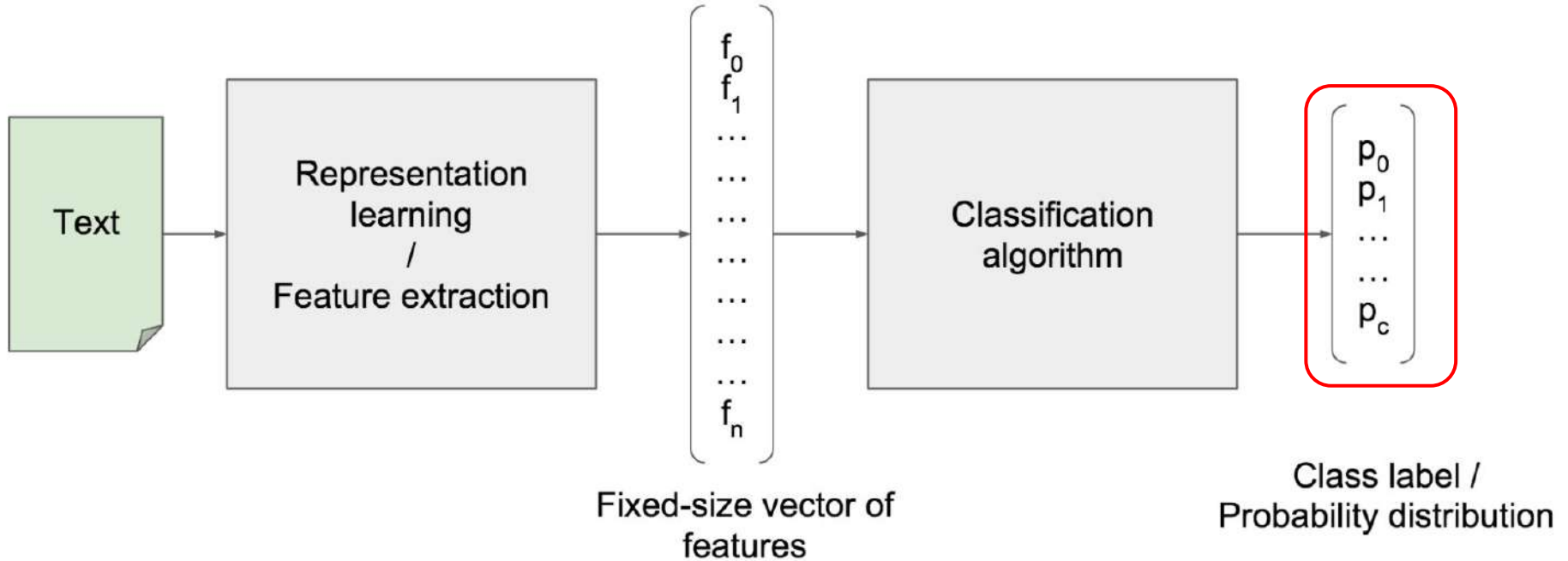
Text classification tasks

Example: sentiment analysis

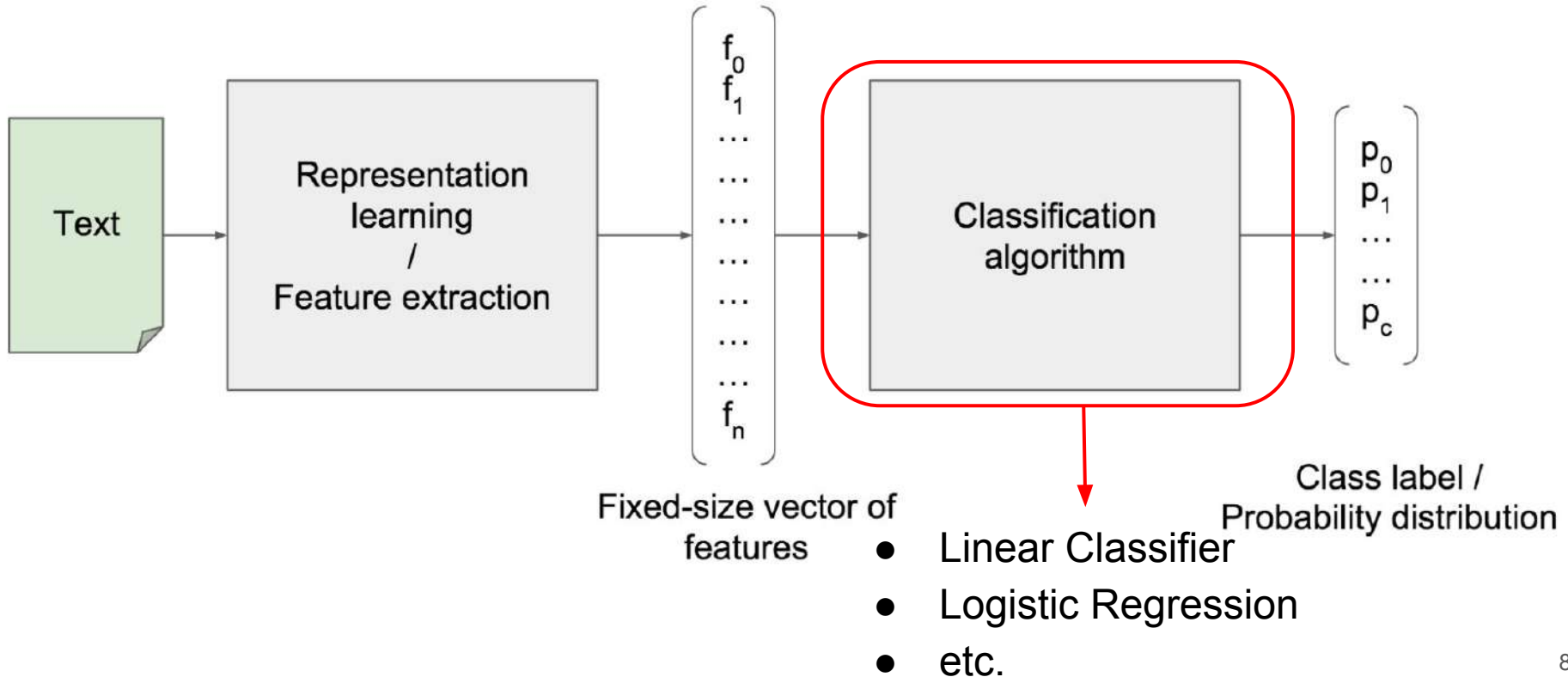


- Discrete labels:
 - Binary
 - spam filtering, sentiment analysis
 - Multi-class
 - categorization of items by its description
 - Multi-label
 - #hashtag prediction
- Continuous labels:
 - Predict product price by its description

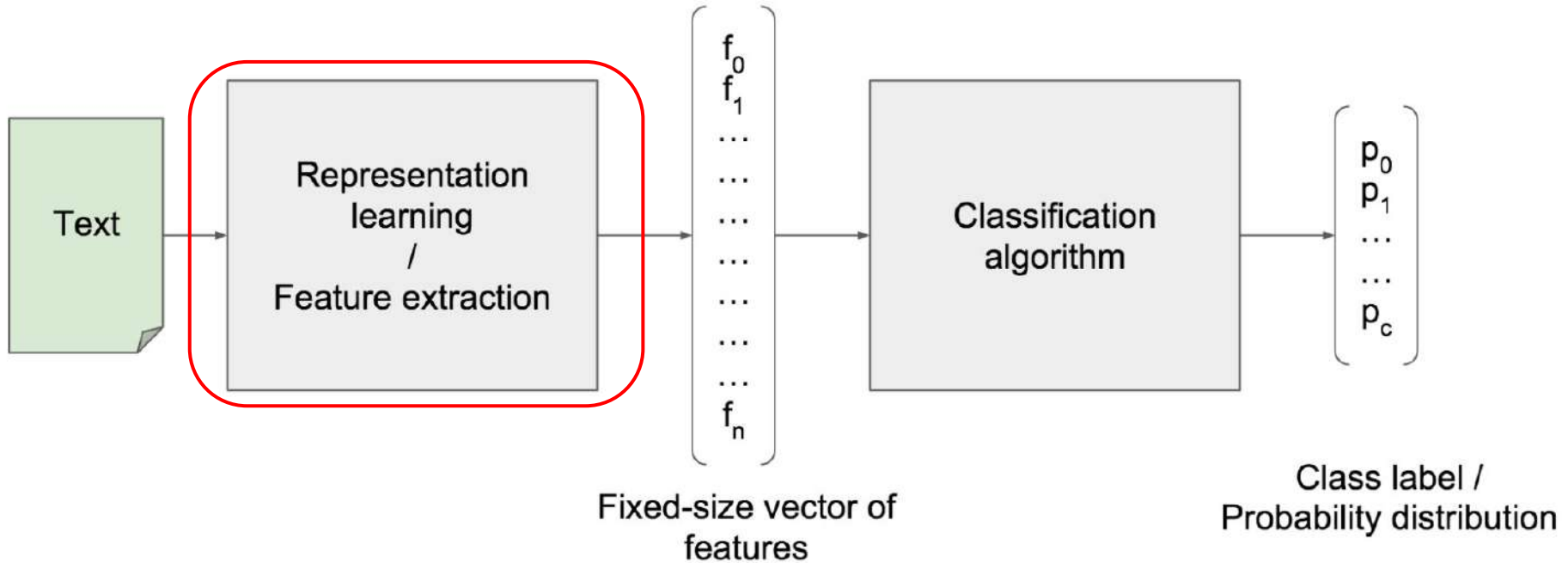
Text classification in general



Text classification in general

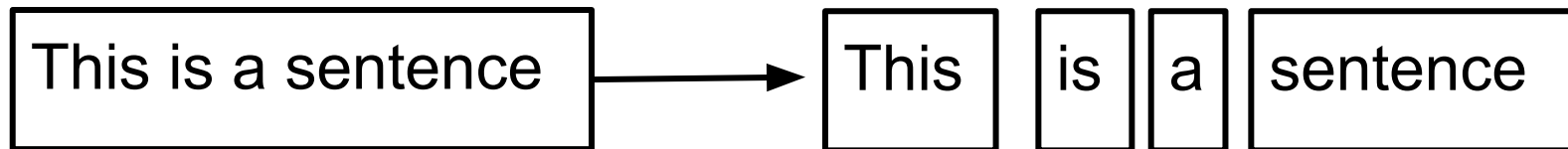


Text classification in general



Feature extraction

- Tokenization: split the input into tokens



the dog is on the table

0	0	1	1	0	1	1	2
are	cat	dog	is	now	on	table	the

- Problems:
 - No information about words order
 - Word vectors are huge and very sparse
 - Word vectors are not normalized
 - Same words can take different forms

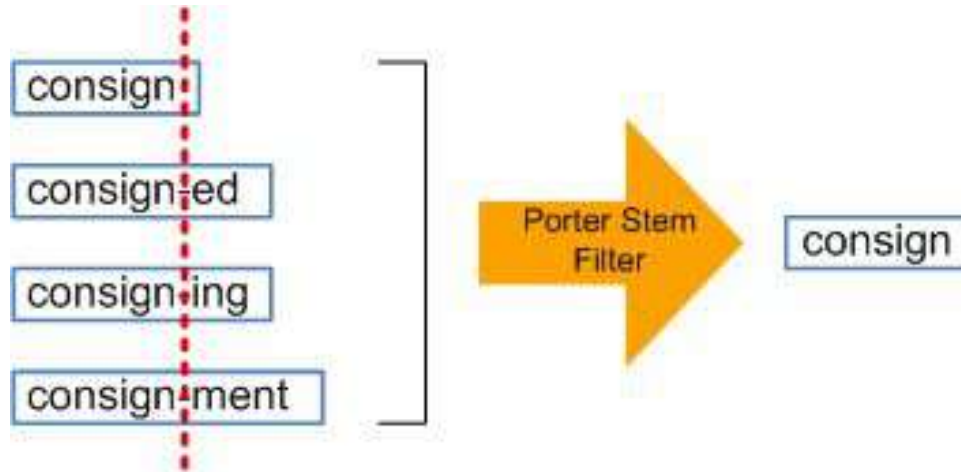
Text Preprocessing

- Token normalization

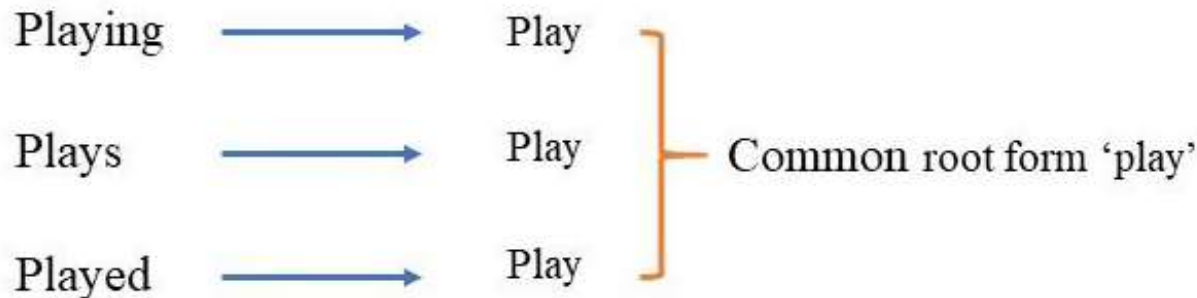
Dog, dogs → dog

Bark, barks → bark

- Token normalization:
 - **Stemming**: removing and replacing suffixes to get to the root of the word (**stem**)



- Token normalization:
 - **Stemming**: removing and replacing suffixes to get to the root of the word (**stem**)
 - **Lemmatization**: to get base or dictionary form of a word (**lemma**)



Stemming: Porter vs Lancaster

Porter stemmer

- Published in 1979
- Base starting option

Snowball stemmer (Porter 2)

- Based on Porter
- More aggressive
- Most popular option now

Lancaster stemmer

- Published in 1990
- The most aggressive
- Easy adding of your own rules

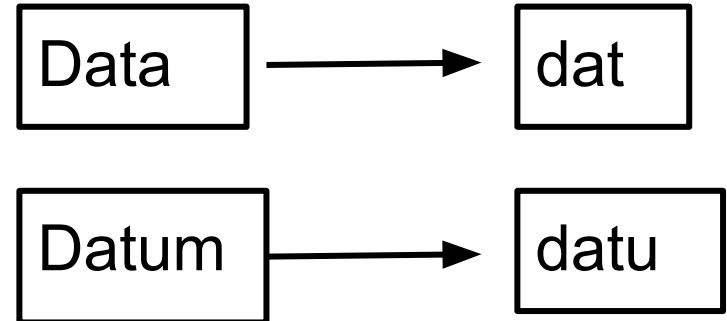
- Porter's stemmer:
 - **Heuristics, applied one-by-one:**
 - SSES - SS (dresses - dress)
 - IES - I (ponies - poni)
 - S - <empty> (dogs - dog)
 - **What's wrong?**
 - **Overstemming and understemming**

Overstemming

- University
- Universal
- Universities
- Universe

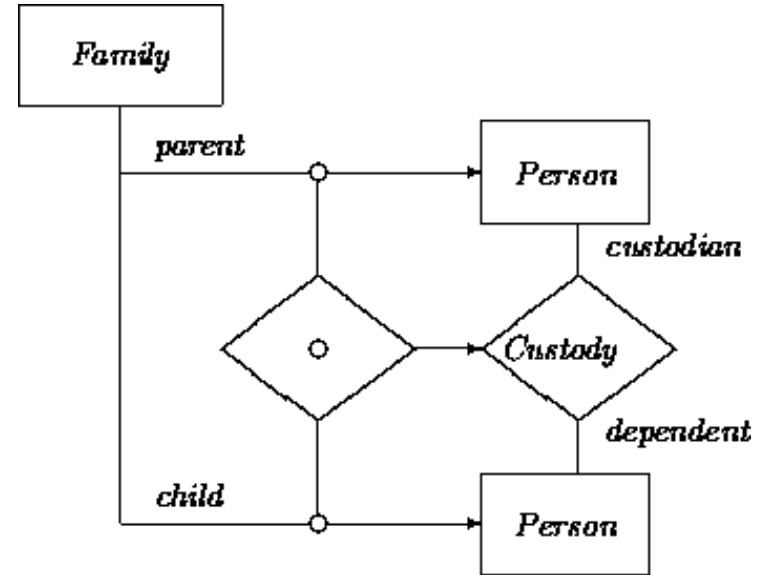
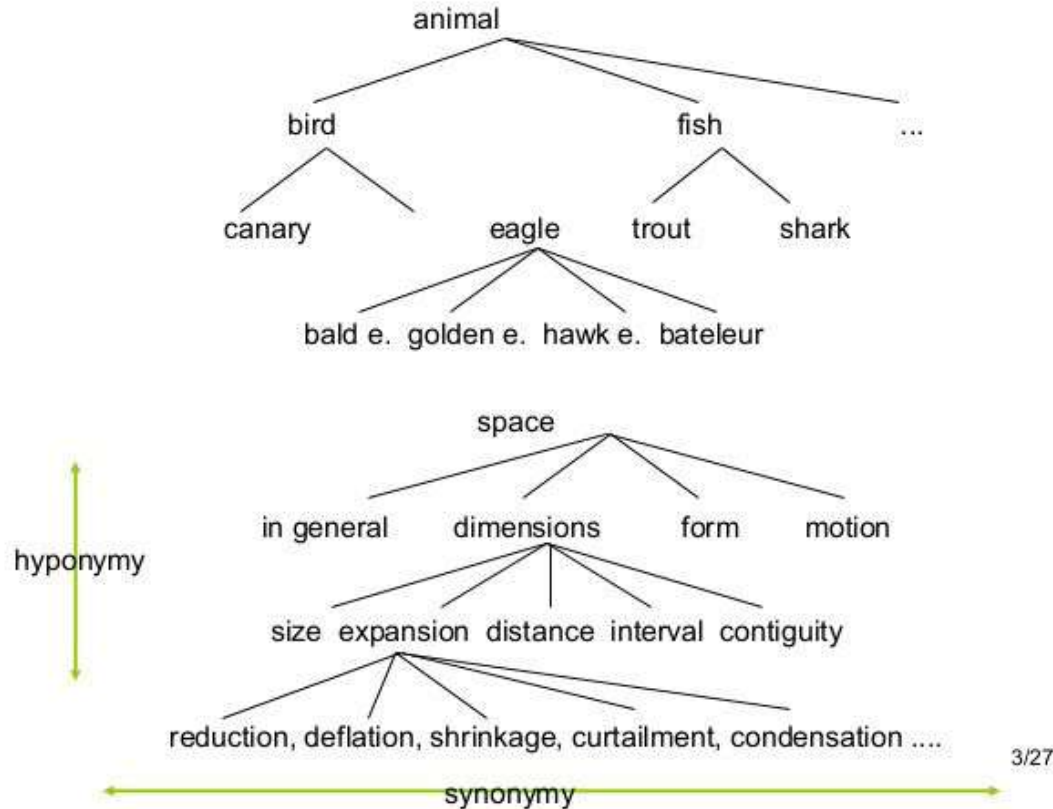
} Univers

Understemming



- Lemmatizer from NLTK:
 - Tries to resolve word to its dictionary form
 - Based on **WordNet** database
 - For the best results feed part-of-speech tagger

BTW, what is WordNet?



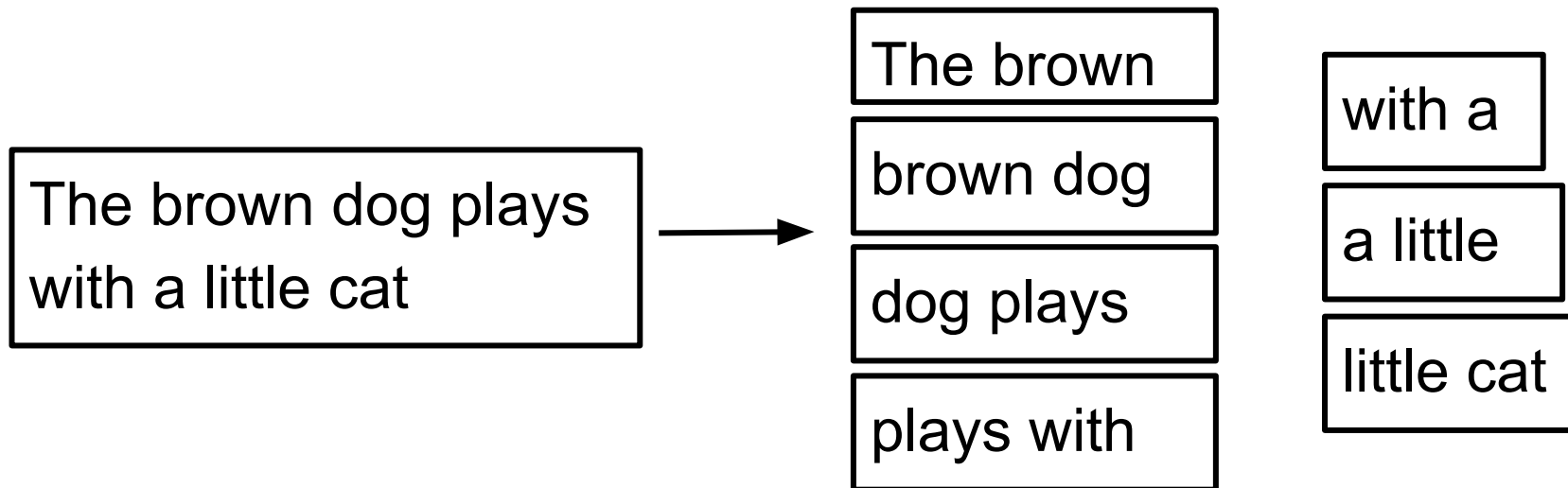
Handful tools for preprocessing

- [NLTK](#)
 - `nltk.stem.SnowballStemmer`
 - `nltk.stem.PorterStemmer`
 - `nltk.stem.WordNetLemmatizer`
 - `nltk.corpus.stopwords`
- [BeautifulSoup](#) (for parsing HTML)
- Regular Expressions (`import re`)
- [Pymorphy2](#)

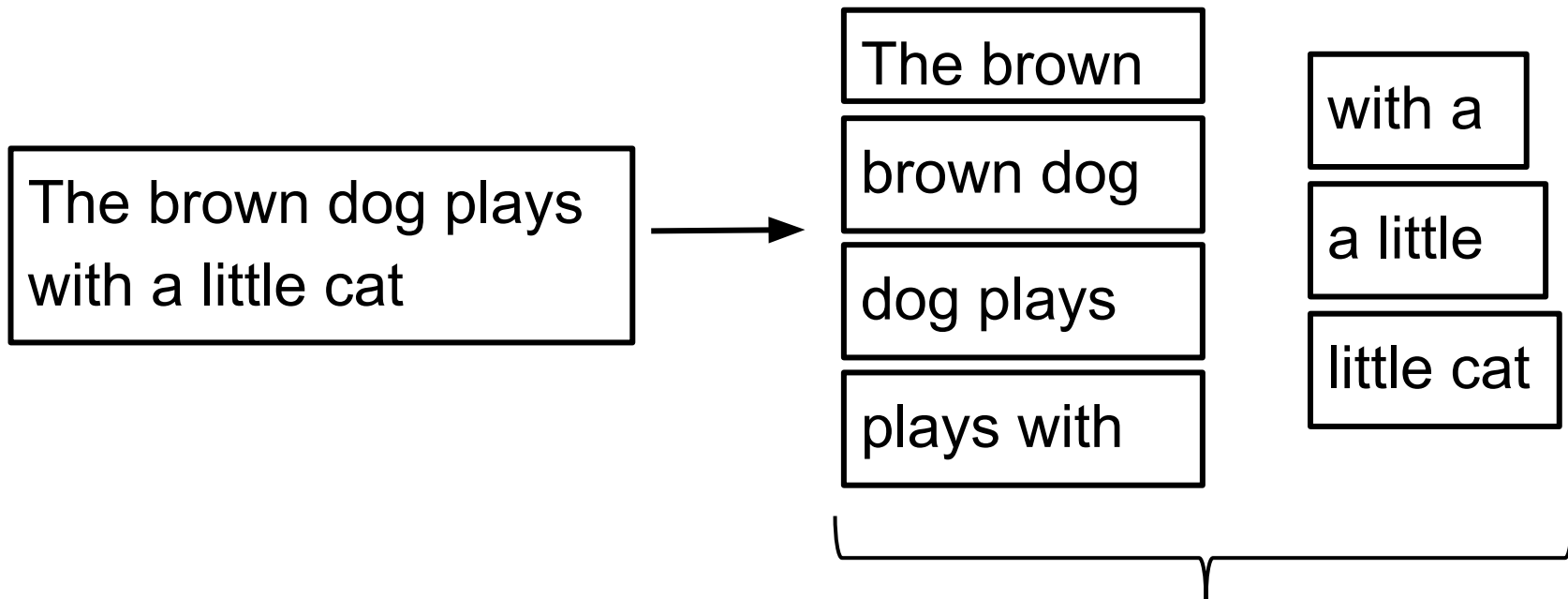
What's left?

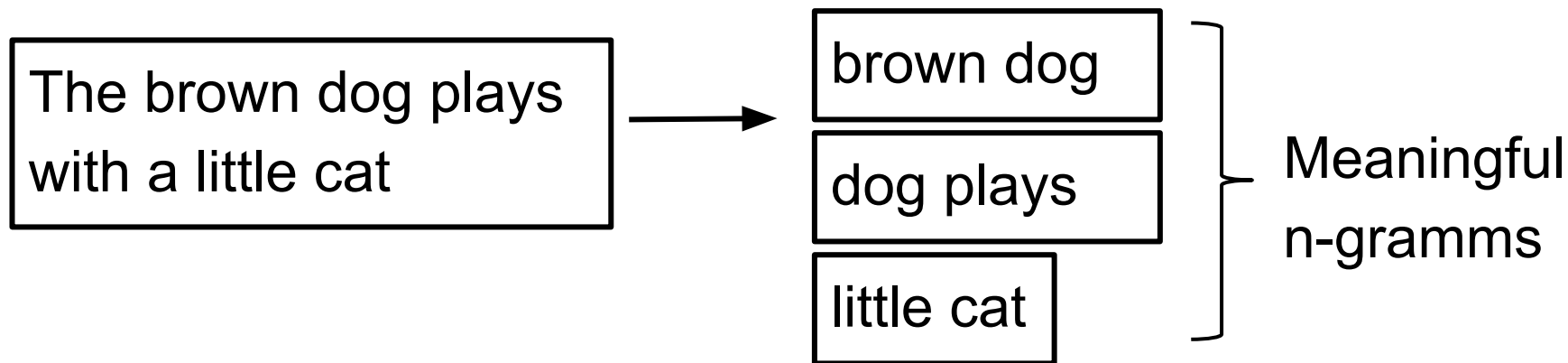
- Capital Letters
- Punctuation
- Contractions (e.g, etc.)
- Numbers (dates, ids, page numbers)
- Stop-words (“the”, “is”, etc.)
- Tags

- How to improve BOW?
 - Use n-gramms instead of words!



Bag-of-Words





Meaningful n-grams are often called **collocations**

How to detect meaningful n-grams?

- Delete:
 - High-frequency n-gramms
 - Articles, prepositions
 - Auxiliary verbs (to be, to have, etc.)
 - General vocabulary
 - Low-frequency n-gramms
 - Typos
 - Combinations that occur 1-2 times in a text

- **Term Frequency (tf):** gives us the frequency of the word in each document in the corpus.

$$\text{tf}(t,d) = f_{t,d}$$

- **Inverse Document Frequency (idf):** used to calculate the weight of rare words across all documents in the corpus. The words that occur rarely in the corpus have a high IDF score.

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

N : total number of documents in the corpus $N = |D|$

$|\{d \in D : t \in d\}|$: number of documents where the term t appears

TF-IDF example

- *Sentence A:* The car is driven on the road.
- *Sentence B:* The truck is driven on the highway.

(each sentence is a separate document)

TF-IDF example

Word	TF		IDF	TF * IDF	
	A	B		A	B
The	1/7	1/7			
Car	1/7	0			
Truck	0	1/7			
Is	1/7	1/7			
Driven	1/7	1/7			
On	1/7	1/7			
The	1/7	1/7			
Road	1/7	0			
Highway	0	1/7			

TF-IDF example

Word	TF		IDF	TF * IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2)=0$		
Car	1/7	0	$\log(2/1)=0.3$		
Truck	0	1/7	$\log(2/1)=0.3$		
Is	1/7	1/7	$\log(2/2)=0$		
Driven	1/7	1/7	$\log(2/2)=0$		
On	1/7	1/7	$\log(2/2)=0$		
The	1/7	1/7	$\log(2/2)=0$		
Road	1/7	0	$\log(2/1)=0.3$		
Highway	0	1/7	$\log(2/1)=0.3$		

TF-IDF example

Word	TF		IDF	TF * IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2)=0$	0	0
Car	1/7	0	$\log(2/1)=0.3$	0.043	0
Truck	0	1/7	$\log(2/1)=0.3$	0	0.043
Is	1/7	1/7	$\log(2/2)=0$	0	0
Driven	1/7	1/7	$\log(2/2)=0$	0	0
On	1/7	1/7	$\log(2/2)=0$	0	0
The	1/7	1/7	$\log(2/2)=0$	0	0
Road	1/7	0	$\log(2/1)=0.3$	0.043	0
Highway	0	1/7	$\log(2/1)=0.3$	0	0.043

TF-IDF example

```
from sklearn.feature_extraction.text  
import TfidfVectorizer
```



Word Embeddings

- **One-hot vectors:**

Rome = $[1, 0, 0, 0, 0, 0, \dots, 0]$

Paris = $[0, 1, 0, 0, 0, 0, \dots, 0]$

Italy = $[0, 0, 1, 0, 0, 0, \dots, 0]$

France = $[0, 0, 0, 1, 0, 0, \dots, 0]$

Problems:

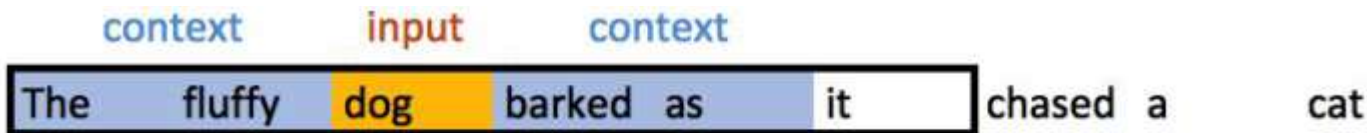
- Huge vectors
- VERY sparse
- No semantics or word similarity information included

Distributional semantics

Does vector similarity imply semantic similarity?

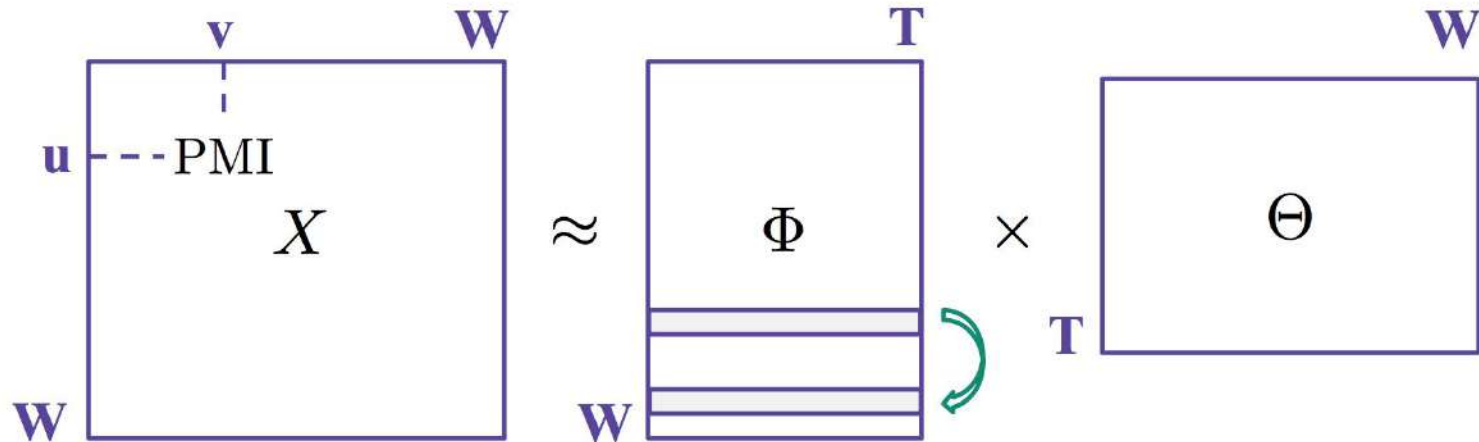
“You shall know a word by the company it keeps”

Firth, 1957



Word representations via matrix factorization

- **Input: PMI, word cooccurrences, etc.**
- **Method: dimensionality reduction (SVD)**
- **Output: word similarities**



- Delete:
 - High-frequency n-gramms
 - Articles, prepositions
 - Auxiliary verbs (to be, to have, etc.)
 - General vocabulary
 - Low-frequency n-gramms
 - Typos
 - Combinations that occur 1-2 times in a text

Collocations: context is all you need

- Cooccurrence counters in a window of fixed size
 - n_{uv} states for the number of times we've seen word u and word v together in the window
- Better solution: Pointwise Mutual Information (PMI)

$$PMI = \log \frac{p(u, v)}{p(u)p(v)} = \log \frac{n_{uv}n}{n_u n_v}$$

- Much better solution: **Positive PMI (pPMI)**

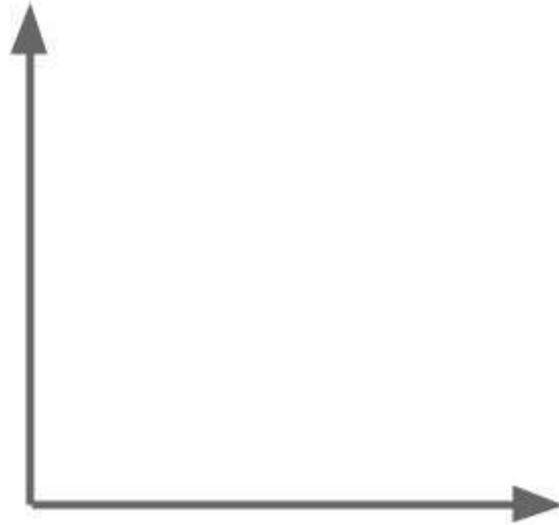
$$pPMI = \max(0, PMI)$$

Frequency With Filter		PMI	T-test With Filter	Chi-Sq Test
(front, desk)	(universal, studios)		(front, desk)	(wi, fi)
(great, location)	(howard, johnson)		(great, location)	(cracker, barrel)
(friendly, staff)	(cracker, barrel)		(friendly, staff)	(howard, johnson)
(hot, tub)	(santa, barbara)		(hot, tub)	(la, quinta)
(clean, room)	(sub, par)	(continental, breakfast)		(front, desk)
(hotel, staff)	(santana, row)	(free, breakfast)		(universal, studios)
(continental, breakfast)	(e, g)	(great, place)		(santa, barbara)
(nice, hotel)	(elk, springs)	(parking, lot)		(santana, row)
(free, breakfast)	(times, square)	(customer, service)		(, more)
(great, place)	(ear, plug)	(desk, staff)		(flat, screen)
(desk, staff)	(la, quinta)	(walk, distance)		(french, quarter)
(parking, lot)	(fire, pit)	(comfortable, bed)		(elk, springs)
(customer, service)	(san, clemente)	(nice, hotel)		(walking, distance)

Why not to learn word vectors?

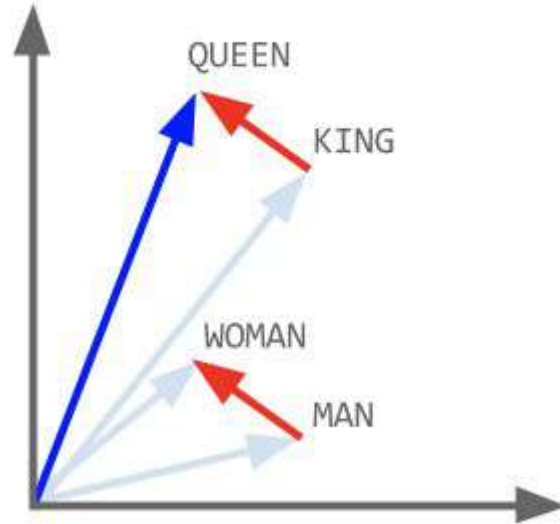
Embeddings: intuition

What is $\text{king} - \text{man} + \text{woman}$?

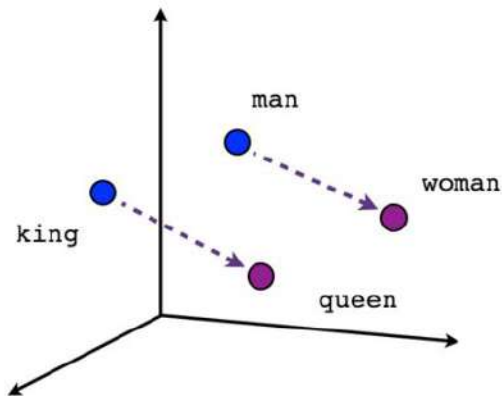


Embeddings: intuition

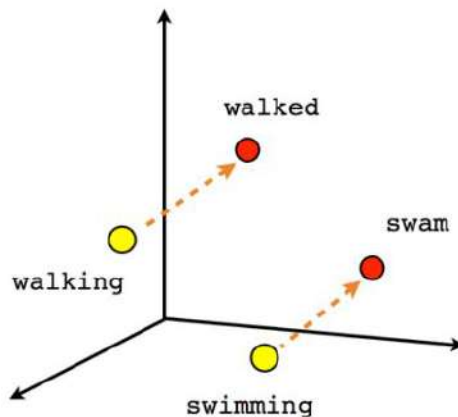
So $\text{king} - \text{man} + \text{woman} = \text{queen!}$



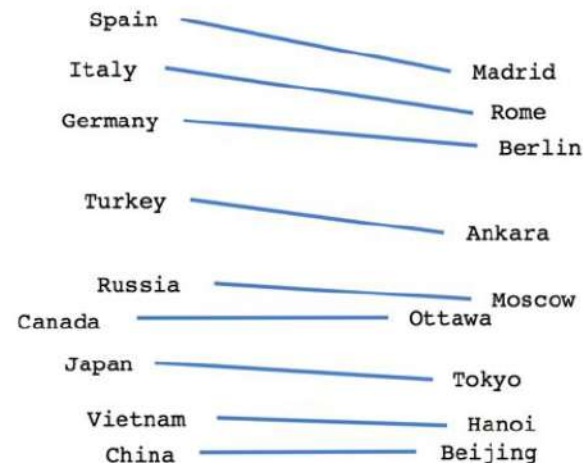
- **Word2vec** (Mikolov et al. 2013) - a framework for learning word embeddings



Male-Female



Verb tense

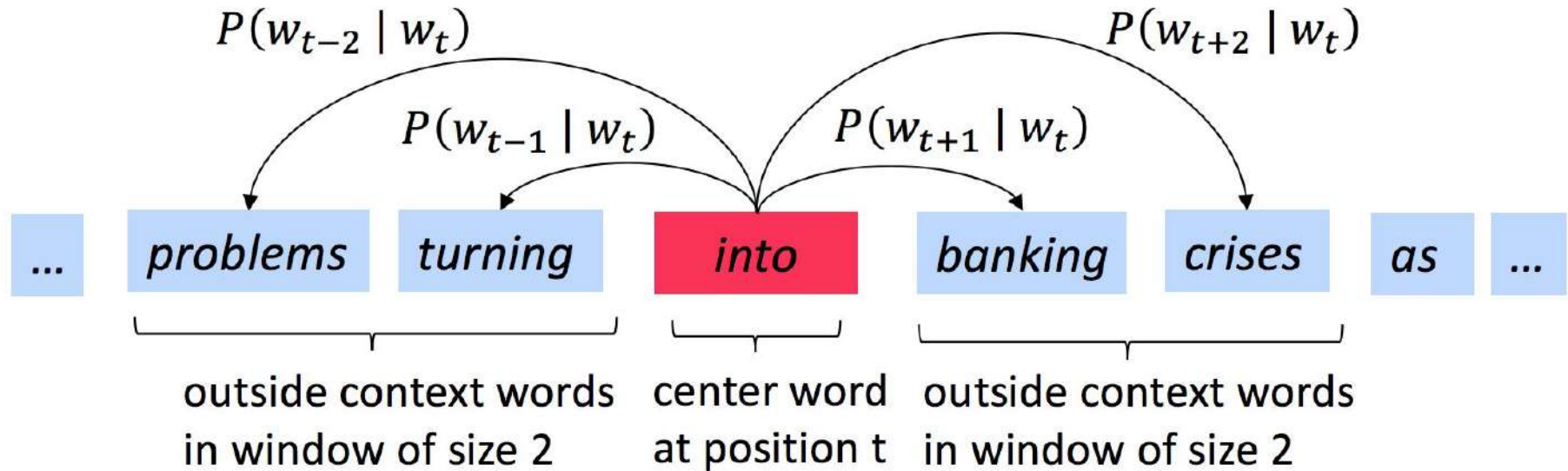


Country-Capital

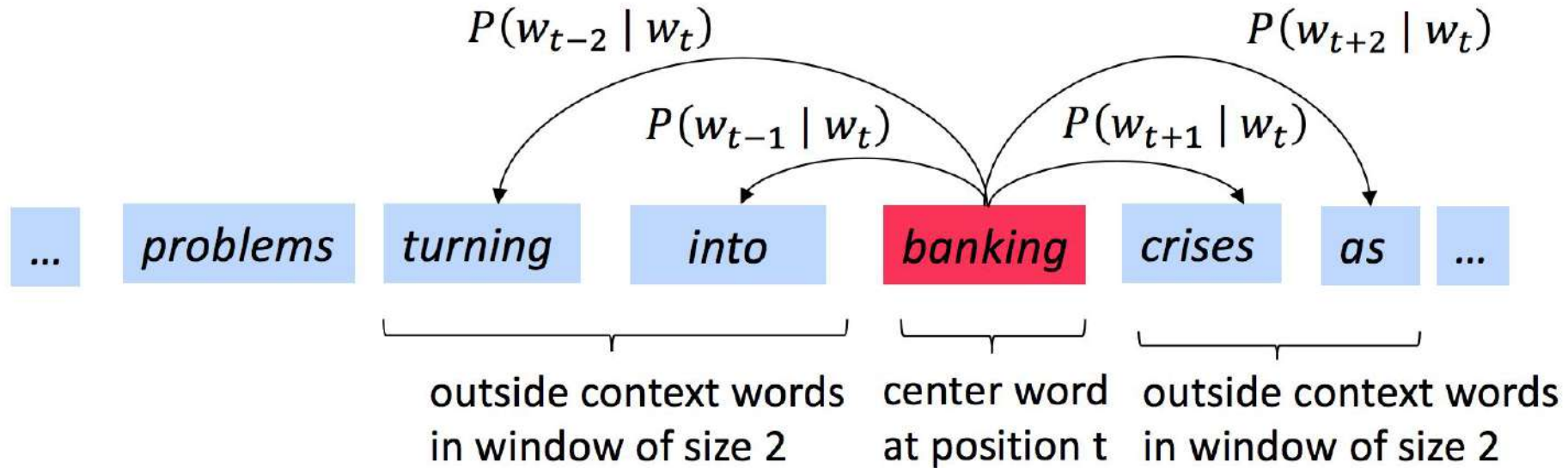
Embeddings: word2vec

Source Text	Training Samples					
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡	The	quick	brown	(the, quick) (the, brown)		
The	quick	brown				
The <table><tr><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)		
quick	brown	fox				
The quick <table><tr><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)		
brown	fox	jumps				
The <table><tr><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡	quick	brown	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
quick	brown	fox	jumps	over		

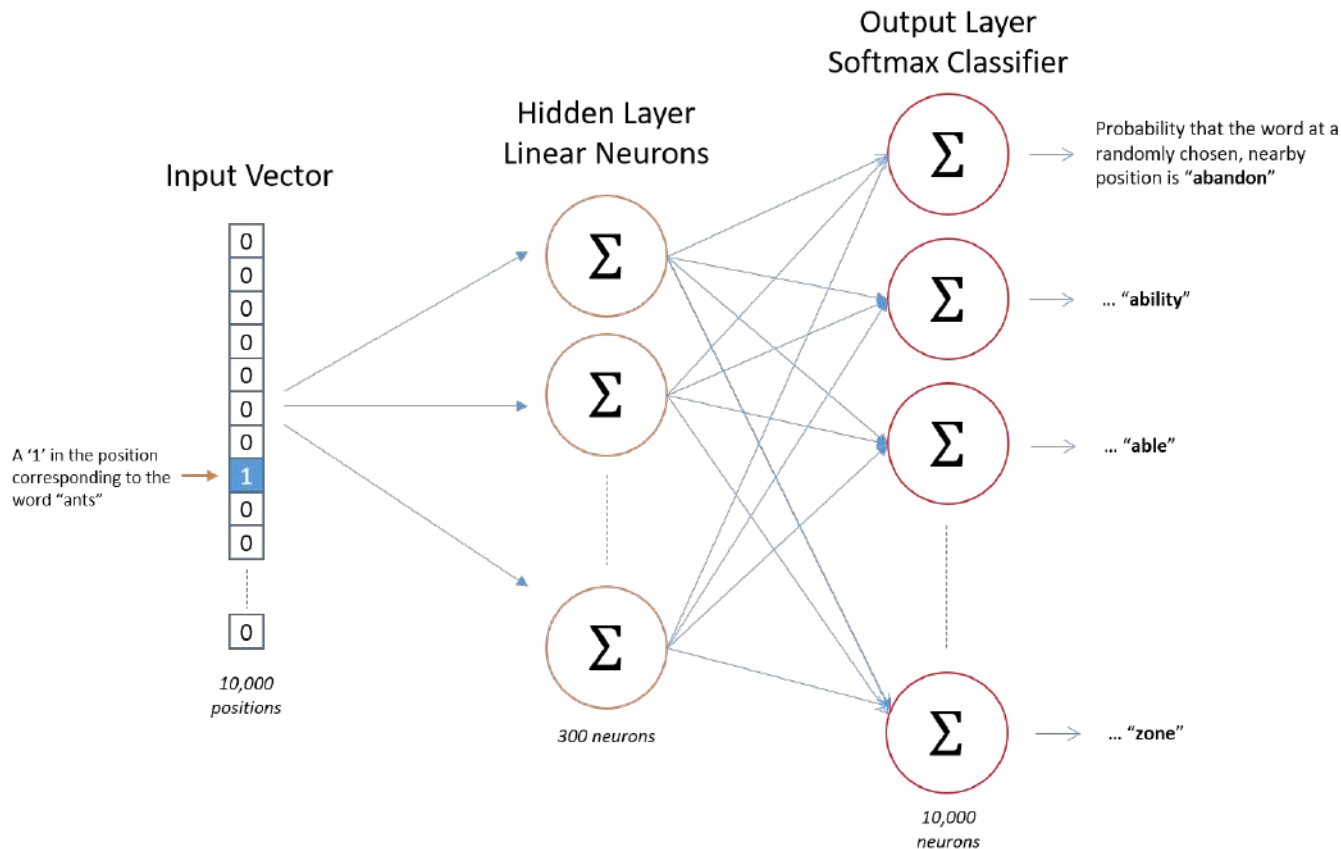
Embeddings: word2vec



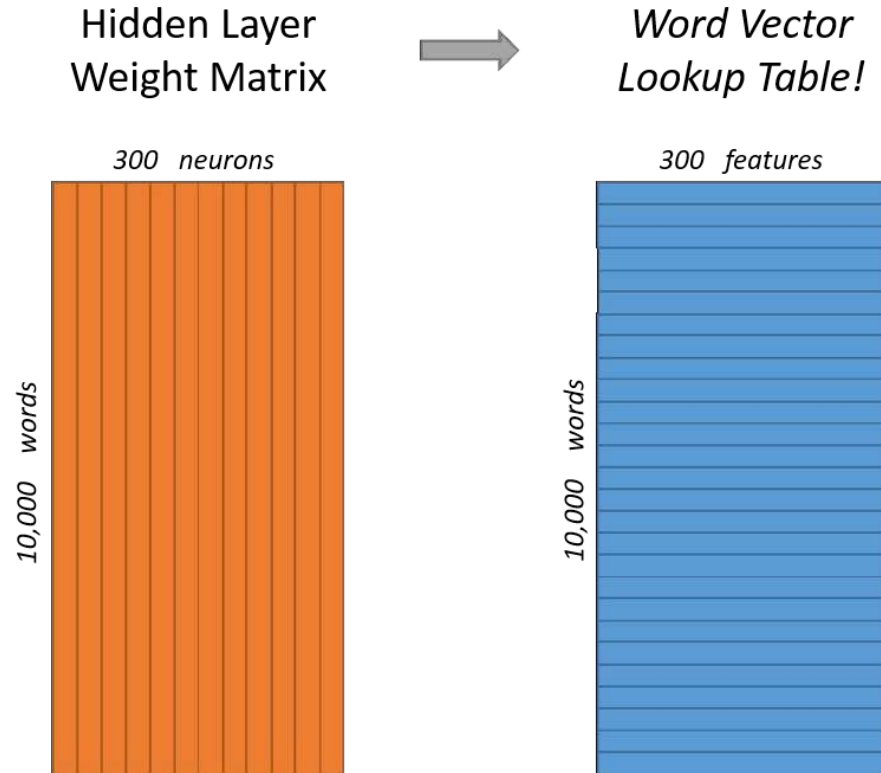
Embeddings: word2vec



Embeddings: word2vec



Embeddings: word2vec



Embeddings: word2vec

- Word vectors with 300 components
- Vocabulary of 10,000 words.
- Weight matrix with $300 \times 10,000 = 3$ million weights each!

Training is too long and computationally expensive

How to fix this?

Basic approaches:

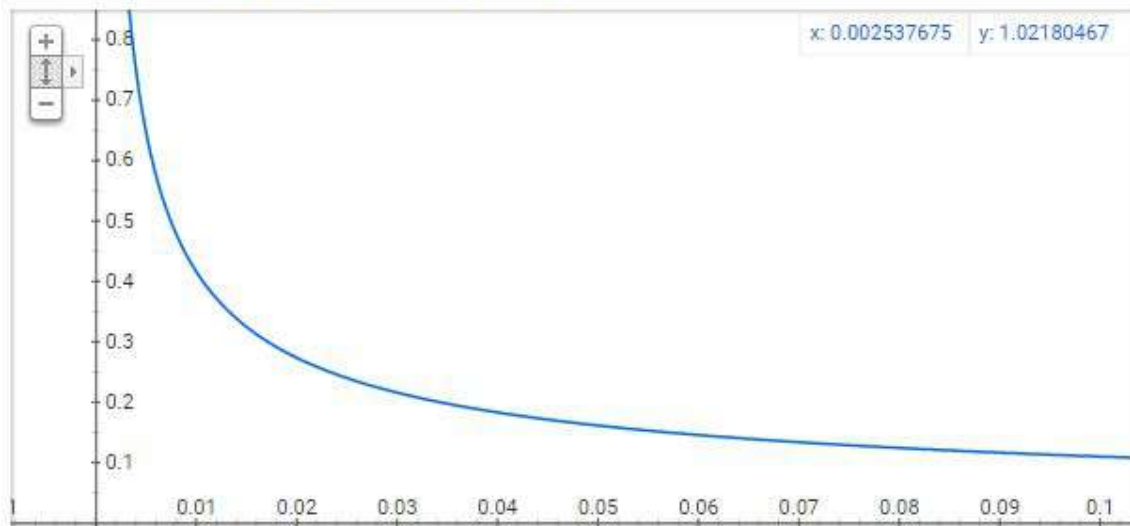
1. Treating common word pairs or phrases as single “words” in their model.
2. Subsampling frequent words to decrease the number of training examples.
3. Modifying the optimization objective with a technique they called “Negative Sampling”, which causes each training sample to update only a small percentage of the model’s weights.

Embeddings: word2vec

Subsampling frequent words.

w_i is the word, $z(w_i)$ is the fraction of this word in the text

Graph for $(\sqrt{x/0.001}+1)*0.001/x$



$P(w_i)$ is the probability of *keeping* the word:

$$P(w_i) = \left(\sqrt{\frac{z(w_i)}{0.001}} + 1 \right) \cdot \frac{0.001}{z(w_i)}$$

Embeddings: negative sampling

Negative Sampling idea: only few words error is computed. All other words have zero error, so no updates by the backprop mechanism.

More frequent words are selected to be negative samples more often. The probability for selecting a word is just its weight divided by the sum of weights for all words.

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_{j=0}^n (f(w_j)^{3/4})}$$

Word2vec: two models

Continuous BOW (CBOW)

$$p(w_i | w_{i-h}, \dots, w_{i+h})$$

Predict center word from
(bag of) context words

- Predicting one word each time
- Relatively fast

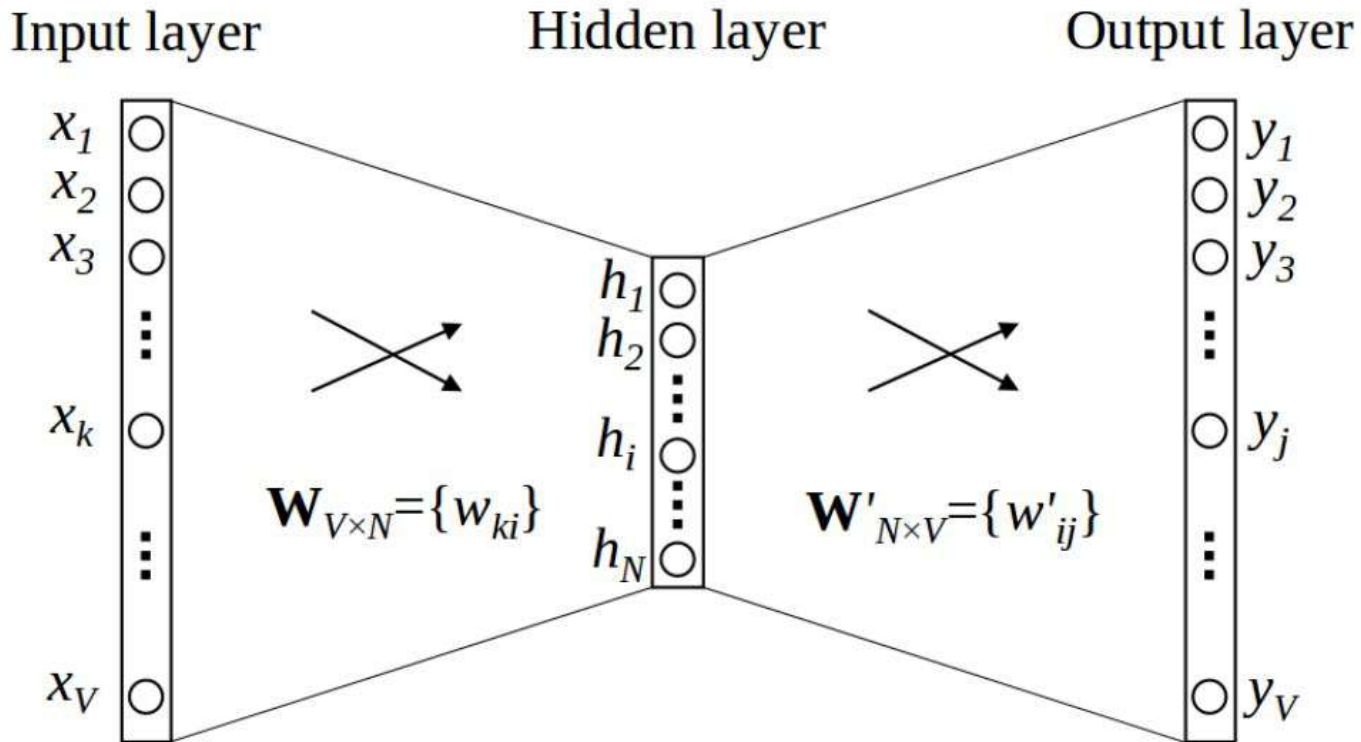
Skip-gram

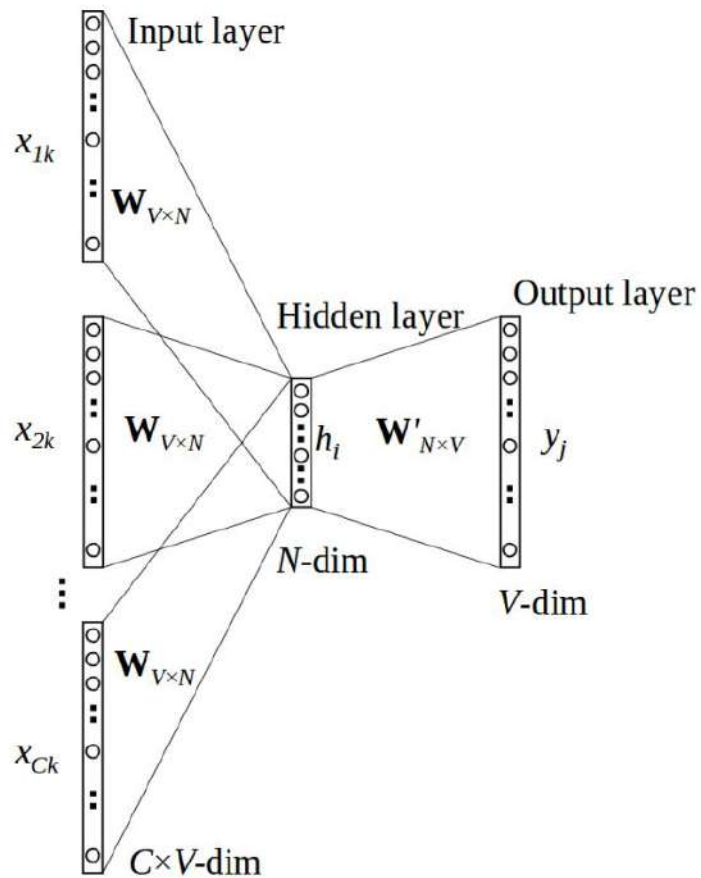
$$p(w_{i-h}, \dots, w_{i+h} | w_i)$$

Predict context ("outside")
words (position independent)
given center word

- Predicting context by one word
- Much slower
- Better with infrequent words

Skip-gram





GloVe (Global Vectors for word representations)

- **PMI / Frequency factorization** - use count-based corpus statistics

GloVe (Global Vectors for word representations)

- **PMI / Frequency factorization** - use count-based corpus statistics
- **Word2Vec** - predict *proximity* with neural embeddings
- **GloVe** - predict *count-based statistics* with neural embeddings

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})$$

GloVe (Global Vectors for word representations)

- **PMI / Frequency factorization** - use count-based corpus statistics
- **Word2Vec** - predict *proximity* with neural embeddings
- **GloVe** - predict *count-based statistics* with neural embeddings

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})$$

Trainable embeddings

Trainable bias vectors

GloVe (Global Vectors for word representations)

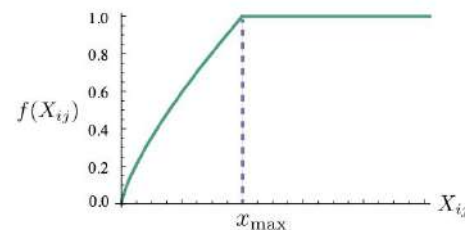
- **PMI / Frequency factorization** - use count-based corpus statistics
- **Word2Vec** - predict *proximity* with neural embeddings
- **GloVe** - predict *count-based statistics* with neural embeddings

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})$$

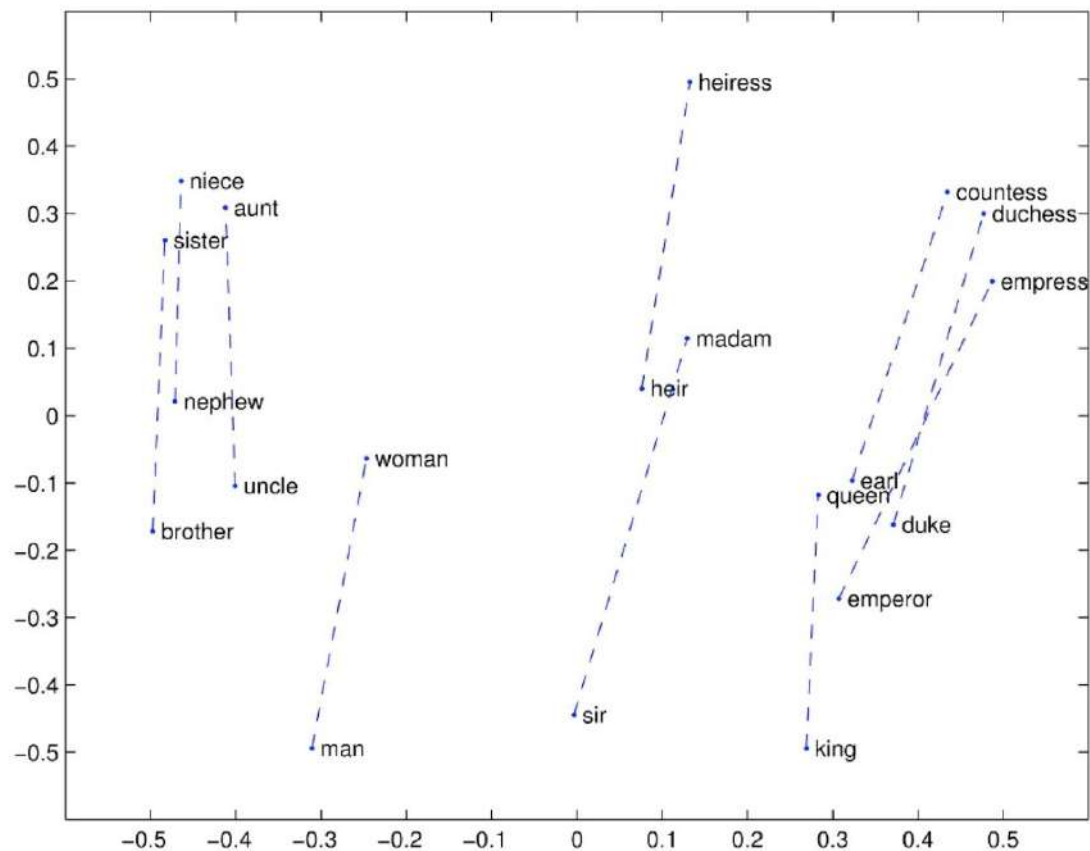
Weighting function
(penalize rare
cooccurrences, regularize
frequent ones)

Trainable embeddings

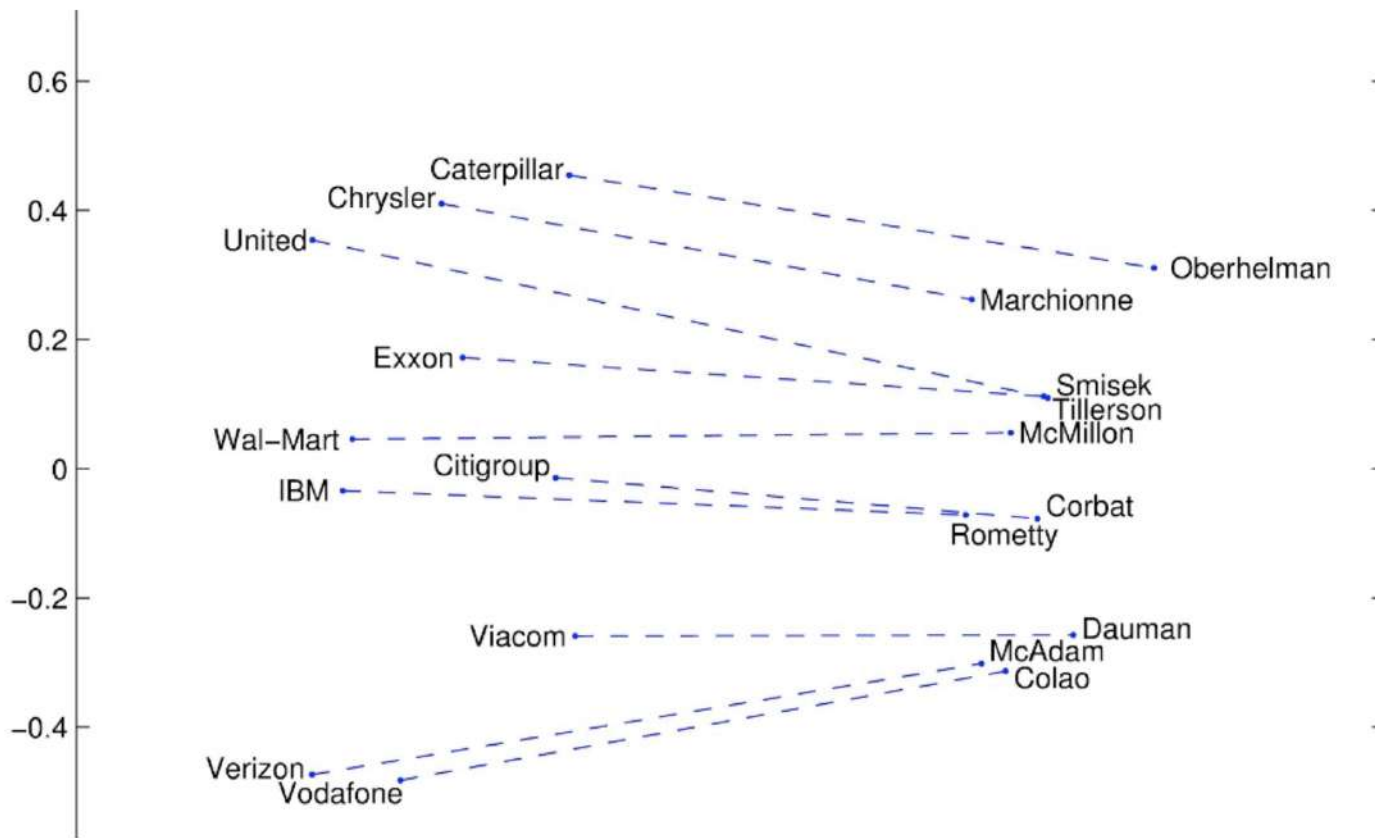
Trainable bias vectors



GloVe Visualizations



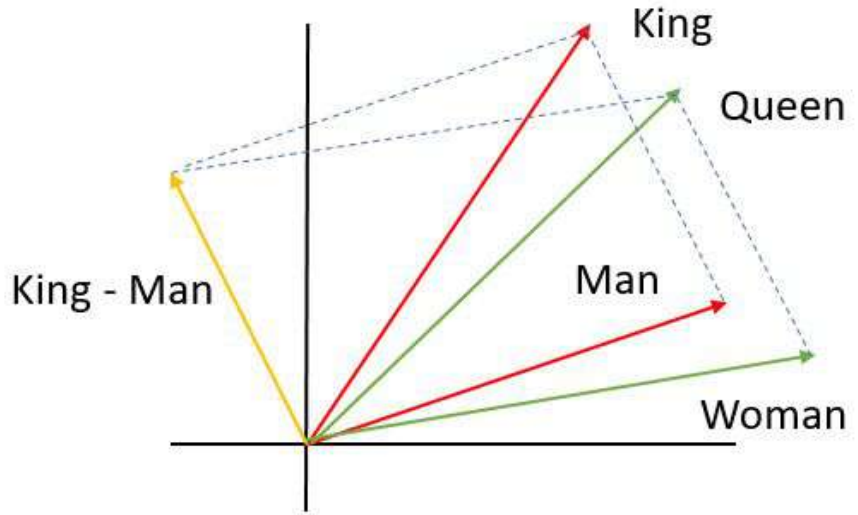
GloVe Visualizations: Company - CEO

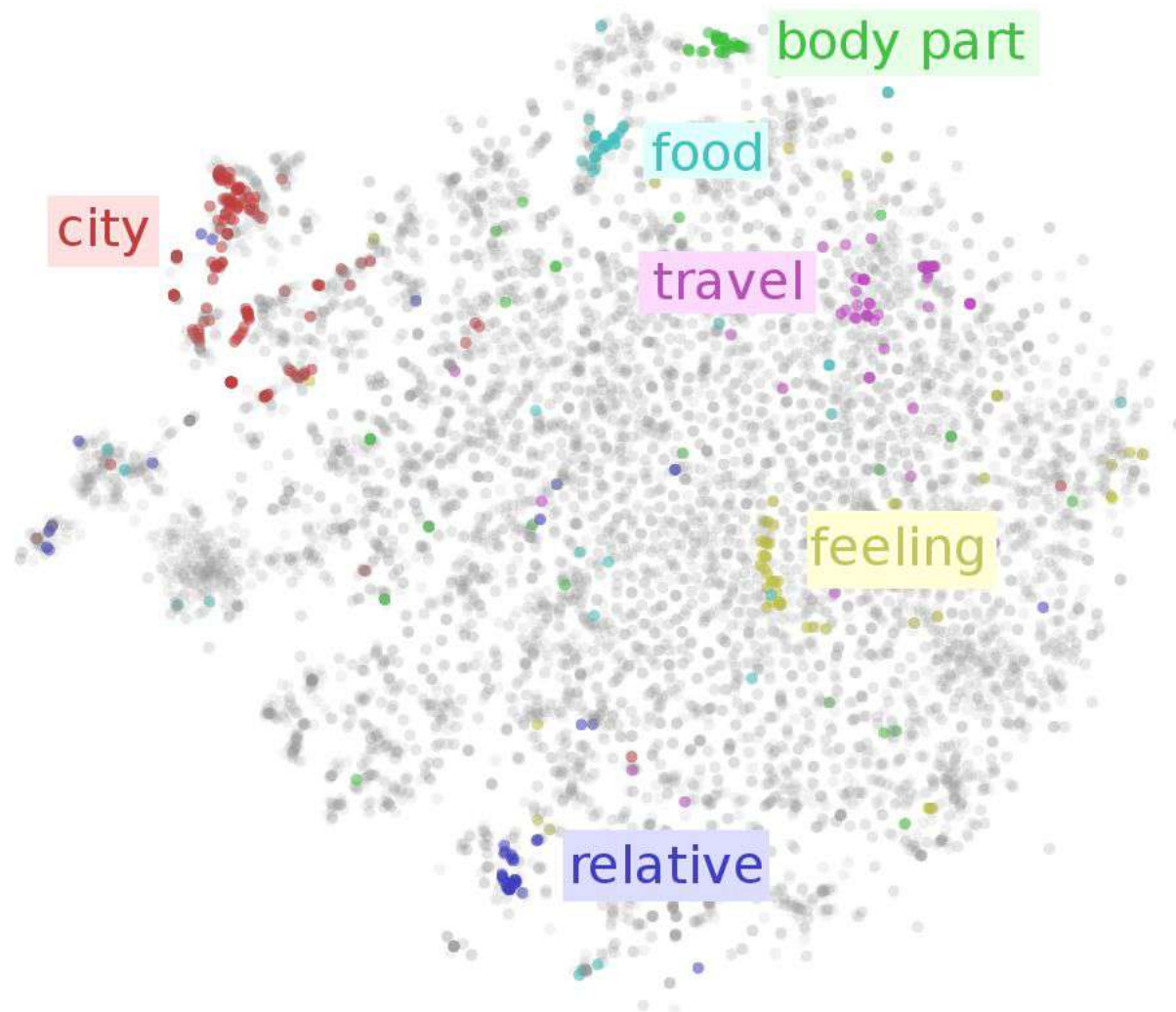


Word2vec: word analogies

King - man + woman = queen
↓ ↓ ↓ ↓
 x y y' $target$

$\cos(x - y + y', target) \rightarrow \max_{target}$





Word vectors are vectors of numbers that represent the meaning of a word

Approaches:

- One-hot encoding
- Bag-of-words models
- Counts of word / context co-occurrences
- TF-IDF
- Neural models for prediction of co-occurrence events (word2vec) or counts (glove)

Lecture 02: CNN for texts, embeddings for different languages

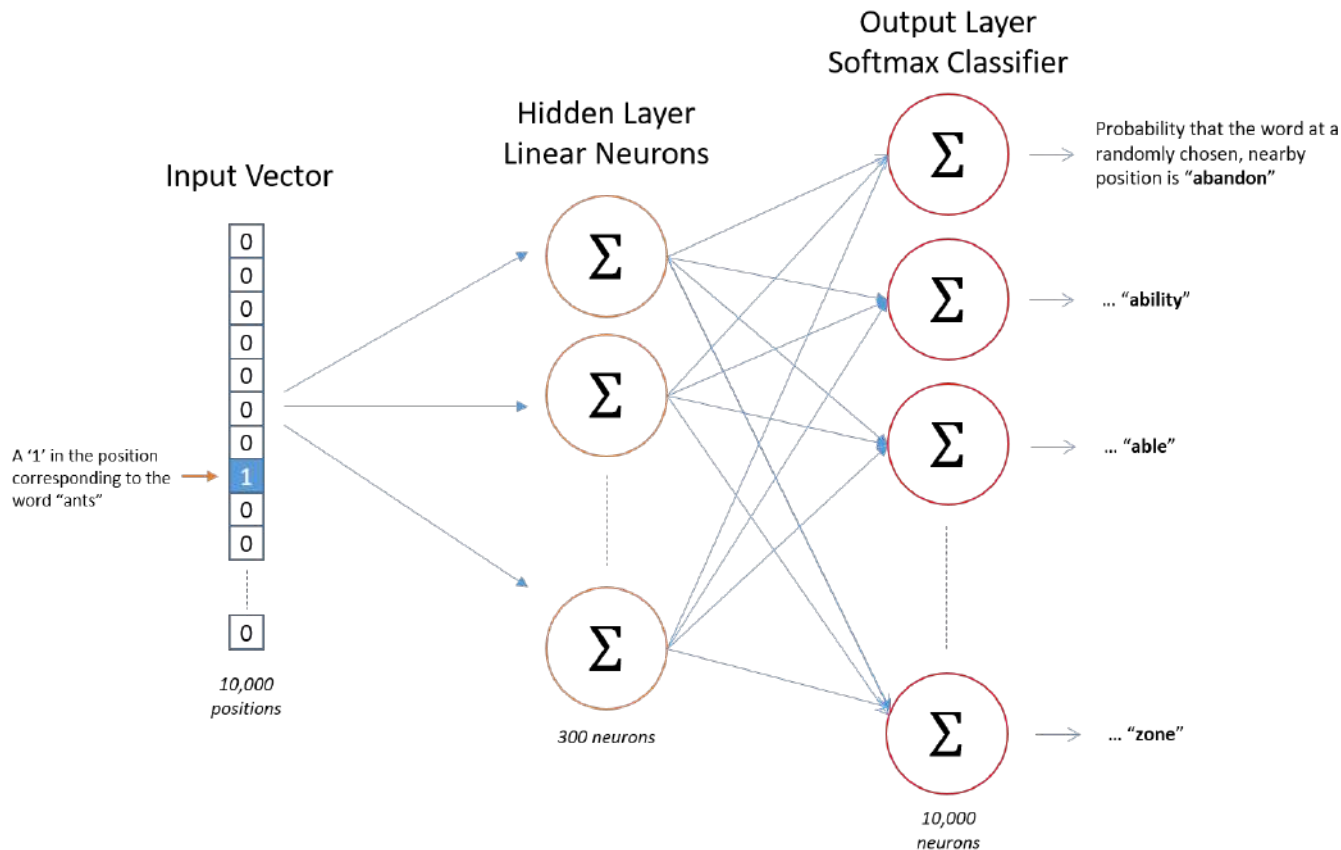
Radoslav Neychev

- Embeddings in the wild
 - Recap
 - Unsupervised translation
- RNNs recap:
- CNNs for text processing

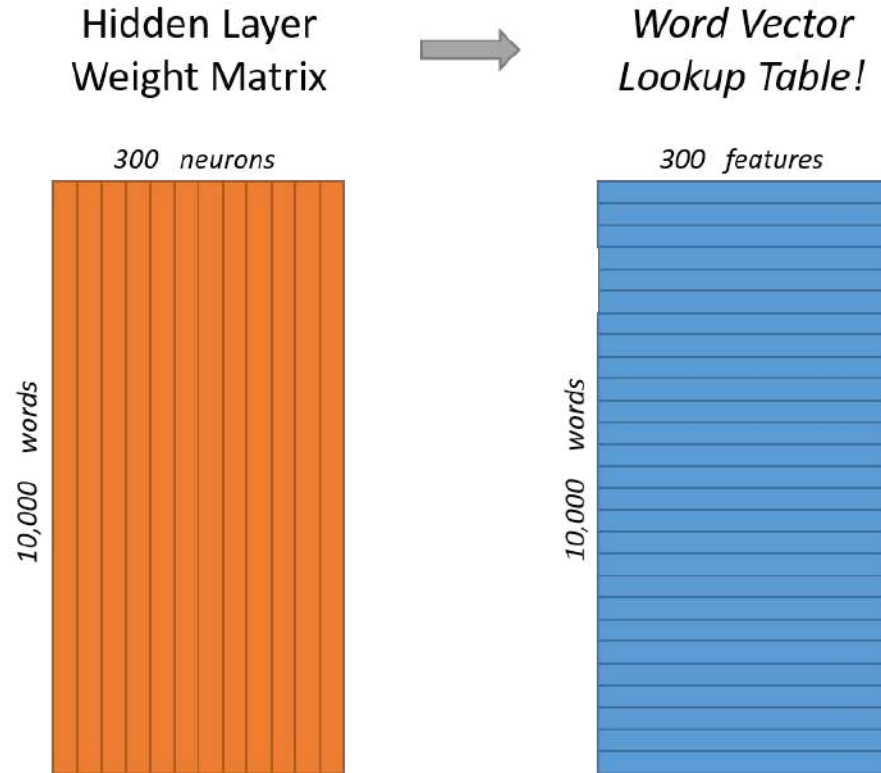
Embeddings: word2vec

Source Text	Training Samples					
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡	The	quick	brown	(the, quick) (the, brown)		
The	quick	brown				
The <table><tr><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)		
quick	brown	fox				
The quick <table><tr><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)		
brown	fox	jumps				
The <table><tr><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡	quick	brown	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
quick	brown	fox	jumps	over		

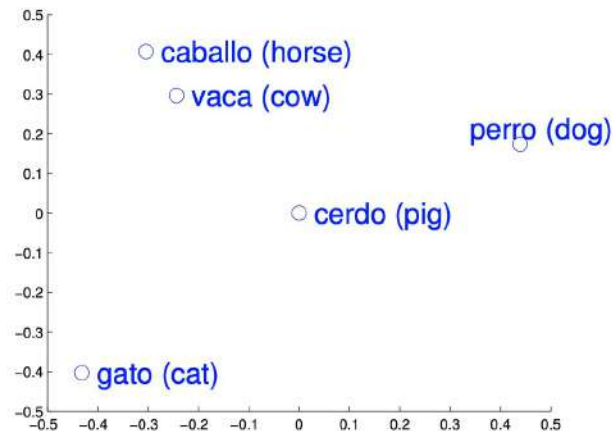
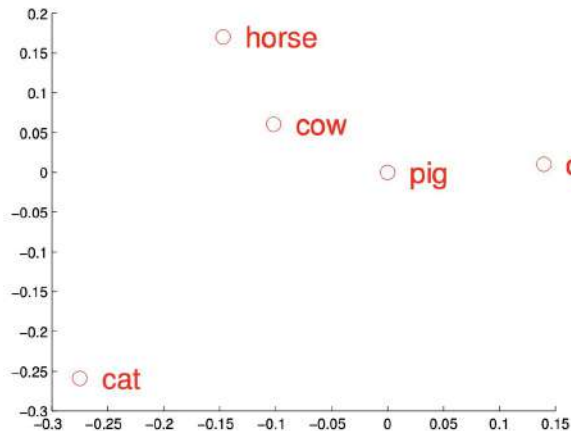
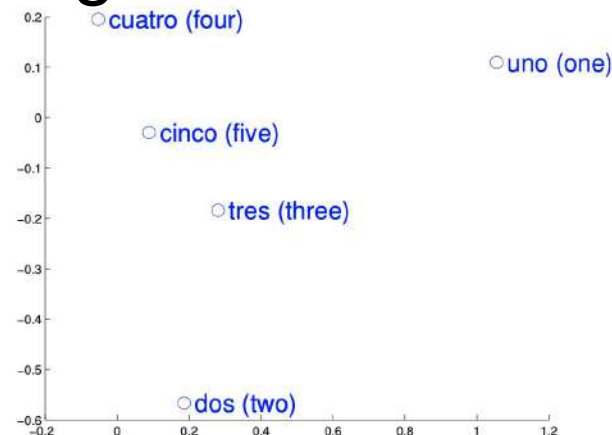
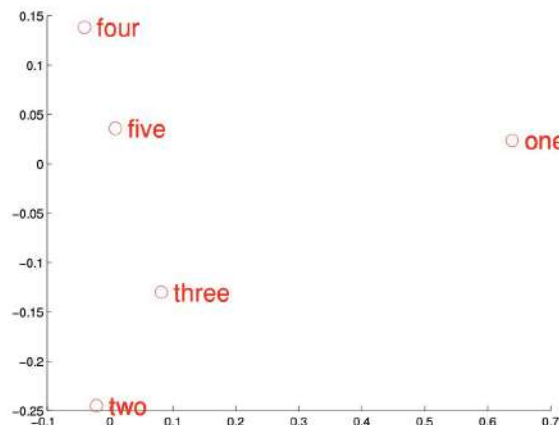
Embeddings: word2vec



Embeddings: word2vec



Word embeddings in different languages



Word embeddings in different languages

- Word embeddings are quite similar for different languages
- Assume there $n = 5000$ word-translation pairs $\{x_i, y_i\}_{i \in \{1, n\}}$
- Learn linear mapping between the source and target spaces

$$W^* = \underset{W \in M_d(\mathbb{R})}{\operatorname{argmin}} \|WX - Y\|_F$$

- The translation of source word is $t = \operatorname{argmax}_t \cos(Wx_s, y_t)$.

Word embeddings in different languages

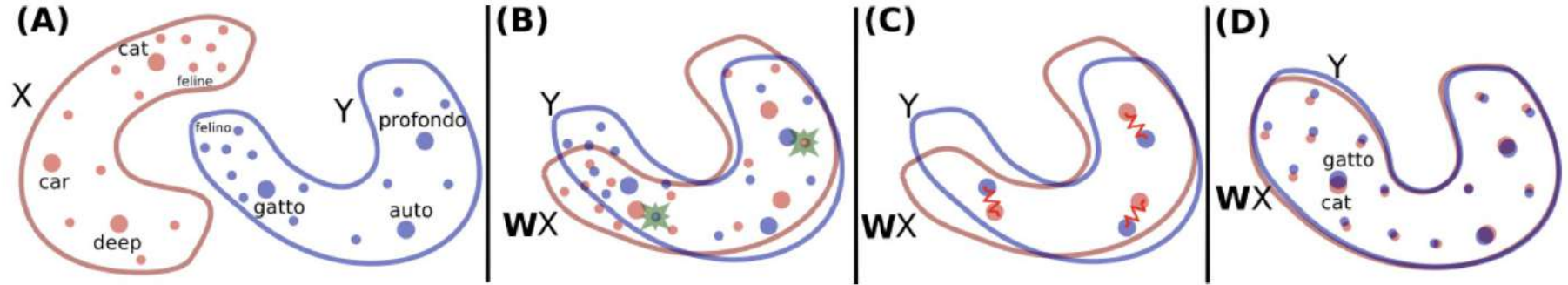
- Word embeddings are quite similar for different languages
- Assume there $n = 5000$ word-translation pairs $\{x_i, y_i\}_{i \in \{1, n\}}$
- Learn linear mapping between the source and target spaces

enforcing an orthogonality constraint on W :

$$W^* = \underset{W \in O_d(\mathbb{R})}{\operatorname{argmin}} \|WX - Y\|_F = UV^T, \text{ with } U\Sigma V^T = \operatorname{SVD}(YX^T).$$

- The translation of source word is $t = \operatorname{argmax}_t \cos(Wx_s, y_t)$.

Word embeddings in different languages



Comment: mapping between two languages can be done completely in unsupervised manner with GANs.

We will meet later.

More info available in the mentioned paper:

Source: [Word translation without parallel data, ICLR 2018](#)

Why cosine distance/similarity?

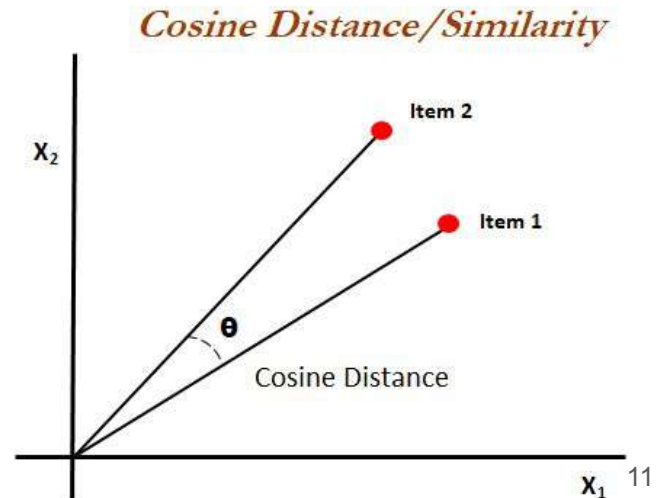
$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Cosine distance focuses on angle between the vectors.

With count-based approaches (e.g. BOW)

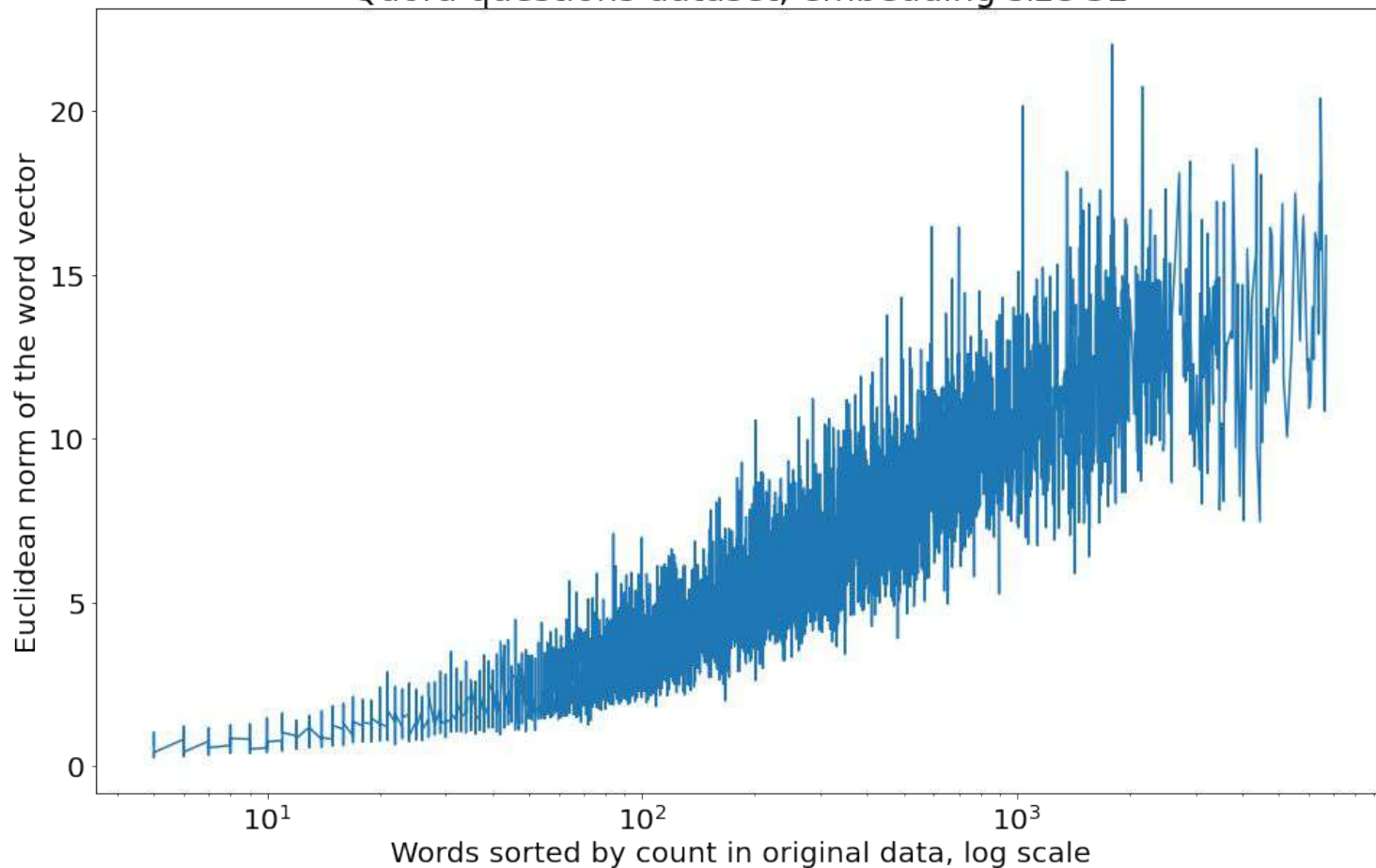
it is really useful.

With word embeddings it is useful as well.



How word frequency affects the embedding vector norm

Quora questions dataset, embedding size 32



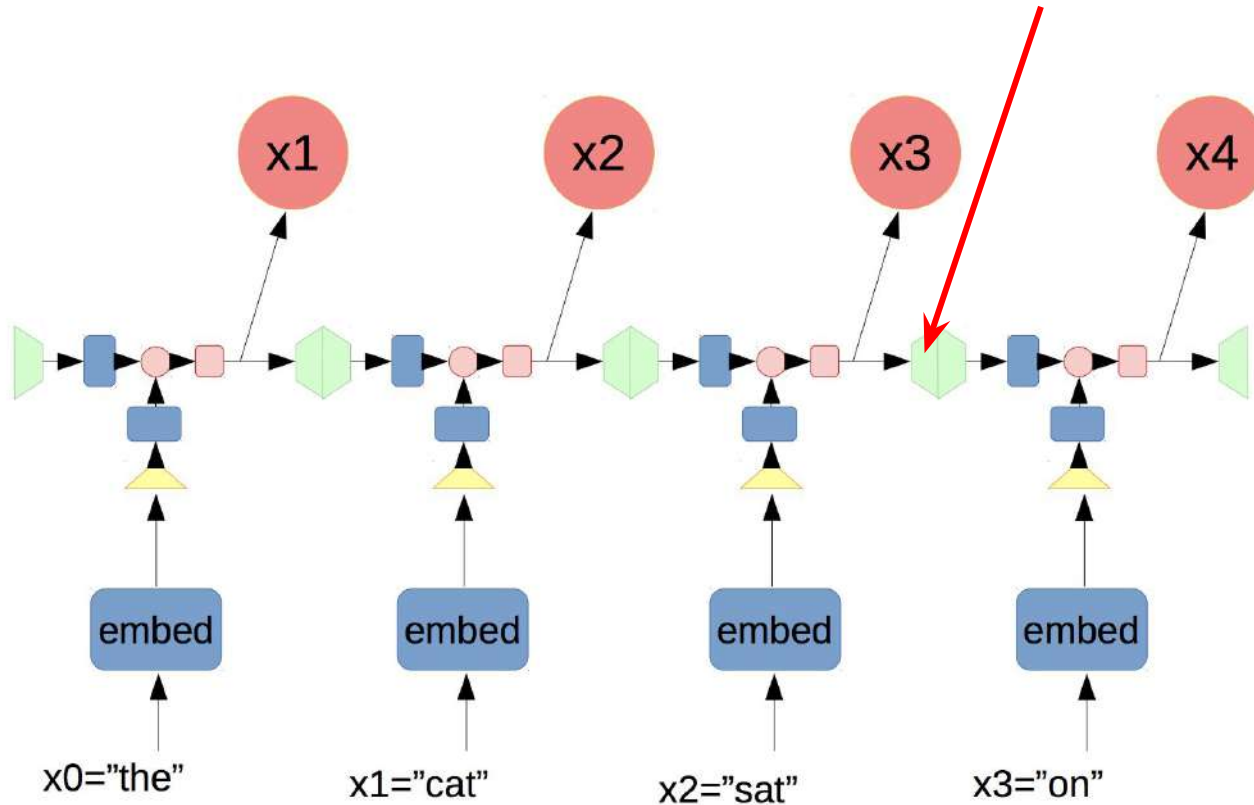
Vector norms for words with no specific context

word	count	vector norm
-----	-----	-----
overheat	11	0.81233
enormous	12	0.807057
dog	1212	11.2591
cat	1545	10.3738
laptop	1906	14.5192
phone	4124	15.7901
a	155726	11.4656
the	252068	8.47355

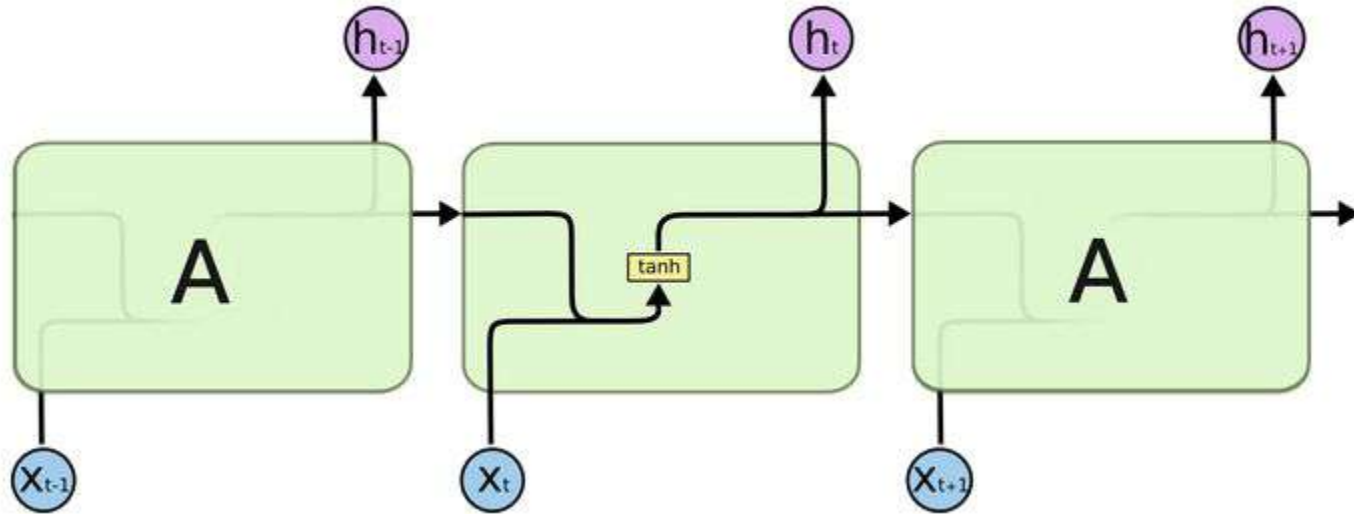
How to deal with texts?

Recap: RNN

Here is the embedding
for phrase [x0, x1, x2]

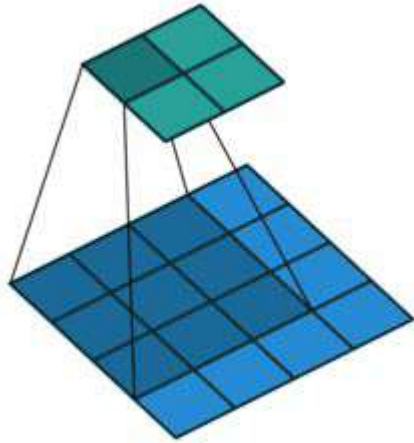


Recap: Vanilla RNN

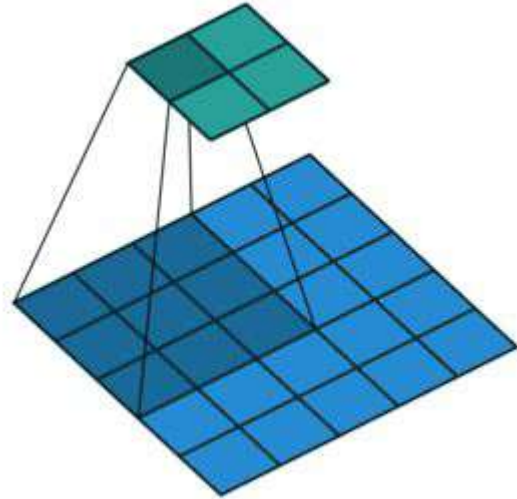


Applying CNNs to texts

CNN simple recap

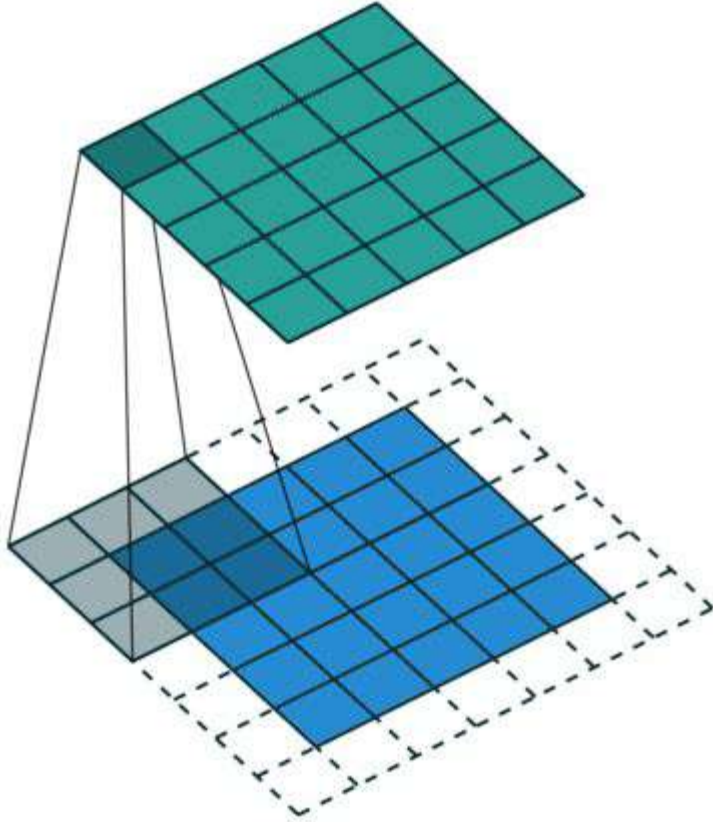


No padding,
no strides

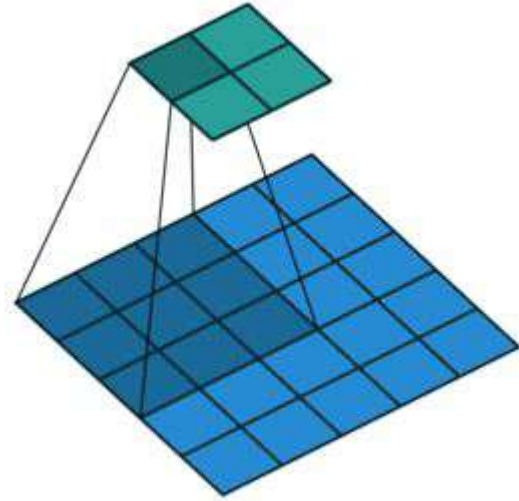


No padding,
with strides

CNN simple recap

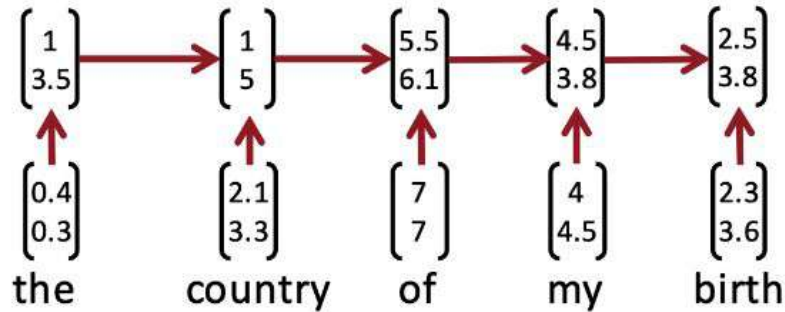


With padding,
no strides



No padding,
with strides

From RNN to CNN

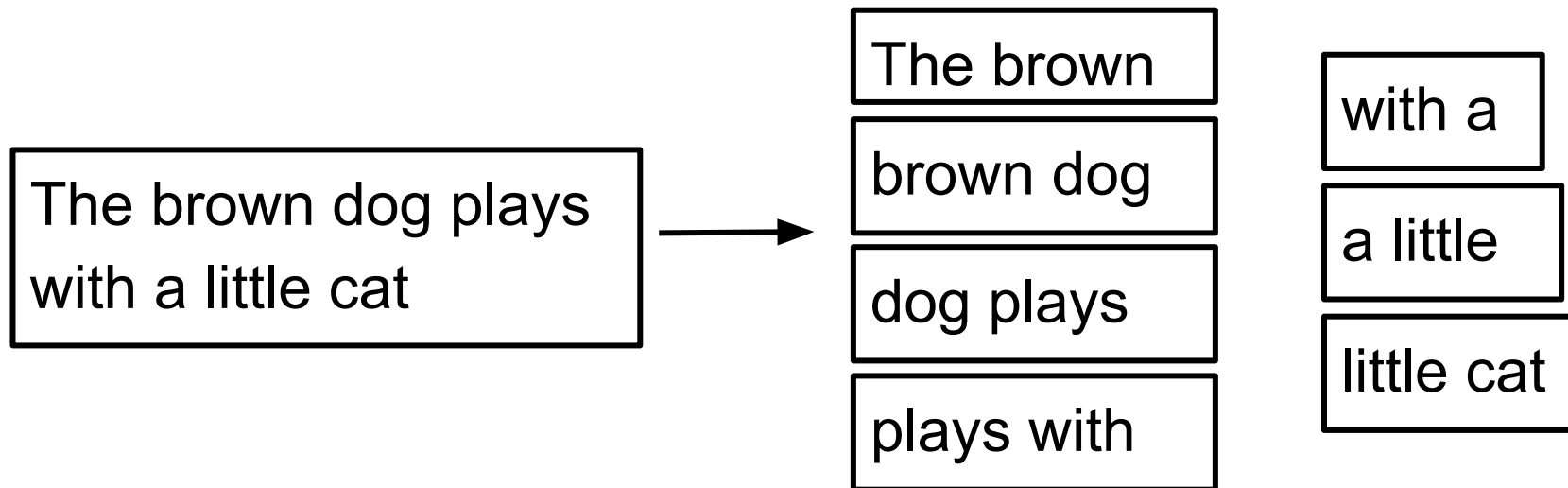


- Recurrent neural nets can not capture phrases without prefix context and often capture too much of last words in final vector

From RNN to CNN

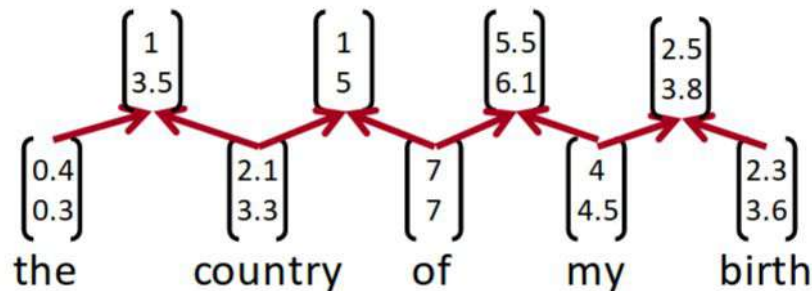
- RNN: Get compositional vectors for grammatical phrases only
- CNN: What if we compute vectors for every possible phrase?
 - Example: “*the country of my birth*” computes vectors for:
 - *the country, country of, of my, my birth, the country of, country of my, of my birth, the country of my, country of my birth*
- Regardless of whether it is grammatical
- Wouldn't need parser
- Not very linguistically or cognitively plausible

Recap: n-gramms



From RNN to CNN

- Imagine using only bigrams



- Same operation as in RNN, but for every pair

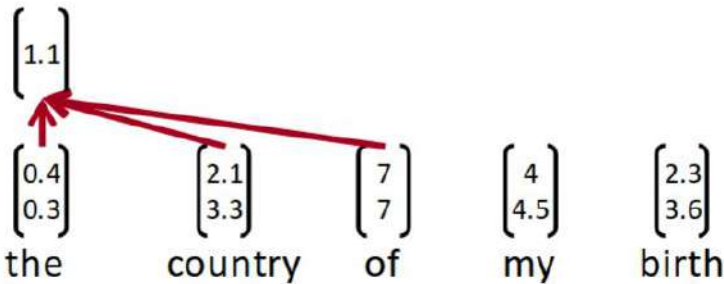
$$p = \tanh \left(W \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} + b \right)$$

- Can be interpreted as convolution over the word vectors

One layer CNN

- Simple convolution + pooling
- Window size may be different (2 or more)
- The feature map based on bigrams:

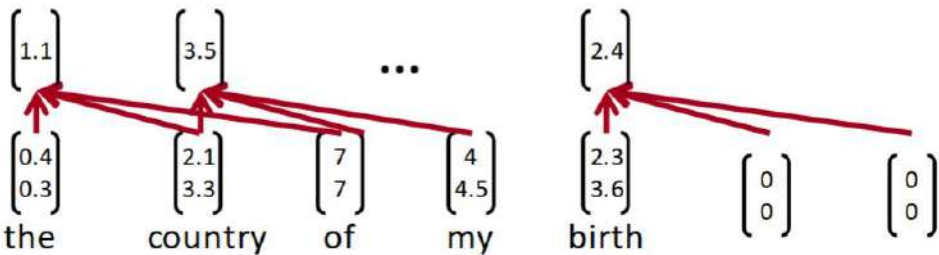
$$c_i = f(\mathbf{w}^T \mathbf{x}_{i:i+h-1} + b)$$



$$\mathbf{c} = [c_1, c_2, \dots, c_{n-h+1}] \in \mathbb{R}^{n-h+1}$$

What's next?

We need more features!



One layer CNN

- Feature representation is based on some applied filter:

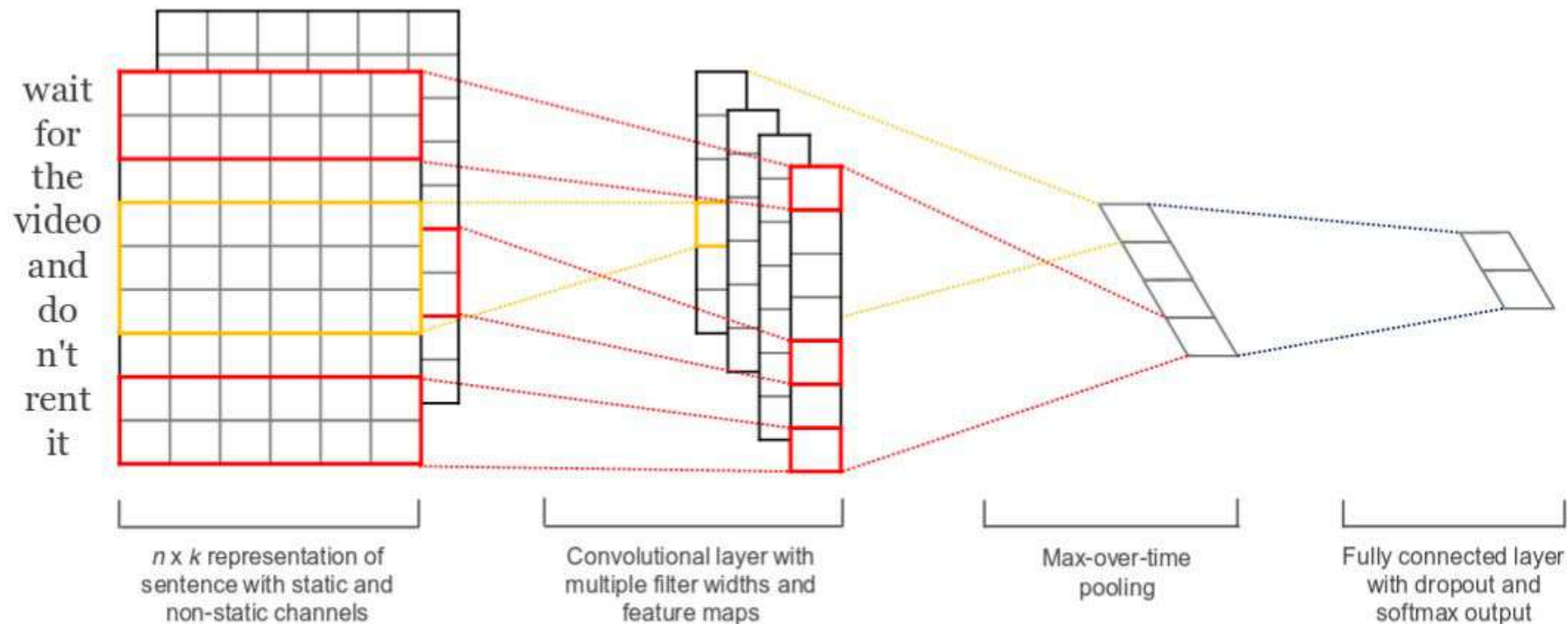
$$\mathbf{c} = [c_1, c_2, \dots, c_{n-h+1}] \in \mathbb{R}^{n-h+1}$$

- Let's use pooling:

$$\hat{c} = \max\{\mathbf{c}\}$$

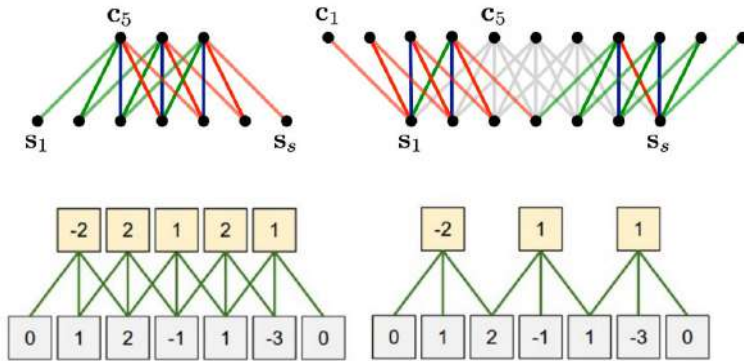
- Now the length of $\mathbf{c}$ is irrelevant!
- So we can use filters based on unigrams, bigrams, tri-grams, 4-grams, etc.

Another example from Kim (2014) paper

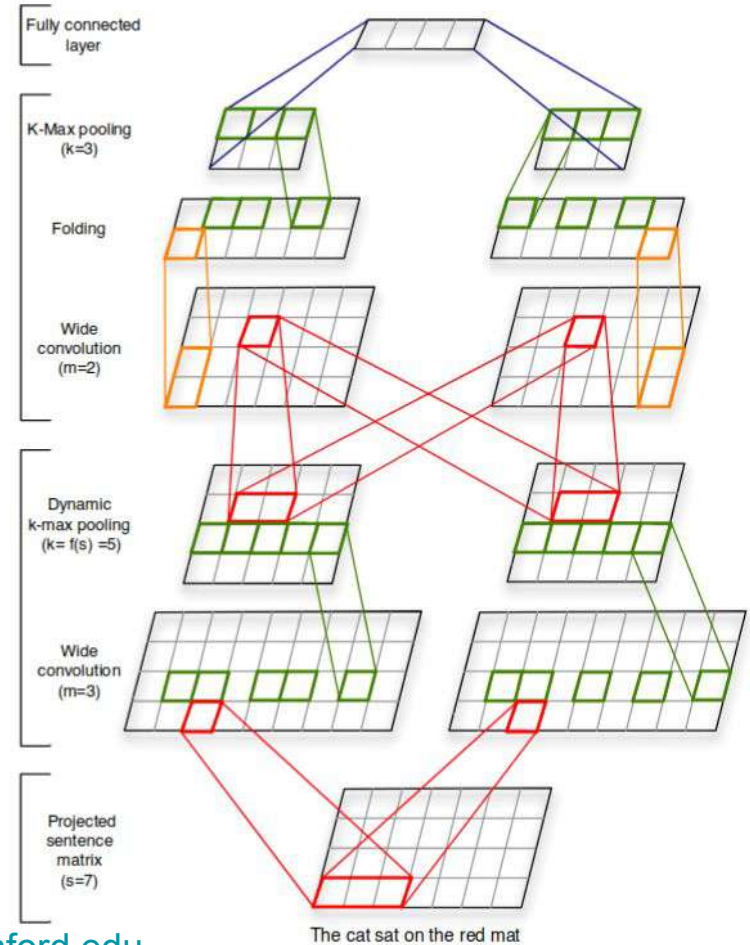


More about CNN

- Narrow vs wide convolution (stride and zero-padding)

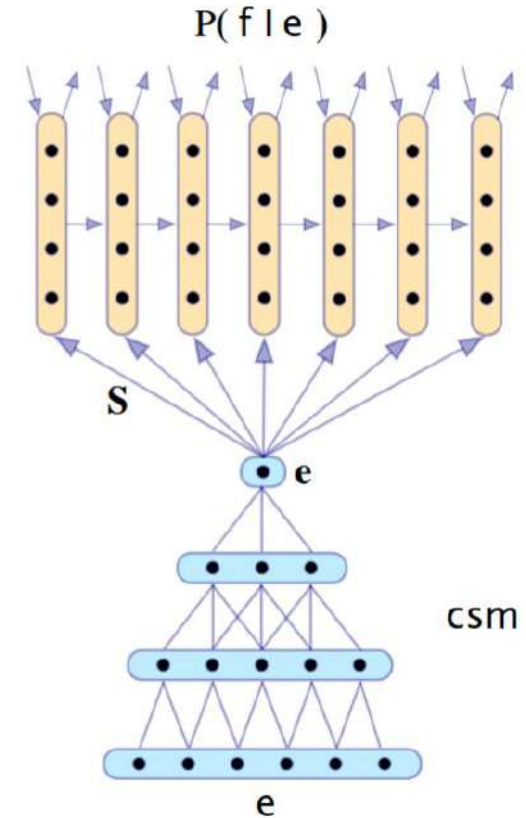


- Complex pooling schemes over sequences
- Great readings (e.g. Kalchbrenner et. al. 2014)



CNN applications

- Neural machine translation: CNN as encoder, RNN as decoder
- One of the first neural machine translation efforts
- Paper: [Recurrent Continuous Translation Models, Kalchbrenner and Blunsom, 2013](#)



Approaches comparison

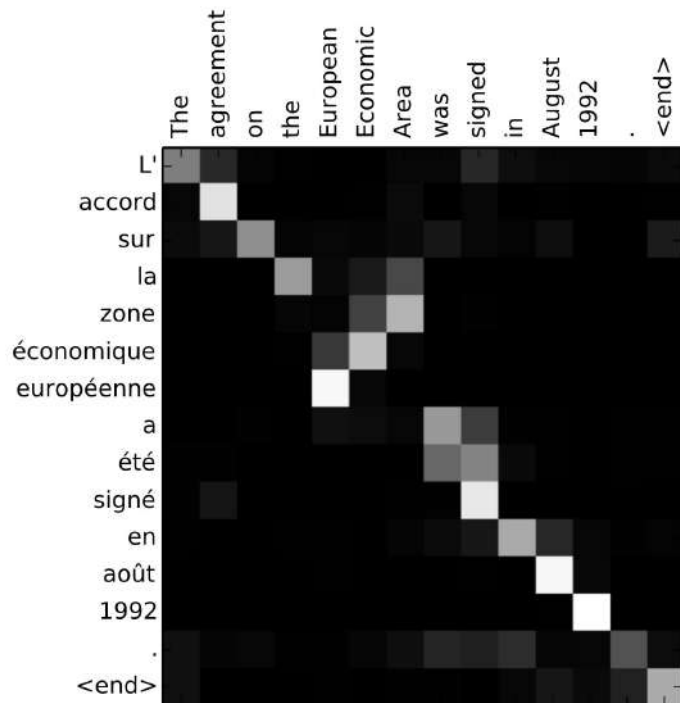
Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-rand	76.1	45.0	82.7	89.6	91.2	79.8	83.4
CNN-static	81.0	45.5	86.8	93.0	92.8	84.7	89.6
CNN-non-static	81.5	48.0	87.2	93.4	93.6	84.3	89.5
CNN-multichannel	81.1	47.4	88.1	93.2	92.2	85.0	89.4
RAE (Socher et al., 2011)	77.7	43.2	82.4	—	—	—	86.4
MV-RNN (Socher et al., 2012)	79.0	44.4	82.9	—	—	—	—
RNTN (Socher et al., 2013)	—	45.7	85.4	—	—	—	—
DCNN (Kalchbrenner et al., 2014)	—	48.5	86.8	—	93.0	—	—
Paragraph-Vec (Le and Mikolov, 2014)	—	48.7	87.8	—	—	—	—
CCAE (Hermann and Blunsom, 2013)	77.8	—	—	—	—	—	87.2
Sent-Parser (Dong et al., 2014)	79.5	—	—	—	—	—	86.3
NBSVM (Wang and Manning, 2012)	79.4	—	—	93.2	—	81.8	86.3
MNB (Wang and Manning, 2012)	79.0	—	—	93.6	—	80.0	86.3
G-Dropout (Wang and Manning, 2013)	79.0	—	—	93.4	—	82.1	86.1
F-Dropout (Wang and Manning, 2013)	79.1	—	—	93.6	—	81.9	86.3
Tree-CRF (Nakagawa et al., 2010)	77.3	—	—	—	—	81.4	86.1
CRF-PR (Yang and Cardie, 2014)	—	—	—	—	—	82.7	—
SVM _S (Silva et al., 2011)	—	—	—	—	95.0	—	—

- Vanishing gradient is present not only in RNNs
 - Use some kind of memory or skip-connections
- LSTM and GRU are both great
 - GRU is quicker, LSTM catch more complex dependencies
- Clip your gradients
- Using CNNs for texts is similar to n-gramm trick
- CNNs are more effective in case of massive computations
- Combining RNN and CNN worlds? Why not

Attention outro

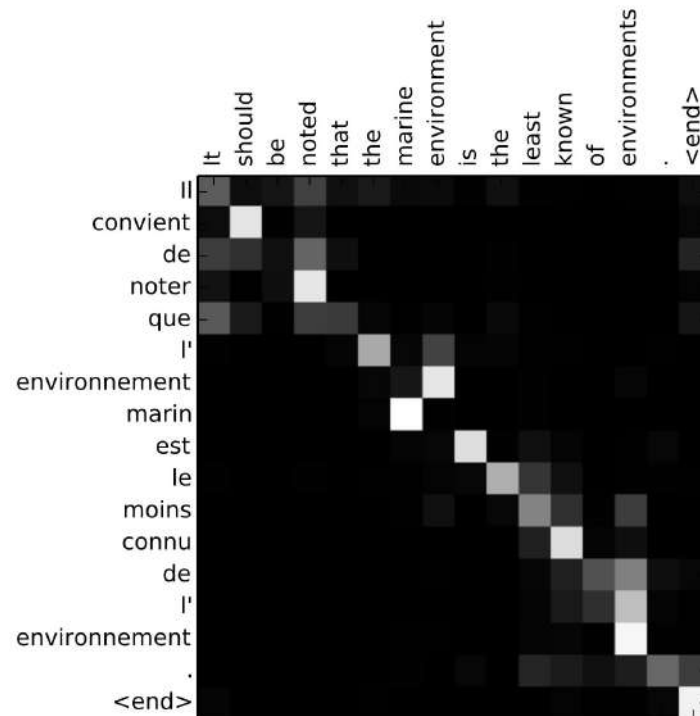
source language (English)

target language (French)



source language (English)

target language (French)



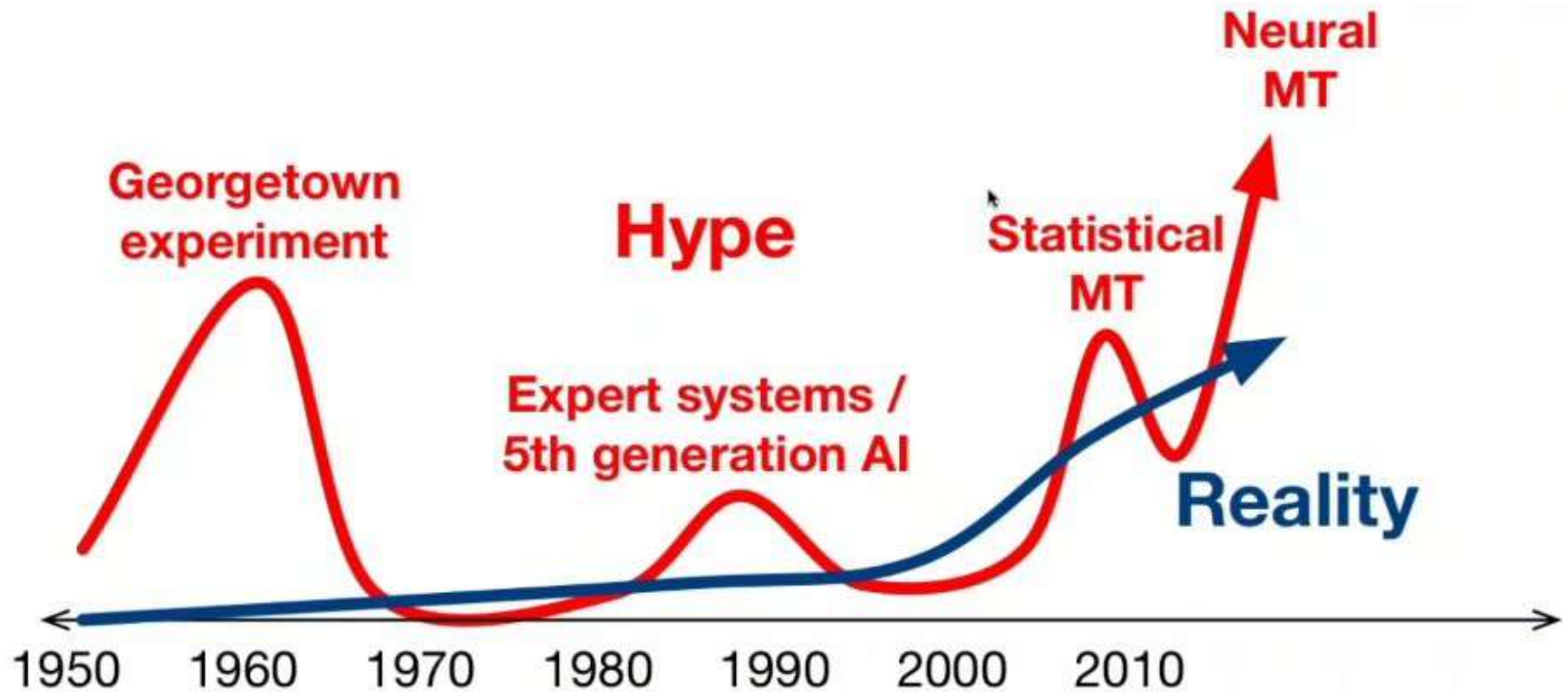
Word2vec embeddings capture only **local** context

Machine Translation Attention Mechanism

Radoslav Neychev

- Machine Translation historical overview
 - Statistical Machine Translation
 - Word alignments
- Neural Machine Translation (NMT)
 - Seq2Seq
 - Beam Search
- Attention mechanism

Historical overview



Before Deep Learning

1950s: first Machine Translation

- Georgetown experiment (7 Jan 1954)
 - Automatic Russian-English translation of 60 sentences
 - 250 vocabulary articles
 - 6 grammar rules
 - Calculated on Mainframe IBM 701
- The same experiment in the USSR (1954 too)
 - Rule-based translation
 - Calculated on BESM

MT Training Data

- Parallel corpora
 - Pairs <source, translation>
 - Typically sentence-level (although sometimes can be paragraph- or doc-level)
 - Can be manually curated by translators, crawled from web or synthetic

Source	角柱を過ぎる粘性流体の乱流をラージエディシミュレーションし、フィルタ幅と数値粘性の影響を調べた。
Reference	<u>The large eddy simulation</u> of a turbulent flow of a viscous fluid passing through a square column was conducted, and the effects of the filter width and numerical viscosity were examined.

1990-2010: Statistical Machine Translation

Want to find best English sentence y , given French sentence x

Let's use Bayes Rule to break this down into two components:

$$\begin{aligned} & \operatorname{argmax}_y P(y|x) \\ &= \operatorname{argmax}_y \underbrace{P(x|y)}_{\text{Translation Model}} \underbrace{P(y)}_{\text{Language Model}} \end{aligned}$$

Translation Model

Models how words and phrases
should be translated (*fidelity*).
Learnt from parallel data.

Language Model

Models how to write
good English (*fluency*).
Learnt from monolingual data.

1990-2010: Statistical Machine Translation

How to learn translation model from the parallel corpus?

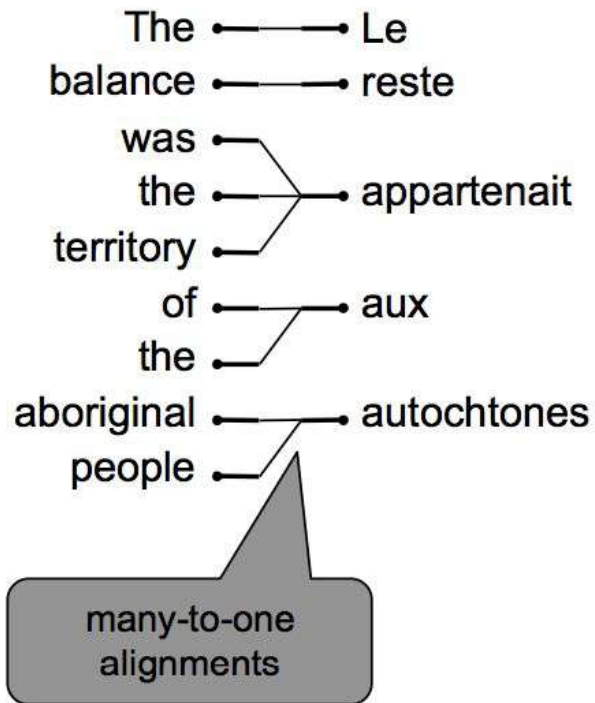
Let's calculate

$$P(x, a|y)$$

Where **a** is an **alignment** (word-level correspondence between French sentence x and English sentence y)

1990-2010: Statistical Machine Translation

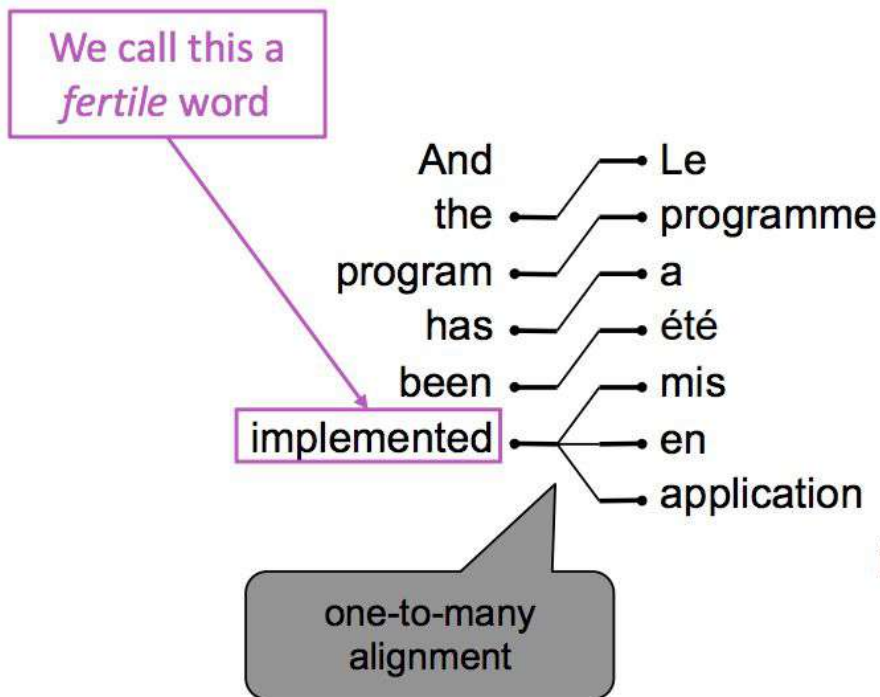
Alignment can be: **many-to-one**



	Le	reste	appartenait	aux	autochtones
The					
balance					
was					
the					
territory					
of					
the					
aboriginal					
people					

1990-2010: Statistical Machine Translation

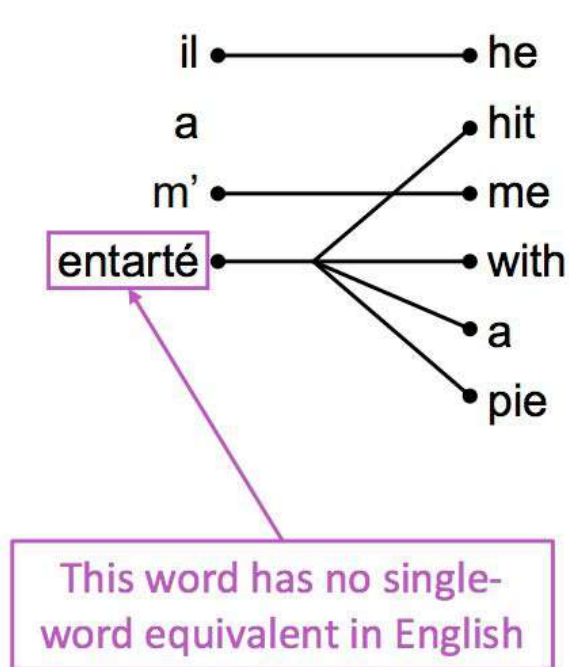
Alignment can be: **one-to-many**



	Le	programme	a	été	mis	en	application
And							
the							
program							
has							
been							
implemented							

1990-2010: Statistical Machine Translation

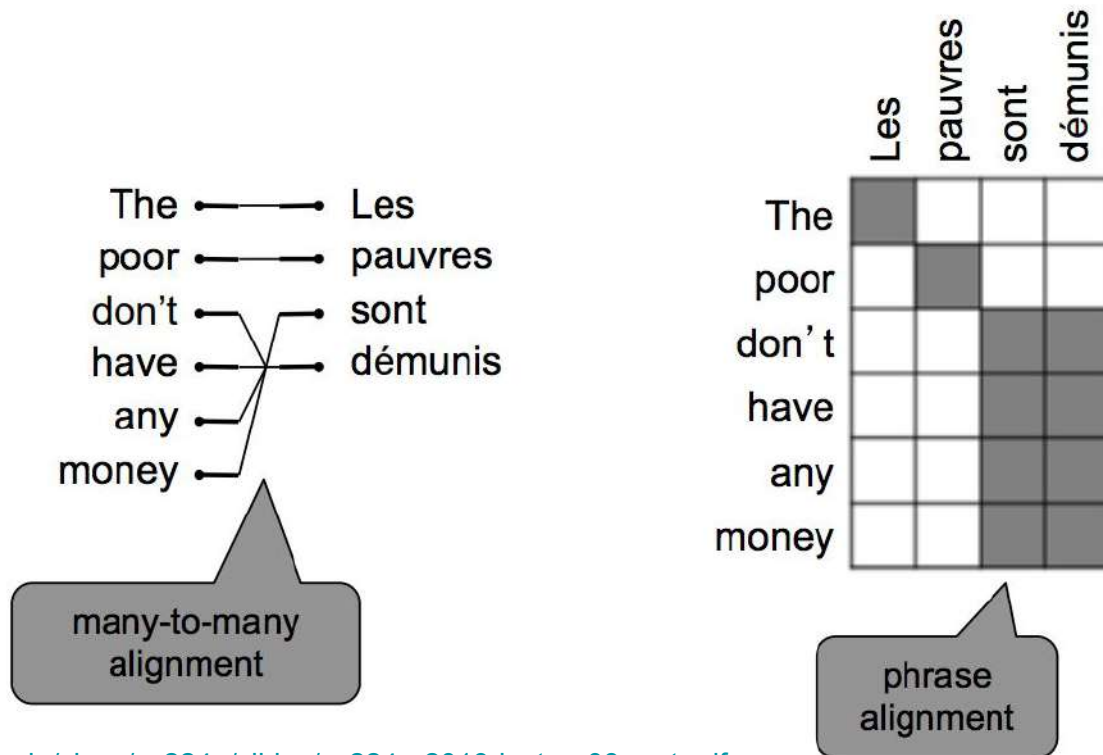
Some words are very fertile!



	he	hit	me	with	a	pie
il						
a						
m'						
entarté						

1990-2010: Statistical Machine Translation

Alignment can be: **many-to-many**



1990-2010: Statistical Machine Translation

$$\operatorname{argmax}_y P(x|y) P(y)$$


The equation is annotated with three colored curly braces underneath it. A purple brace is under the argmax_y term, a blue brace is under the $P(x|y)$ term, and a green brace is under the $P(y)$ term. Arrows point from these braces to the text labels below.

Question:

How to compute
this argmax?

Translation Model

Language Model

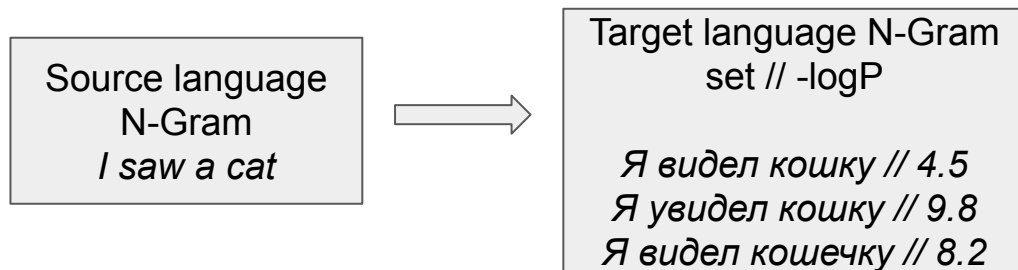
Enumerate every possible y and calculate the probability? No!

Use a heuristic search algorithm to search for the best translation, discarding hypotheses that are too low-probability

PBMT

- PBMT: phrase-based machine translation
 - N-Gram phrase table
 - N-Gram language model

Phase tables



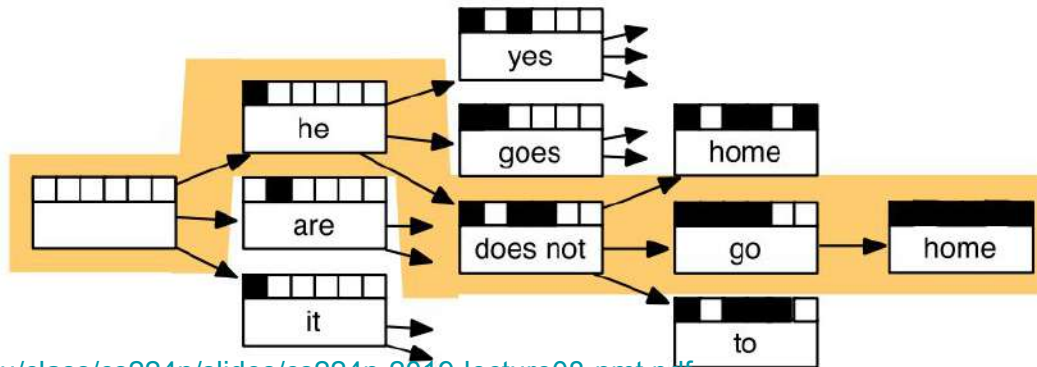
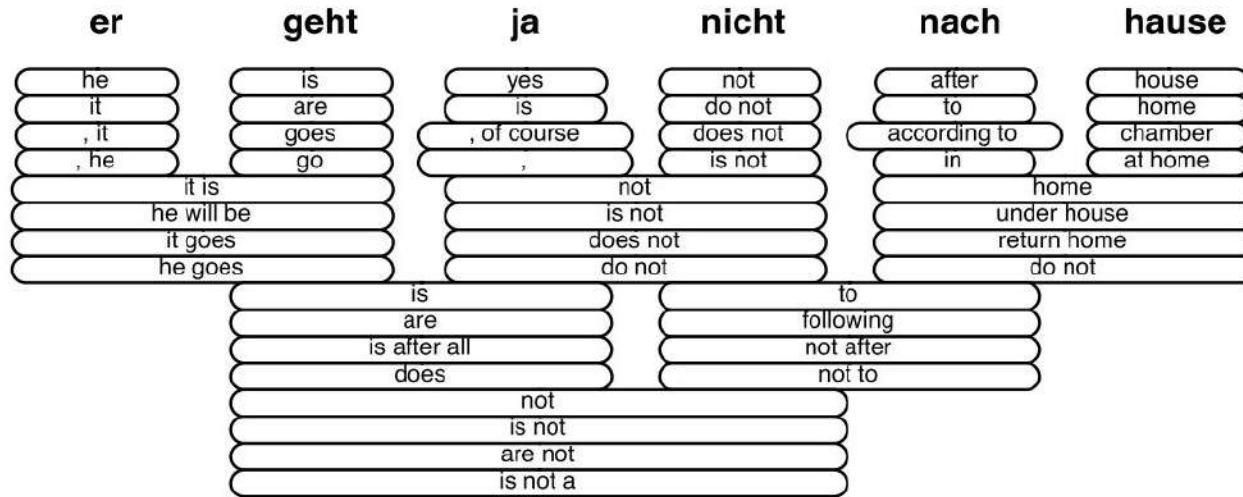
PBMT

- PBMT: phrase-based machine translation
 - N-Gram phrase table
 - N-Gram language model

NGram LM's

Source language
N-Gram // $-\log P$
(next | prefix)

I saw a cat
I saw a dog
I saw a bicycle



1990-2010: Statistical Machine Translation

- Systems had many separately-designed subcomponents
- Lots of feature engineering
- Need to design features to capture particular language phenomena
- Require compiling and maintaining extra resources (tables of equivalent phrases)
- Lots of human effort to maintain
- Repeated effort for each language pair!

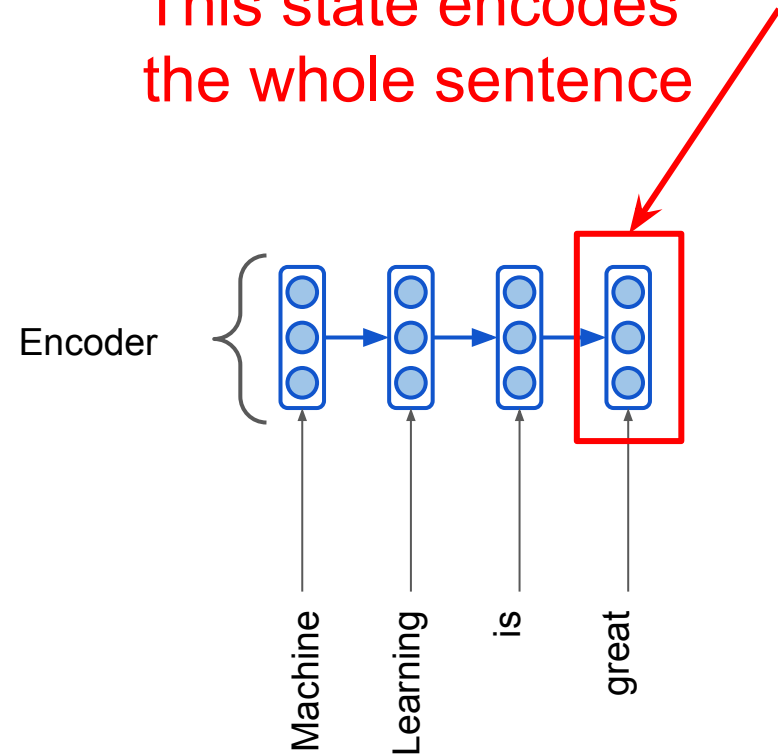
Neural Machine Translation

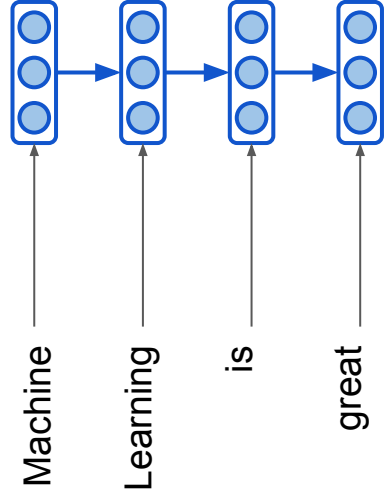
What is Neural Machine Translation?

- Neural Machine Translation (NMT) is a way to do Machine Translation with a single neural network
- The neural network architecture is called sequence-to-sequence (aka **seq2seq**), it involves two **RNNs**

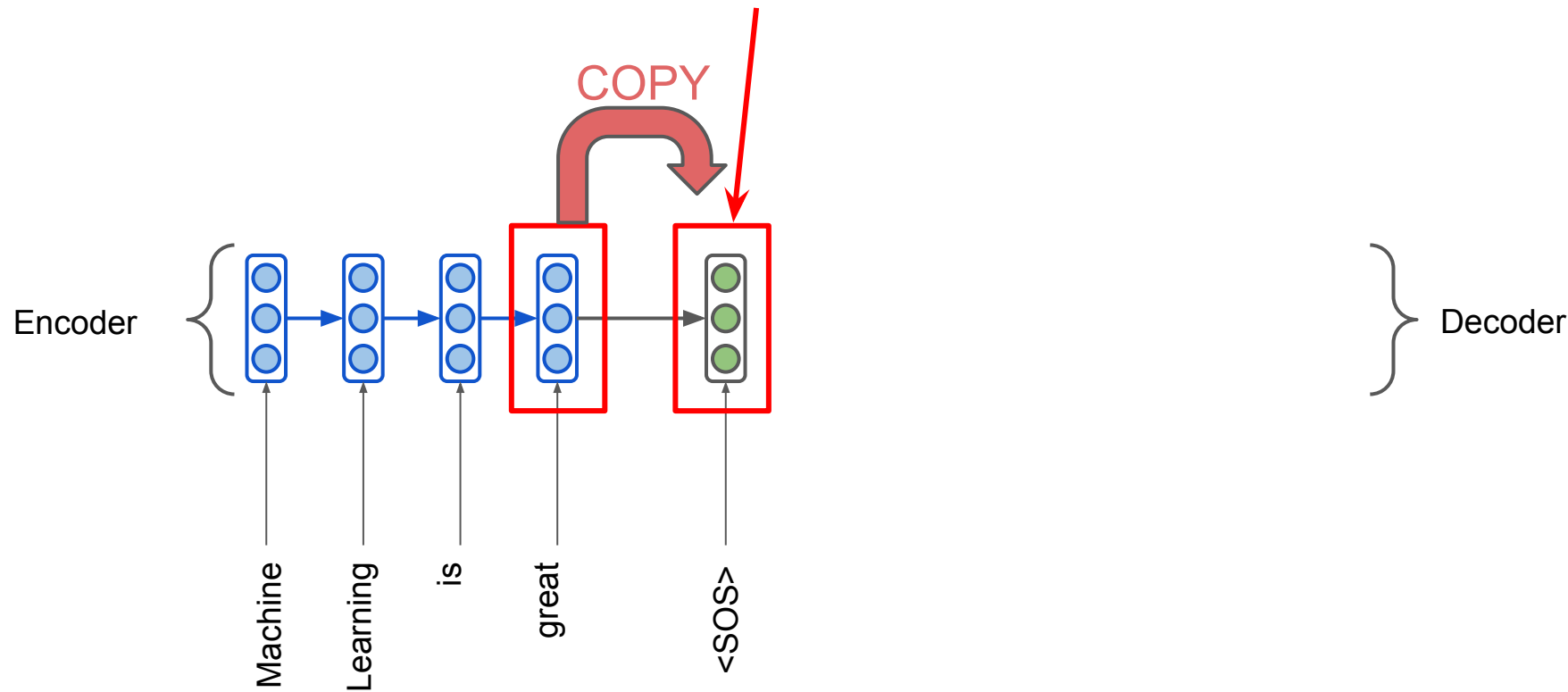
Seq2seq NMT

This state encodes
the whole sentence

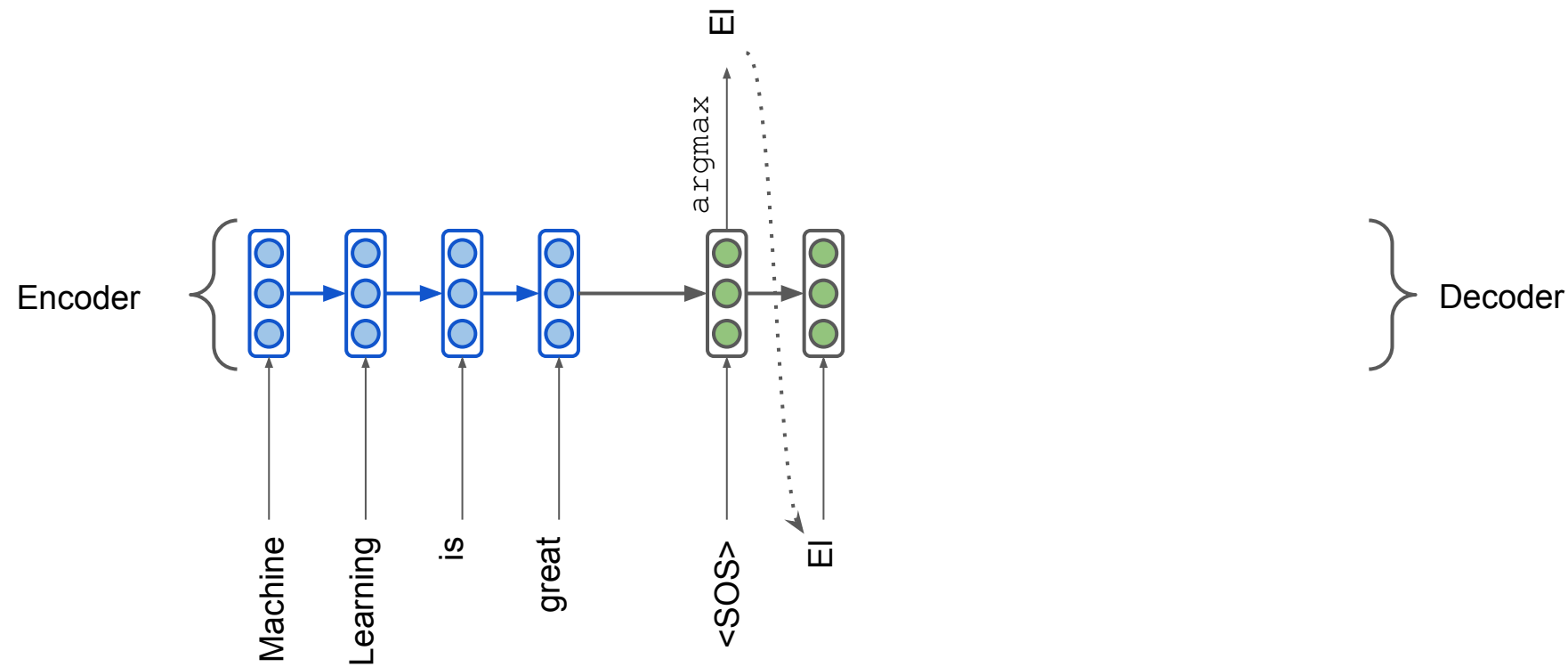




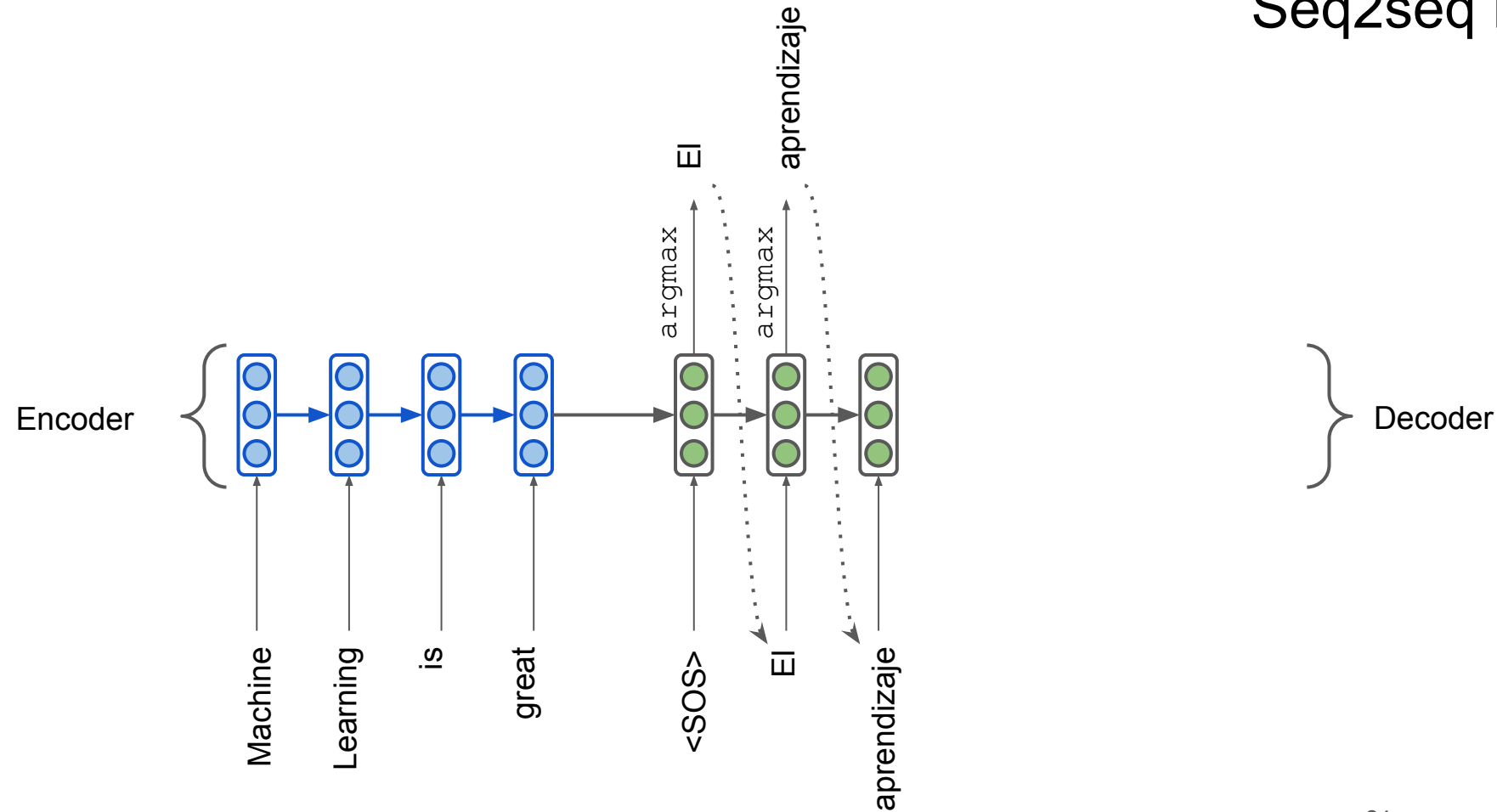
Forwarded as initial
hidden state to decoder



Seq2seq NMT

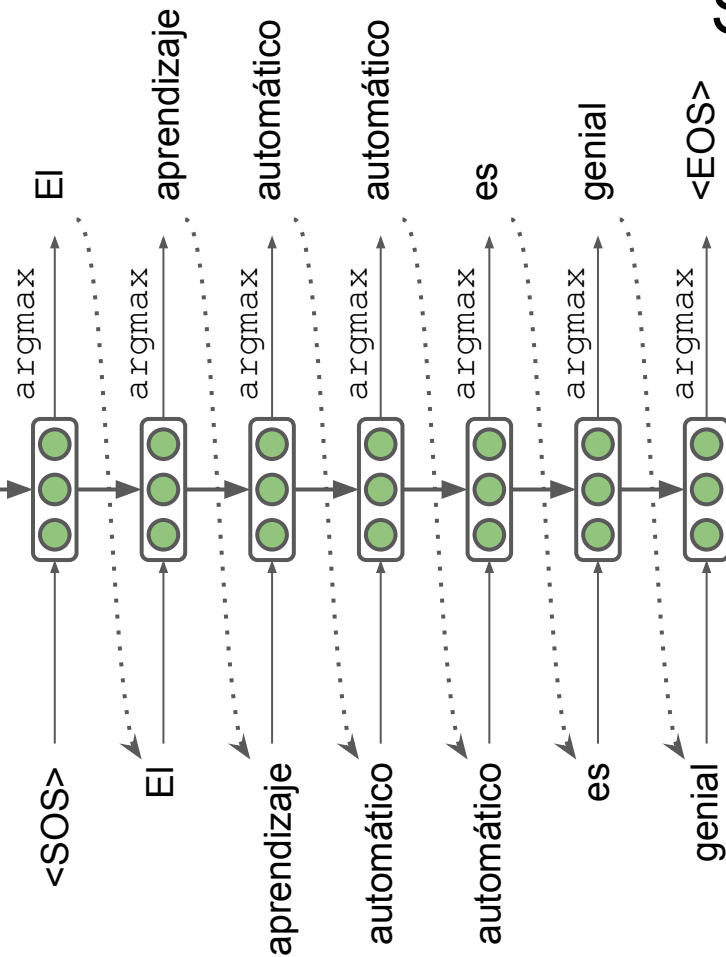
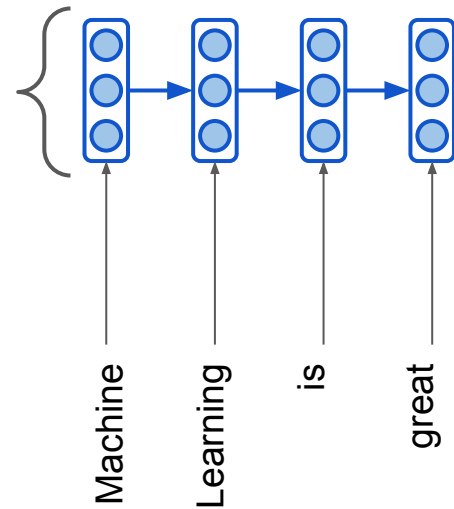


Seq2seq NMT



Seq2seq NMT

Encoder



Decoder

NMT: how does it work?

- NMT directly calculates $P(y|x)$
 - y – target sentence, x – source sentence

$$P(y|x) = P(y_2|y_1, x)P(y_3|y_1, y_2, x) \dots \underbrace{P(y_T|y_1, y_2, \dots, x)}$$

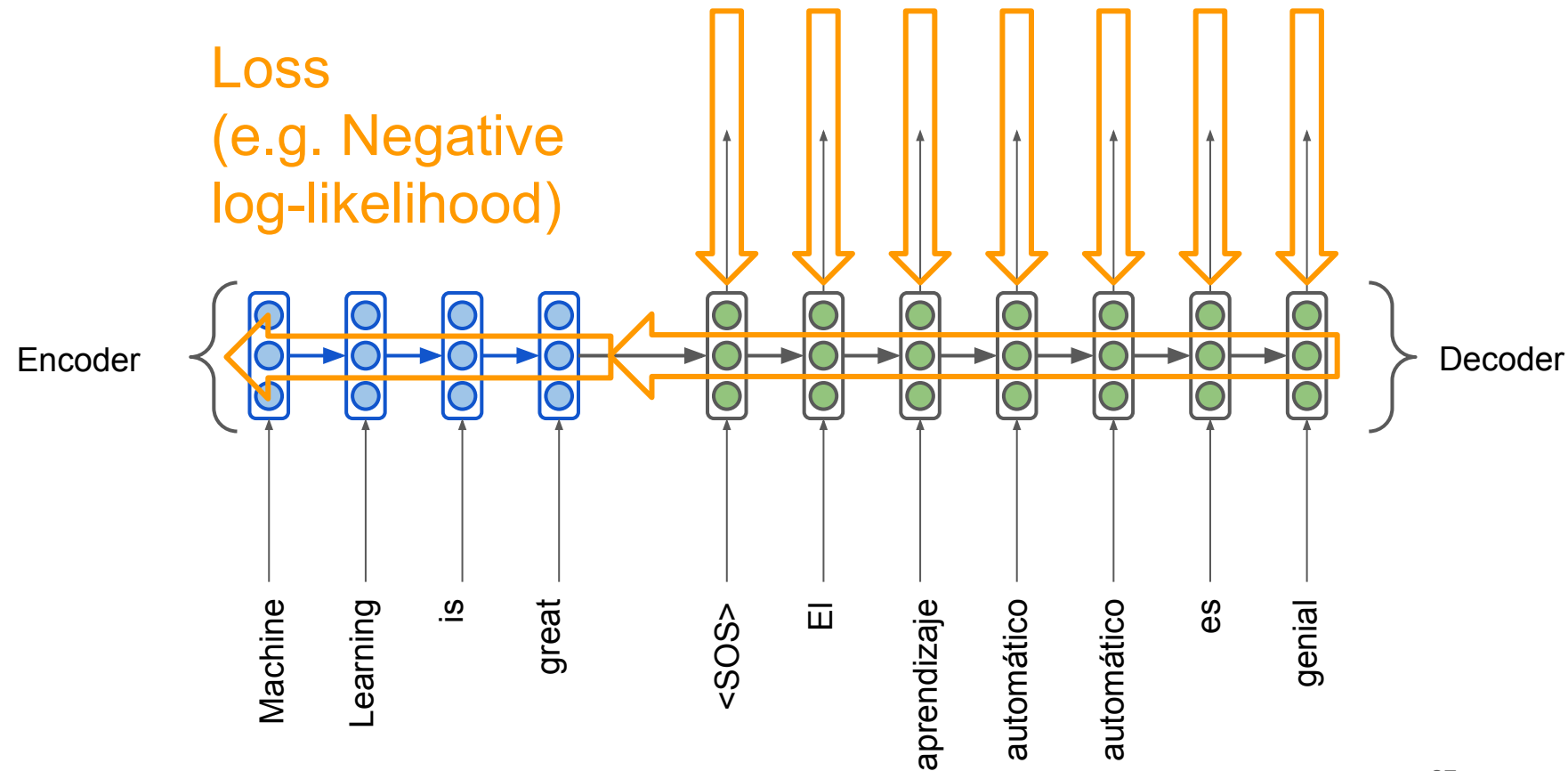
Probability of next word
in target language



- To train it we need a huge parallel corpus.

Seq2seq is trained end-to-end

Loss
(e.g. Negative
log-likelihood)

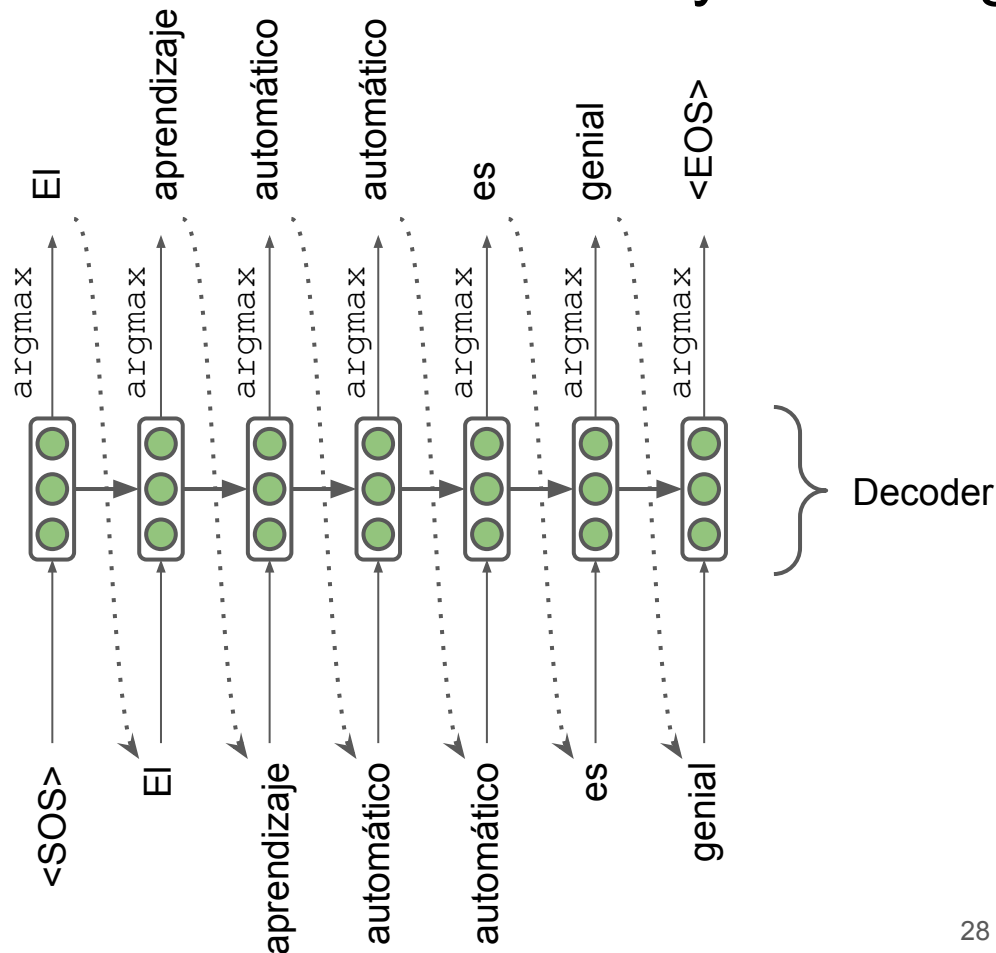


Greedy decoding

- Decoder predicts the most probable token (argmax) on each step
- The approach is **greedy**

Any problems with it?

Any mistake is treated as input on the next step!



- We want the translation that maximizes the likelihood:

$$P(y|x) = P(y_1|x) \prod_{t=2}^T P(y_t|y_1, \dots, y_{t-1}, x)$$

- We cannot compute all the possible sequences (exponential complexity)

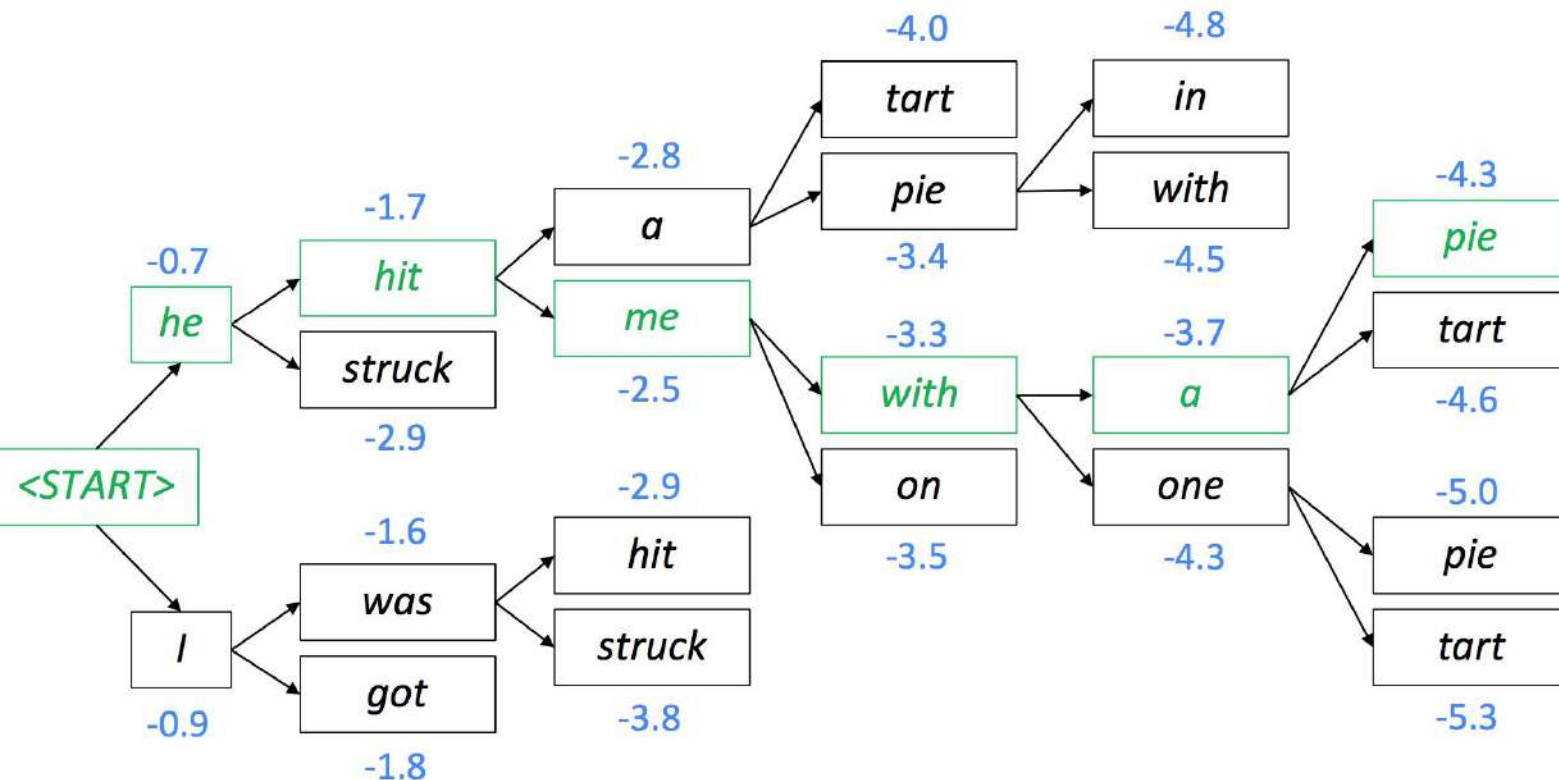
- On each step of decoder, keep track of the k most probable partial translations (which we call hypotheses)
- k is the beam size (in practice around 5 to 10)
- A hypothesis has a score which is its log probability:

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- We search for high-scoring hypotheses, tracking top k on each step
- Beam search does not guarantee finding optimal solution

Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Beam search decoding: stopping criterion

- In **greedy decoding**, usually we decode until the model produces <EOS> token
- In **beam search decoding**, different hypotheses may produce <EOS> tokens on different timesteps
 - When a hypothesis produces <EOS>, that hypothesis is complete.
 - Place it aside and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
 - We reach pre-defined timestep T
 - We have at least n completed hypotheses

Beam search decoding: finishing up

- How to select top one with highest score?
- Each hypothesis on our list has a score:

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- **Problems?**

Longer hypotheses have lower scores

- **Fix:** Normalize by length. Use this to select top one instead:

$$\frac{1}{t} \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

NMT: Quality Evaluation

BLEU (Bilingual Evaluation Understudy) compares the machine-written translation to human-written translation, and computes a similarity score based on:

- n-gram precision
- penalty for too-short system translations (brevity penalty)

$$BLEU = \text{brevity penalty} \cdot \left(\prod_{i=1}^n \text{precision}_i \right)^{1/n} \cdot 100\%$$

$$\text{brevity penalty} = \min \left(1, \frac{\text{output length}}{\text{reference length}} \right)$$

BLEU (Bilingual Evaluation Understudy) compares the machine-written translation to human-written translation, and computes a similarity score based on:

- n-gram precision
- brevity penalty

SYSTEM A: Israeli officials responsibility of airport safety
 2-GRAM MATCH 1-GRAM MATCH

REFERENCE: Israeli officials are responsible for airport security

SYSTEM B: airport security Israeli officials are responsible
 2-GRAM MATCH 4-GRAM MATCH

Metric	System A	System B
precision (1gram)	3/6	6/6
precision (2gram)	1/5	4/5
precision (3gram)	0/4	2/4
precision (4gram)	0/3	1/3
brevity penalty	6/7	6/7
BLEU	0%	52%

$$BLEU = \text{brevity penalty} \cdot \left(\prod_{i=1}^n \text{precision}_i \right)^{1/n} \cdot 100\%$$

BLEU is imperfect:

- There are many valid ways to translate a sentence
- So a good translation may get a poor BLEU score just because of low n-gram overlap with the human translation

- **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation)
- **METEOR** (Metric for Evaluation of Translation with Explicit ORdering)
 - Uses synonyms from WordNet

Quality metrics for MT

- **Human evaluation**

Side-by-side (SbS HE)

Q: Pick the best translation

Src: Beam search decoding

System1: декодирование с поиском луча декодирование поиска луча

System2: декодирование поиска луча

Direct Assessment (DA)

Q: Rate this translation of 5-point scale

Src: Beam search decoding

Translation: декодирование поиска луча

Quality metrics for MT

Human eval is expensive

Solution: ML models trained to approximate HE

1. BLEURT (Google) - BERT finetuned on DA ratings
2. XCOMET (Unbabel) - XLM finetuned on DA + MQM ratings
3. LLM-based estimators - prompt LLM to estimate MT quality

- Better performance
 - More fluent
 - Better use of context
 - Better use of phrase similarities
- A single neural network to be optimized end-to-end
 - No subcomponents to be individually optimized
- Requires much less human engineering effort
 - No feature engineering
 - Same method for all language pairs

- NMT is less interpretable
 - Hard to debug
- NMT is difficult to control
 - For example, can't easily specify rules or guidelines for translation
 - Safety concerns!

NMT: disadvantages



?

NMT: disadvantages

Somali ▾
↔

English ▾
📄 🔊

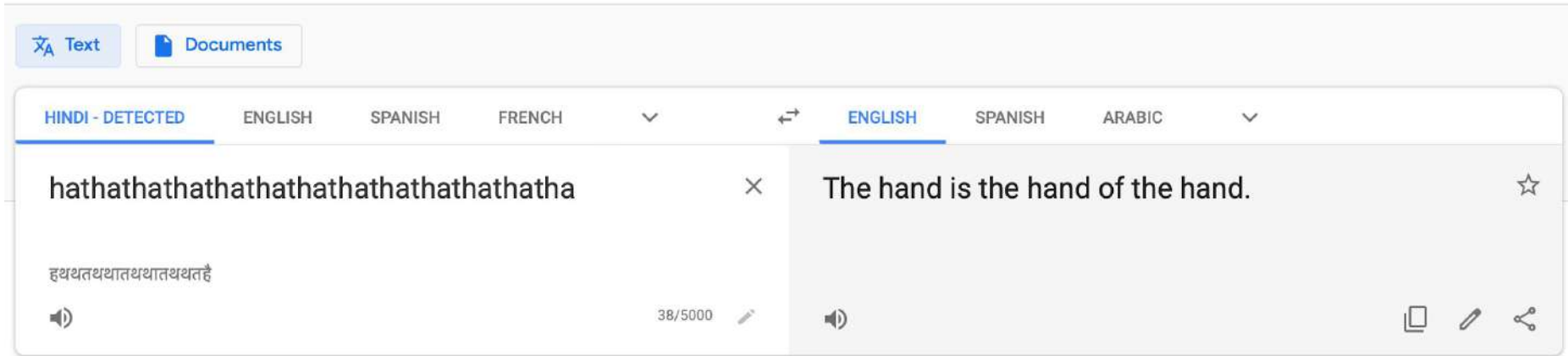
Translate from Irish

ag ag ag ag ag ag ag ag ag ag ag ag ag ag ag ag
ag ag ag ag ag ag ag ag ag ag ag ag ag ag ag
ag Edit

As the name of the LORD was written
in the Hebrew language, it was written
in the language of the Hebrew Nation

Feedback

NMT: disadvantages



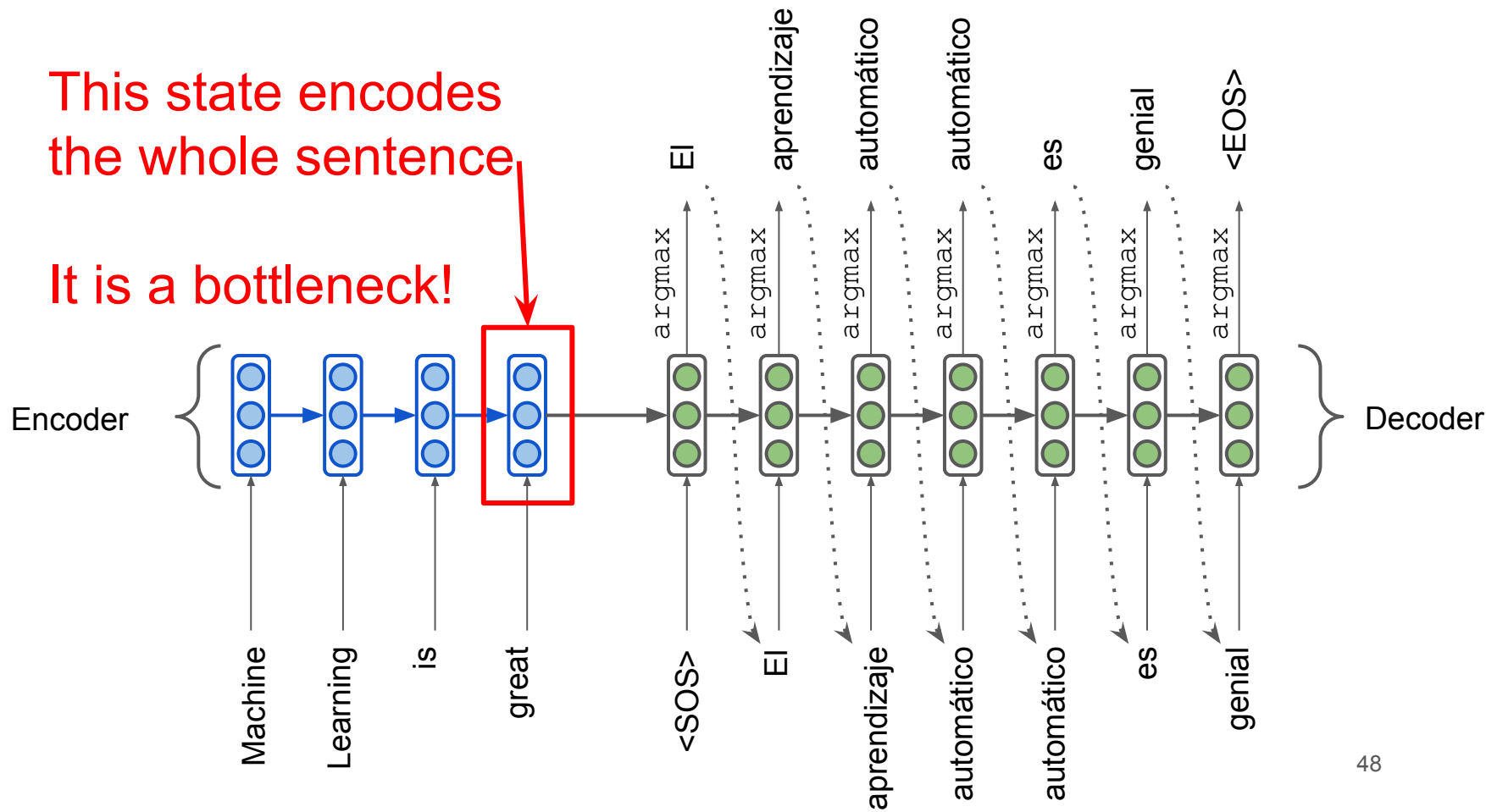
Is Machine Translation solved?

- Challenges yet to be resolved
 - Use of large context in translation
 - Low-resource language pairs (no big parallel corpora)
 - Slang and narrow domains
 - Robustness to errors and typos

Attention

This state encodes
the whole sentence

It is a bottleneck!

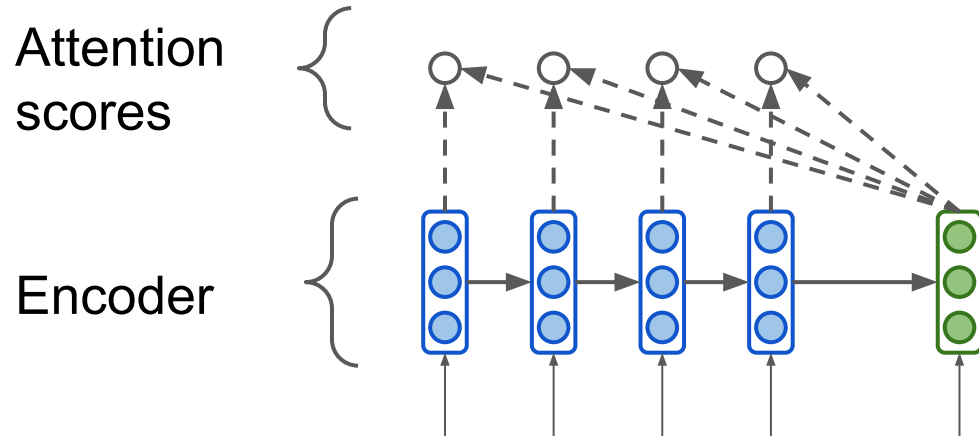


Main idea:

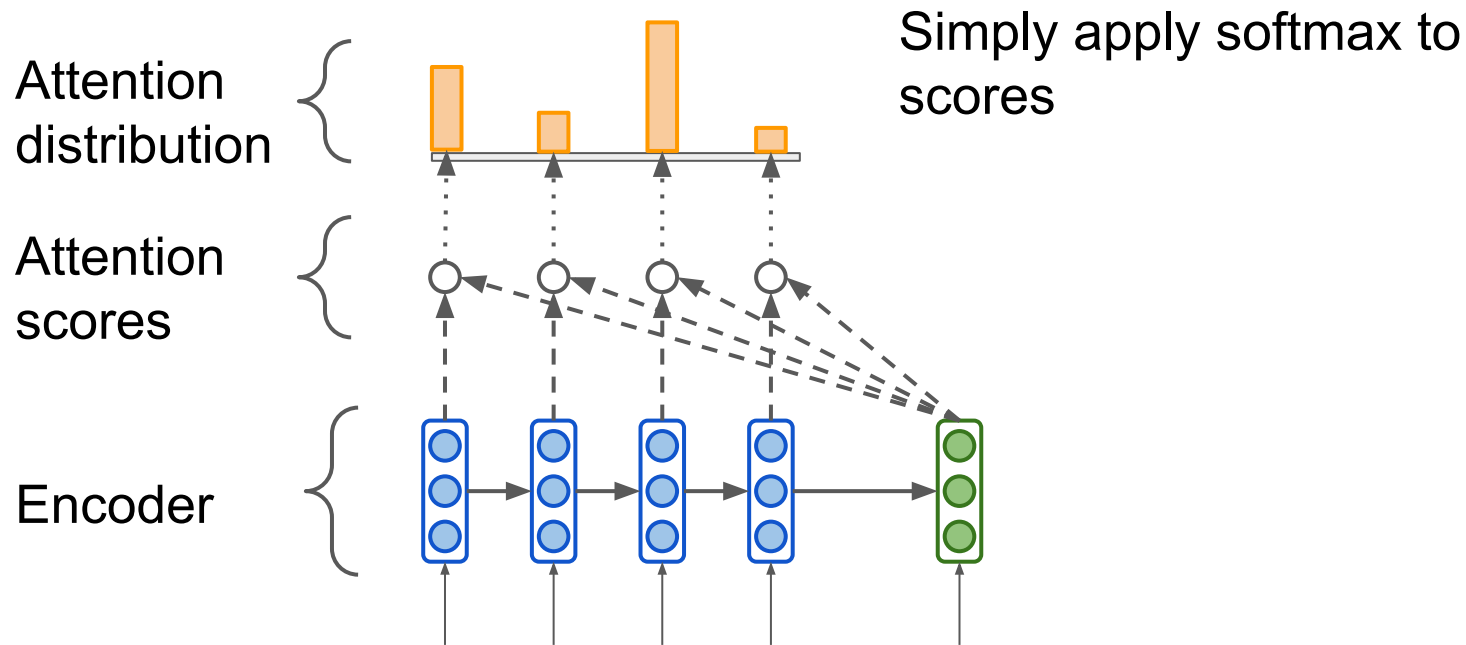
on each step of the **decoder**, use **direct connection to the encoder** to focus on a particular part of the source sequence



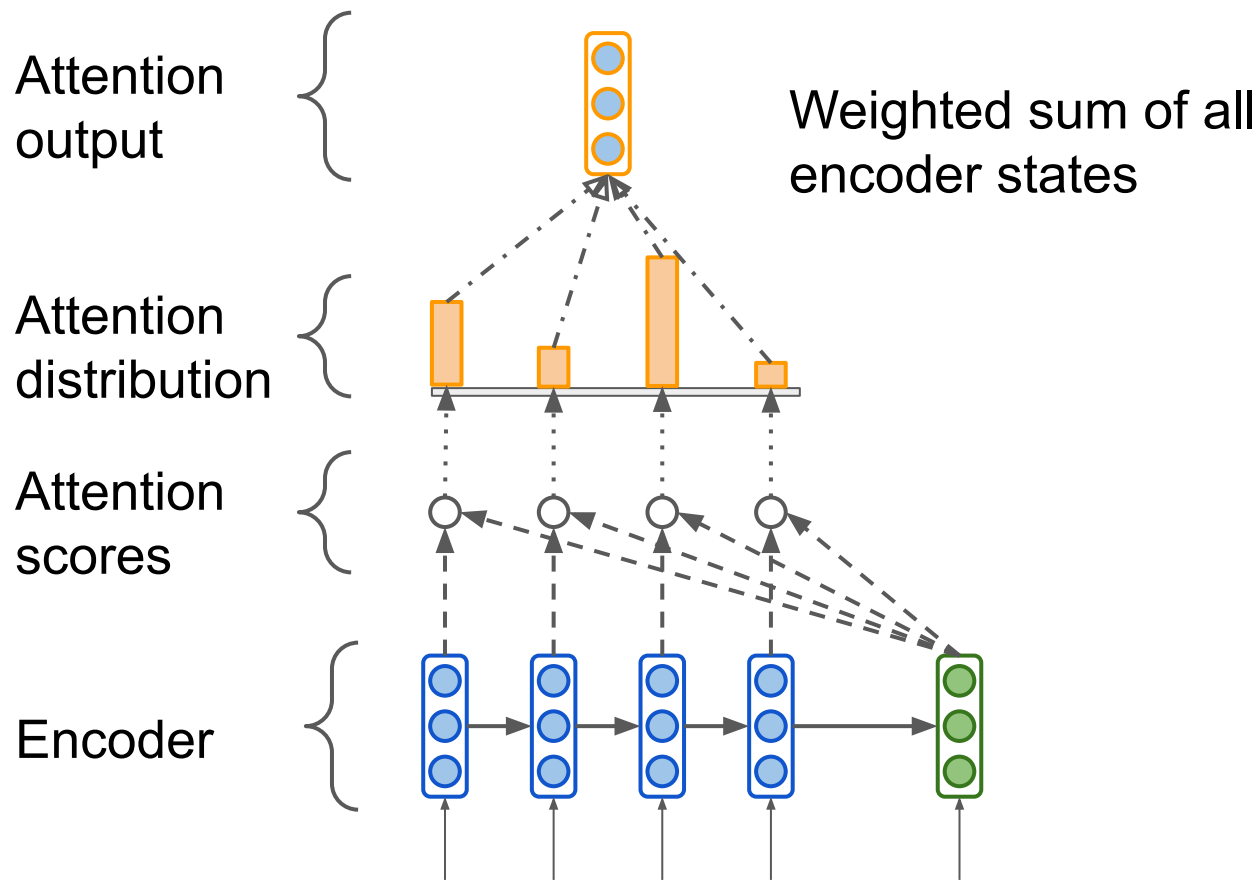
Seq2seq with attention



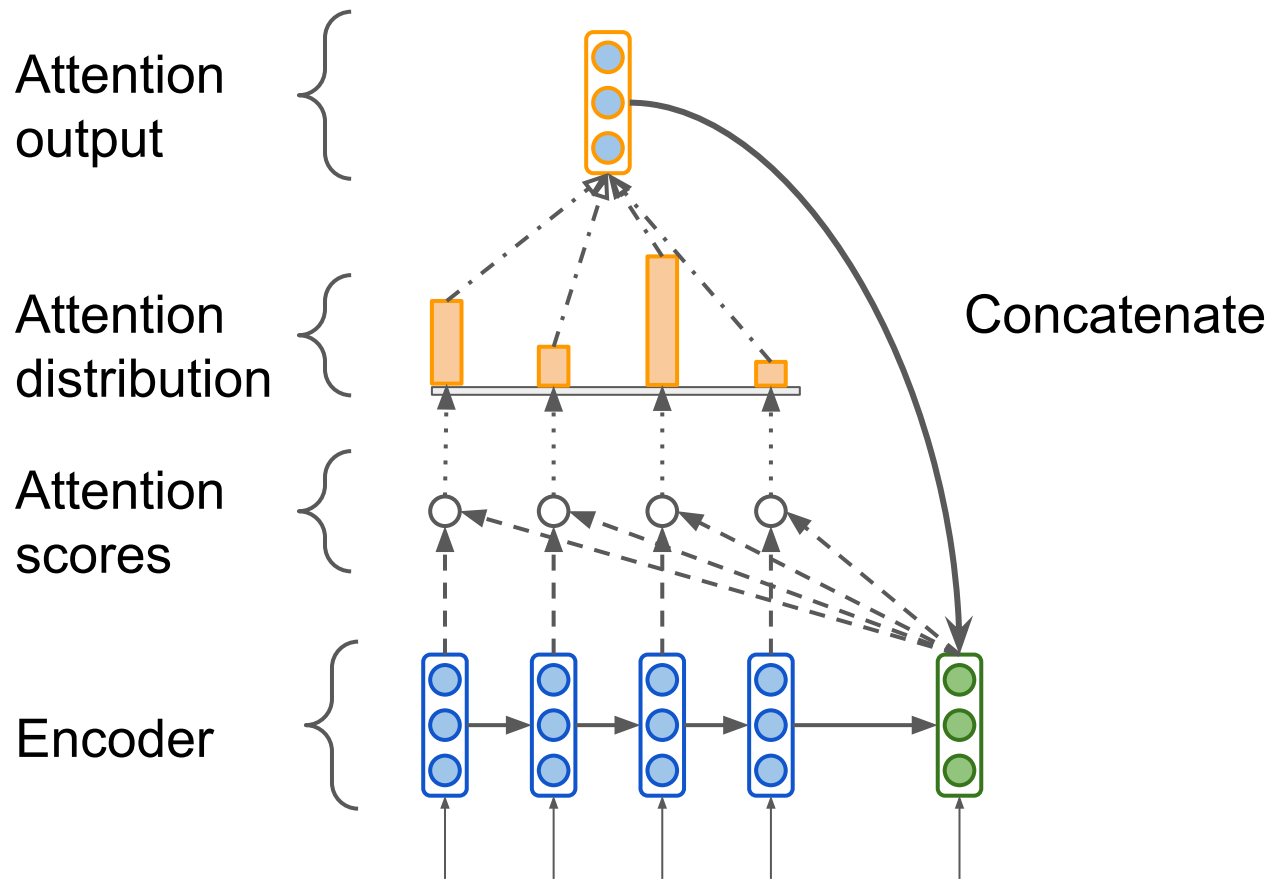
Seq2seq with attention



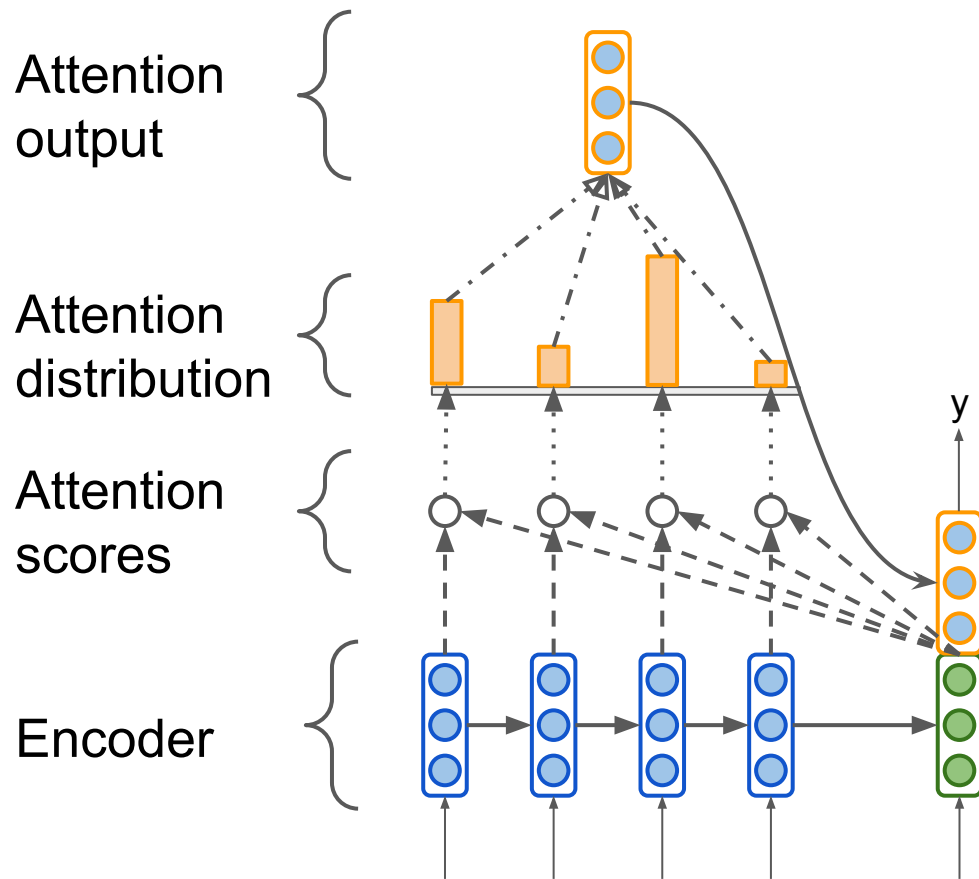
Seq2seq with attention



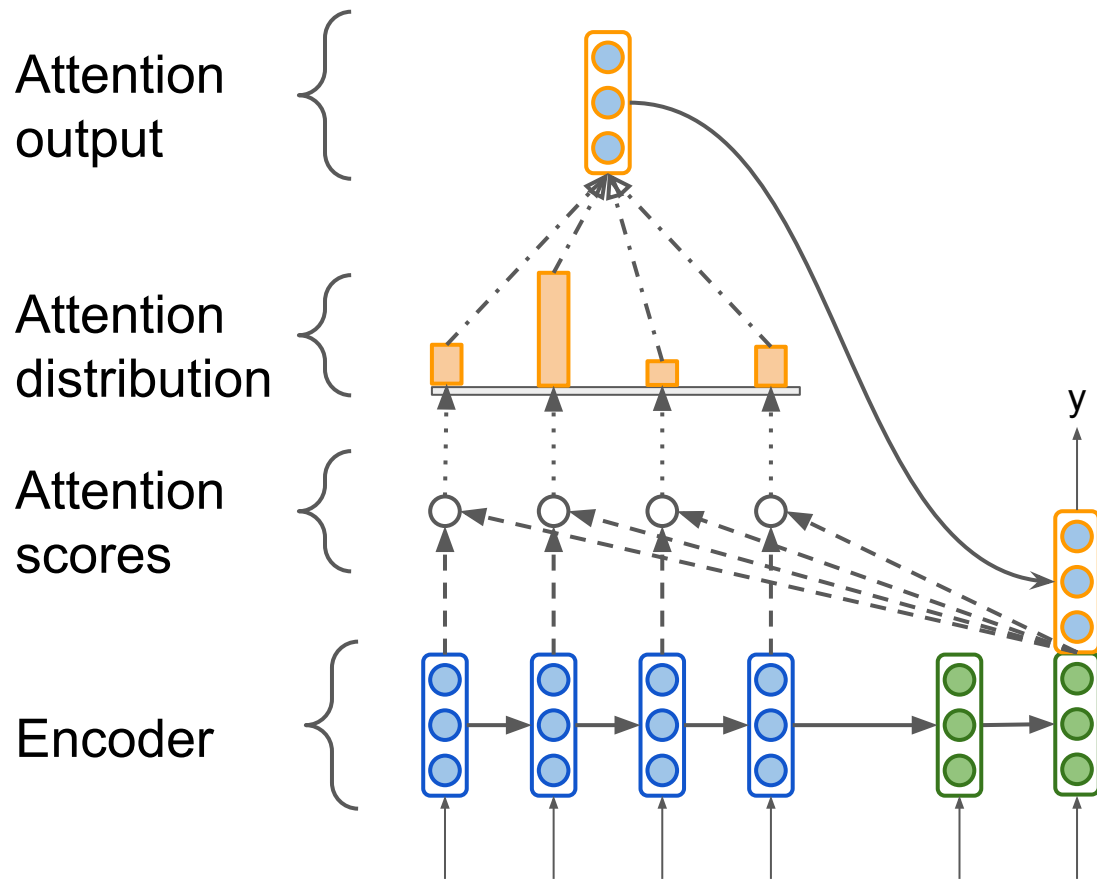
Seq2seq with attention



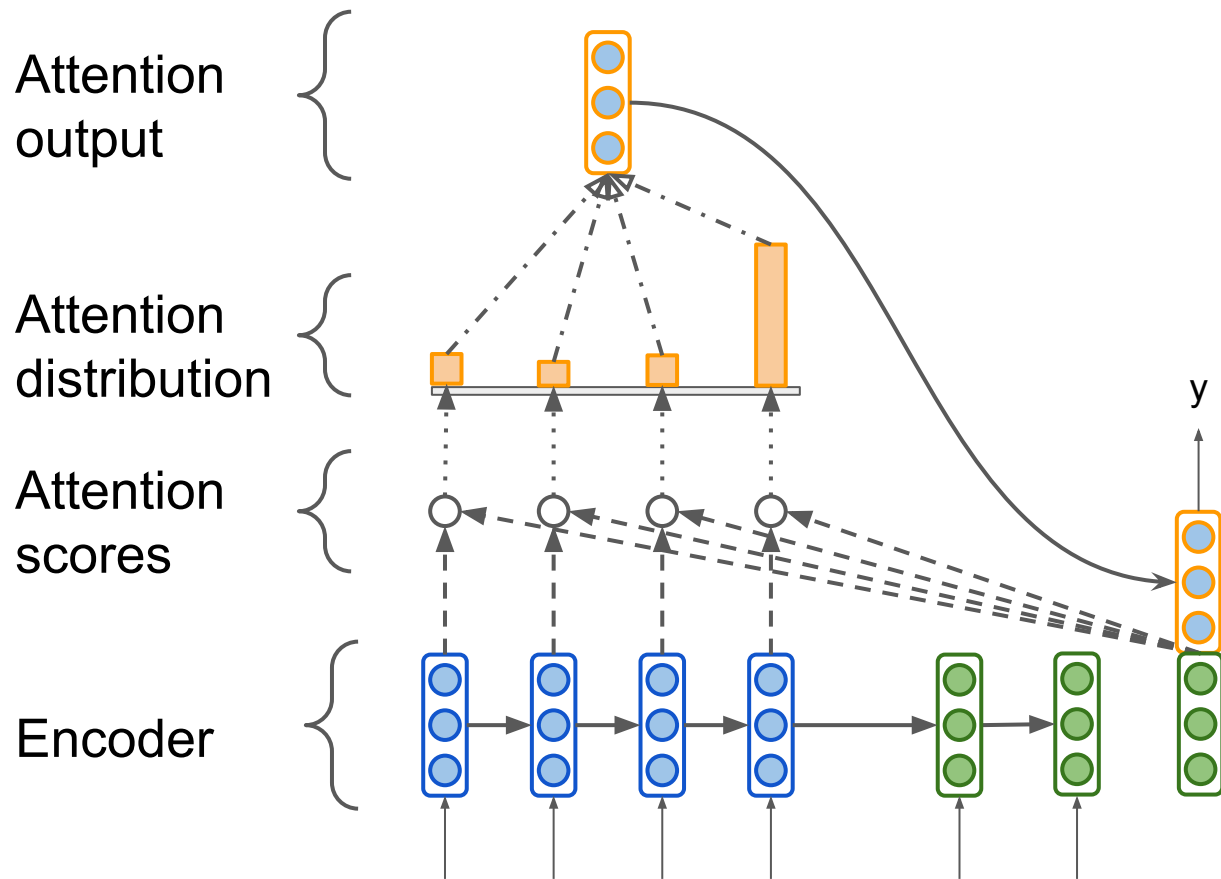
Seq2seq with attention



Seq2seq with attention



Seq2seq with attention



Denote encoder hidden states $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^k$
and decoder hidden state at time step t $\mathbf{s}_t \in \mathbb{R}^k$

The attention scores $\mathbf{e}^t$ can be computed as dot product

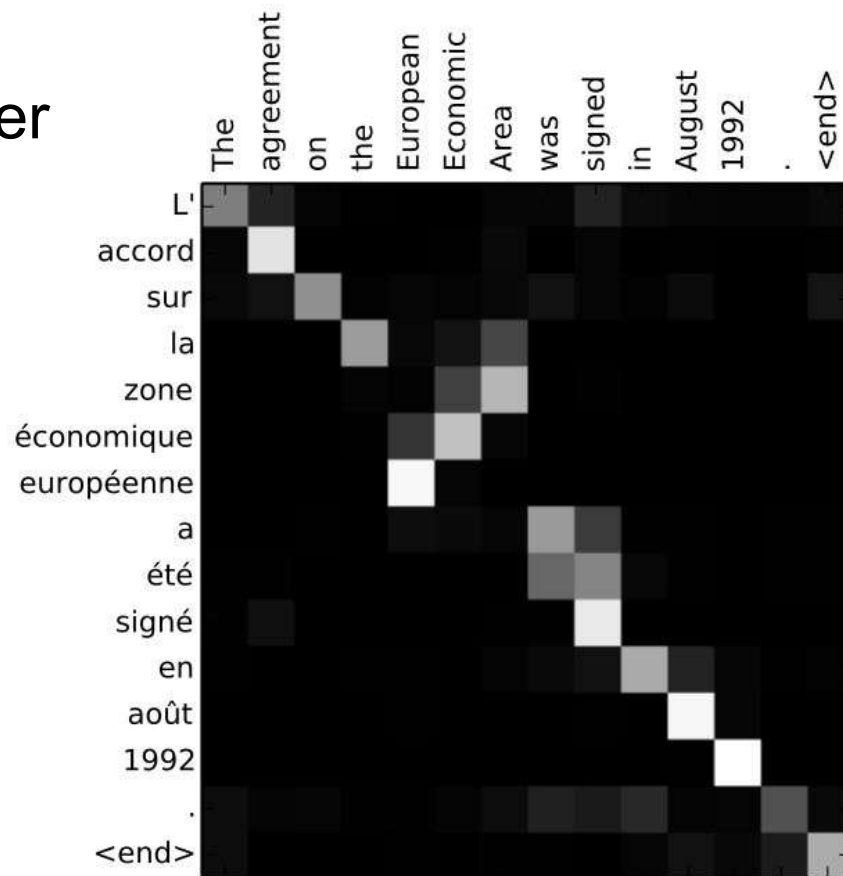
$$\mathbf{e}^t = [\mathbf{s}^T \mathbf{h}_1, \dots, \mathbf{s}^T \mathbf{h}_N]$$

Then the attention vector is a linear combination of encoder states

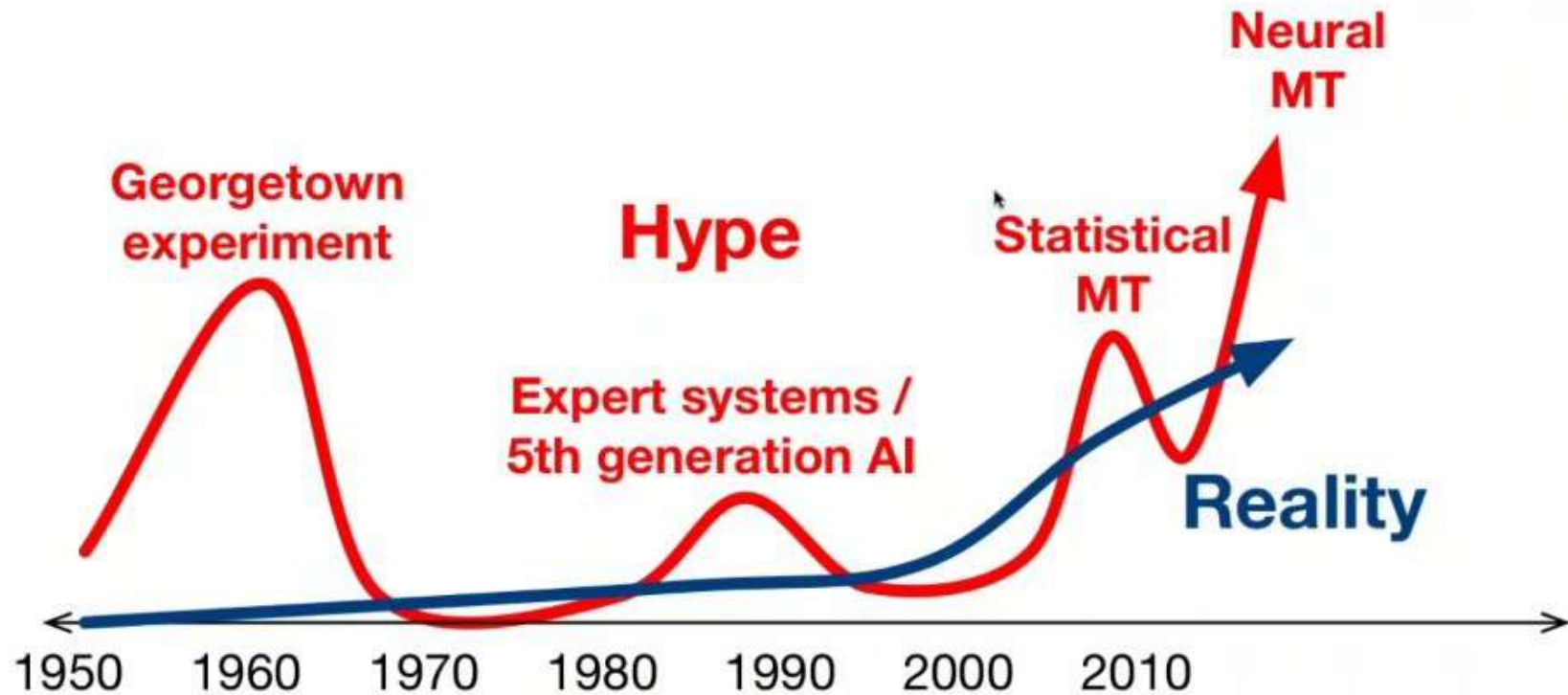
$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t \mathbf{h}_i \in \mathbb{R}^k, \text{ where } \boldsymbol{\alpha}_t = \text{softmax}(\mathbf{e}_t)$$

Attention provides interpretability

- We may see what the decoder was focusing on
- We get word alignment for free!



- Basic dot-product (the one discussed before): $e_i = s^T h_i \in \mathbb{R}$
- Multiplicative attention: $e_i = s^T W h_i \in \mathbb{R}$
 - $W \in \mathbb{R}^{d_2 \times d_1}$ - weight matrix
- Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$
 - $W_1 \in \mathbb{R}^{d_3 \times d_1}, W_2 \in \mathbb{R}^{d_3 \times d_2}$ - weight matrices
 - $v \in \mathbb{R}^{d_3}$ - weight vector



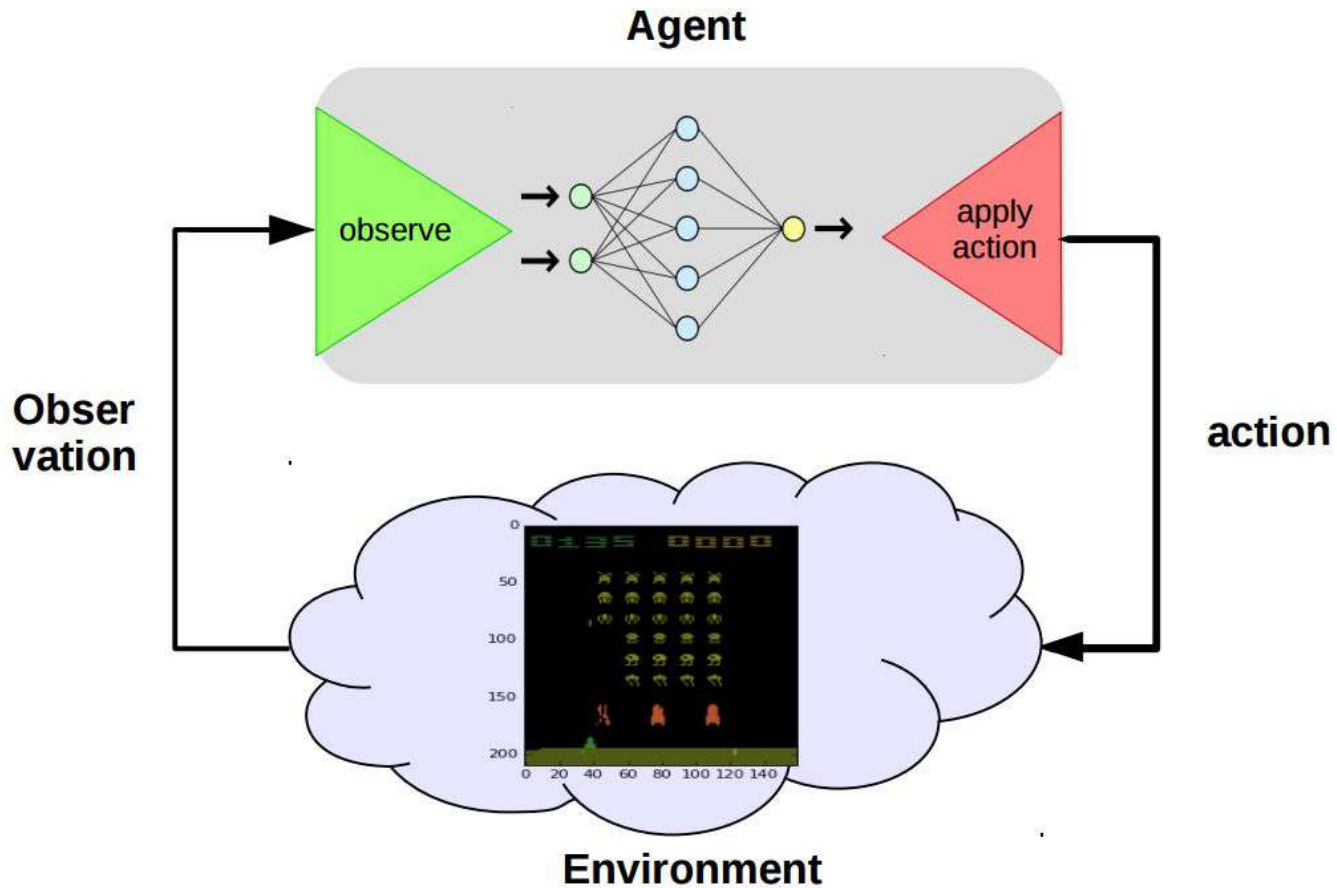
Lecture 9: Deep Reinforcement Learning

Radoslav Neychev

References

These slides are almost the exact copy of Practical RL course week 4 slides.
Special thanks to YSDA team for making them publicly available.

Original slides link: [week04_approx_rl](#)



$$G_t = \sum_{t'=t}^T \gamma^{(t'-t)} r_{t'}$$

$$Q^\pi(s, a) = E_\pi[G_t | s_t = s, a_t = a]$$

$$V^\pi(s) = E_\pi[G_t | s_t = s]$$

Recurrent relations

$$Q^\pi(s, a) = E_{s_{t+1}}[r_t + \gamma V^\pi(s_{t+1})]$$

$$Q^\pi(s, a) = E_{s_{t+1}, a_{t+1} \sim \pi}[r_t + \gamma Q^\pi(s_{t+1}, a_{t+1})]$$

For all π, s, a : $Q^{\pi^*}(s, a) \geq Q^{\pi}(s, a)$

$$\pi^*(s) = \operatorname{argmax}_a Q^{\pi^*}(s, a)$$

Bellman optimality equation

$$Q^*(s_t, a) = E_{s_{t+1}}[r_t + \max_{a'} Q^*(s_{t+1}, a')]$$

Training step

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

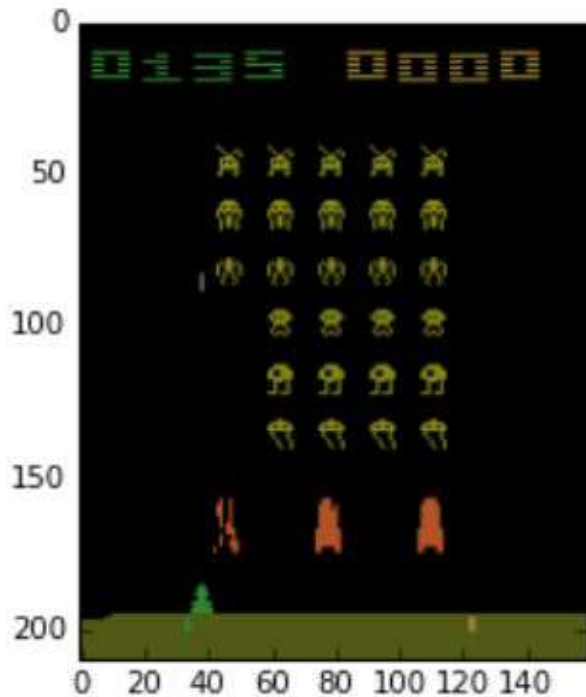
Q-learning as MSE minimization

$$L = (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))^2$$

Const

$$\nabla L = 2 \cdot (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

What's wrong here?



How many states are there?
approximately

$$|S| = 2^{210 \cdot 160 \cdot 8 \cdot 3}$$

State space is usually large,
sometimes continuous.

Two solutions:

- Binarize state space (last week)
- Approximate agent with a function (crossentropy method)

Which one would you prefer for atari?

State space is usually large,
sometimes continuous.

And so is action space;

Two solutions:

- Binarize state space  Too many bins or handcrafted features
- Approximate agent with a function  Let's pick this one

From tables to approximations

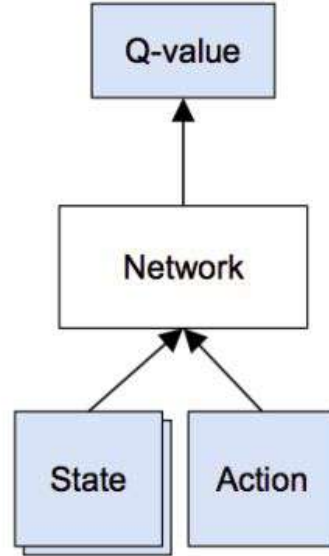
- Before:
 - For all states, for all actions, remember $Q(s,a)$
- Now:
 - Approximate $Q(s,a)$ with some function
 - e.g. linear model over state features

$$\operatorname{argmin}_{w,b} (Q(s_t, a_t) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')])^2$$

Question: should we use **classification** or **regression** model?
(e.g. logistic regression Vs linear regression)

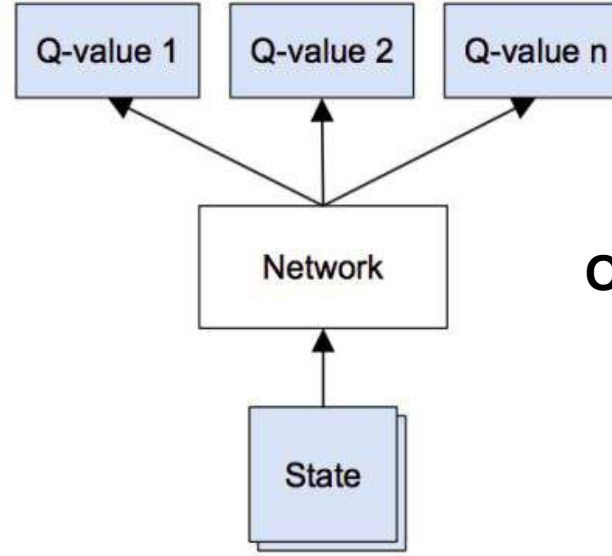
Possible architectures

**Continuous
control or large
number of
actions**



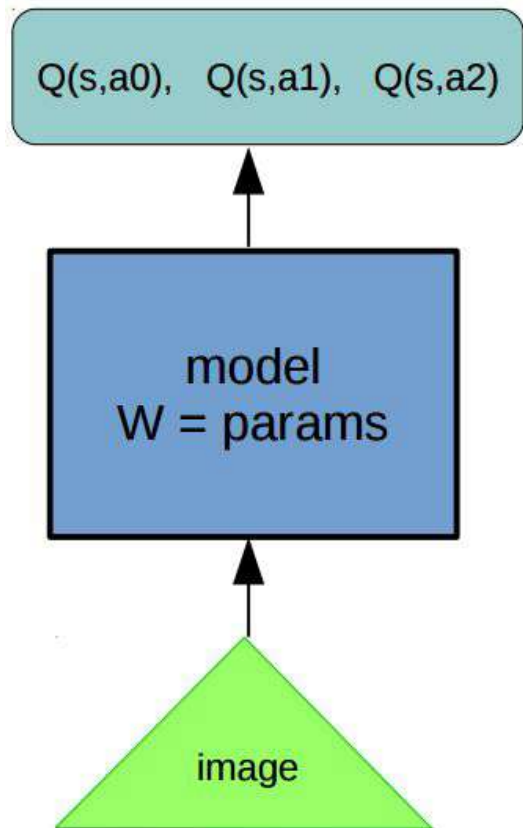
Given **(s,a)**
Predict $Q(s,a)$

**One pass for all
actions**



Given **s** predict all q-values
 $Q(s,a_0)$, $Q(s,a_1)$, $Q(s,a_2)$

Approximate Q-learning



Q-values:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$$

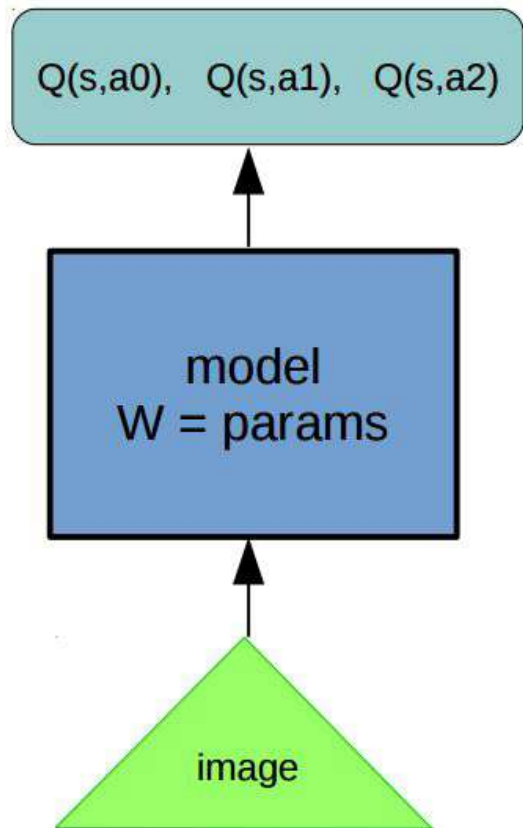
Objective:

$$L = (Q(s_t, a_t) - [r + \gamma \cdot \max_{a'} \underbrace{Q(s_{t+1}, a')}_{\text{Const}}])^2$$

Gradient step:

$$w_{t+1} = w_t - \alpha \cdot \frac{\delta L}{\delta w}$$

Approximate Q-learning



Objective:

$$L = (Q(s_t, a_t) - \hat{Q}(s_t, a_t))^2$$

consider const

Q-learning:

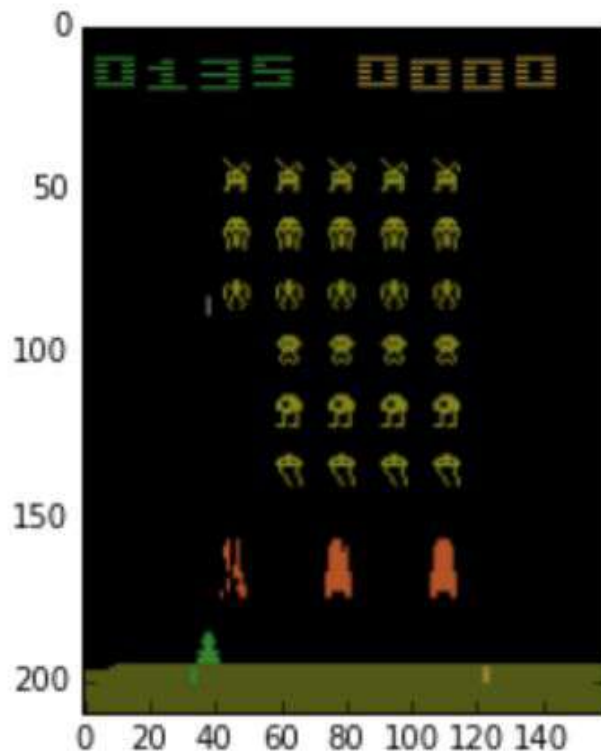
$$\hat{Q}(s_t, a_t) = r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$$

SARSA:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot Q(s_{t+1}, a_{t+1})$$

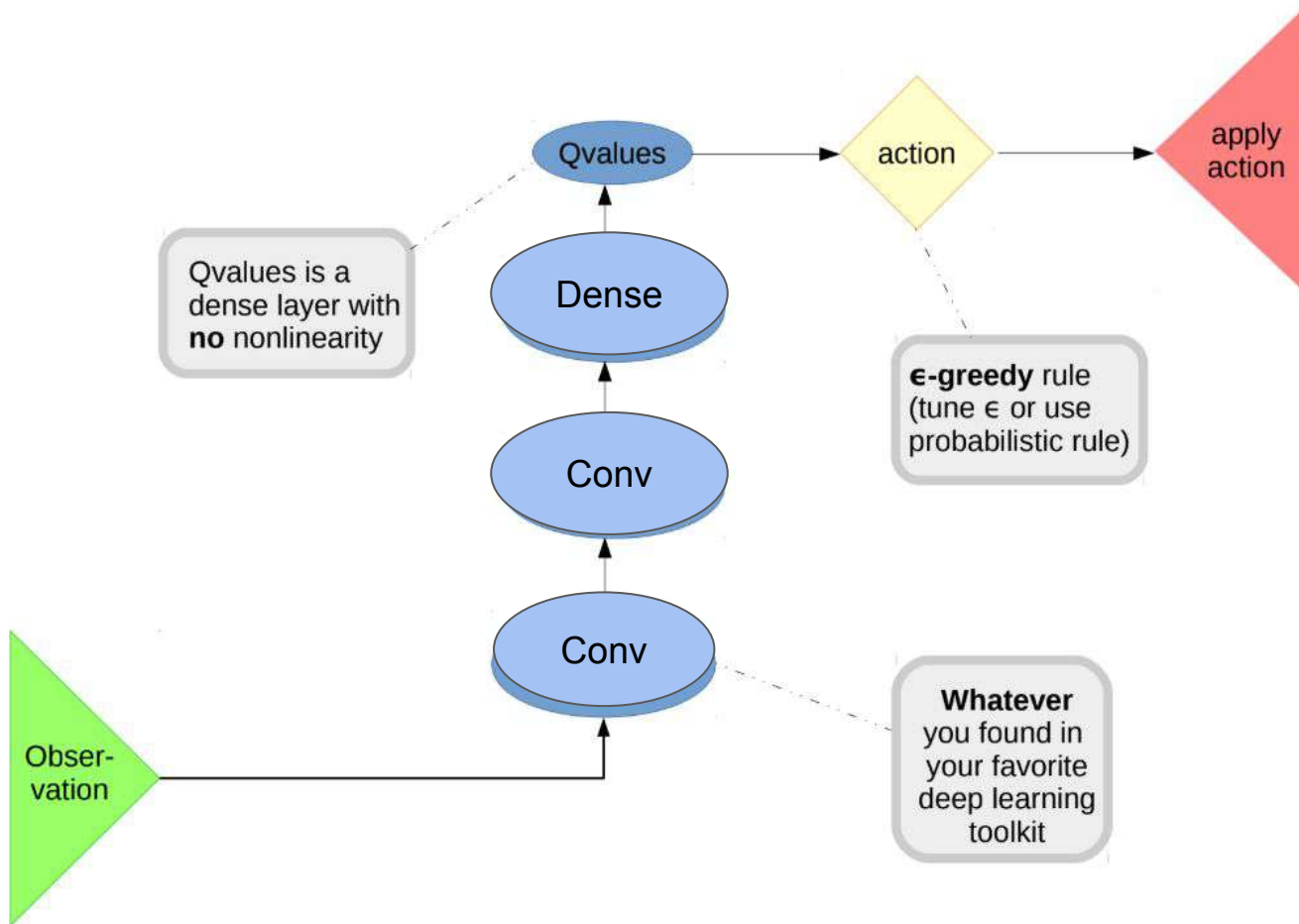
Expected Value SARSA:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot E_{a' \sim \pi(a|s)} Q(s_{t+1}, a')$$

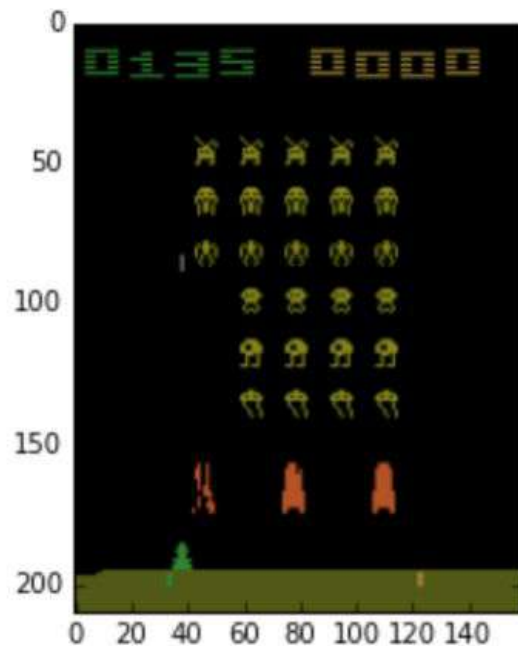


What kind of network digests images well?

Basic deep Q-learning

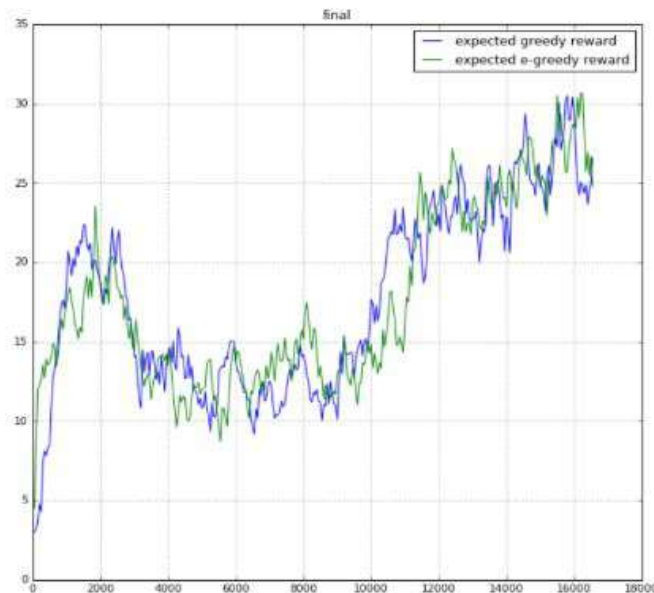






How bad it is if agent spends
next 1000 ticks under the left rock?
(while training)

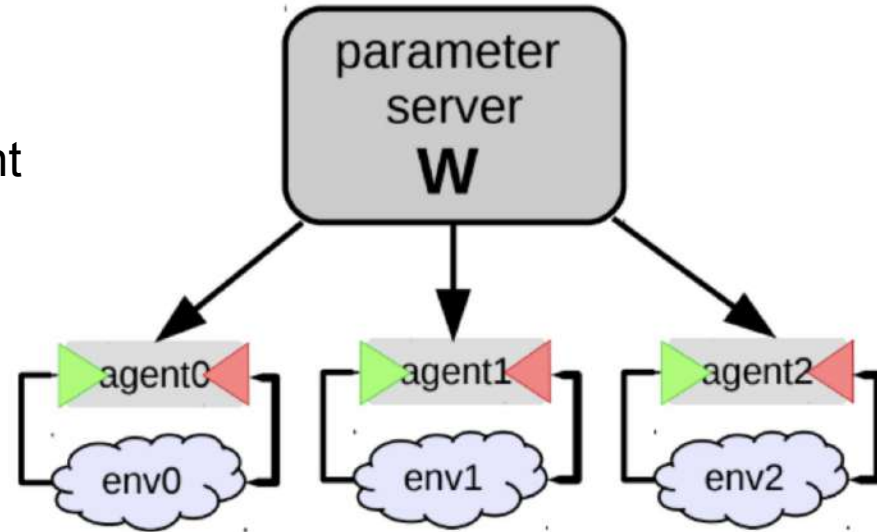
- Training samples are **not** “i.i.d”,
- Model forgets parts of environment it hasn't visited for some time
- Drops on learning curve
- **Any ideas?**



Multiple agents trick

Idea: Throw in several agents with shared W .

- Chances are, they will be exploring different parts of the environment
- More stable training
- Requires a lot of interaction



Question: your agent is a real robot car.
Will there be any problems?



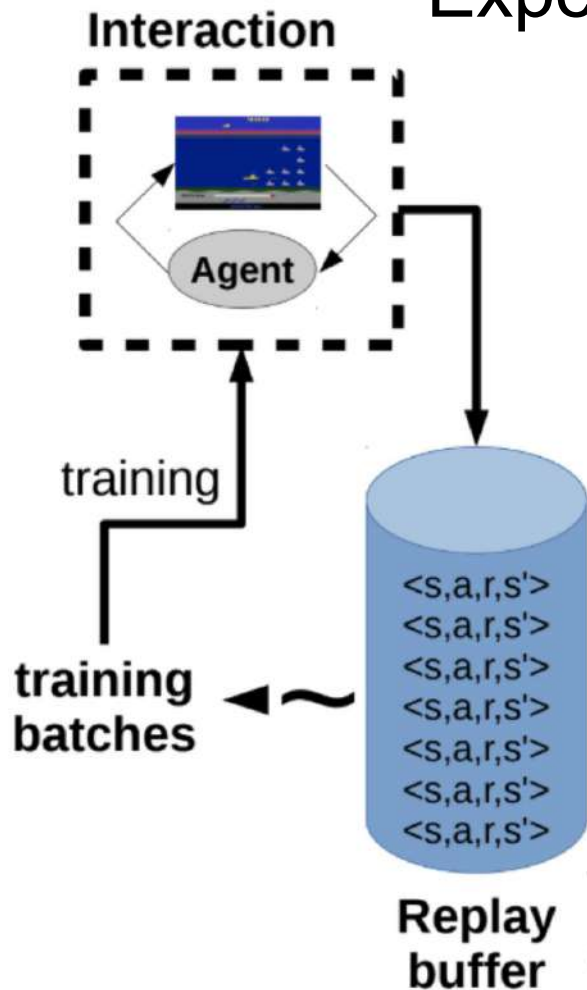
Experience replay

Idea: store several past interactions

$\langle s, a, r, s' \rangle$

Train on random subsamples

- Atari DQN $> 10^5$ interactions
- Closer to i.i.d.
pool contains several sessions
- Older interactions were obtained
under weaker policy



Experience replay

- You approximate $Q(s, a)$ with a neural network
- You use **experience replay** when training

Question: which of those algorithms will fail?

- Q-learning
- SARSA
- CEM
- Expected Value SARSA

Experience replay

- You approximate $Q(s, a)$ with a neural network
- You use **experience replay** when training

Agent trains off-policy on an older version of himself

Question: which of those algorithms will fail?

Off-policy methods work, On-policy methods are super slow (fail)

- Q-learning
- **SARSA**
- **CEM**
- Expected Value SARSA

When training with on-policy methods,

- use no (or small) experience replay
- compensate with parallel game sessions



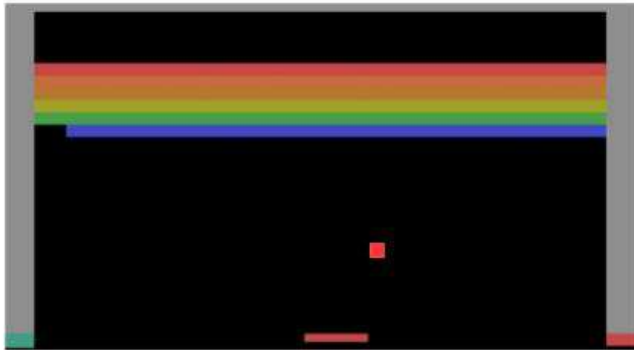
Left or right?

Idea:

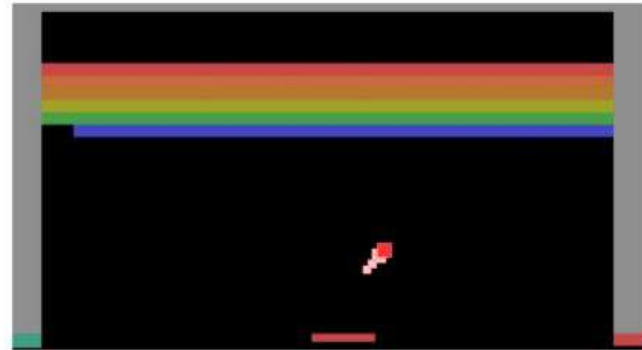
$$s_t \neq o(s_t)$$

$$s_t \approx (o(s_{t-n}), a_{t-n}, \dots, o(s_{t-1}), a_{t-1}, o(s_t))$$

e.g. ball movement in breakout



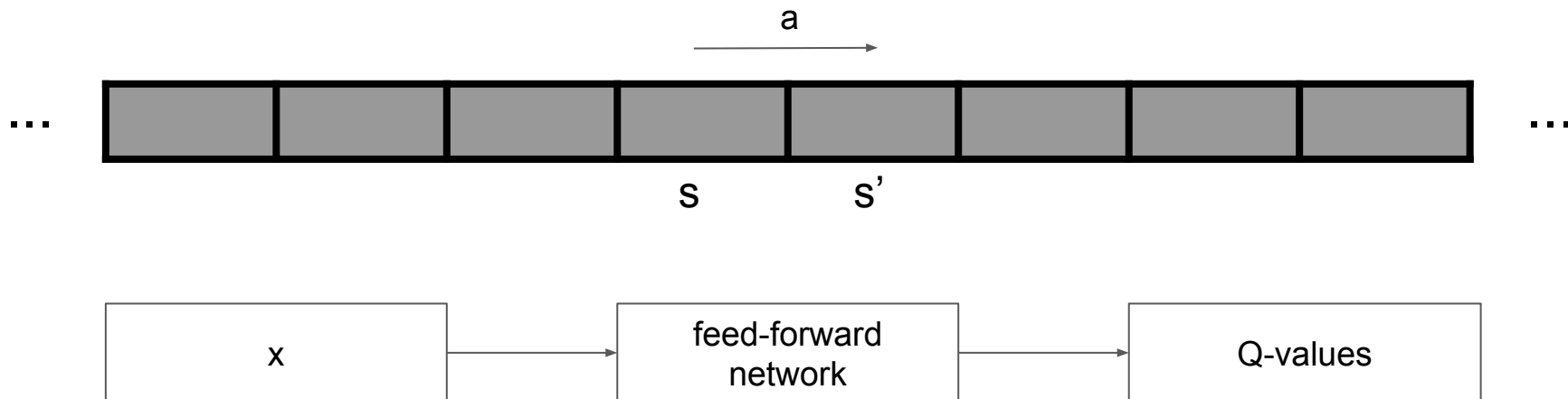
· One frame



· Several frames

- Nth-order markov assumption
- Works for velocity/timers
- Fails for anything longer than N frames
- Impractical for large N

Autocorrelation



Target is based on prediction

$Q(s, a)$ correlates with $Q(s', a)$

Target network

Idea: use network with frozen weights to compute the target

$$L(\Theta) = E_{s \sim S, a \sim A} [(Q(s, a, \Theta) - (r + \gamma \max_{a'} Q(s', a', \Theta^-)))^2]$$

where Θ^- is the frozen weights

↑
Const

Hard target network:

Update Θ^- every **n** steps and set its weights as Θ

Target network

Idea: use network with frozen weights to compute the target

$$L(\Theta) = E_{s \sim S, a \sim A} [(Q(s, a, \Theta) - (r + \gamma \max_{a'} Q(s', a', \Theta^-)))^2]$$

where Θ^- is the frozen weights

↑
Const

Hard target network:

Update Θ^- every **n** steps and set its weights as Θ

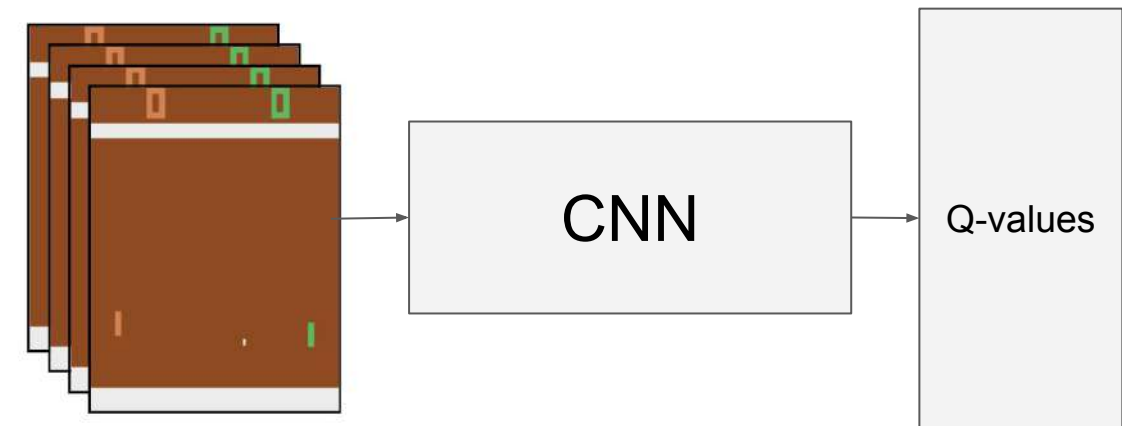
Soft target network:

Update Θ^- every step:

$$\Theta^- = (1 - \alpha)\Theta^- + \alpha\Theta$$

Playing Atari with Deep Reinforcement Learning

(2013, Deepmind)



4 last frames as input

Update weights using:

$$L(\Theta) = E_{s \sim S, a \sim A} [(Q(s, a, \Theta) - (r + \gamma \max_{a'} Q(s', a', \Theta^-)))^2]$$

Update Θ^- every 5000 train steps

Experience replay



10^6 last transitions

Problem of overestimation

We use “max” operator to compute the target

$$L(s, a) = (Q(s, a) - (r + \gamma \max_{a'} Q(s', a')))^2$$

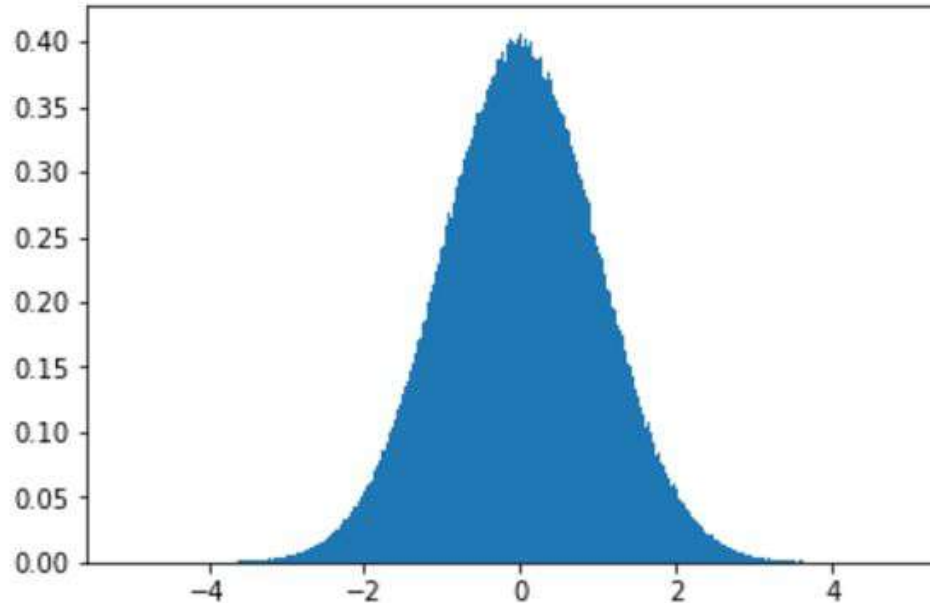
We have a problem

(although we want $E_{s \sim S, a \sim A}[L(s, a)]$ to be equal zero)

Problem of overestimation

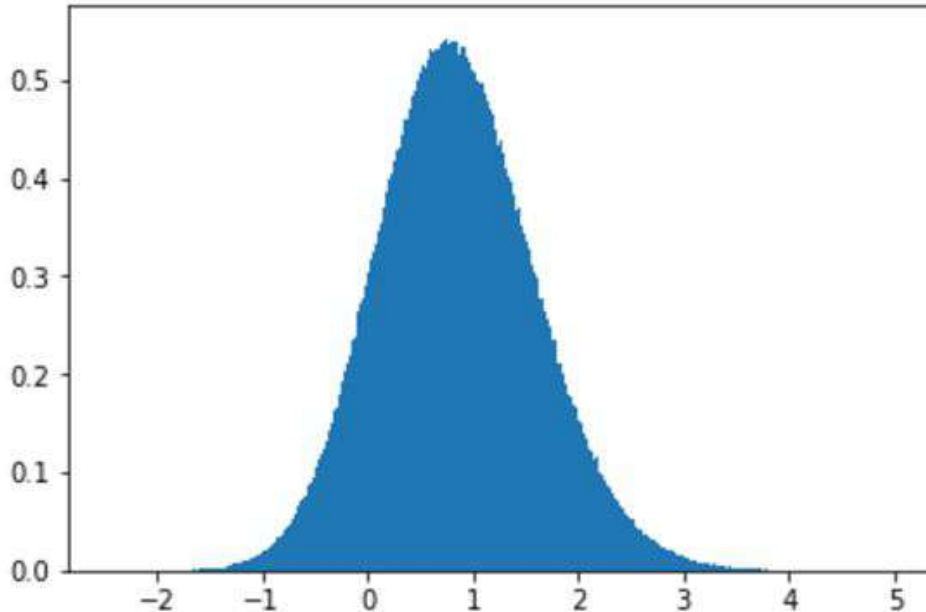
Normal distribution
 $3 \cdot 10^6$ samples

mean: ~ 0.0004



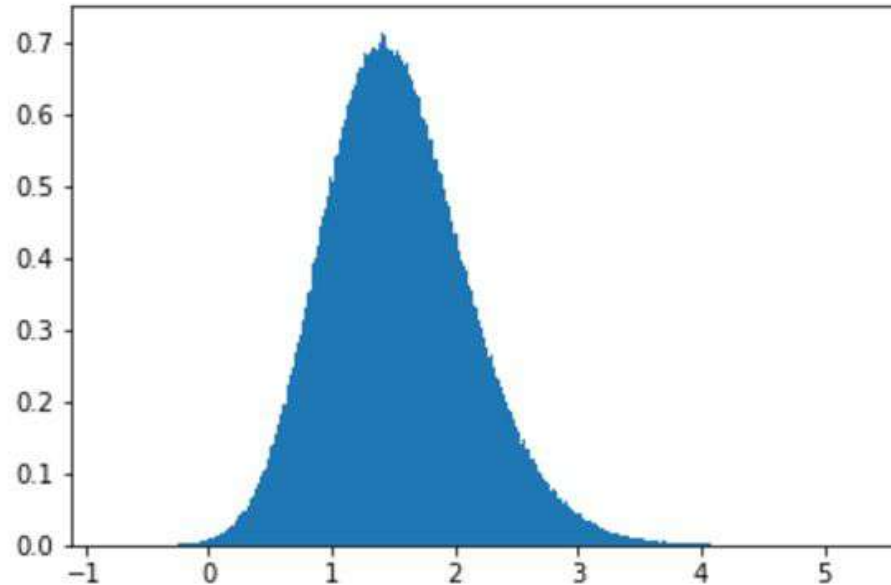
Problem of overestimation

Normal distribution
 $3 \cdot 10^6 \times 3$ samples
Then take maximum of every tuple
mean: ~ 0.8467

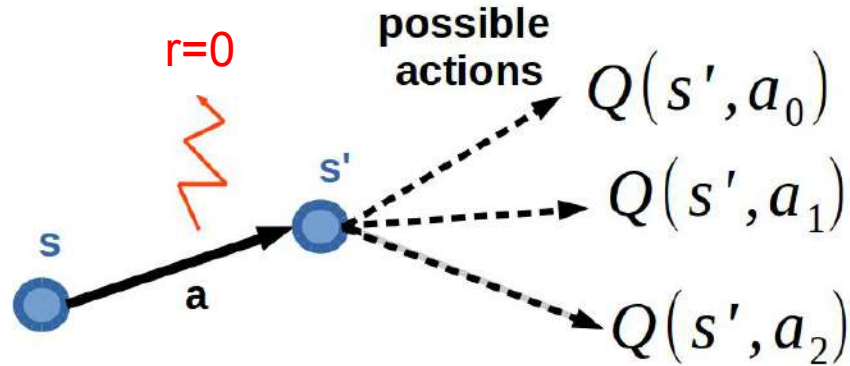


Problem of overestimation

Normal distribution
 $3 \times 10^6 \times 10$ samples
Then take maximum of every tuple
mean: ~ 1.538



Problem of overestimation

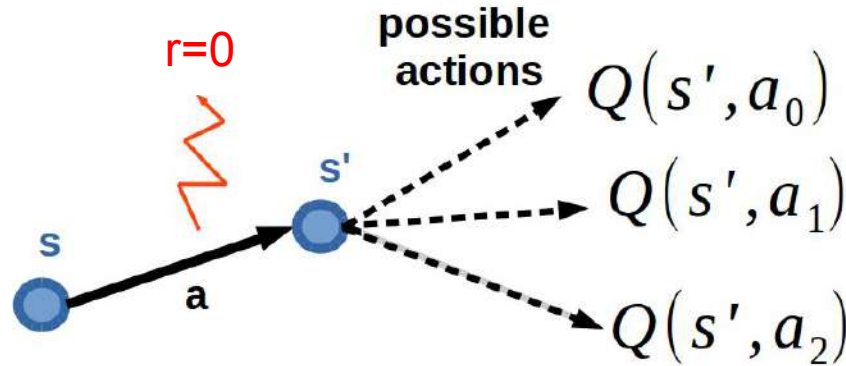


Suppose true $Q(s', a')$ are equal to $\mathbf{0}$ for all a'

But we have an approximation (or other) error $\sim N(0, \sigma^2)$

So $Q(s, a)$ should be equal to $\mathbf{0}$

Problem of overestimation



But if we update $Q(s, a)$ towards $r + \gamma \max_{a'} Q(s', a')$
we will have overestimated $Q(s, a) > \mathbf{0}$ because

$$E[\max_{a'} Q(s', a')] \geq \max_{a'} E[Q(s', a')]$$

Double Q-learning

(NIPS 2010)

$$y = r + \gamma \max_{a'} Q(s', a')$$

- Q-learning target

$$y = r + \gamma Q(s', \operatorname{argmax}_{a'} Q(s', a'))$$

- Rewritten Q-learning target

Idea: use two estimators of q-values: Q^A, Q^B

They should compensate mistakes of each other because they will be independent
Let's get argmax from another estimator!

$$y = r + \gamma Q^A(s', \operatorname{argmax}_a Q^B(s', a'))$$

- Double Q-learning target

Algorithm 1 Double Q-learning

```
1: Initialize  $Q^A, Q^B, s$ 
2: repeat
3:   Choose  $a$ , based on  $Q^A(s, \cdot)$  and  $Q^B(s, \cdot)$ , observe  $r, s'$ 
4:   Choose (e.g. random) either UPDATE(A) or UPDATE(B)
5:   if UPDATE(A) then
6:     Define  $a^* = \arg \max_a Q^A(s', a)$ 
7:      $Q^A(s, a) \leftarrow Q^A(s, a) + \alpha(s, a) (r + \gamma Q^B(s', a^*) - Q^A(s, a))$ 
8:   else if UPDATE(B) then
9:     Define  $b^* = \arg \max_a Q^B(s', a)$ 
10:     $Q^B(s, a) \leftarrow Q^B(s, a) + \alpha(s, a) (r + \gamma Q^A(s', b^*) - Q^B(s, a))$ 
11:   end if
12:    $s \leftarrow s'$ 
13: until end
```

Can we combine this algorithm with DQN?

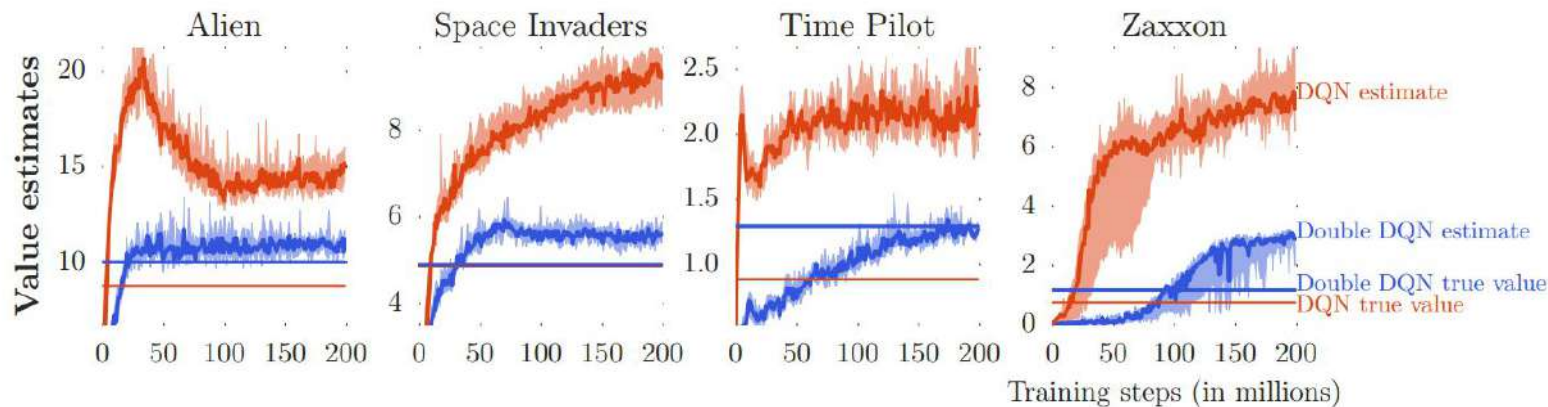
Deep RL with Double Q-learning

(Deepmind, 2015)

Idea: use main network to choose action!

$$y_{dqn} = r + \gamma \max_{a'} Q(s', a', \Theta^-)$$

$$y_{ddqn} = r + \gamma Q(s', \operatorname{argmax}_{a'} Q(s', a', \Theta), \Theta^-)$$



	DQN	Double DQN	Double DQN (tuned)
Median	47.5%	88.4%	116.7%
Mean	122.0%	273.1%	475.2%

Double Q-learning visualization



Dueling Network Architectures for Deep RL

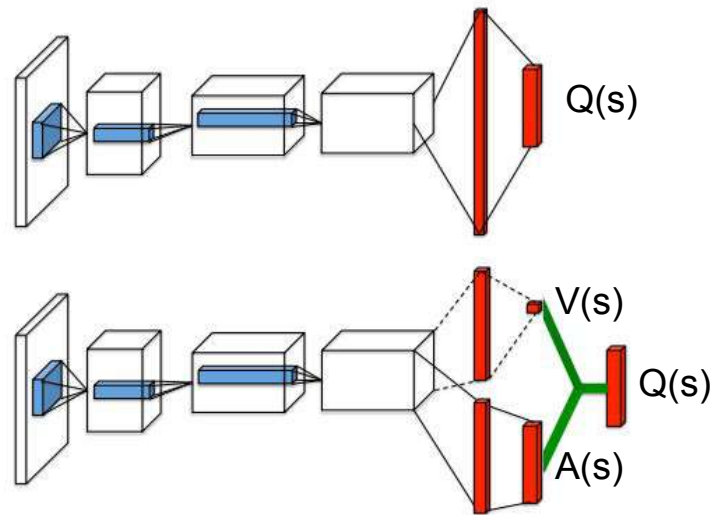
(2016, Deepmind)

Idea: change the network's architecture.

Recall:

Advantage Function $A(s,a) = Q(s,a) - V(s)$

So, $Q(s,a) = A(s,a) + V(s)$



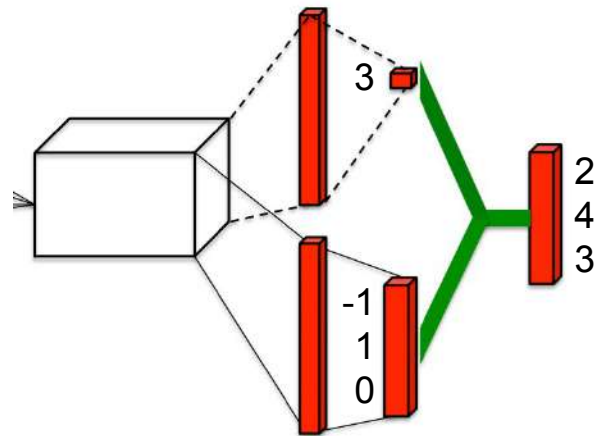
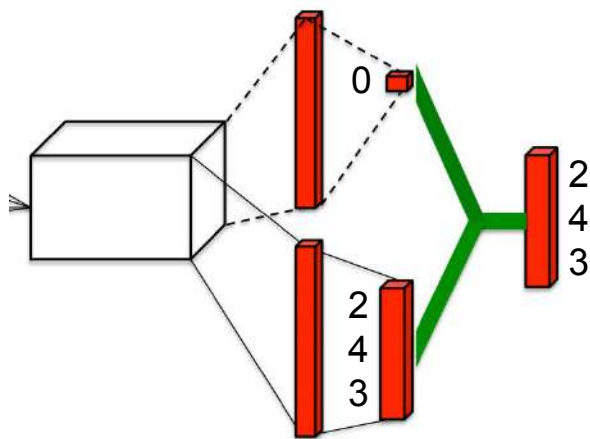
Do you see the problem?

Dueling Network Architectures for Deep RL

(2016, Deepmind)

Here is one extra freedom degree!

Example:



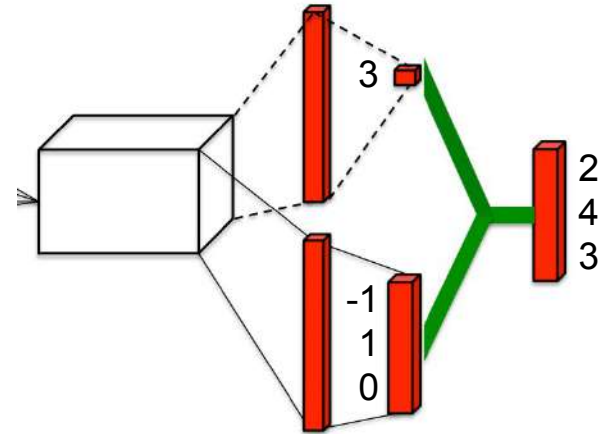
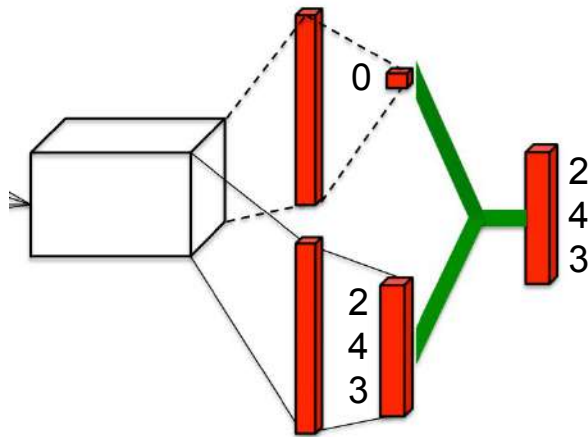
Which one is good?

Dueling Network Architectures for Deep RL

(2016, Deepmind)

Here is one extra freedom degree!

Example:



No one

Dueling Network Architectures for Deep RL

(2016, Deepmind)

Solution: require $\max_{a' \in |\mathcal{A}|} A(s, a'; \theta, \alpha)$ to be equal to zero!

So the **Q-function** is computed as:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \max_{a' \in |\mathcal{A}|} A(s, a'; \theta, \alpha) \right)$$

Dueling Network Architectures for Deep RL

(2016, Deepmind)

Solution: require $\max_{a' \in |\mathcal{A}|} A(s, a'; \theta, \alpha)$ to be equal to zero!

So the **Q-function** is computed as:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \max_{a' \in |\mathcal{A}|} A(s, a'; \theta, \alpha) \right)$$

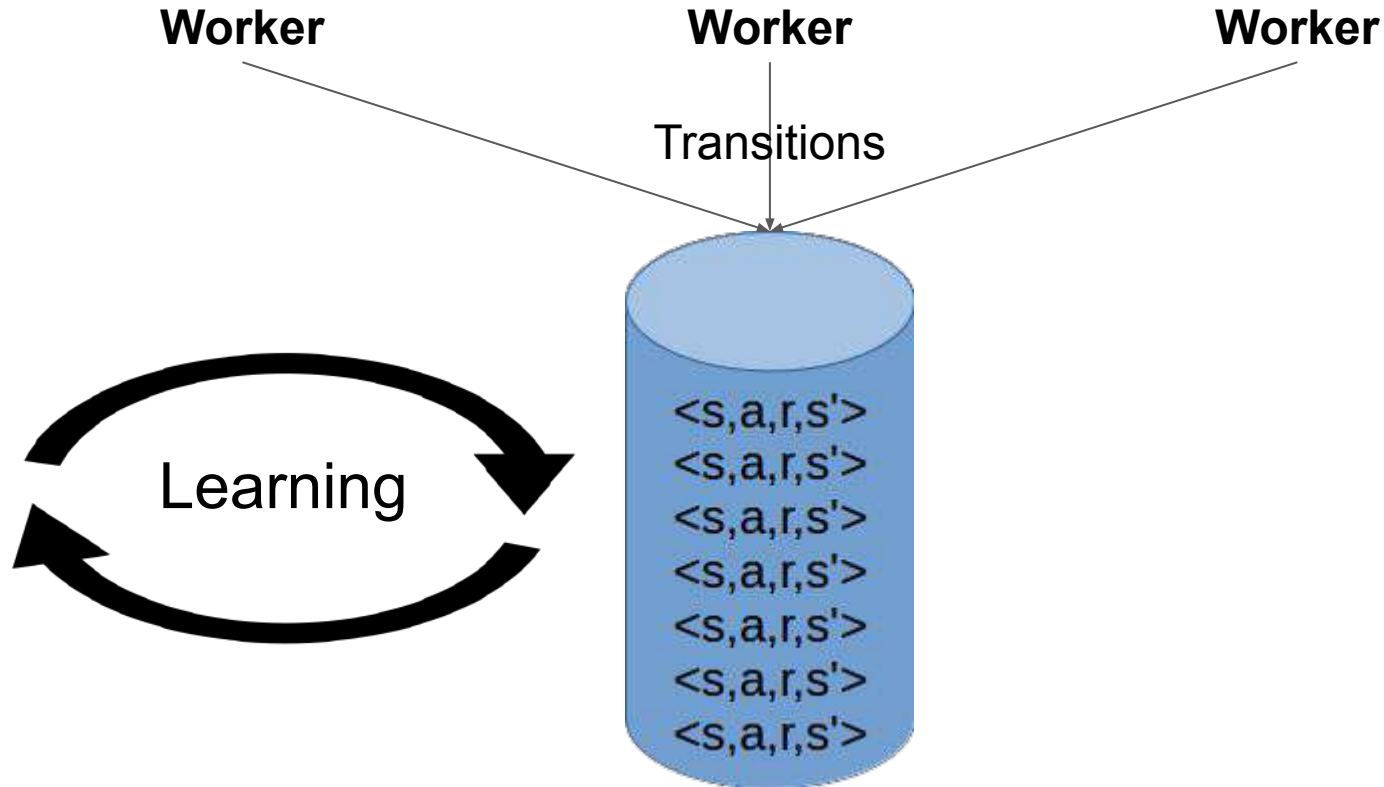
Authors of this papers also introduced this way to compute Q-values:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right)$$

They wrote that this variant increases stability of the optimization
(The fact that this loses the original semantics of Q doesn't matter)

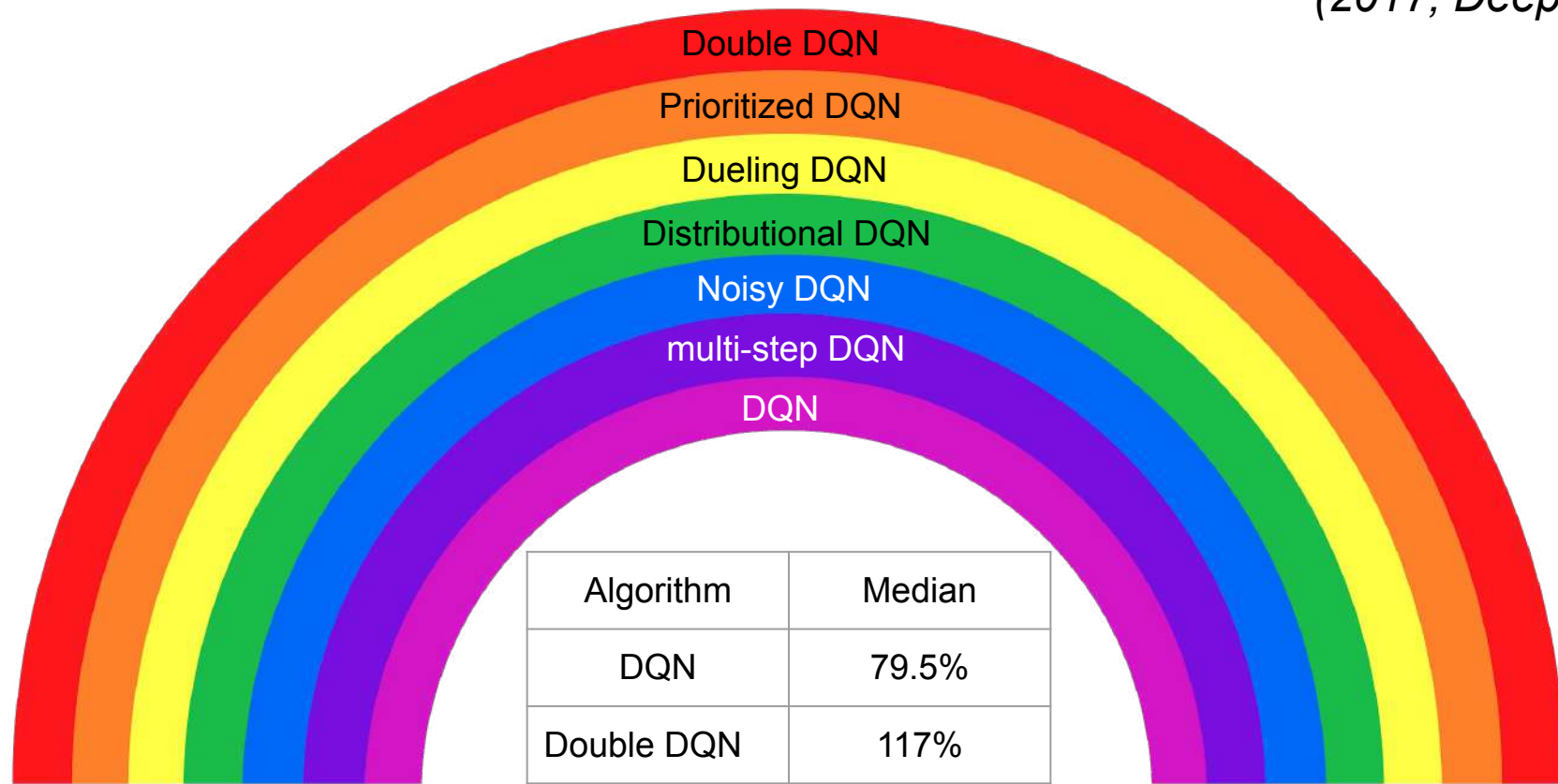
Asynchronous Methods for Deep RL

(2016, Deepmind)



Rainbow

(2017, Deepmind)



Algorithm	Median
DQN	79.5%
Double DQN	117%
Rainbow	223%

R2D2

(2018, Deepmind)

LSTM

Reward re-scaling

Distributed Prioritized
Experience Replay

Double DQN

n-step DQN

Dueling DQN



Median performance: **1920%** of human performance!

Q & A

Backlog

Experience Replay

State	Action	Reward	Next state
s ₀	a ₀	0	s ₁
s ₁	a ₁	0	s ₂
...	...	...	...
s _(n-1)	a _(n-1)	0	s _n
s_n	a_n	100	s_(n+1)
s _(n+1)	a _(n+1)	0	s _(n+2)
...	...	...	...

Prioritized Experience Replay

(2016, Deepmind)

Idea: sample transitions from xp-replay cleverly

We want to set probability for every transition. Let's use the absolute value of TD-error of transition as a probability!

$$\text{TD-error } \delta = Q(s, a) - (r + \gamma Q(s', \arg\max_{a'} Q(s', a', \Theta), \Theta^-))$$

$$p = |\delta|$$

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \text{ where } \alpha \text{ is the priority parameter (when } \alpha \text{ is 0 it's the uniform case)}$$

Do you see the problem?

Transitions become non i.i.d. and therefore we introduce the bias.

Prioritized Experience Replay

(2016, Deepmind)

Solution: we can correct the bias by using importance-sampling weights

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta \quad \text{where } \beta \text{ is the parameter}$$

So we sample using $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$ and multiply error by w_i

Prioritized Experience Replay

(2016, Deepmind)

Additional details

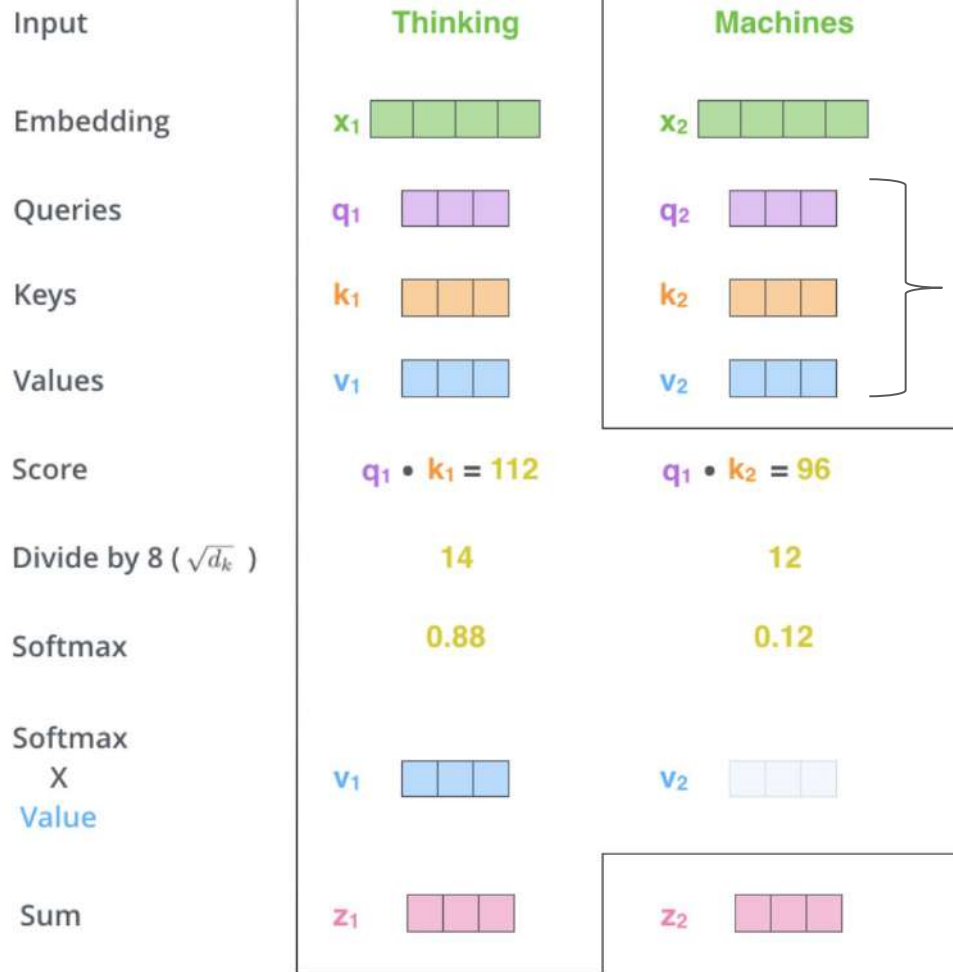
We also normalize weights by $1 / \max_i w_i$ (here is no mathematical reason)

When we put transition into experience replay, we set maximal priority $p_t = \max_{i < t} p_i$

Lecture 05: Transfer Learning, BERT, GPT

Self-Attention: recap

Self-Attention in Detail



STEP 1: create Query, Key, Value

STEP 2: calculate scores

STEP 3: divide by $\sqrt{d_k}$

STEP 4: softmax

STEP 5: multiply each value vector by the softmax score

STEP 6: sum up the weighted value vectors

Self-Attention: Matrix Calculation

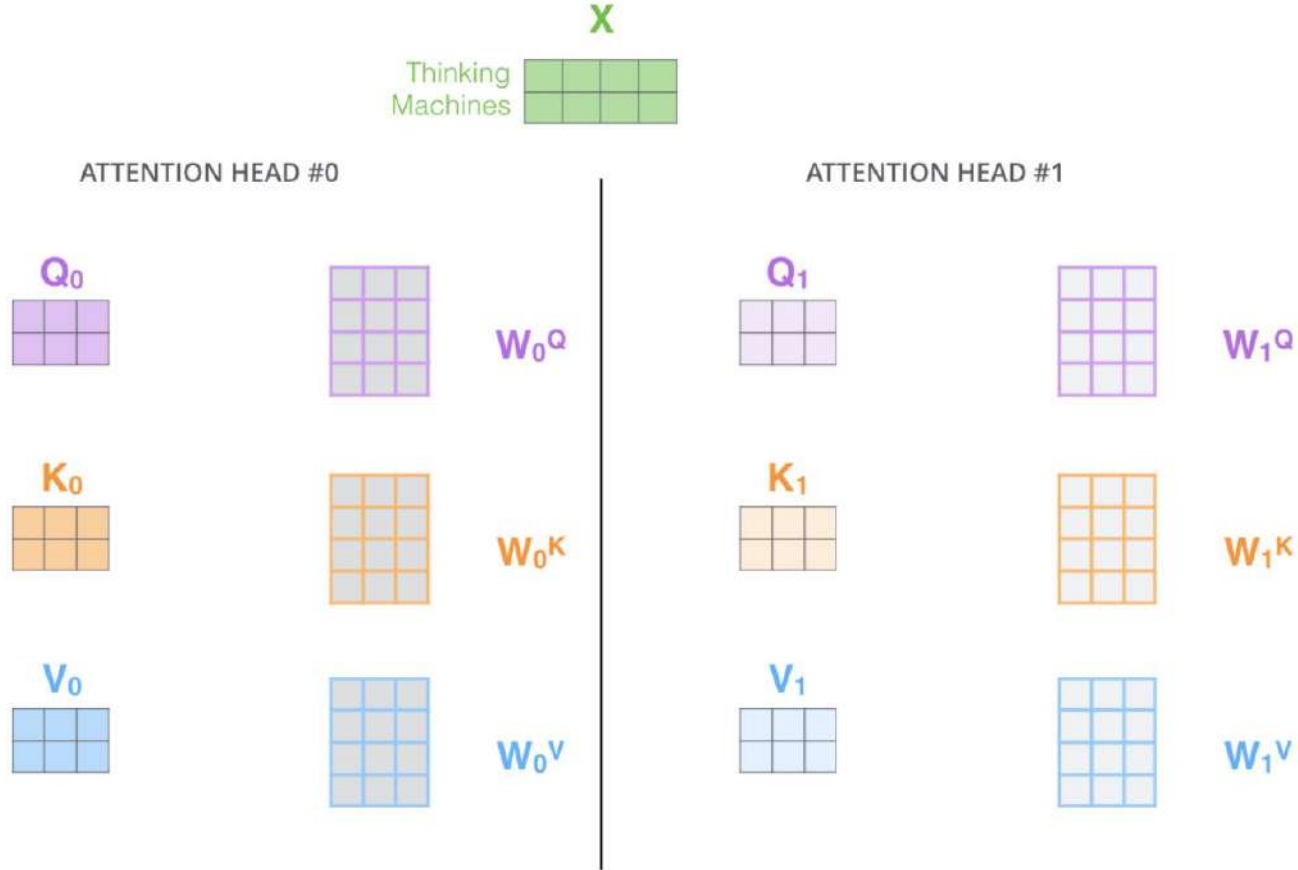
$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

=

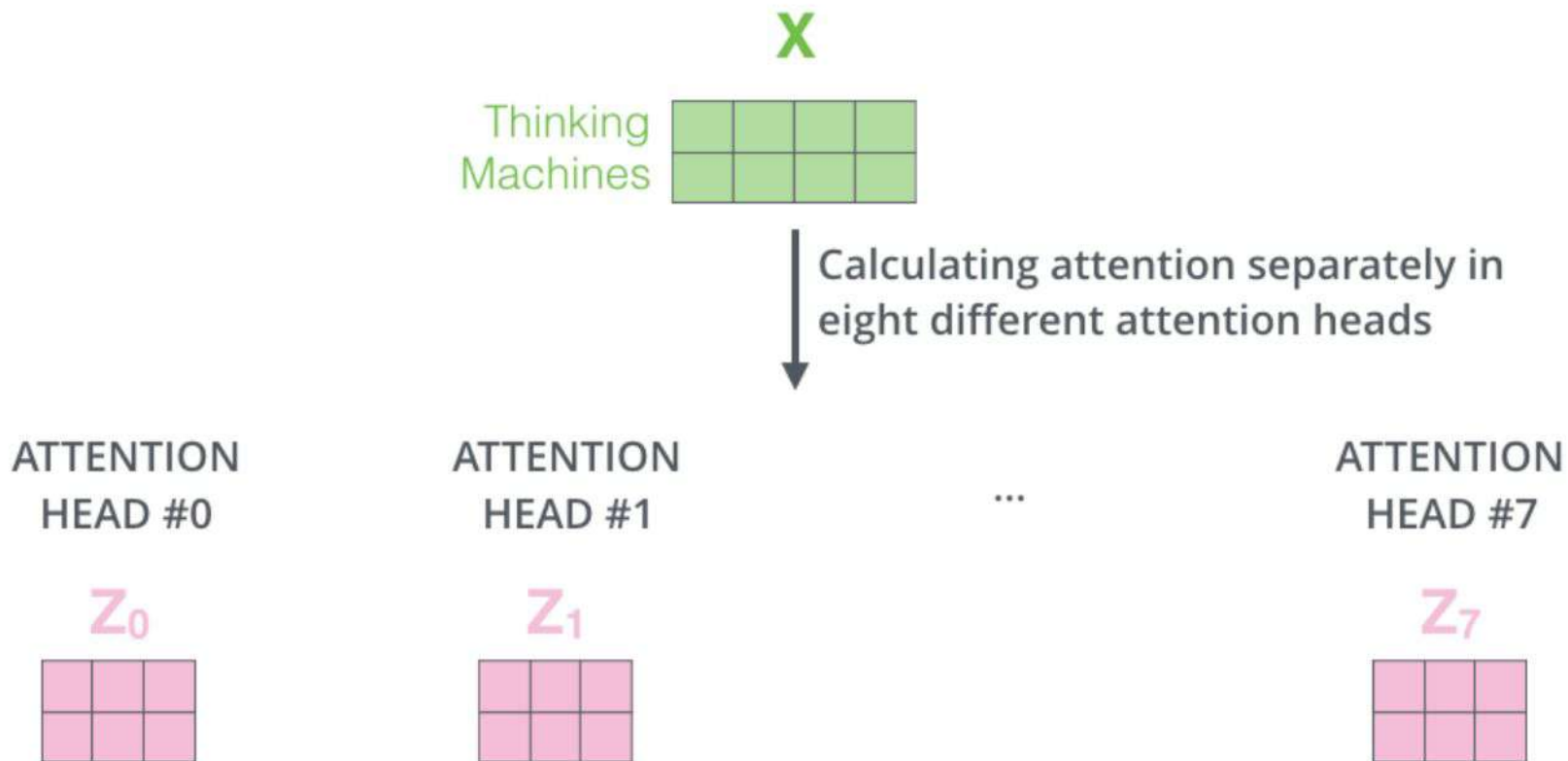
Z

$\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}$

Multi-Head Attention

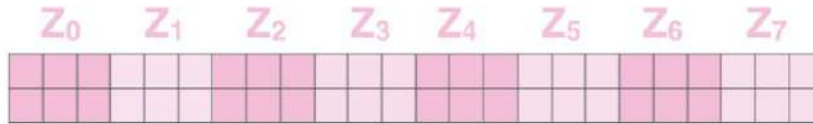


Multi-Head Attention



Multi-Head Attention

1) Concatenate all the attention heads

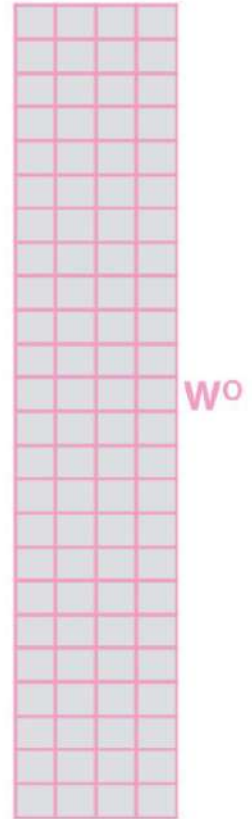


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



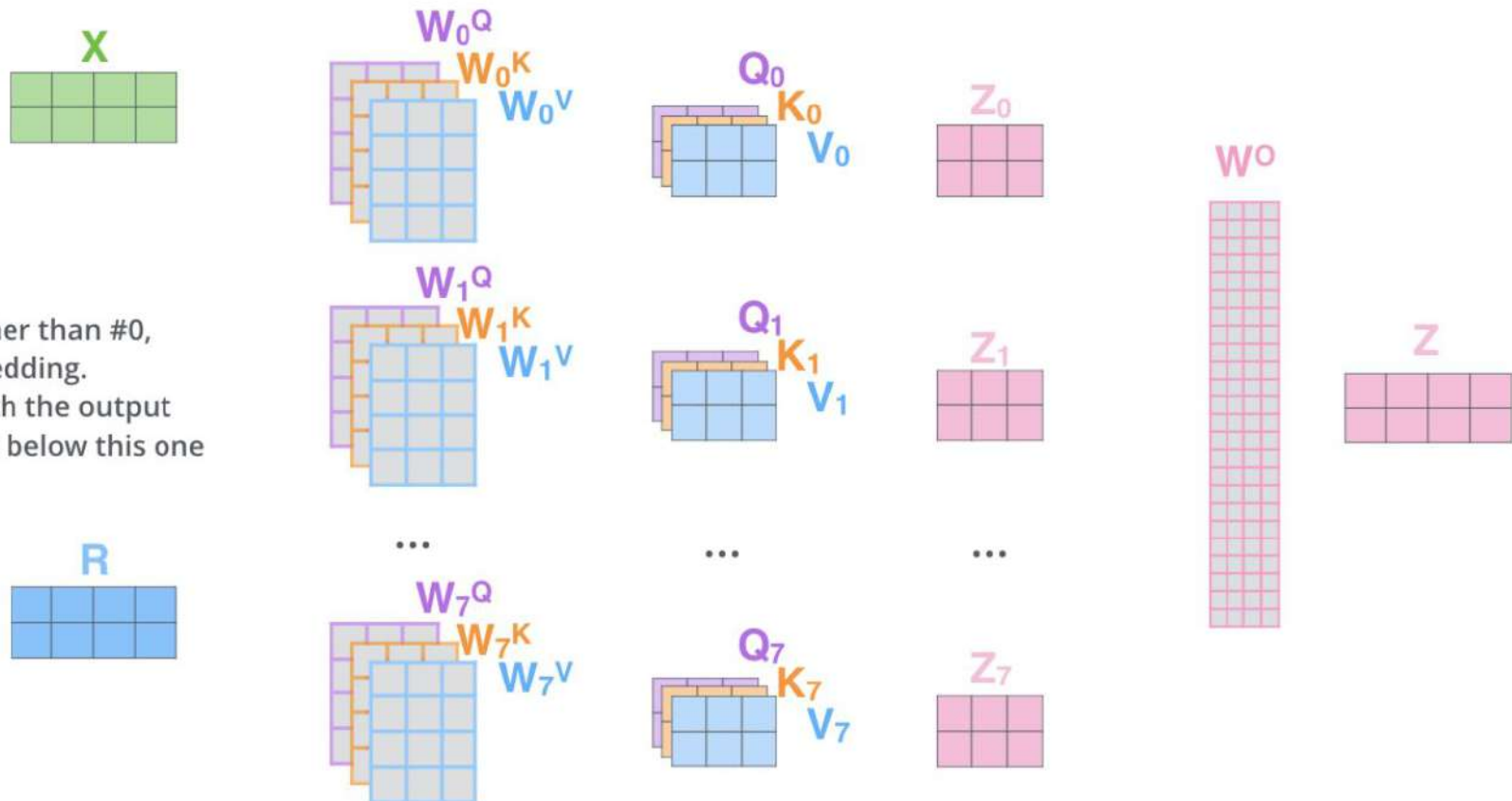
2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$



- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

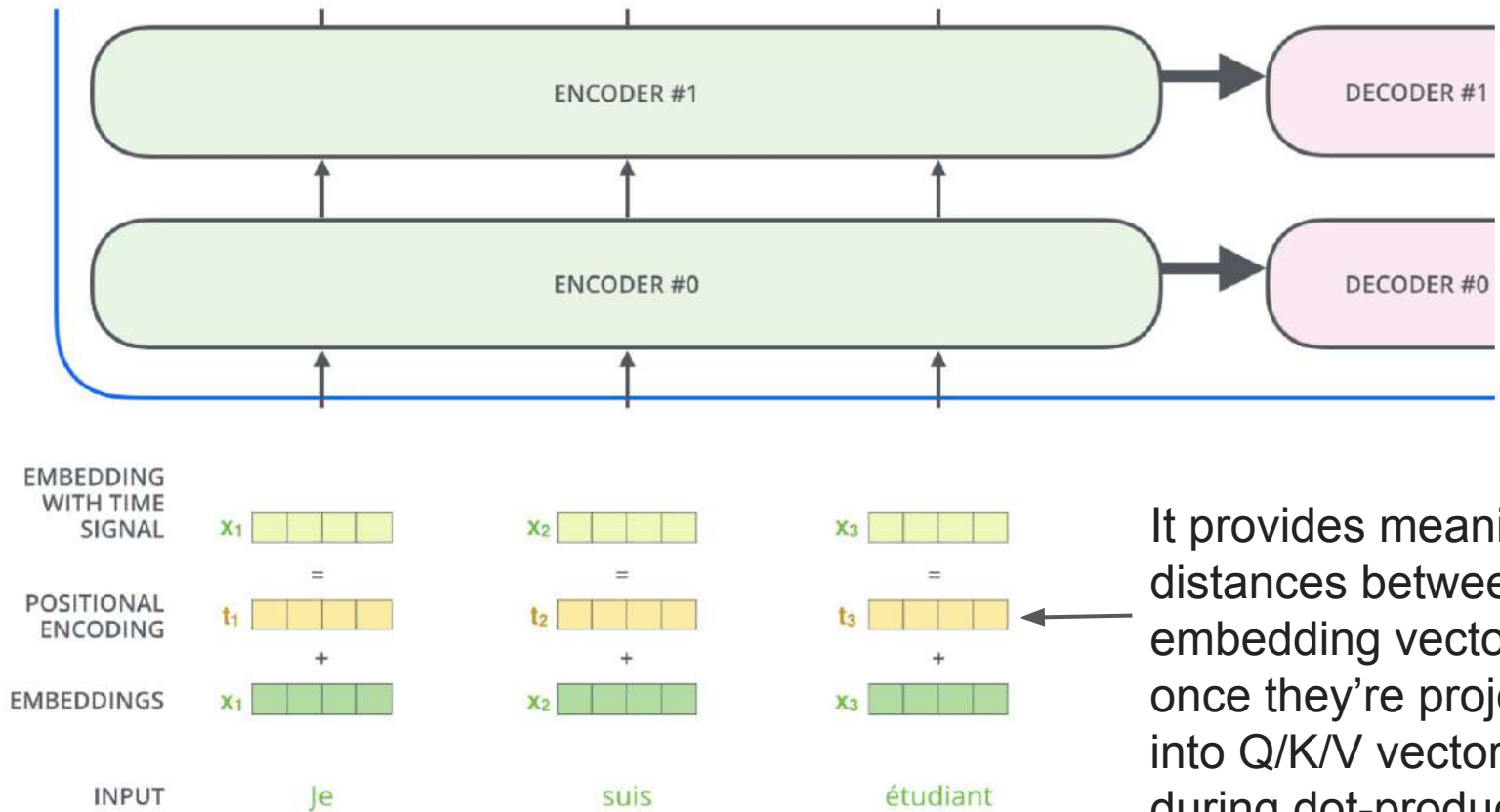
Thinking
Machines



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

Positional Encoding

Positional Encoding



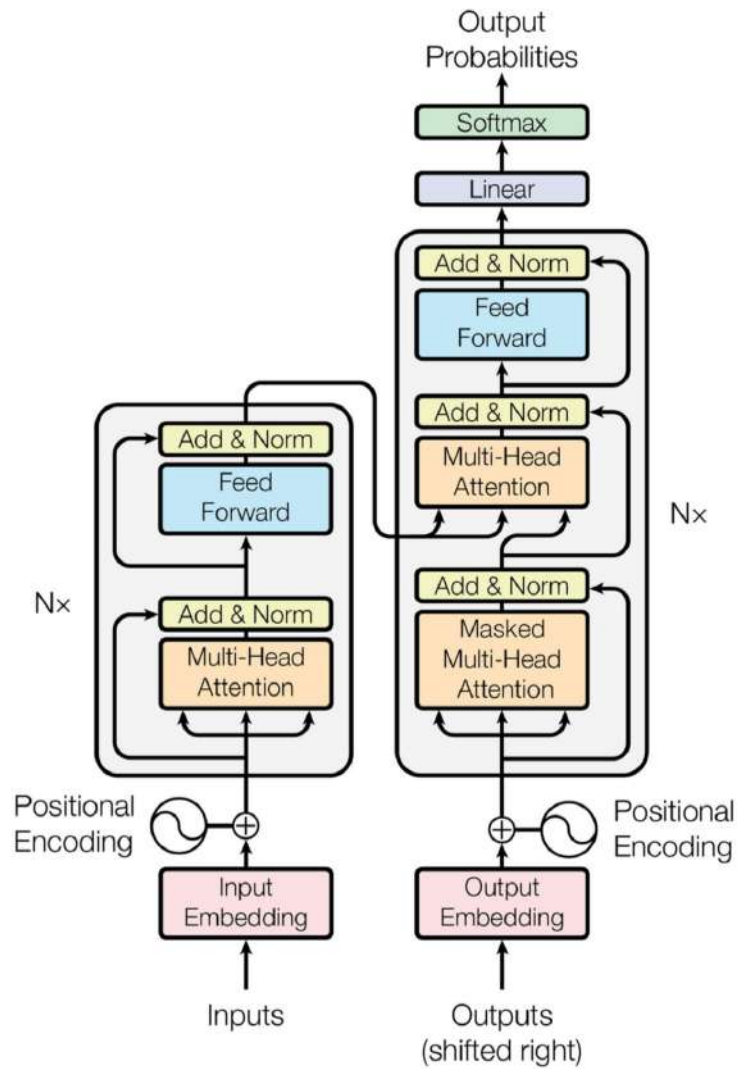
It provides meaningful distances between the embedding vectors once they're projected into Q/K/V vectors and during dot-product attention

Positional Encoding

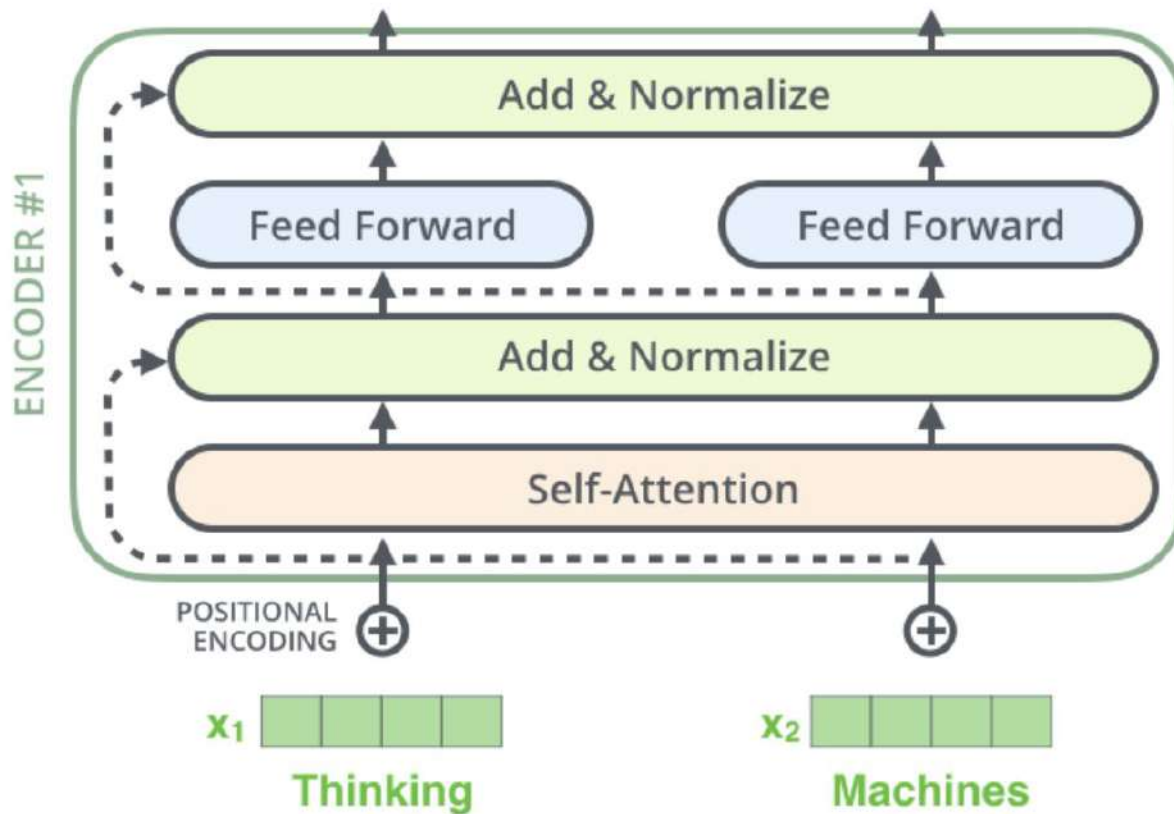


Layer Normalization

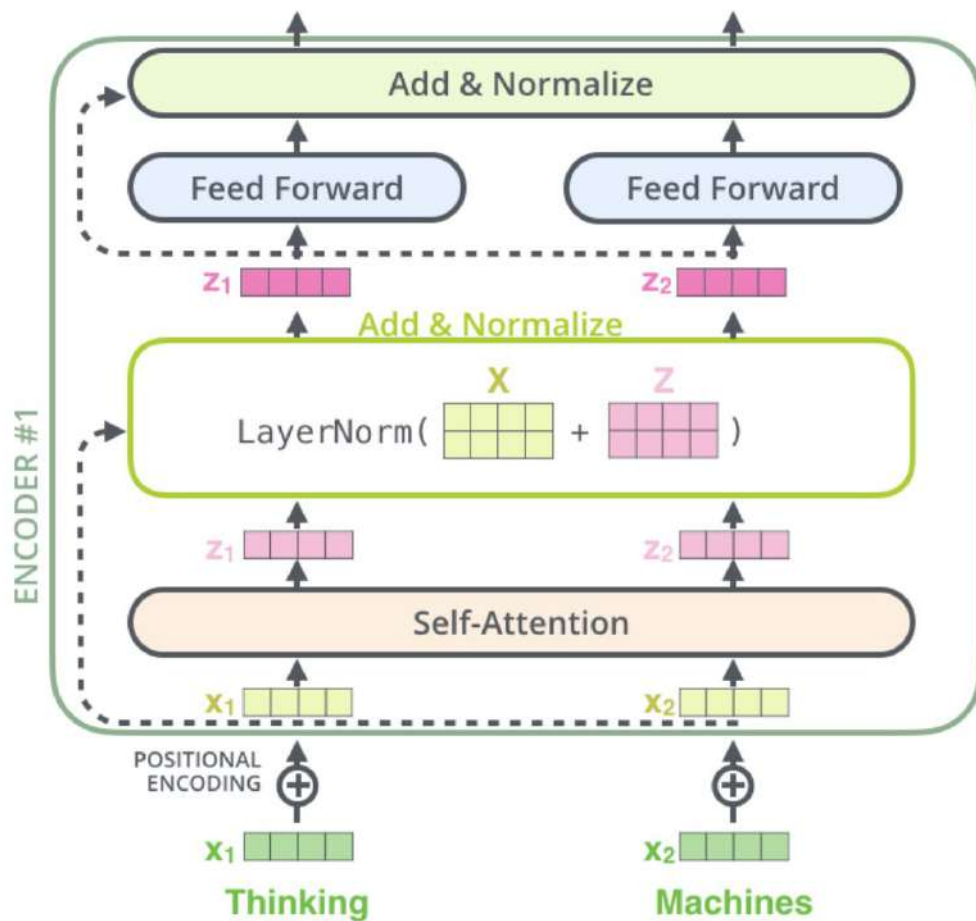
The Transformer: recap



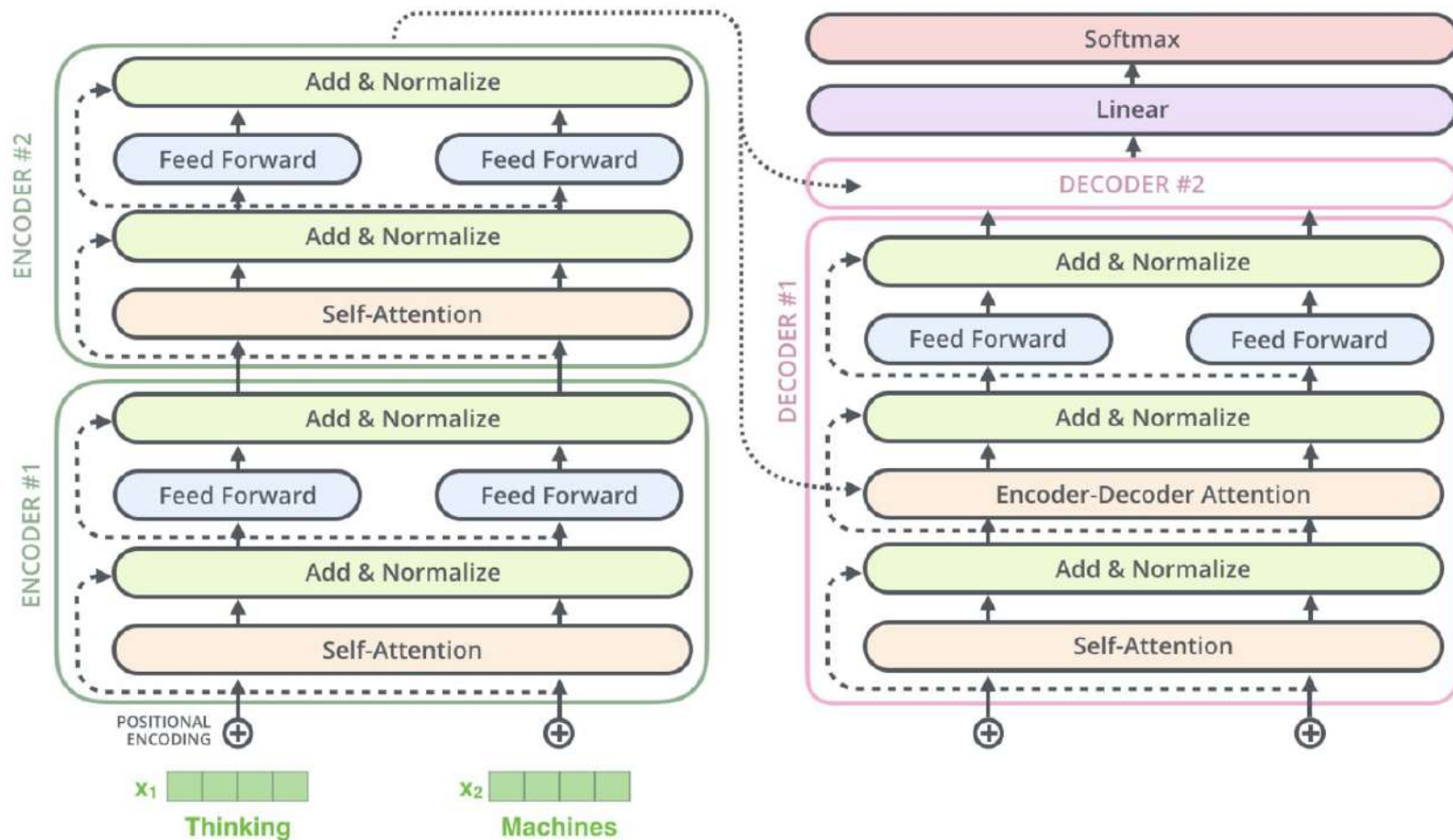
Layer Normalization



Layer Normalization



Layer Normalization



OpenAI Transformer: Pre-training Decoder for Language Modeling

OpenAI Transformer

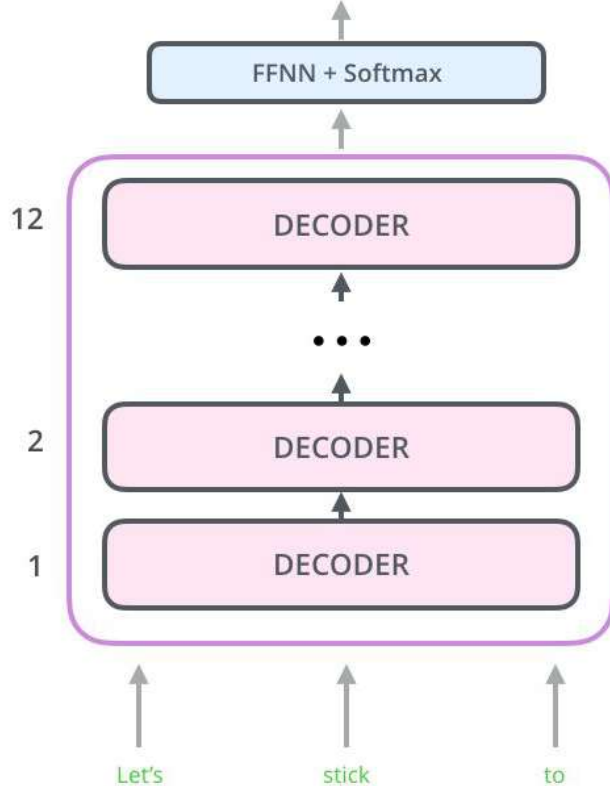
- The Encoder-Decoder structure of the transformer made it perfect for machine translation
- But what about sentence classification?
- **Main goal: pre-train a language model that can be fine-tuned for other tasks**



OpenAI Transformer

Possible classes:
All English words

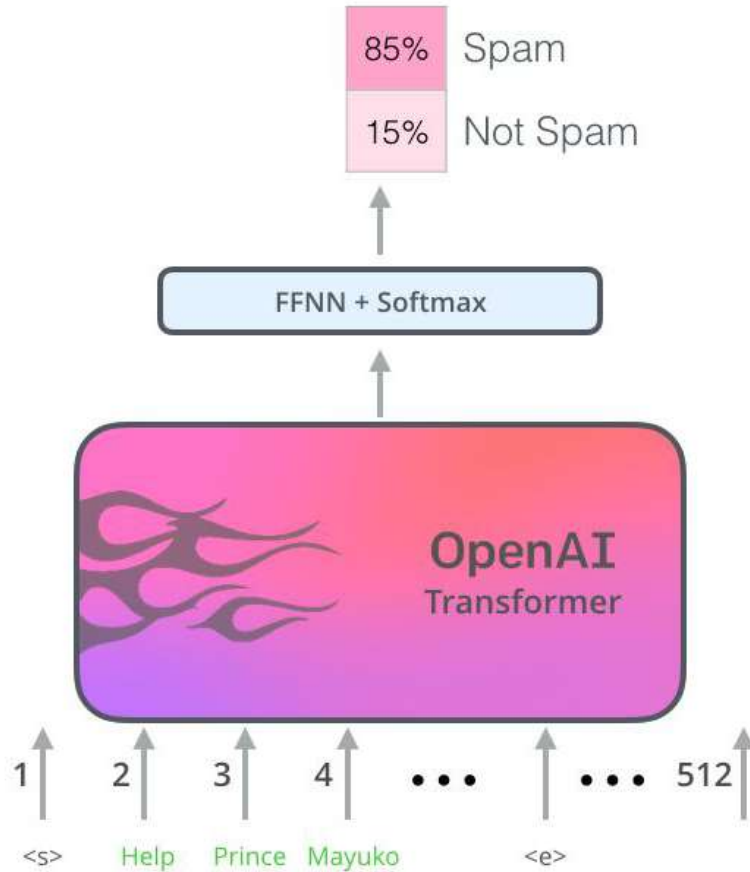
0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzzzyva



Differences from vanilla Transformer:

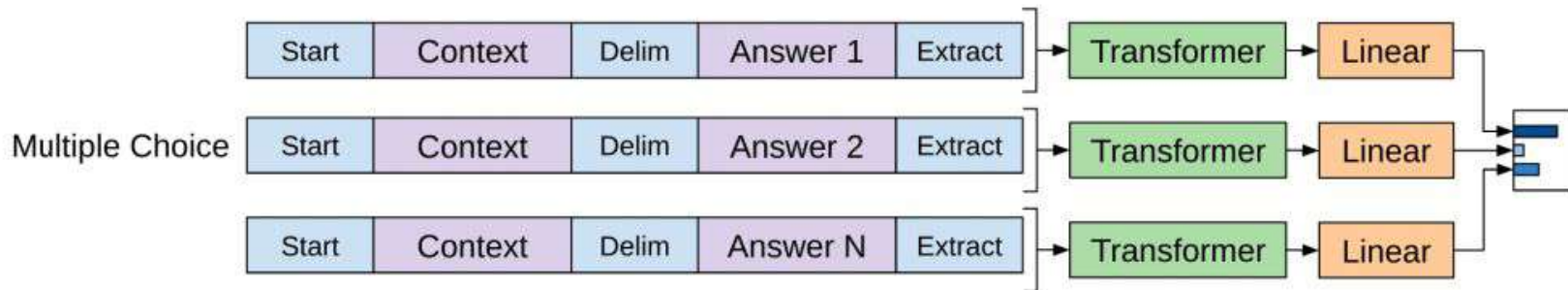
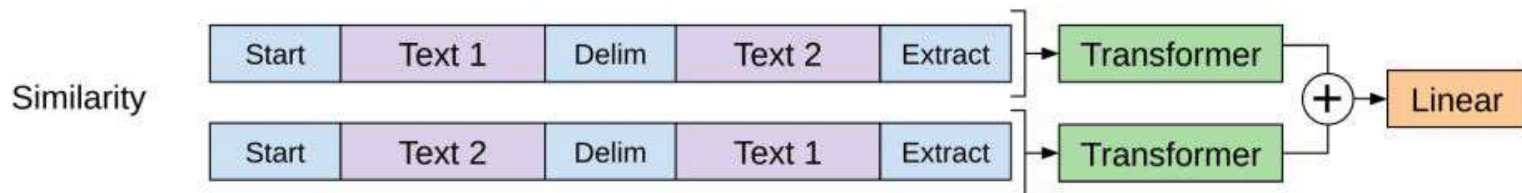
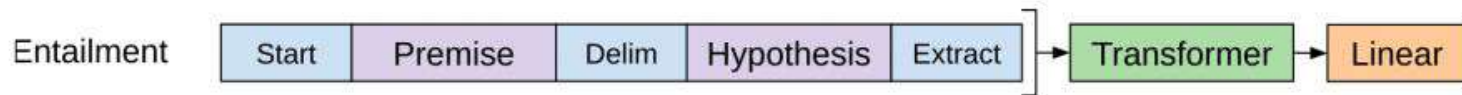
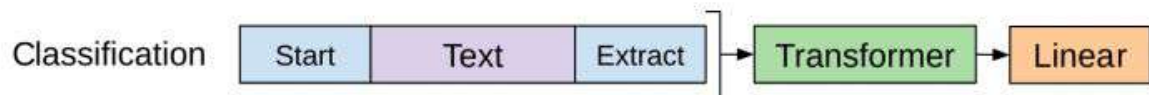
- no encoder
- decoder layers would not have the encoder-decoder attention sublayer
- Pre-train the model on predicting the next word using massive (unlabeled) datasets

OpenAI Transformer



- During pre-training phase layers have been tuned to reasonably handle language
- Now let's use it for downstream tasks (e.g. sentence classification)

Input transformations for different tasks



ELMo: context that matters

ELMo: contextualized word embeddings

“Why not give it an embedding based on the context it’s used in – to both capture the word meaning in that context as well as other contextual information?”



[Peters et. al., 2017](#), [McCann et. al., 2017](#),
and yet again [Peters et. al., 2018 in the ELMo paper](#)

ELMo - deep contextualized word representations

What does it stand for?



1. **E**xpedited **L**abour **M**arket **O**pinion
2. **E**lectric **L**ight **M**achine **O**rganization
3. **E**nough **L**et's **M**ove **O**n

What does it stand for?



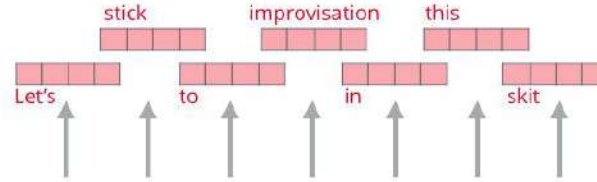
1. **E**xpedited **L**abour **M**arket **O**pinion
2. **E**lectric **L**ight **M**achine **O**rganization
3. **E**nough **L**et's **M**ove **O**n
4. **E**mbellishments from **L**anguage **M**odels

ELMo: contextualized word embeddings



ELMo: Contextualized word embeddings

ELMo
Embeddings

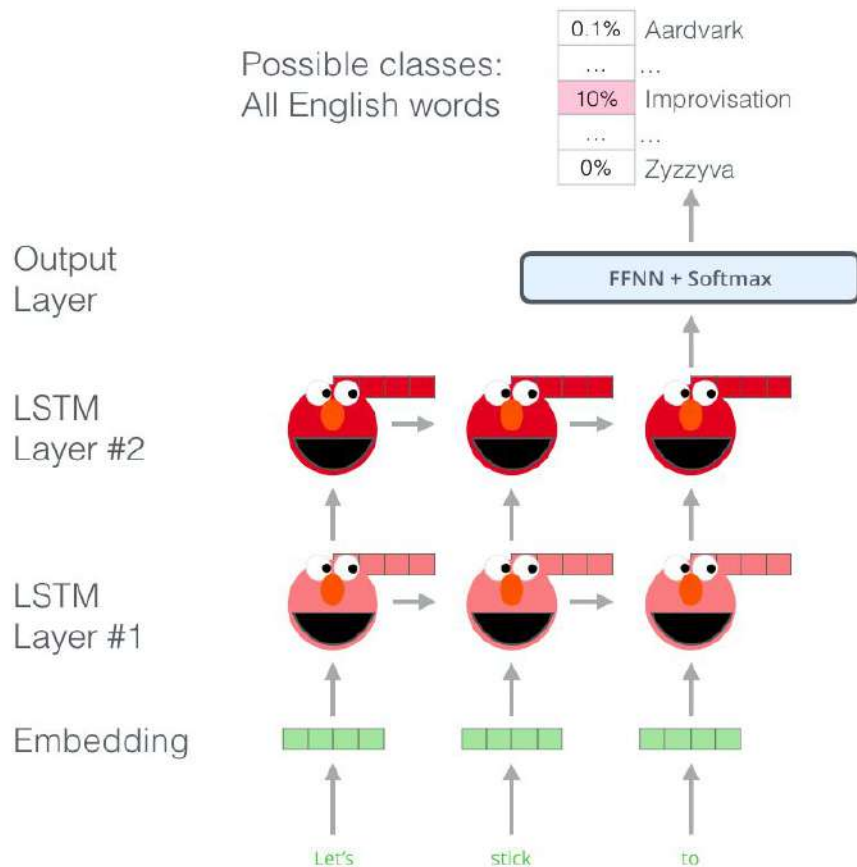


Words to embed



- uses a bi-directional LSTM trained on Language Modeling task
- a model can learn without labels

Bidirectional Language Models (biLMs)



biLMs consist of forward and backward LMs:

- forward:

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_1, t_2, \dots, t_{k-1})$$

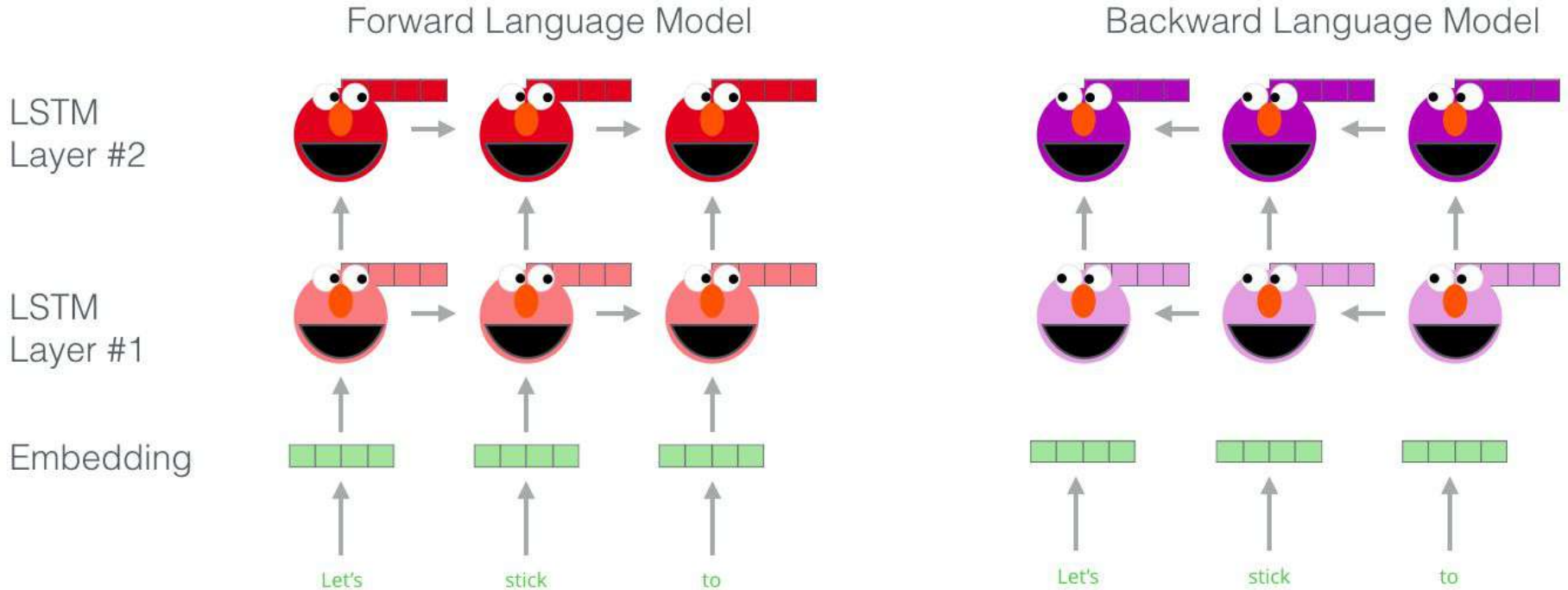
- Backward:

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_{k+1}, t_{k+2}, \dots, t_N)$$

LSTM predicts next word in both directions to build biLMs

ELMo: main pipeline

Embedding of “stick” in “Let’s stick to” - Step #1

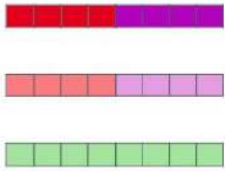


ELMo: main pipeline

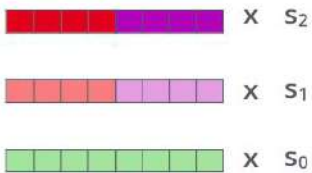
ELMo represents a word as a linear combination of corresponding hidden layers:

Embedding of “stick” in “Let’s stick to” - Step #2

1- Concatenate hidden layers



2- Multiply each vector by a weight based on the task

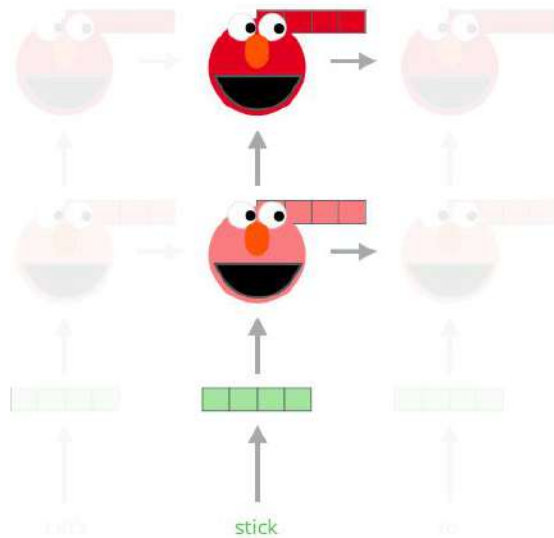


3- Sum the (now weighted) vectors

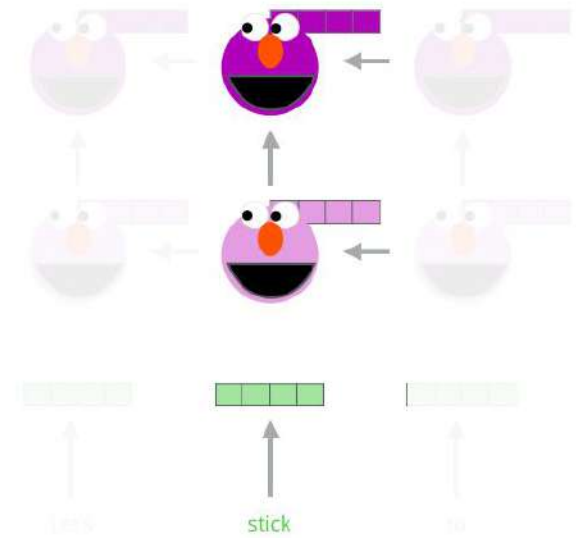


ELMo embedding of “stick” for this task in this context

Forward Language Model

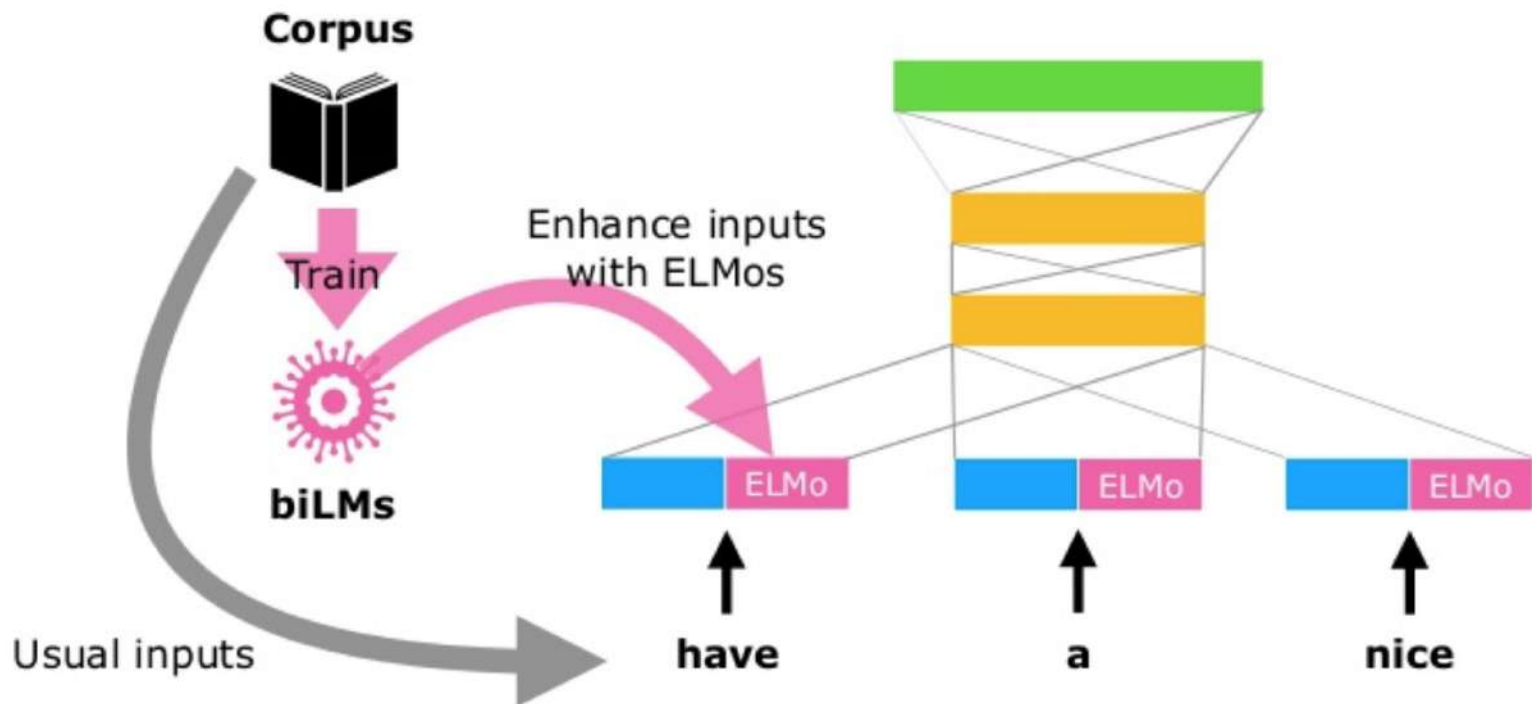


Backward Language Model



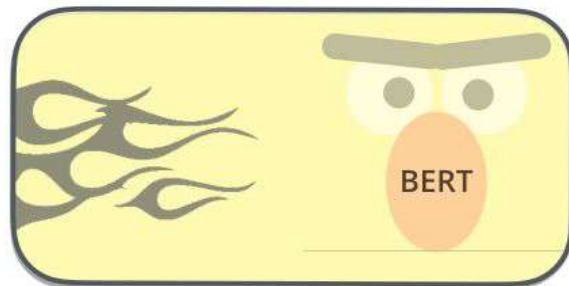
ELMo

ELMo can be integrated to almost all neural NLP tasks with simple concatenation to the embedding layer



ELMo: overview

- Pretrained ELMo models: <http://allennlp.org/elmo>
- AllenNLP is a library on the top of PyTorch
- Higher levels seems to catch semantics while lower layer probably capture syntactic features



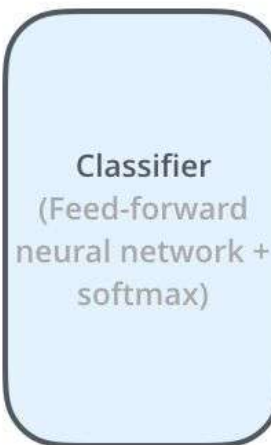
BERT

Bidirectional Encoder Representations from Transformers

BERT

Input
Features

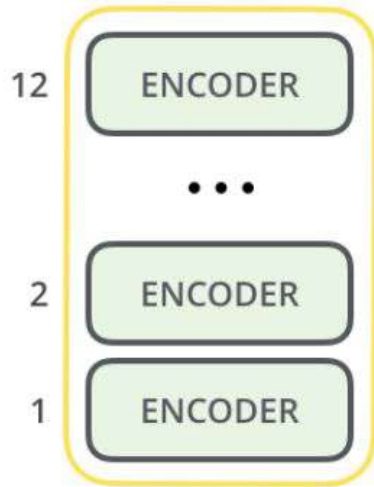
Help Prince Mayuko Transfer
Huge Inheritance



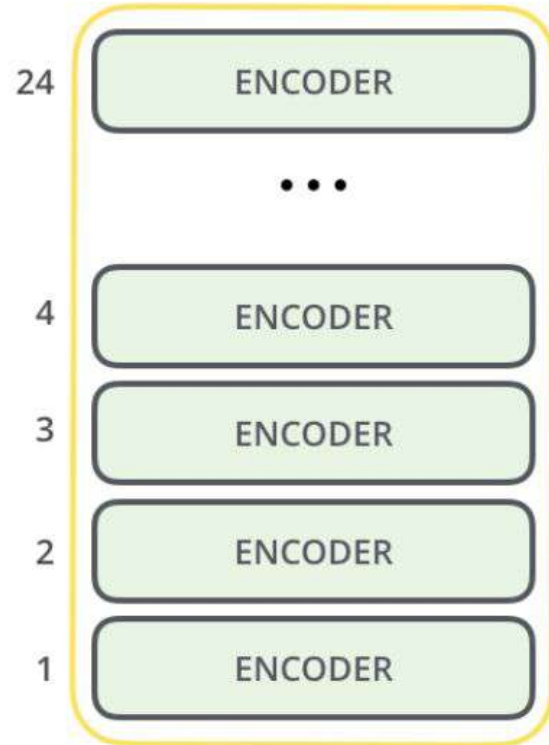
Output
Prediction

85%	Spam
15%	Not Spam

BERT: base and large

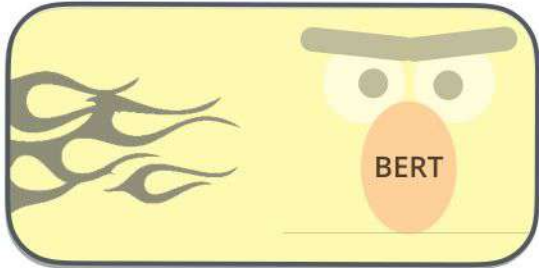


BERT_{BASE}

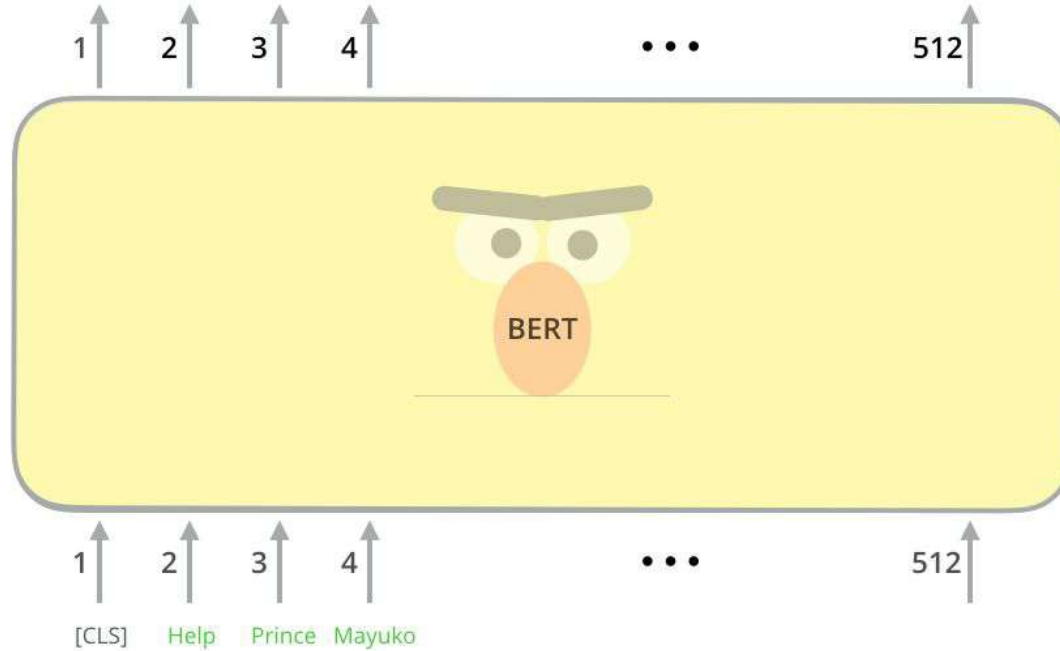


BERT_{LARGE}

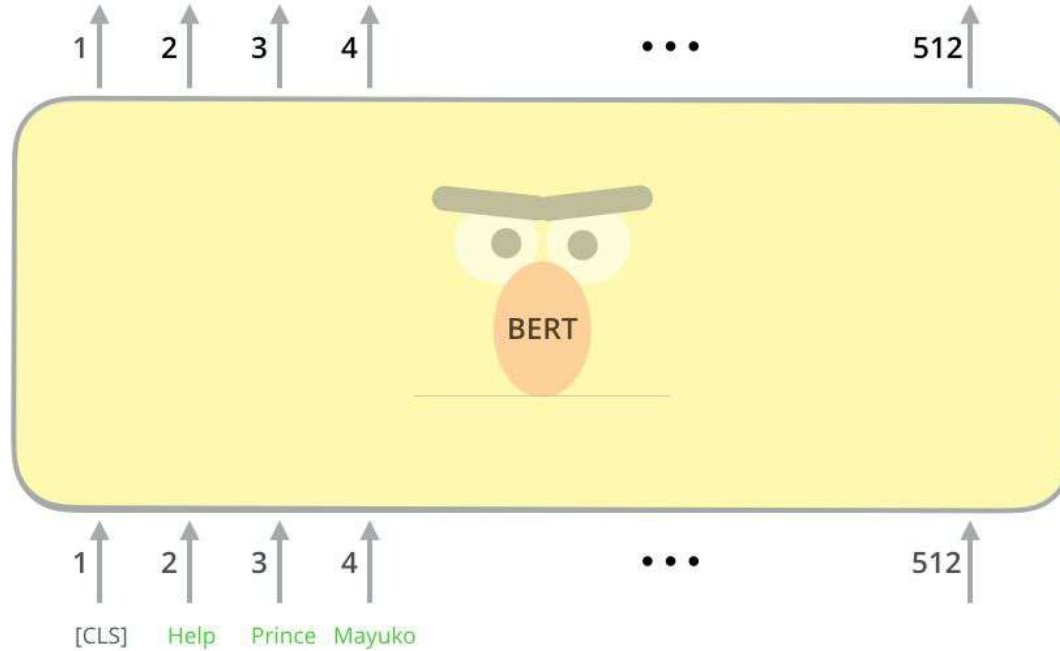
BERT vs. Transformer

			
		Base BERT	Large BERT
Encoders	6	12	24
Units in FFN	512	768	1024
Attention Heads	8	12	16

Model inputs

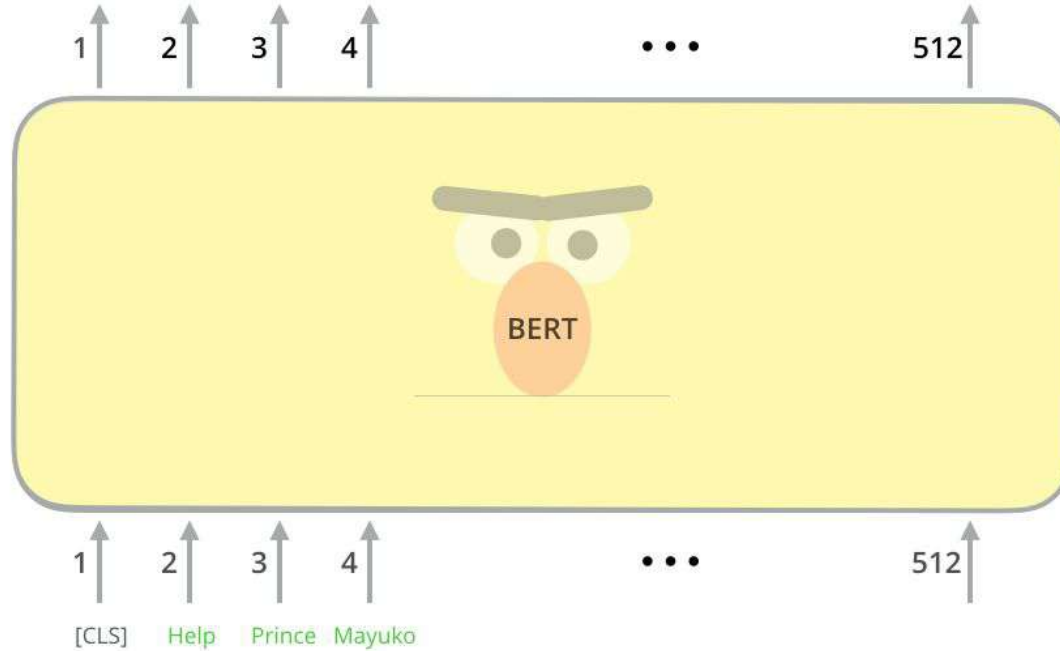


Model inputs



Identical to the Transformer up until this point

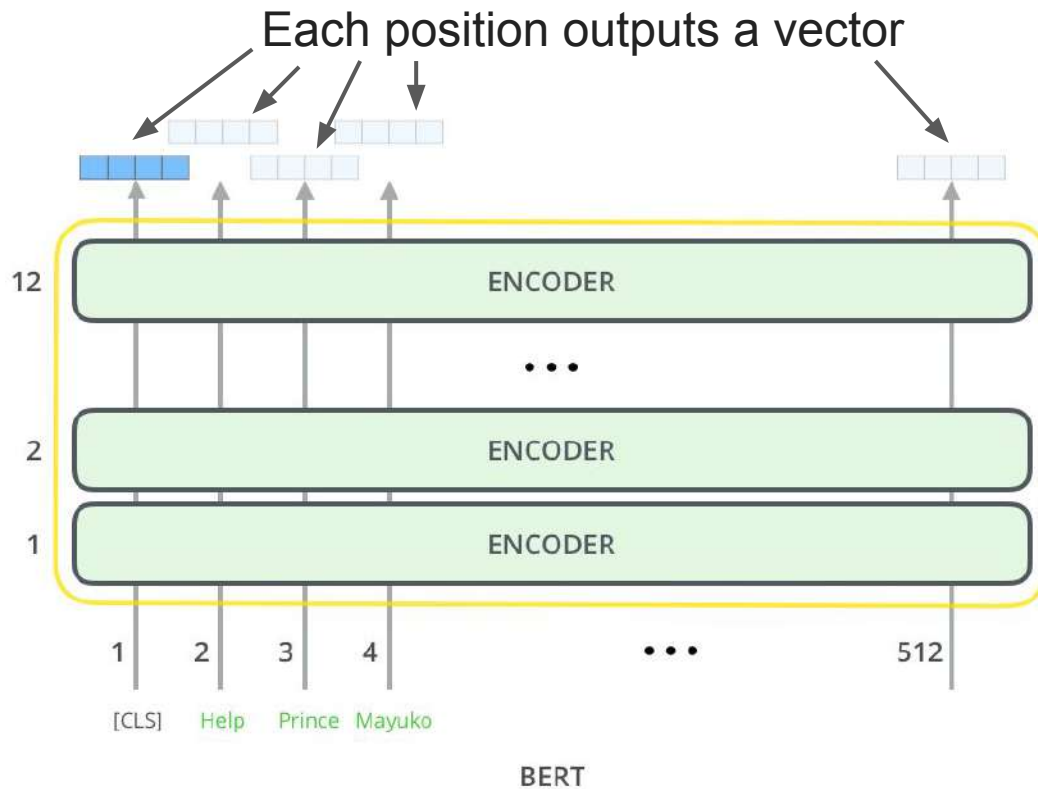
Model inputs



Identical to the Transformer up until this point

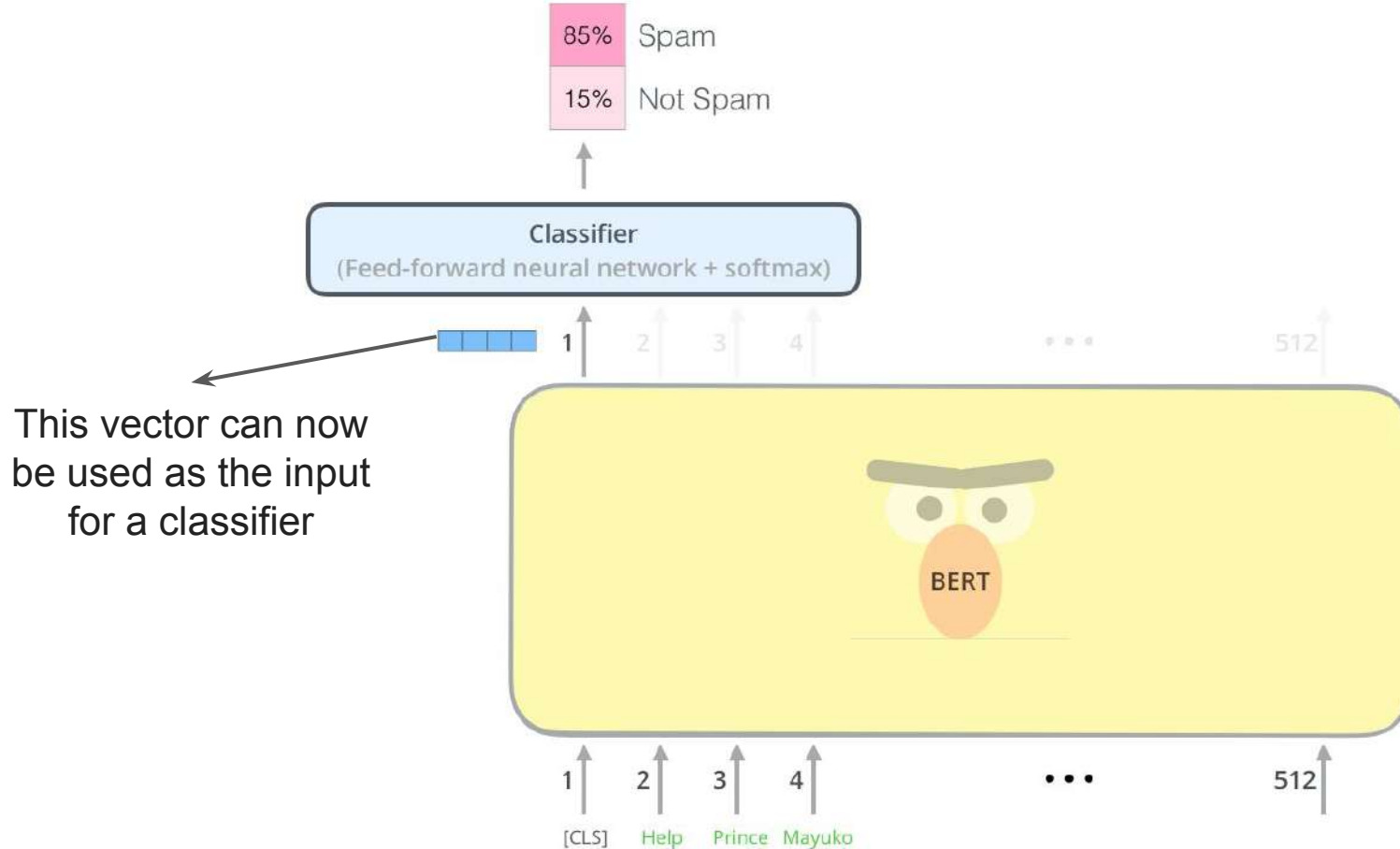
Why is BERT so special?

Model outputs



For sentence classification we focus on the first position (that we passed [CLS] token to)

Model inputs

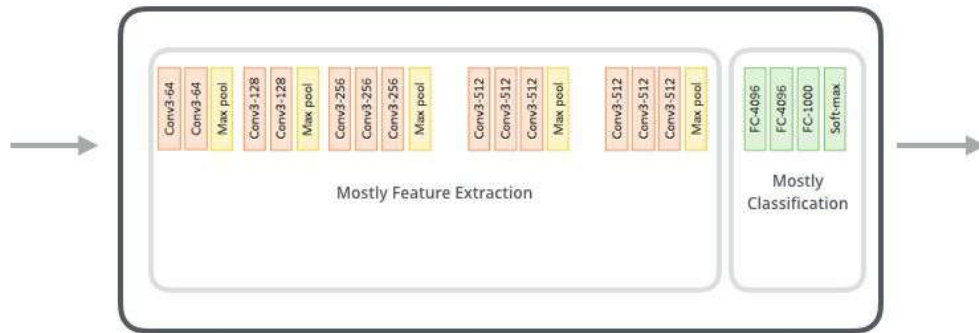


Similar to CNN concept!

Input
Features



VGG-16



Output
Prediction

0.2%	Kit fox
0.1%	English setter
95%	Egyptian cat
1%	Great Dane
...	...
0%	Hotdog

BERT: pre-training

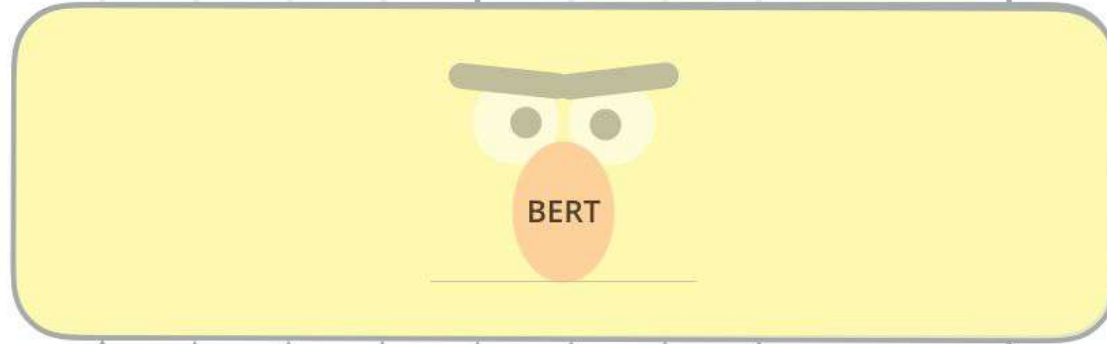
Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzzzva

FFNN + Softmax

1 2 3 4 5 6 7 8 ... 512



Randomly mask
15% of tokens

1 2 3 4 5 6 7 8 ... 512
[CLS] Let's stick to [MASK] in this skit

Input

[CLS] Let's stick to improvisation in this skit

BERT: pre-training

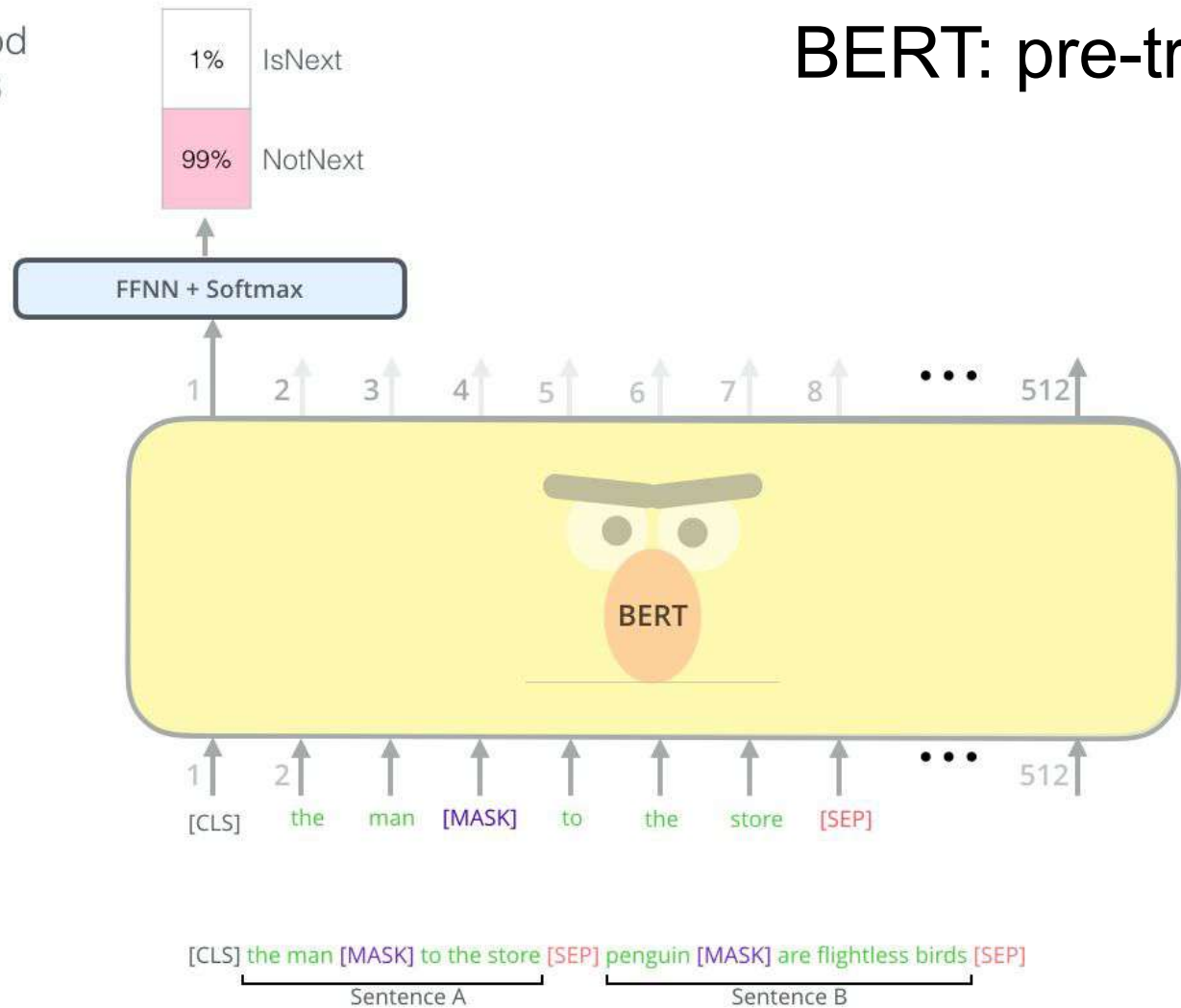
- “Masked Language Model” approach
- To make BERT better at handling relationships between multiple sentences, the pre-training process includes an additional task:
“Given two sentences (A and B), is B likely to be the sentence that follows A, or not?”

BERT: pre-training

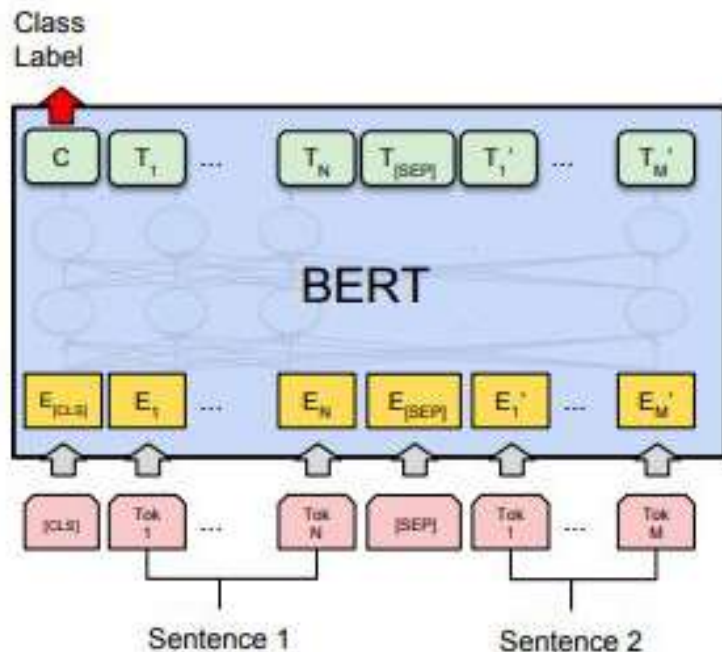
Predict likelihood
that sentence B
belongs after
sentence A

Tokenized
Input

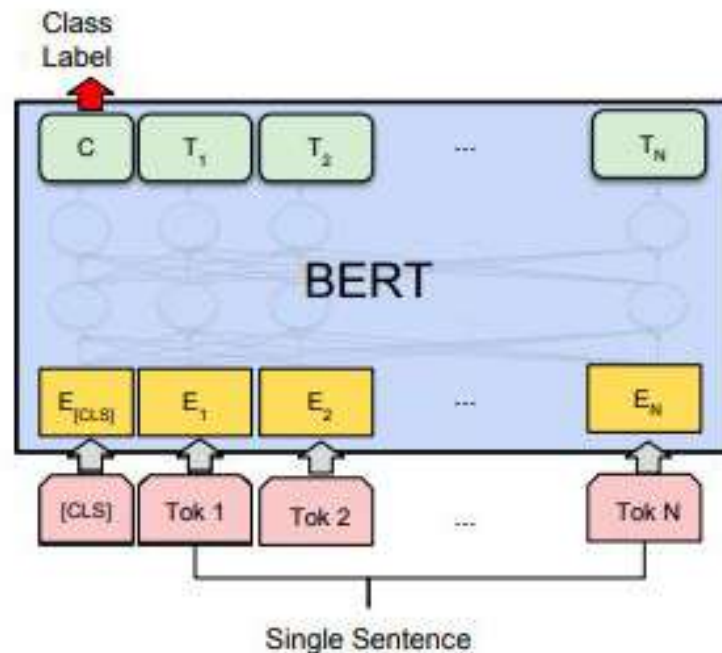
Input



BERT: fine-tuning for different tasks

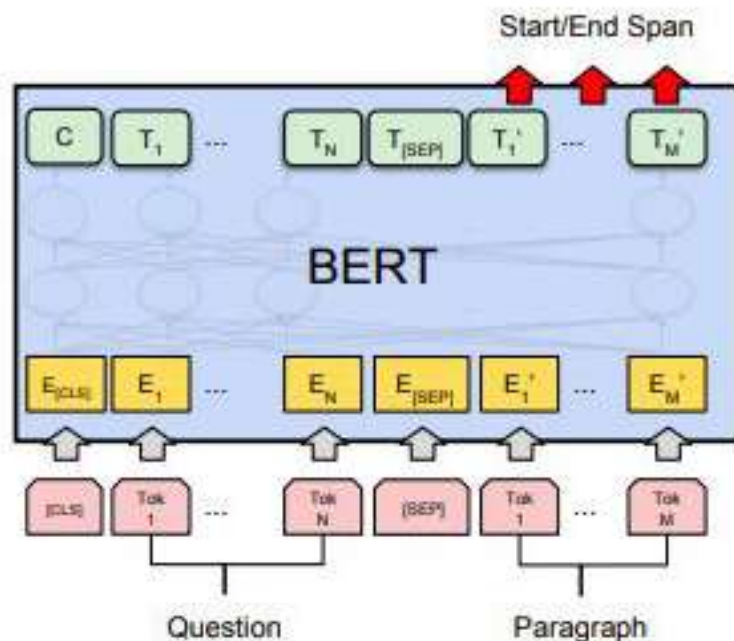


(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG

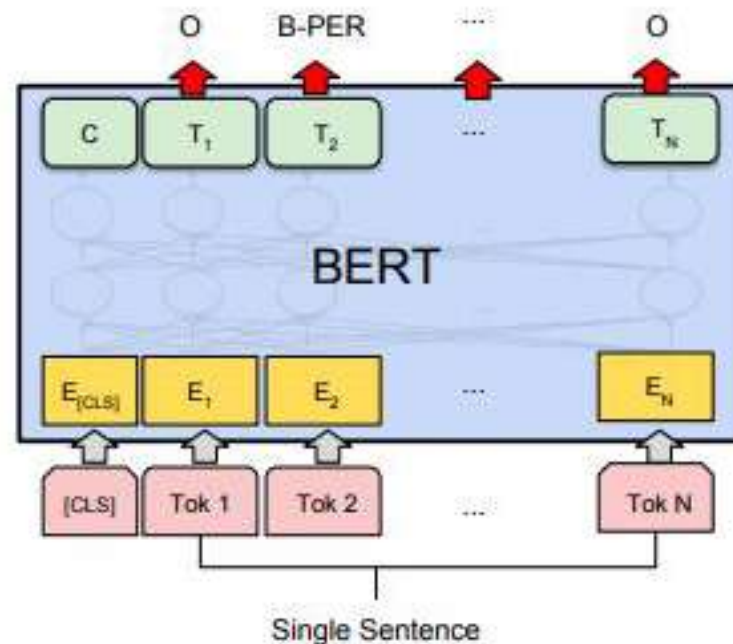


(b) Single Sentence Classification Tasks:
SST-2, CoLA

BERT: fine-tuning for different tasks

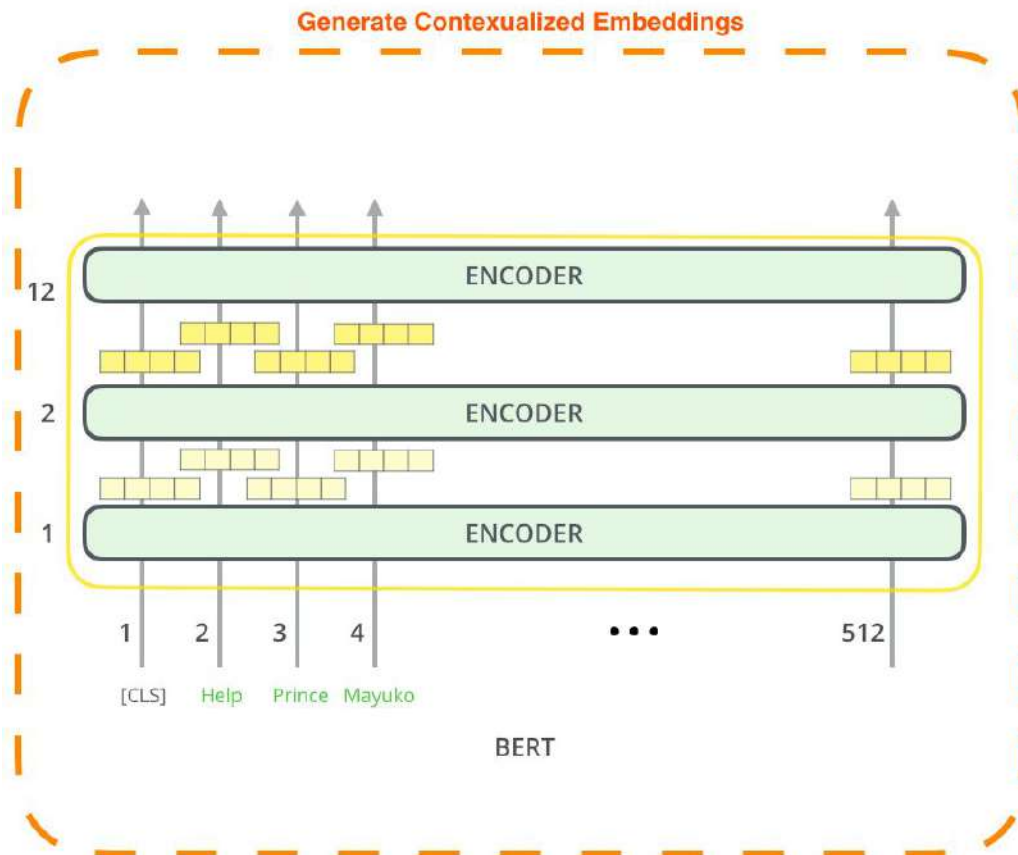


(c) Question Answering Tasks:
SQuAD v1.1

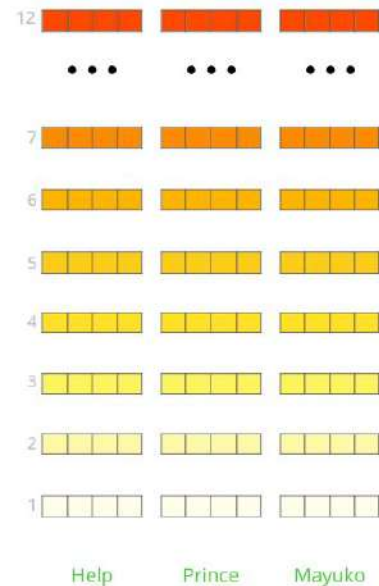


(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

BERT for feature extraction



The output of each encoder layer along each token's path can be used as a feature representing that token.

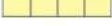


But which one should we use?

BERT for feature extraction

What is the best contextualized embedding for “Help” in that context?

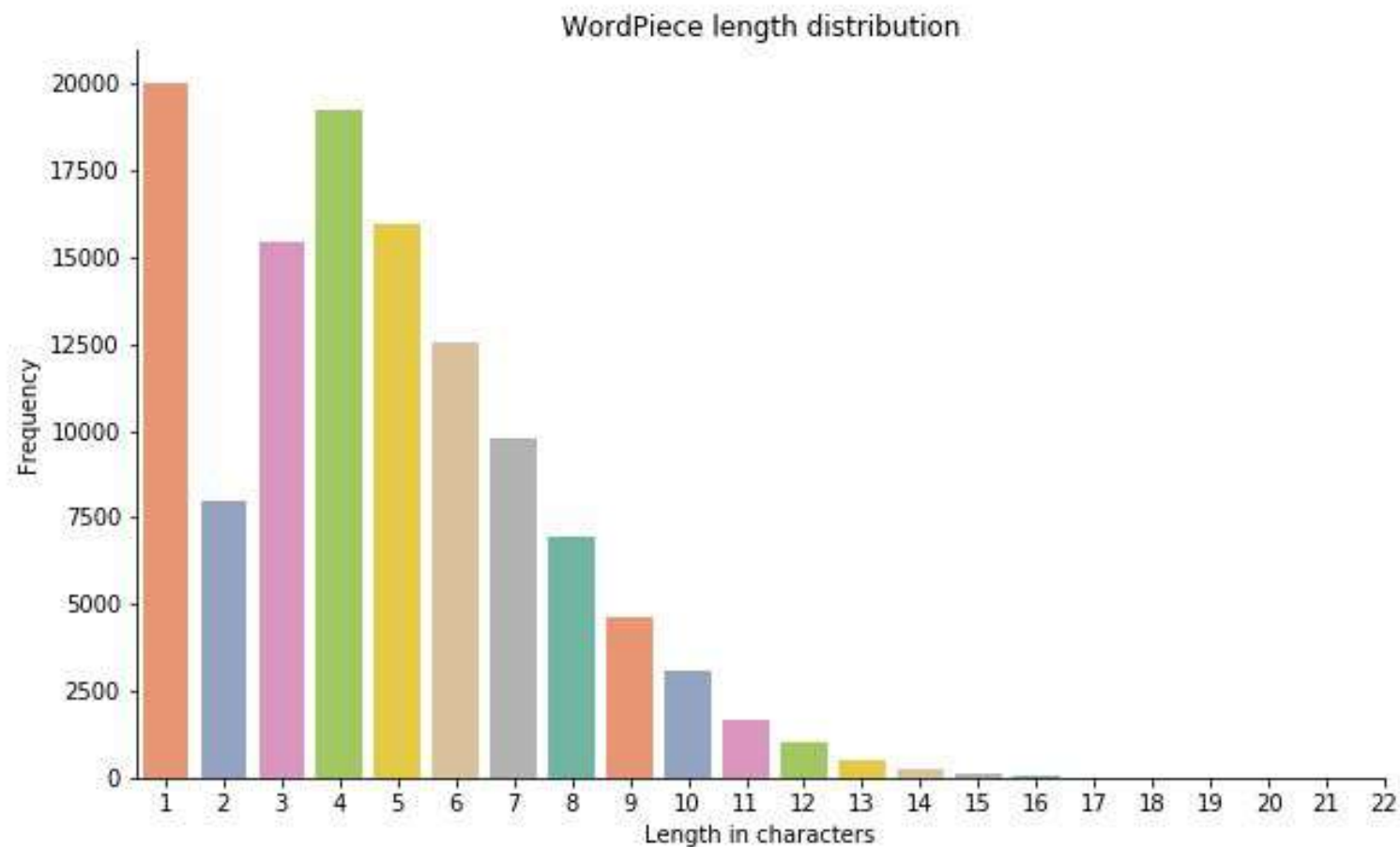
For named-entity recognition task CoNLL-2003 NER

		Dev F1 Score
12 	First Layer	91.0
• • •	Last Hidden Layer	94.9
7 		
6 	Sum All 12 Layers	95.5
5 		
4 		
3 		
2 		
1 		
		
Help		
	Second-to-Last Hidden Layer	95.6
	Sum Last Four Hidden	95.9
	Concat Last Four Hidden	96.1

Example: Unaffable -> un, ##aff, ##able

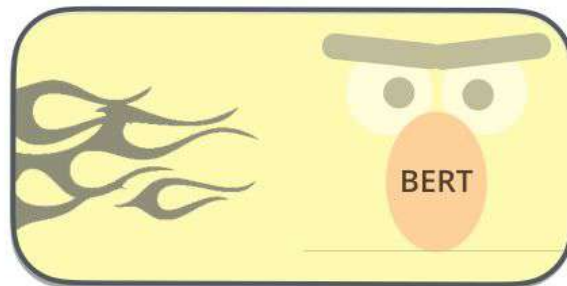
- Single model for 104 languages with a large shared vocabulary (119,547 [WordPiece](#) model)
- Non-word-initial units are prefixed with ##
- The first 106 symbols: constants like PAD and UNK
- 36.5% of the vocabulary are non-initial word pieces
- The alphabet consists of 9,997 unique characters that are defined as word-initial (C) and continuation symbols (##C), which together make up 19,994 word pieces
- The rest are multicharacter word pieces of various length.

BERT: tokenization



BERT: overview

- [BERT repo](#)
- [Try out BERT on TPU](#)
- [WordPieces Tokenizer](#)
- [PyTorch Implementation of BERT](#)



Contrastive learning recap

Radoslav Neychev

Senior researcher



Self-supervised Learning

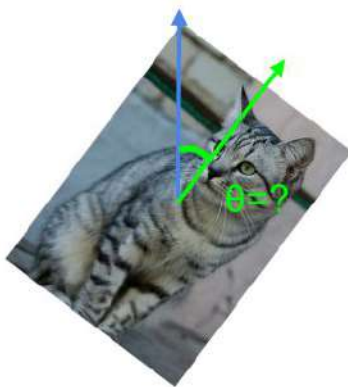
- Both aim to learn from data without manual label annotation.
- Self-supervised learning methods solve “pretext” tasks that produce good features for downstream tasks.
 - Learn with supervised learning objectives, e.g., classification, regression.
 - Labels of these pretext tasks are generated automatically

Self-supervised pretext tasks

Example: learn to predict image transformations / complete corrupted images



image completion



rotation prediction



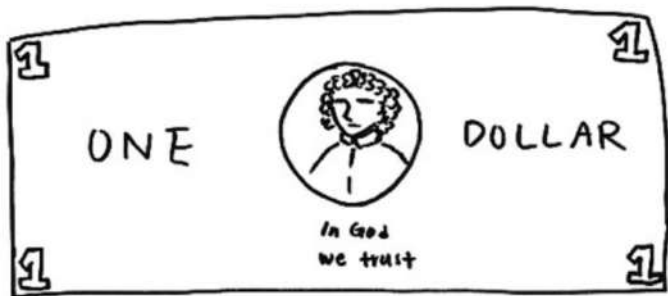
"jigsaw puzzle"



colorization

1. Solving the pretext tasks allow the model to learn good features.
2. We can automatically generate labels for the pretext tasks.

Generative vs. Self-supervised Learning



Left: Drawing of a dollar bill from memory. Right: Drawing subsequently made with a dollar bill present. Image source: [Epstein, 2016](#)

Learning to generate pixel-level details is often unnecessary; learn high-level semantic features with pretext tasks instead

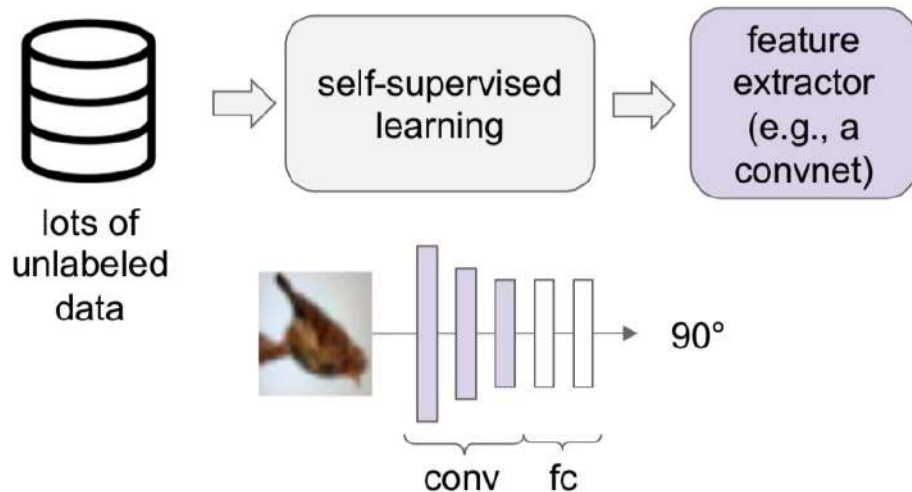
How to evaluate a self-supervised learning method?

We usually don't care about the performance of the self-supervised learning task, e.g., we don't care if the model learns to predict image rotation perfectly.

Evaluate the learned feature encoders on downstream target tasks

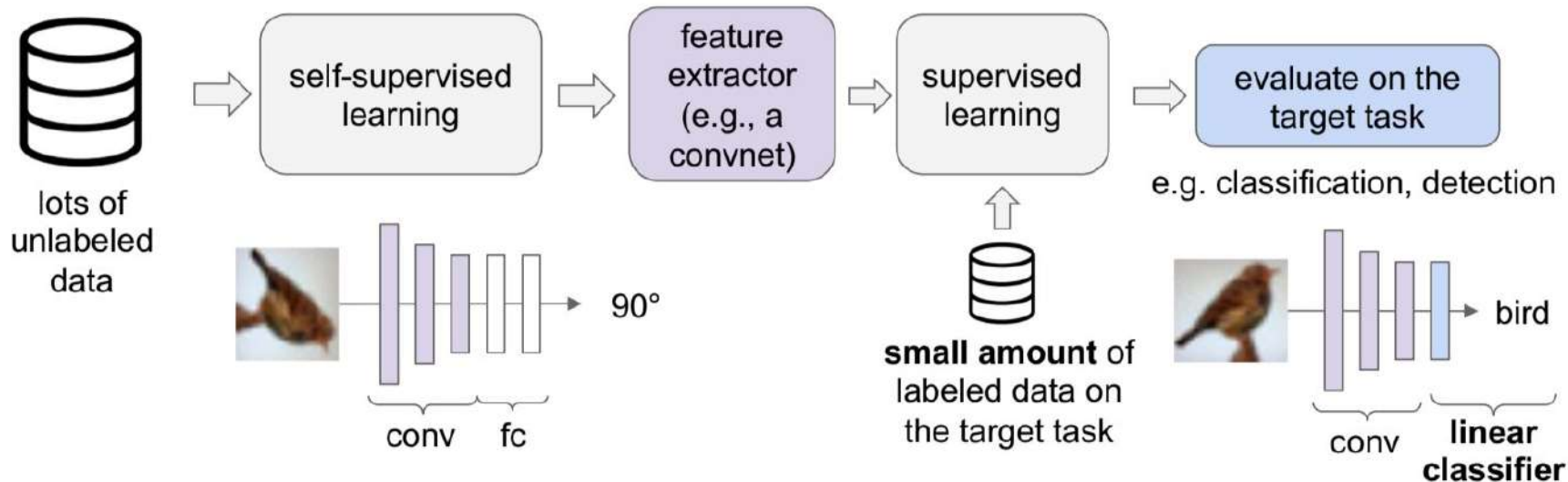


How to evaluate a self-supervised learning method?



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

How to evaluate a self-supervised learning method?

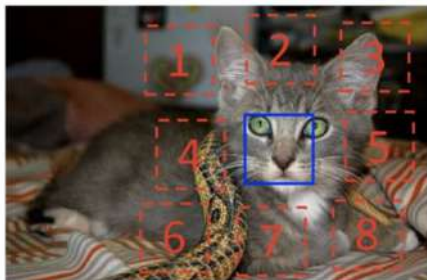


1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

2. Attach a shallow network on the feature extractor; train the shallow network on the target task with small amount of labeled data

Broader picture

computer vision



Doersch et al., 2015

robot / reinforcement learning



Dense Object Net (Florence and Manuelli et al., 2018)

Today's lecture

language modeling

GPT-4 Technical Report

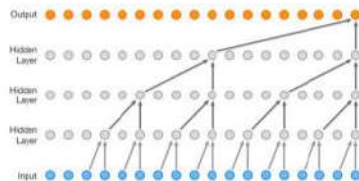
OpenAI*

Abstract

We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.

GPT-4 (OpenAI 2023)

speech synthesis



Wavenet (van den Oord et al., 2016)

...

Today's Agenda

- Pretext tasks from image transformations
 - Rotation, inpainting, rearrangement, coloring
- Contrastive representation learning
 - Intuition and formulation
 - Instance contrastive learning: SimCLR and MOCO
 - Sequence contrastive learning: CPC

Today's Agenda

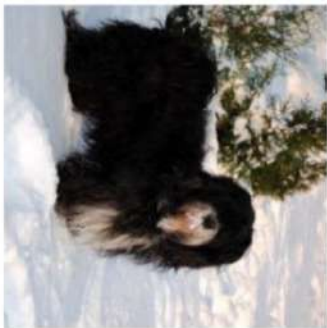
- Pretext tasks from image transformations
 - Rotation, inpainting, rearrangement, coloring
- Contrastive representation learning
 - Intuition and formulation
 - Instance contrastive learning: SimCLR and MOCO
 - Sequence contrastive learning: CPC



Pretext task: predict rotations



90° rotation



270° rotation



180° rotation



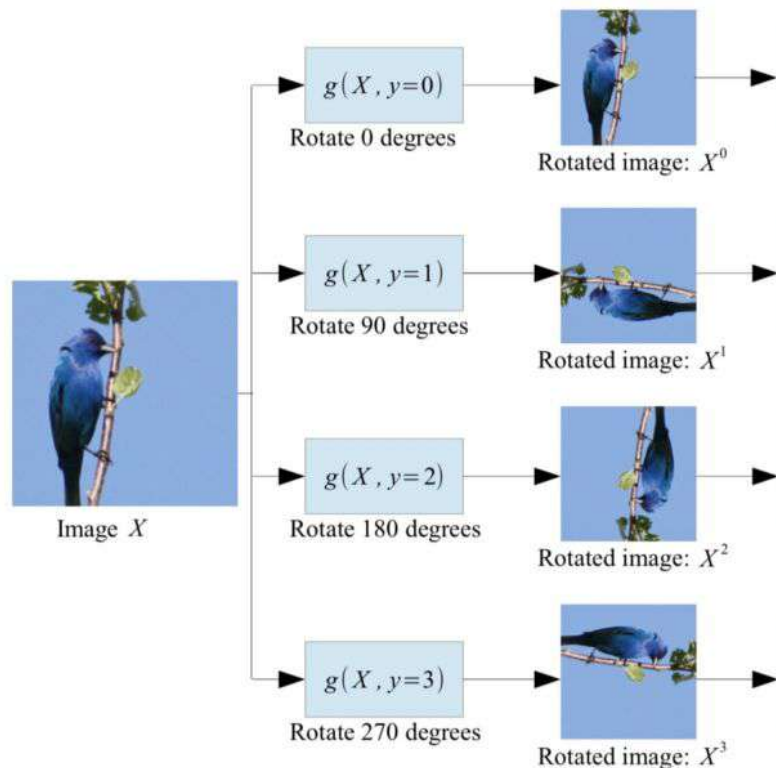
0° rotation



270° rotation

Hypothesis: a model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

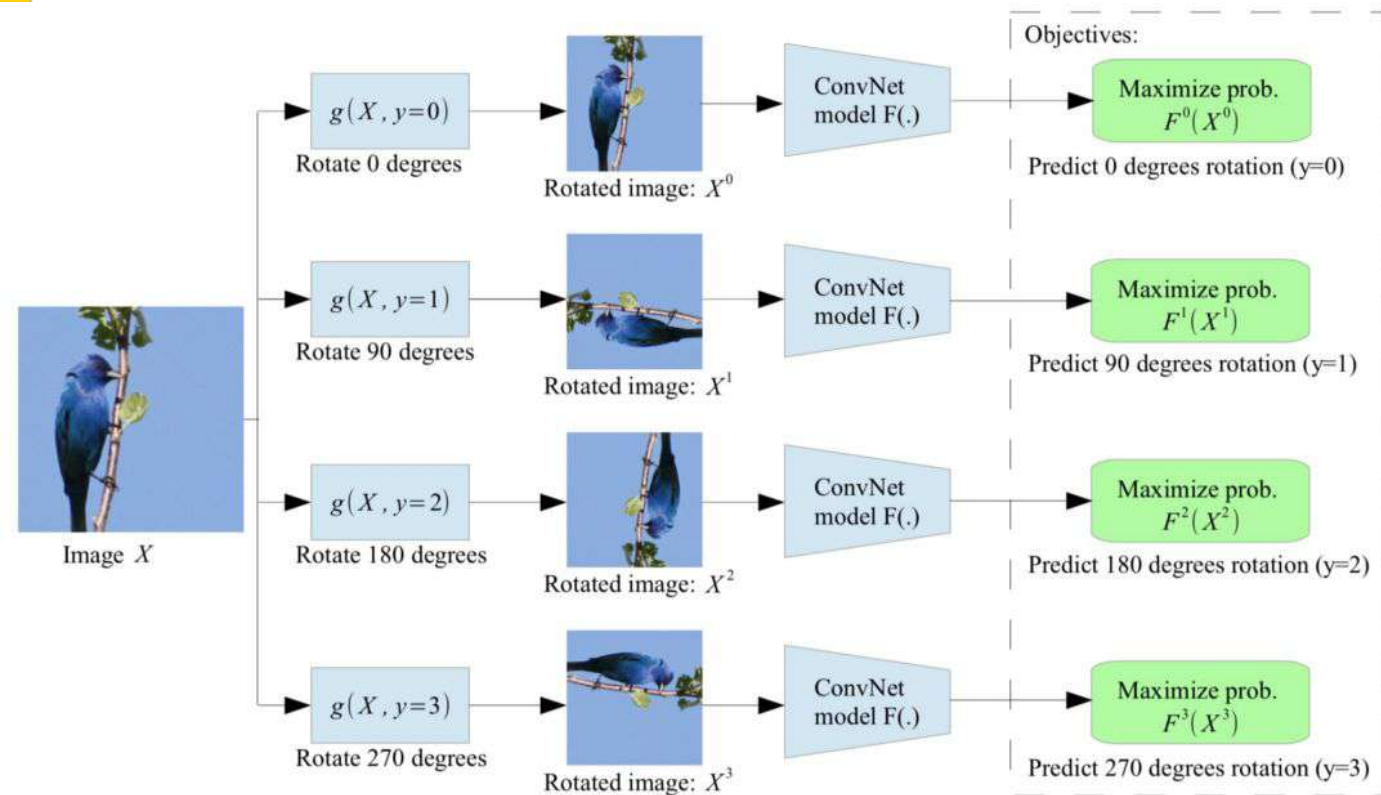
Pretext task: predict rotations



Self-supervised learning by rotating the entire input images.

The model learns to predict which rotation is applied (4-way classification)

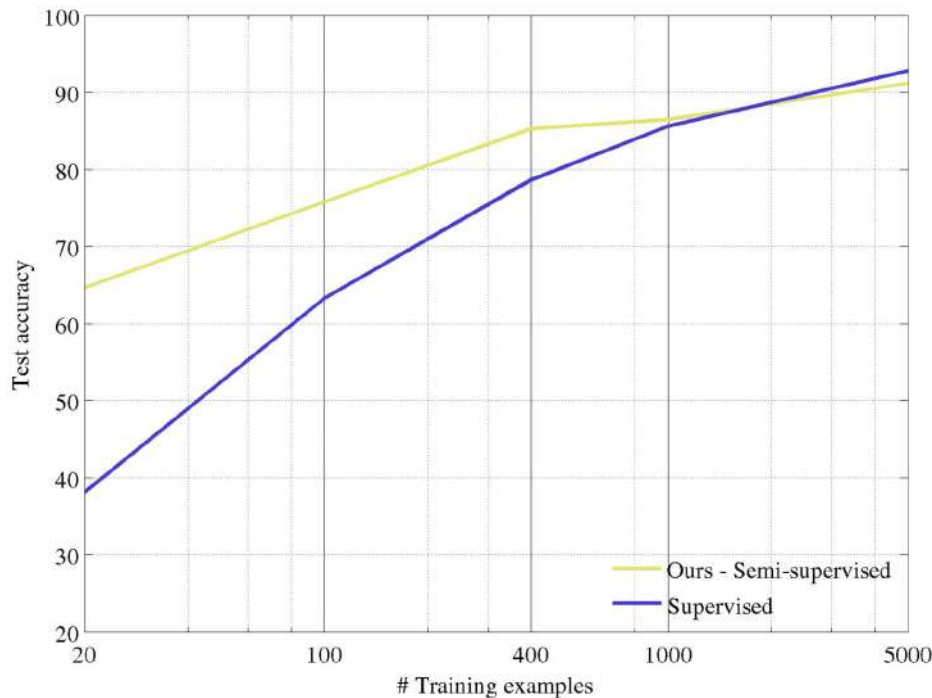
Pretext task: predict rotations



Self-supervised learning by rotating the entire input images.

The model learns to predict which rotation is applied (4-way classification)

Evaluation on semi-supervised learning



Self-supervised learning on CIFAR10 (entire training set).

Freeze conv1 + conv2

Learn conv3 + linear layers with subset of labeled CIFAR10 data (classification).

Transfer learned features to supervised learning

	Classification (%mAP)		Detection (%mAP)	Segmentation (%mIoU)
Trained layers	fc6-8	all	all	all
ImageNet labels	78.9	79.9	56.8	48.0
Random		53.3	43.4	19.8
Random rescaled Krähenbühl et al. (2015)	39.2	56.6	45.6	32.6
Egomotion (Agrawal et al., 2015)	31.0	54.2	43.9	
Context Encoders (Pathak et al., 2016b)	34.6	56.5	44.5	29.7
Tracking (Wang & Gupta, 2015)	55.6	63.1	47.4	
Context (Doersch et al., 2015)	55.1	65.3	51.1	
Colorization (Zhang et al., 2016a)	61.5	65.6	46.9	35.6
BIGAN (Donahue et al., 2016)	52.3	60.1	46.9	34.9
Jigsaw Puzzles (Noroozi & Favaro, 2016)	-	67.6	53.2	37.6
NAT (Bojanowski & Joulin, 2017)	56.7	65.3	49.4	
Split-Brain (Zhang et al., 2016b)	63.0	67.1	46.7	36.0
ColorProxy (Larsson et al., 2017)		65.9		38.4
Counting (Noroozi et al., 2017)	-	67.7	51.4	36.6
(Ours) RotNet	70.87	72.97	54.4	39.1

Pretrained with full ImageNet supervision

No pretraining

Self-supervised learning on ImageNet (entire training set) with AlexNet.

Finetune on labeled data from Pascal VOC 2007.

Self-supervised learning with rotation prediction

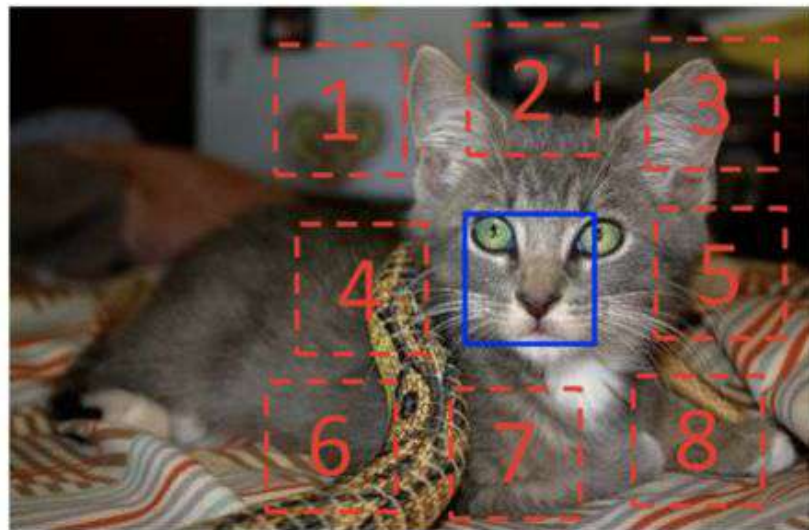
Visualize learned visual attentions



Conv1 27×27 Conv3 13×13 Conv5 6×6

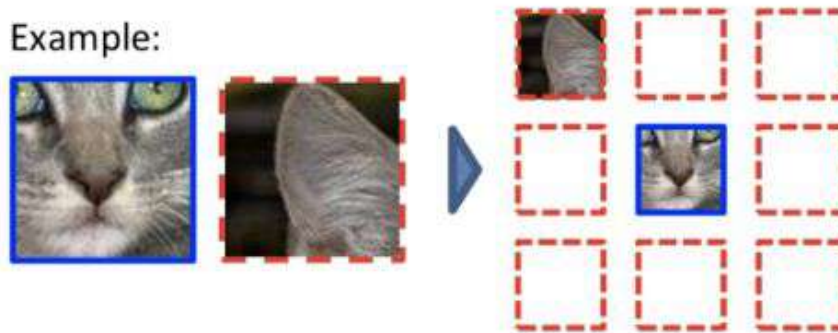
Conv1 27×27 Conv3 13×13 Conv5 6×6

Pretext task: predict relative patch locations



$$X = \left(\begin{array}{c} \text{Patch 4} \\ \text{Patch 7} \end{array} \right); Y = 3$$

Example:



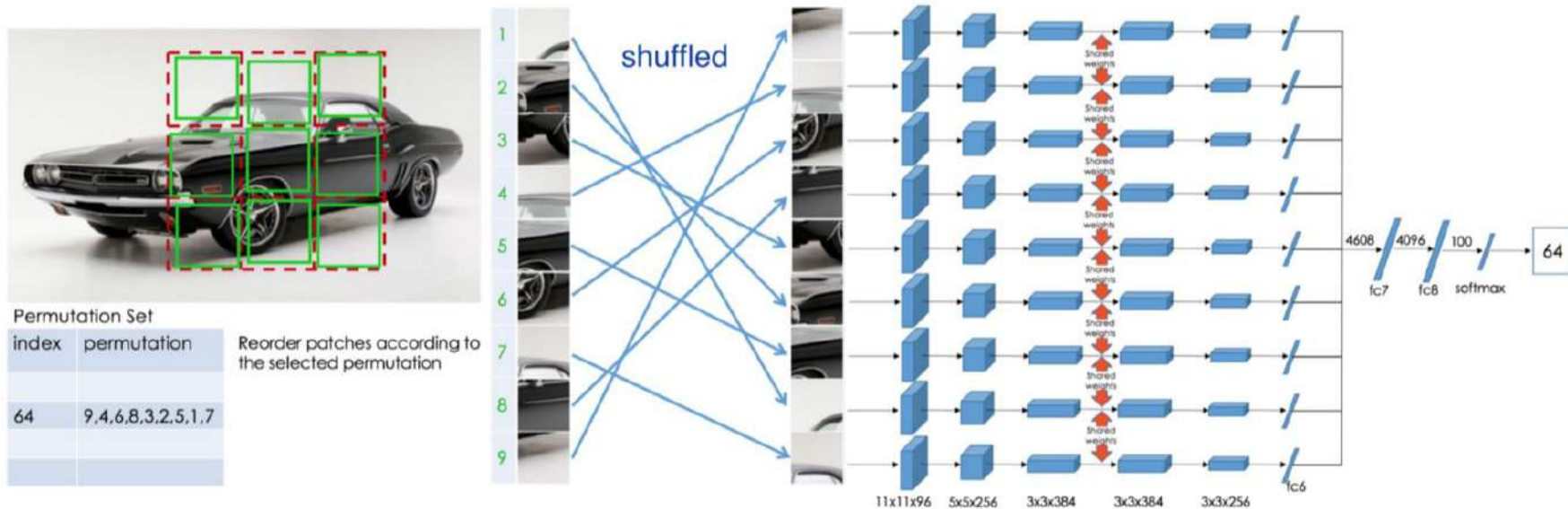
Question 1:



Question 2:



Pretext task: solving “jigsaw puzzles”



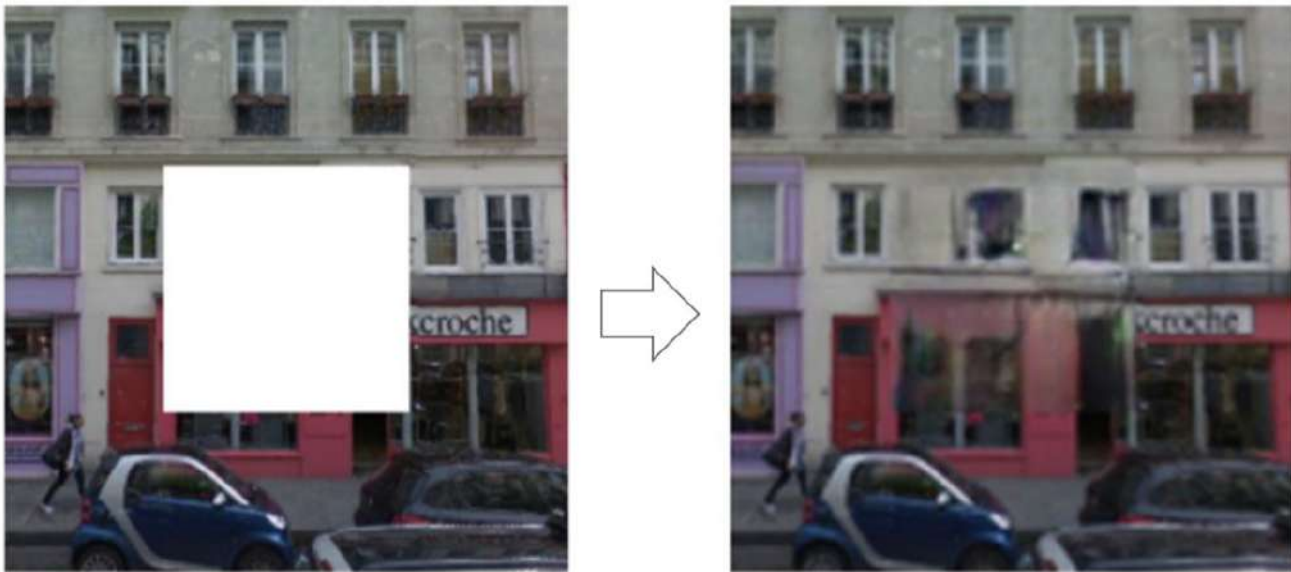
Transfer learned features to supervised learning

Table 1: Results on PASCAL VOC 2007 Detection and Classification. The results of the other methods are taken from Pathak *et al.* [30].

Method	Pretraining time	Supervision	Classification	Detection	Segmentation
Krizhevsky <i>et al.</i> [25]	3 days	1000 class labels	78.2%	56.8%	48.0%
Wang and Gupta[39]	1 week	motion	58.4%	44.0%	-
Doersch <i>et al.</i> [10]	4 weeks	context	55.3%	46.6%	-
Pathak <i>et al.</i> [30]	14 hours	context	56.5%	44.5%	29.7%
Ours	2.5 days	context	67.6%	53.2%	37.6%

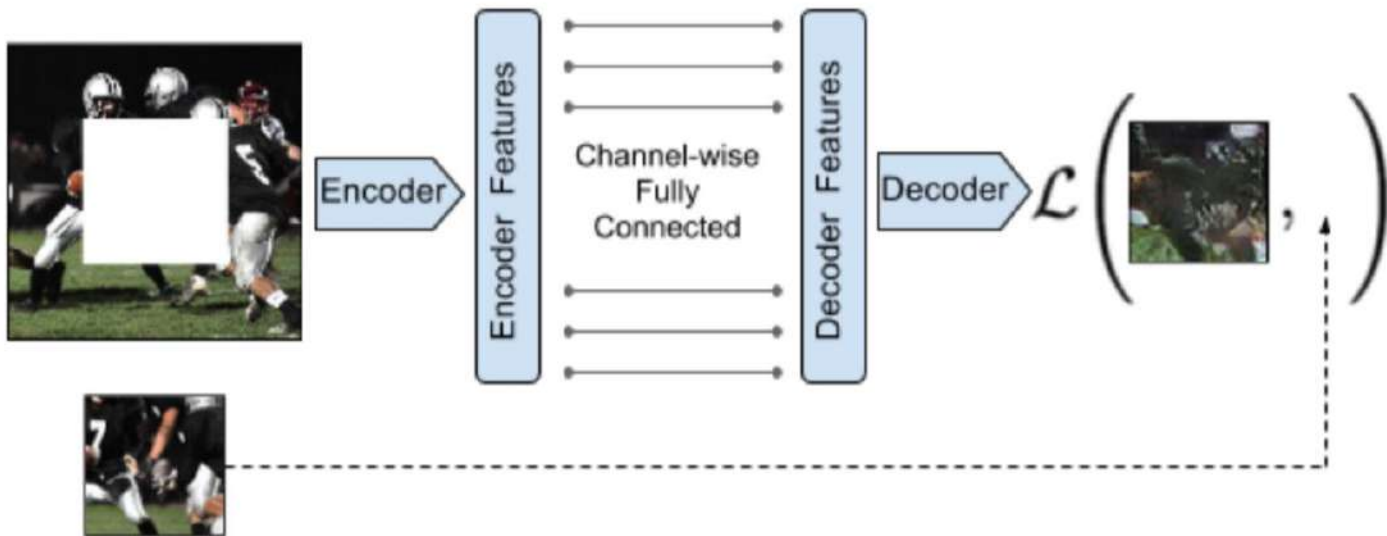
“Ours” is feature learned from solving image Jigsaw puzzles (Noroozi & Favaro, 2016). Doersch *et al.* is the method with relative patch location

Pretext task: predict missing pixels (inpainting)



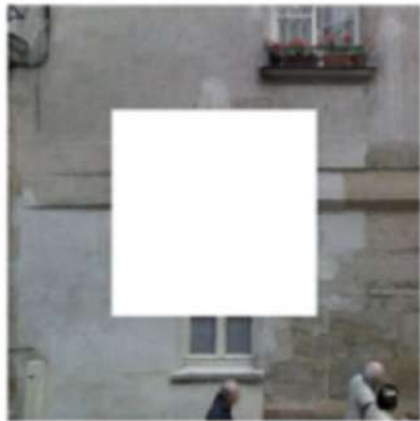
Context Encoders: Feature Learning by Inpainting (Pathak et al., 2016)

Learning to inpaint by reconstruction



Learning to reconstruct the missing pixels

Inpainting evaluation



Input (context)



reconstruction



Learning to inpaint by reconstruction

Loss = reconstruction + adversarial learning

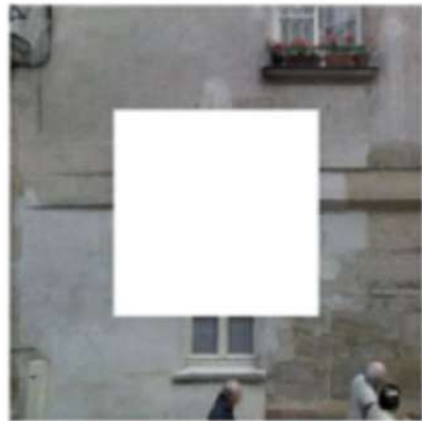
$$L(x) = L_{recon}(x) + L_{adv}(x)$$

$$L_{recon}(x) = ||M * (x - F_{\theta}((1 - M) * x))||_2^2$$

$$L_{adv} = \max_D \mathbb{E}[\log(D(x))] + \log(1 - D(F((1 - M) * x)))]$$

Adversarial loss between “real” images and inpainted images

Inpainting evaluation



Input (context)



reconstruction



adversarial



recon + adv

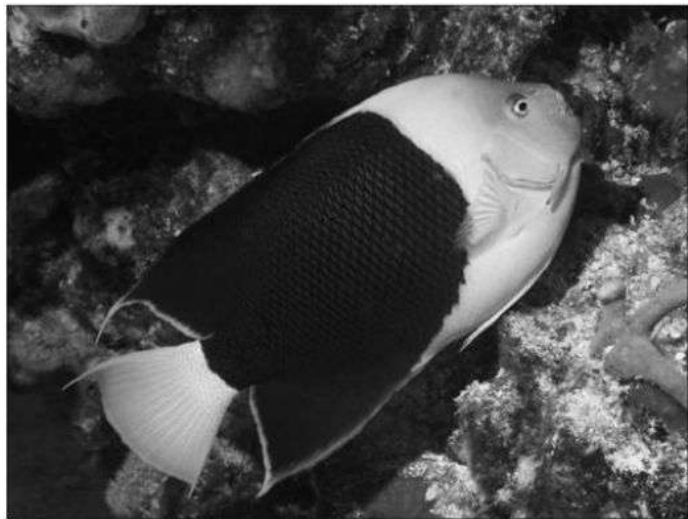


Transfer learned features to supervised learning

Pretraining Method	Supervision	Pretraining time	Classification	Detection	Segmentation
ImageNet [26]	1000 class labels	3 days	78.2%	56.8%	48.0%
Random Gaussian	initialization	< 1 minute	53.3%	43.4%	19.8%
Autoencoder	-	14 hours	53.8%	41.9%	25.2%
Agrawal <i>et al.</i> [1]	egomotion	10 hours	52.9%	41.8%	-
Wang <i>et al.</i> [39]	motion	1 week	58.7%	47.4%	-
Doersch <i>et al.</i> [7]	relative context	4 weeks	55.3%	46.6%	-
Ours	context	14 hours	56.5%	44.5%	30.0%

Self-supervised learning on ImageNet training set, transfer to classification (Pascal VOC 2007), detection (Pascal VOC 2007), and semantic segmentation (Pascal VOC 2012)

Pretext task: image coloring



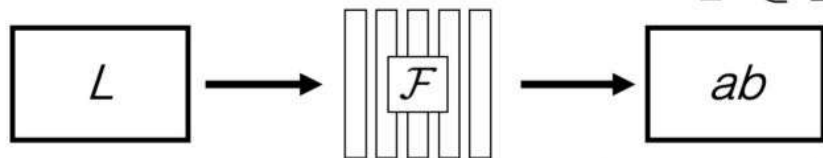
Grayscale image: L channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$

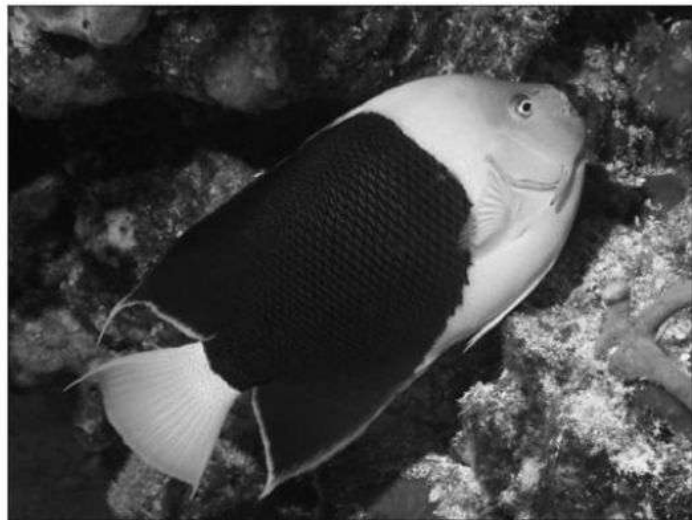


Color information: ab channels

$$\hat{\mathbf{Y}} \in \mathbb{R}^{H \times W \times 2}$$

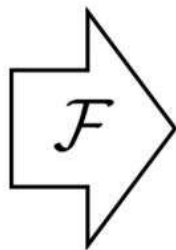


Pretext task: image coloring



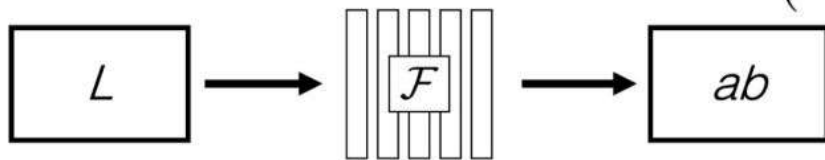
Grayscale image: L channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$



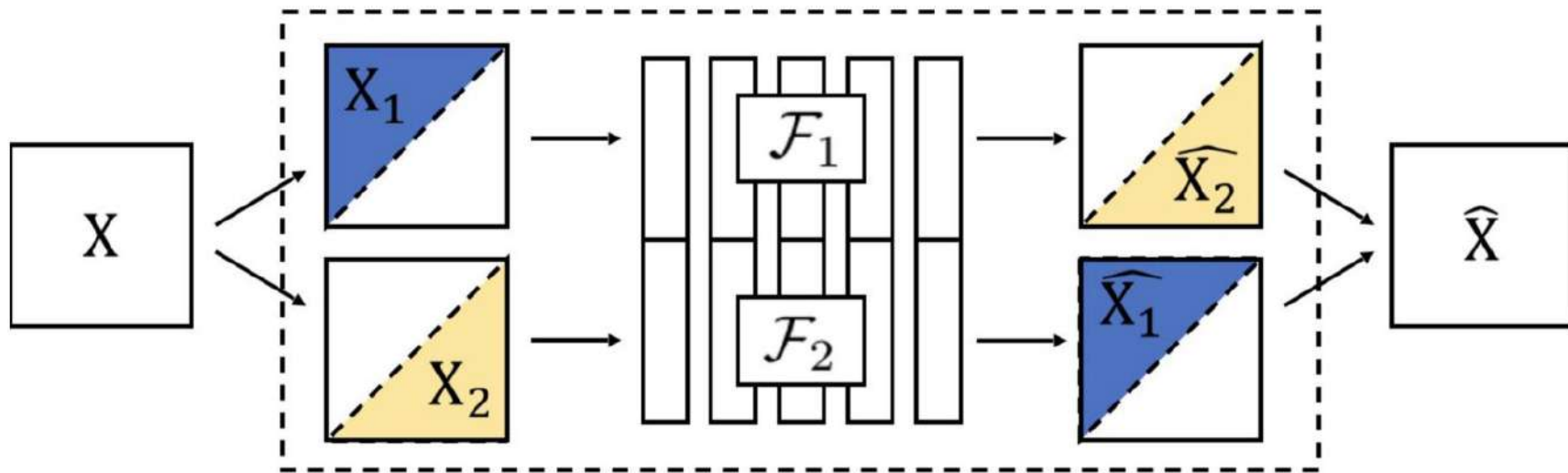
Concatenate (L, ab) channels

$$(\mathbf{X}, \hat{\mathbf{Y}})$$



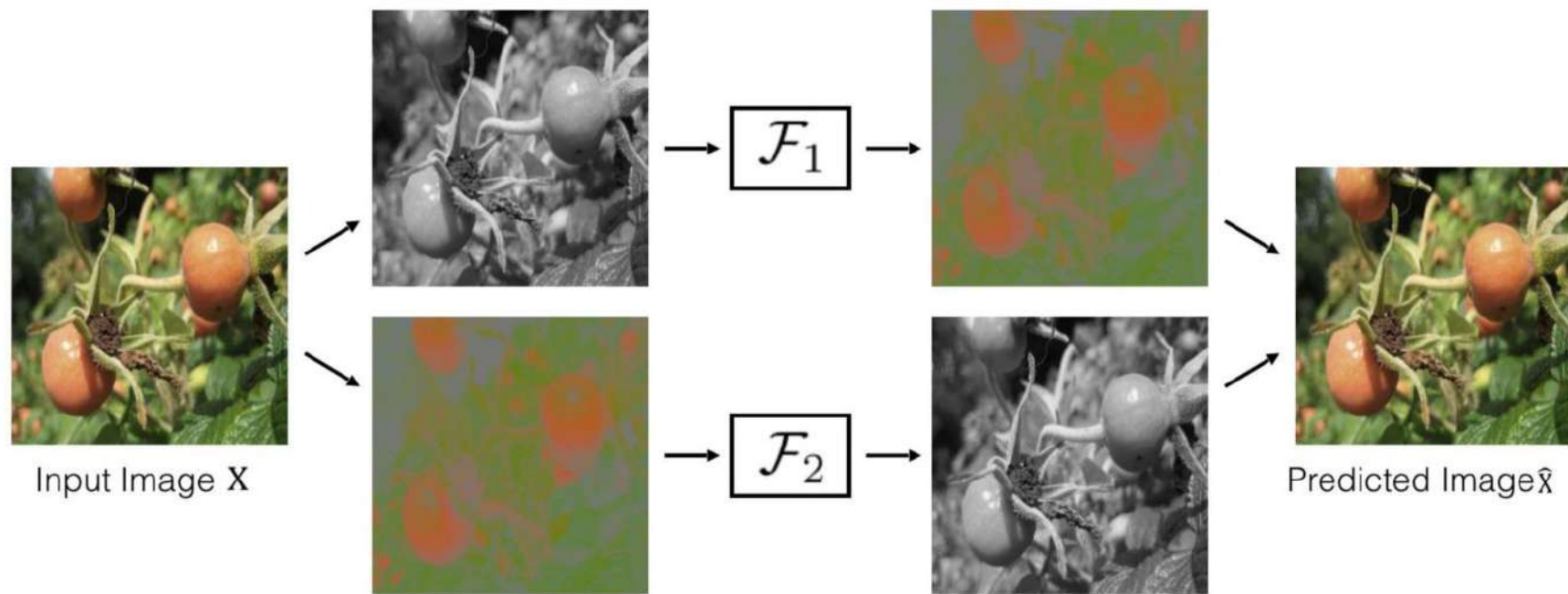
Learning features from colorization: Split-brain Autoencoder

Idea: cross-channel predictions

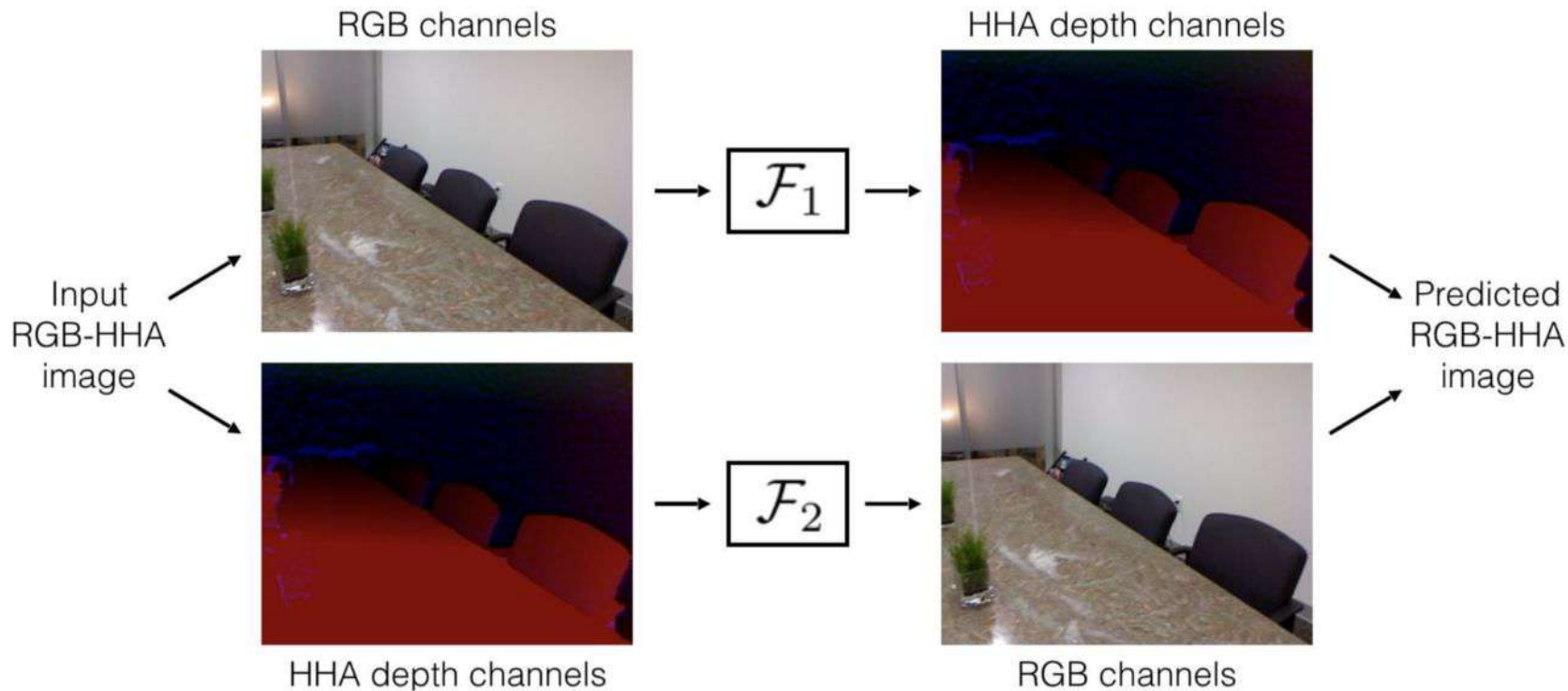


Split-Brain Autoencoder

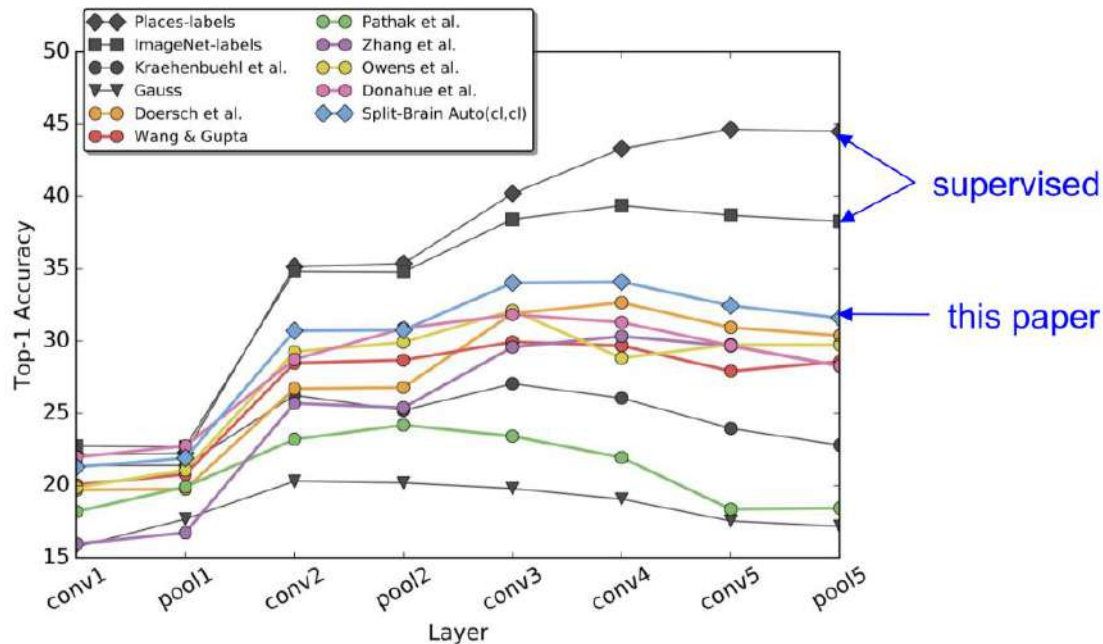
Learning features from colorization: Split-brain Autoencoder



Learning features from colorization: Split-brain Autoencoder



Transfer learned features to supervised learning



Self-supervised learning on ImageNet (entire training set).

Use concatenated features from F_1 and F_2

Labeled data is from the Places (Zhou 2016).

Pretext task: image coloring



Pretext task: image coloring



Pretext task: video coloring

Idea: model the temporal coherence of colors in videos

reference frame



$t = 0$

how should I color these frames?



$t = 1$



$t = 2$

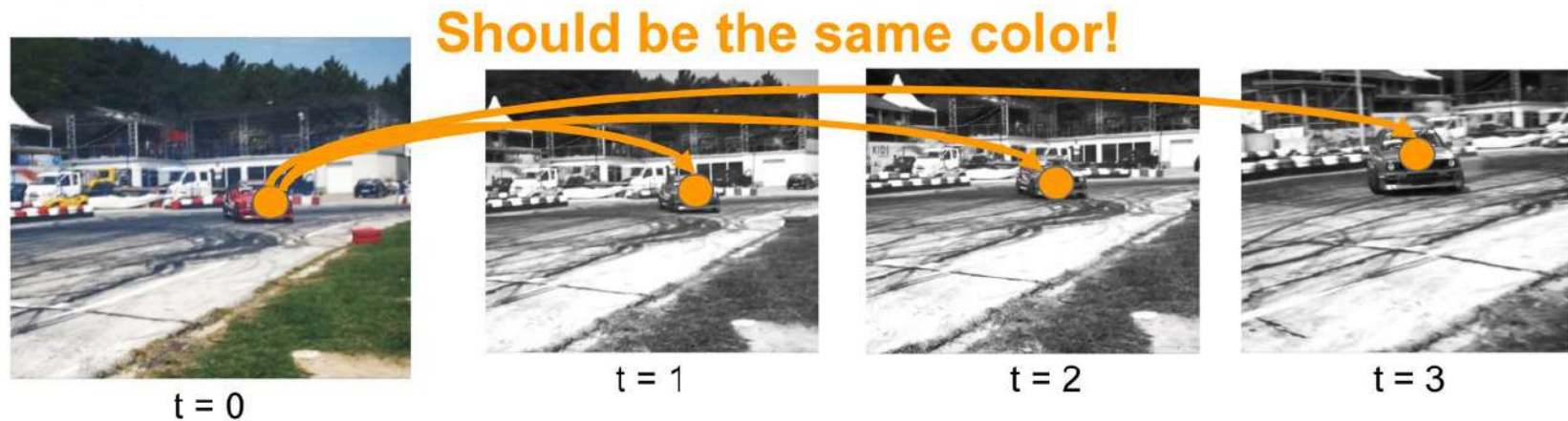


$t = 3$

...

Pretext task: video coloring

Idea: model the temporal coherence of colors in videos



Hypothesis: learning to color video frames should allow model to learn to track regions or objects without labels!

Learning to color videos



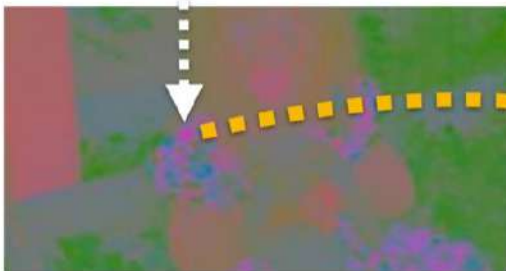
Reference Frame



Input Frame



Pointer



Reference Colors

Copy



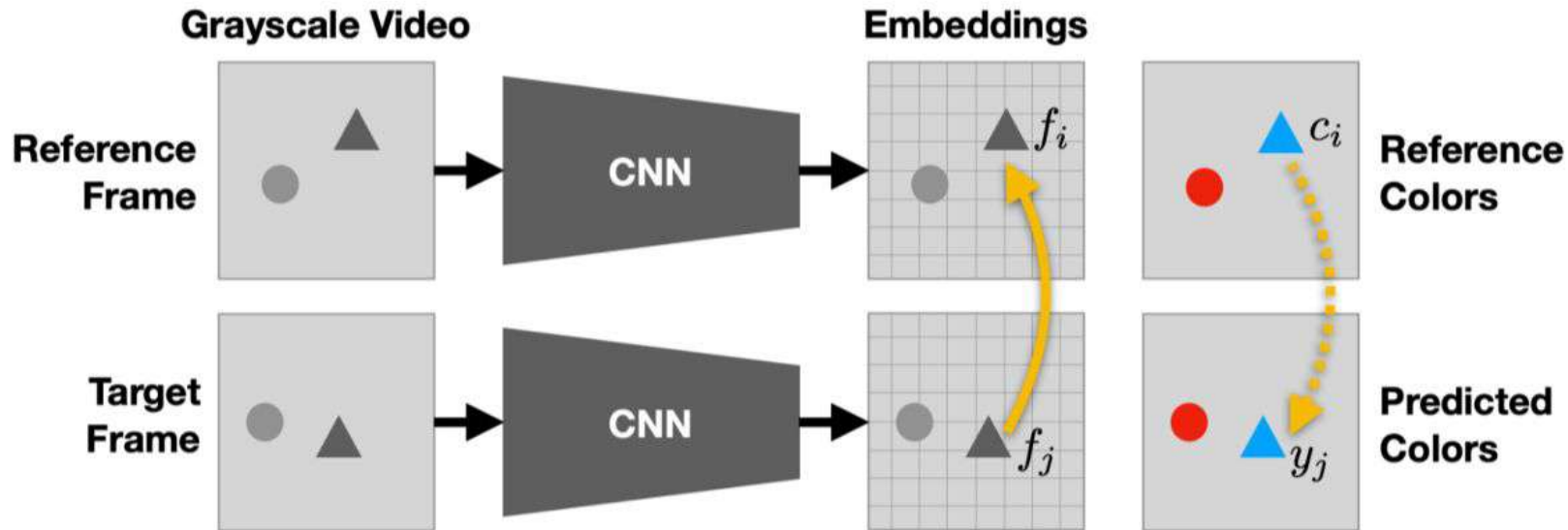
Target Colors

Learning objective:

Establish mappings between reference and target frames in a learned feature space.

Use the mapping as “pointers” to copy the correct color (LAB).

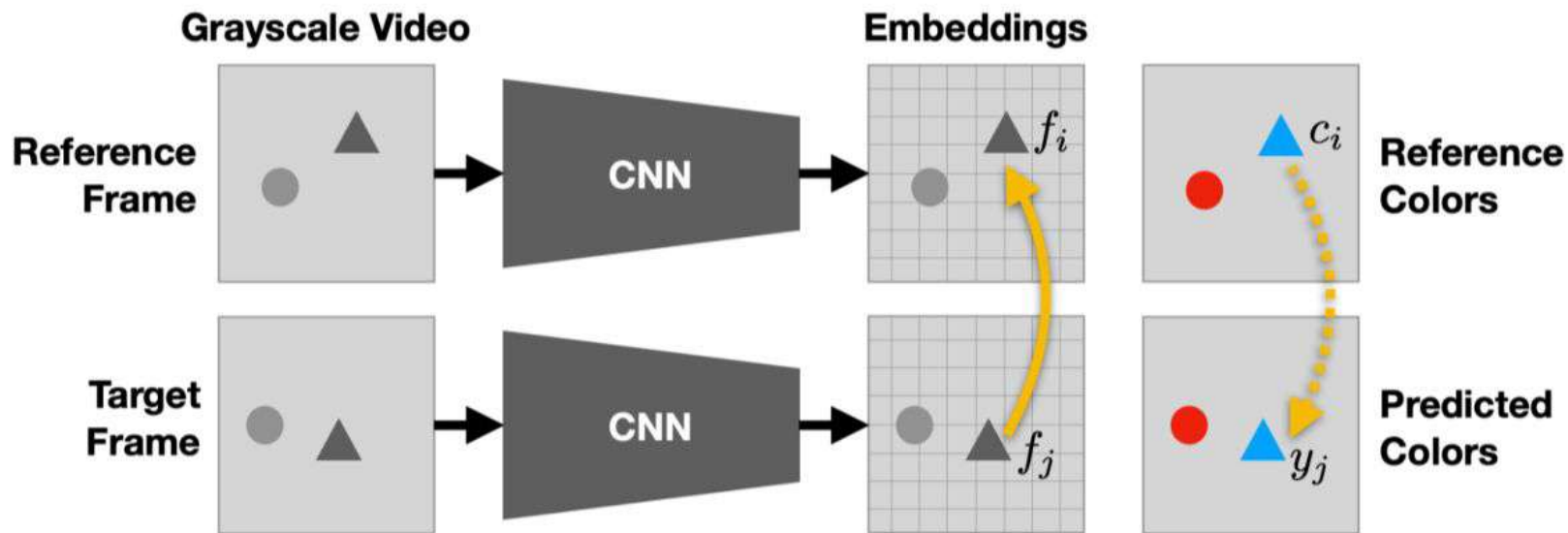
Learning to color videos



attention map on
the reference frame

$$A_{ij} = \frac{\exp(f_i^T f_j)}{\sum_k \exp(f_k^T f_j)}$$

Learning to color videos



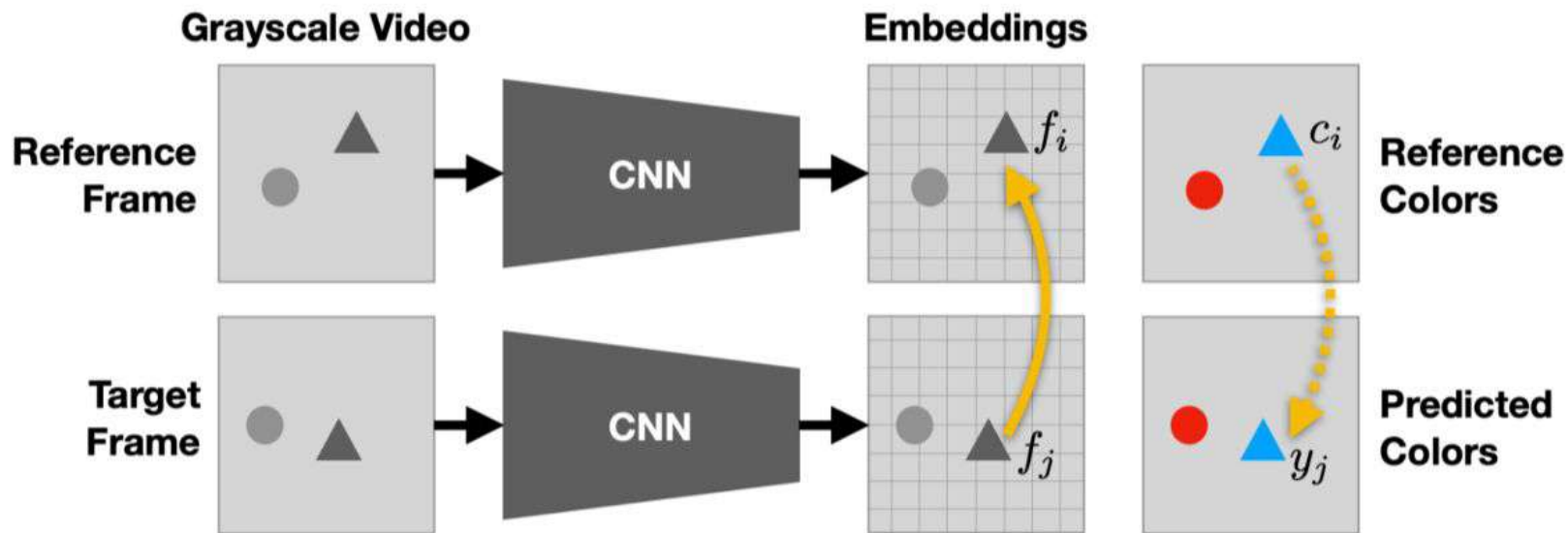
attention map on
the reference frame

predicted color = weighted
sum of the reference color

$$A_{ij} = \frac{\exp(f_i^T f_j)}{\sum_k \exp(f_k^T f_j)}$$

$$y_j = \sum_i A_{ij} c_i$$

Learning to color videos



attention map on
the reference frame

predicted color = weighted
sum of the reference color

loss between predicted color
and ground truth color

$$A_{ij} = \frac{\exp(f_i^T f_j)}{\sum_k \exp(f_k^T f_j)}$$

$$y_j = \sum_i A_{ij} c_i$$

$$\min_{\theta} \sum_j \mathcal{L}(y_j, c_j)$$

Colorizing videos (qualitative)



reference frame



target frames (gray)



predicted color



Colorizing videos (qualitative)



reference frame



target frames (gray)

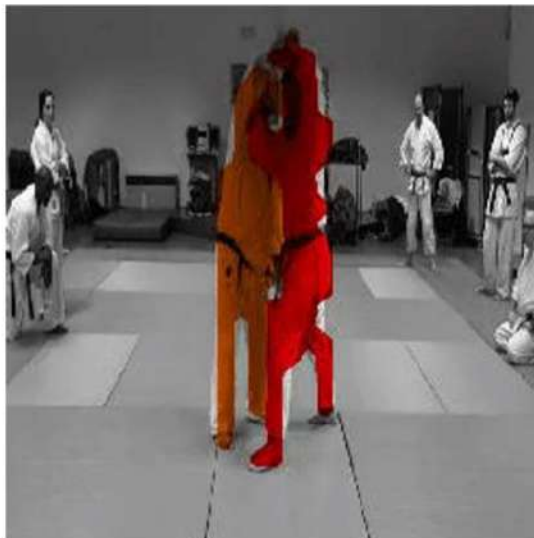


predicted color



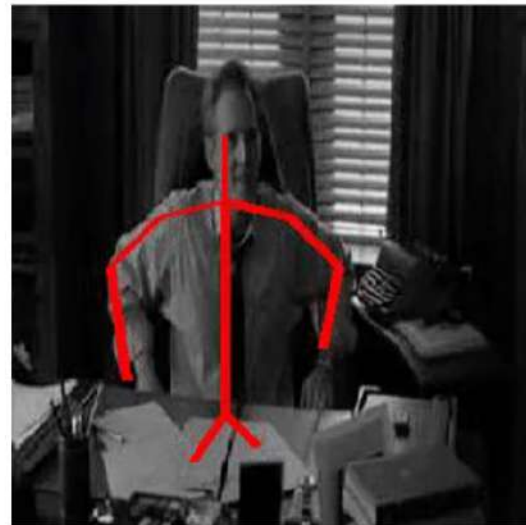
Tracking emerges from colorization

Propagate segmentation masks using learned attention



Tracking emerges from colorization

Propagate pose keypoints using learned attention



Summary: pretext tasks from image transformations

- Pretext tasks focus on “visual common sense”, e.g., predict rotations, inpainting, rearrangement, and colorization.
- The models are forced learn good features about natural images, e.g., semantic representation of an object category, in order to solve the pretext tasks.
- We don't care about the performance of these pretext tasks, but rather how useful the learned features are for downstream tasks (classification, detection, segmentation).

Summary: pretext tasks from image transformations

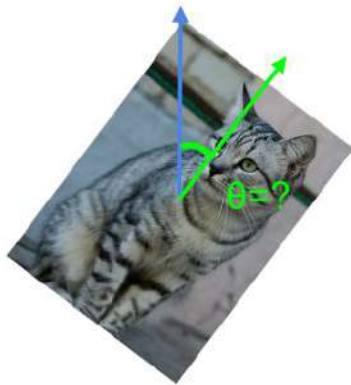
- Pretext tasks focus on “visual common sense”, e.g., predict rotations, inpainting, rearrangement, and colorization.
- The models are forced learn good features about natural images, e.g., semantic representation of an object category, in order to solve the pretext tasks.
- We don't care about the performance of these pretext tasks, but rather how useful the learned features are for downstream tasks (classification, detection, segmentation).
- Problems: 1) coming up with individual pretext tasks is tedious, and 2) the learned representations may not be general.



Pretext tasks from image transformations



image completion



rotation prediction



“jigsaw puzzle”

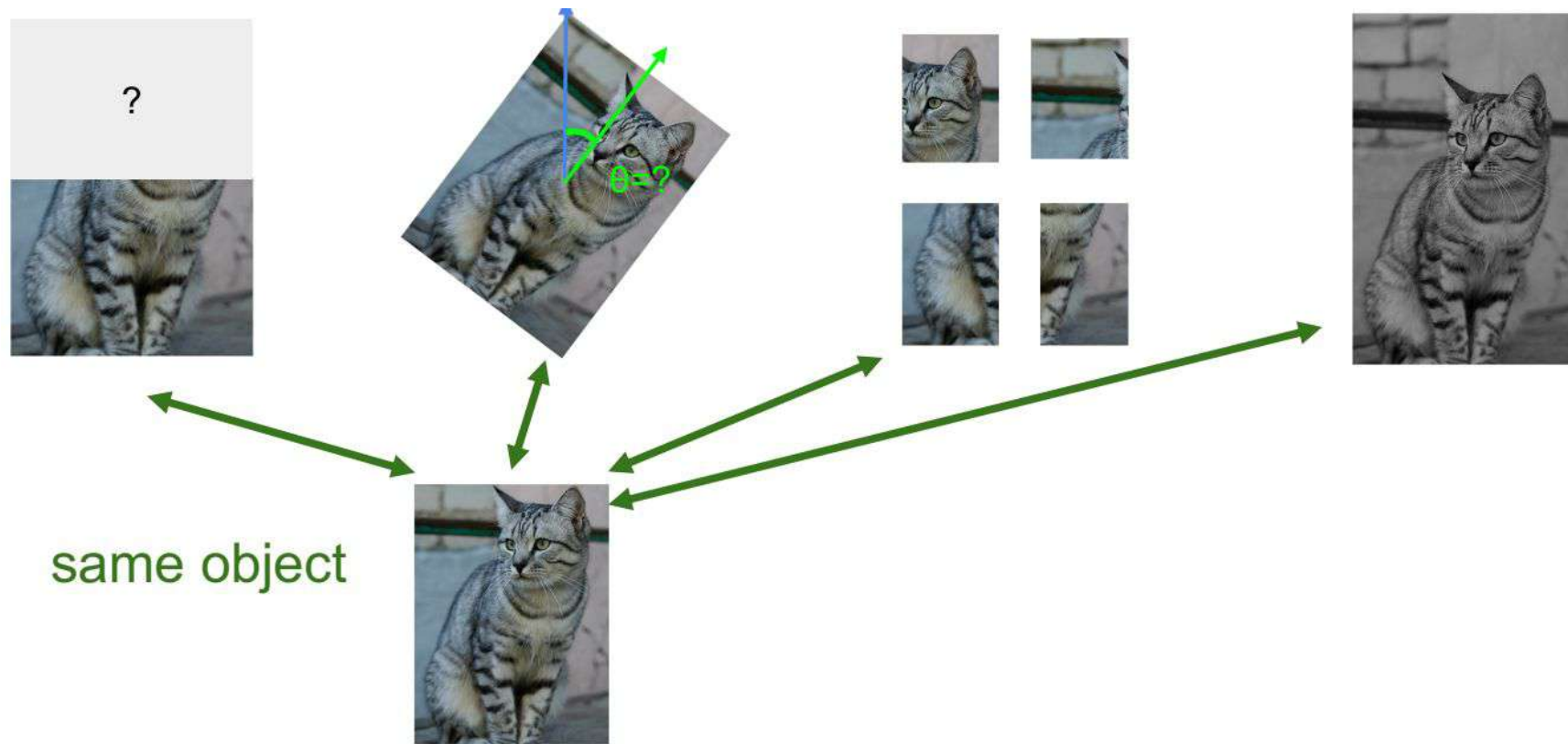


colorization

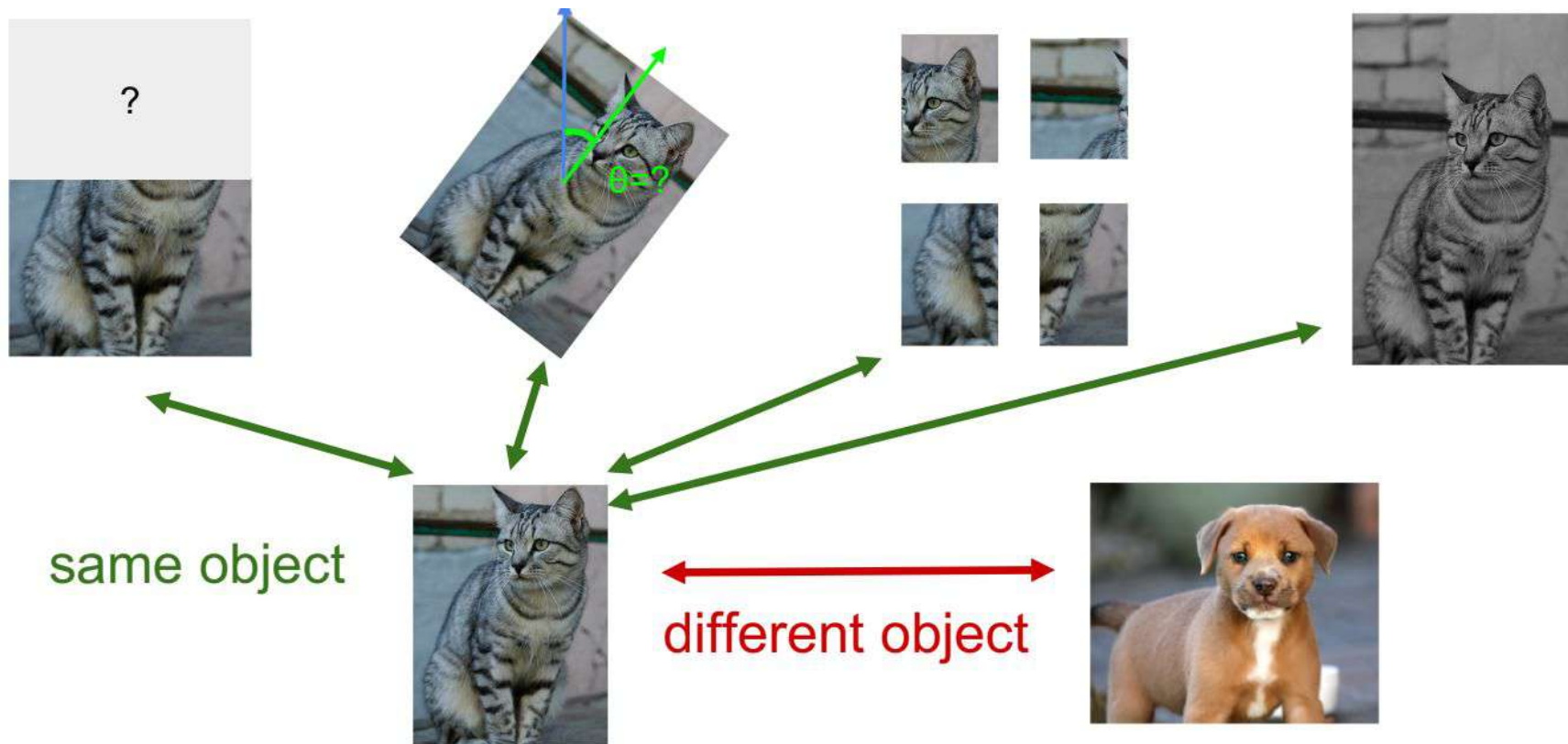
Learned representations may be tied to a specific pretext task!

Can we come up with a more general pretext task?

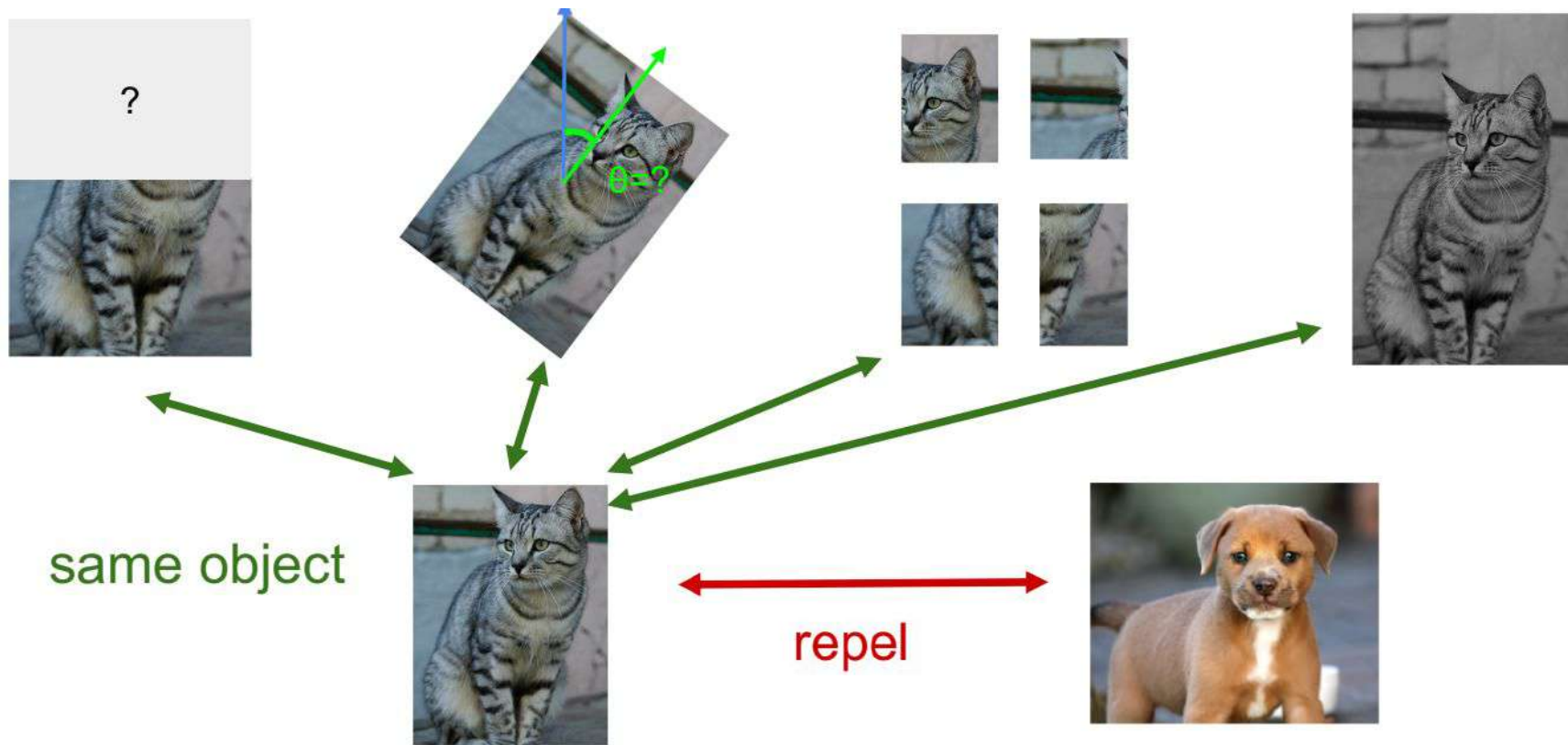
A more general pretext task?



A more general pretext task?



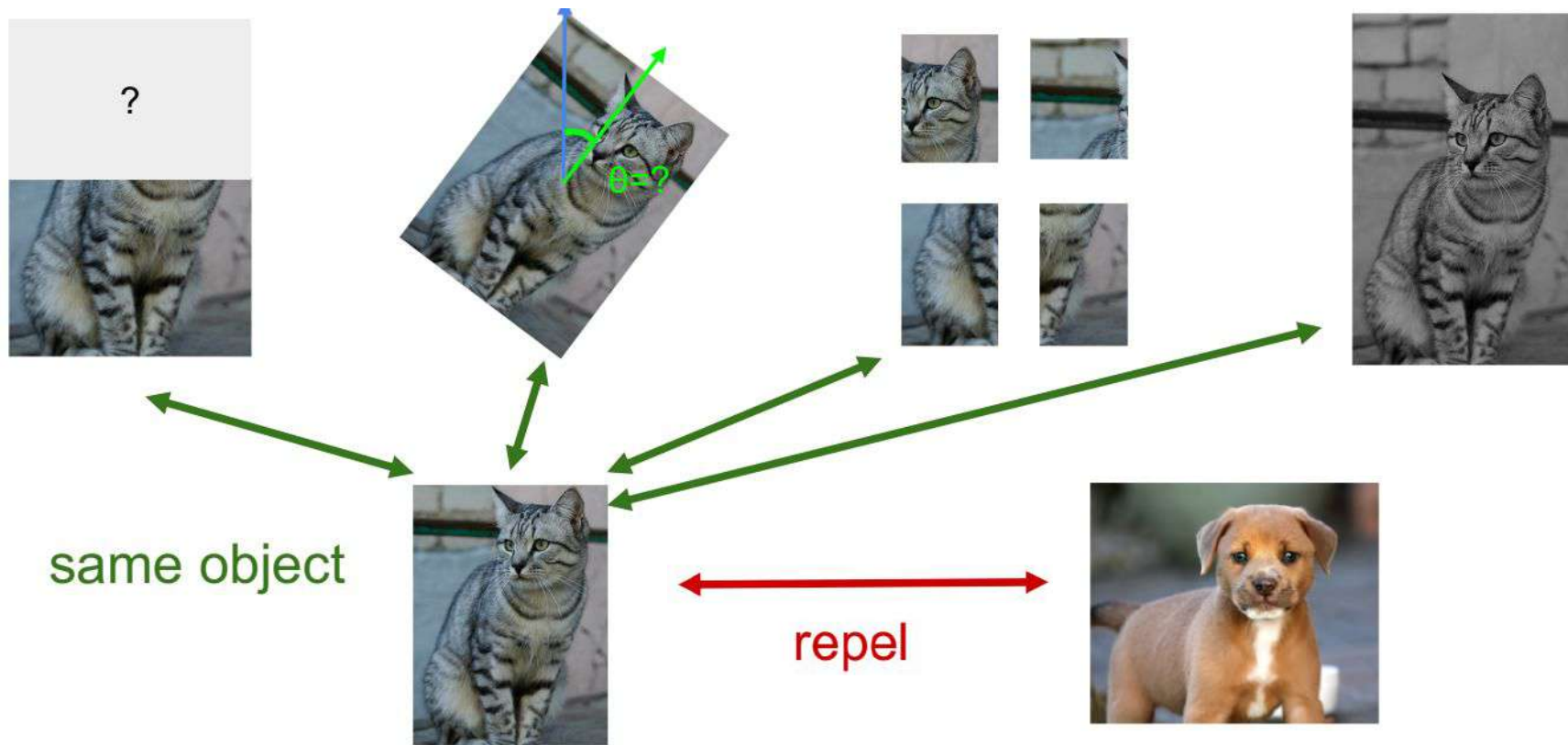
Contrastive Representation Learning



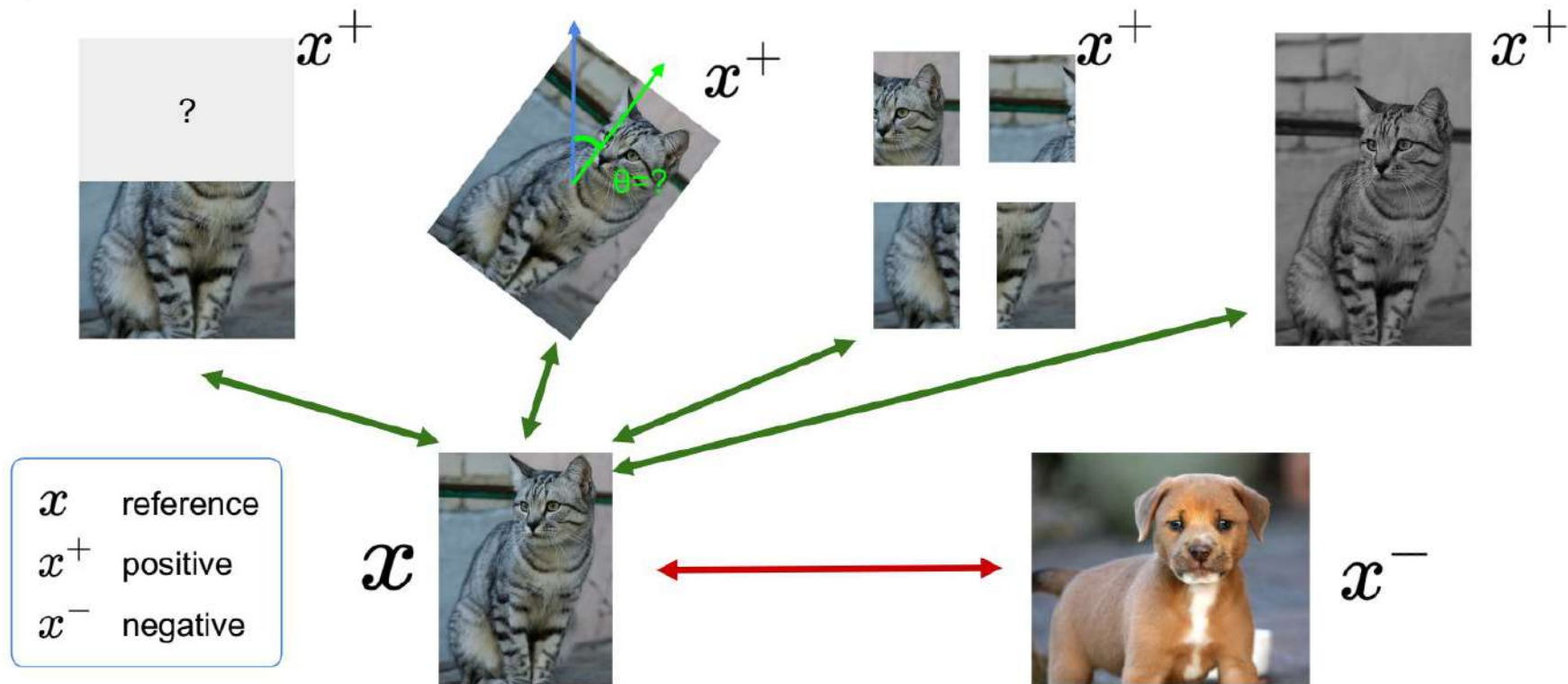
Today's Agenda

- Pretext tasks from image transformations
 - Rotation, inpainting, rearrangement, coloring
- Contrastive representation learning
 - Intuition and formulation
 - Instance contrastive learning: SimCLR and MOCO
 - Sequence contrastive learning: CPC

Contrastive Representation Learning



Contrastive Representation Learning



A formulation of contrastive learning

What we want:

$$\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-))$$

x : reference sample; x^+ positive sample; x^- negative sample

Given a chosen score function, we aim to learn an encoder function f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-) .

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$



x



x^+



x



x_1^-



x_2^-



x_3^-

...

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{score for the positive pair}}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{score for the positive pair}} + \underbrace{\sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for the } N-1 \text{ negative pairs}}} \right]$$

score for the positive pair

score for the $N-1$ negative pairs

This seems familiar ...

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}}{\underbrace{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}} \right]$$

score for the positive pair score for the $N-1$ negative pairs

This seems familiar ...

Cross entropy loss for a N -way softmax classifier!

I.e., learn to find the positive sample from the N samples

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

Commonly known as the InfoNCE loss ([van den Oord et al., 2018](#))

A lower bound on the mutual information between $f(x)$ and $f(x^+)$

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

The larger the negative sample size (N), the tighter the bound

SimCLR: A Simple Framework for Contrastive Learning

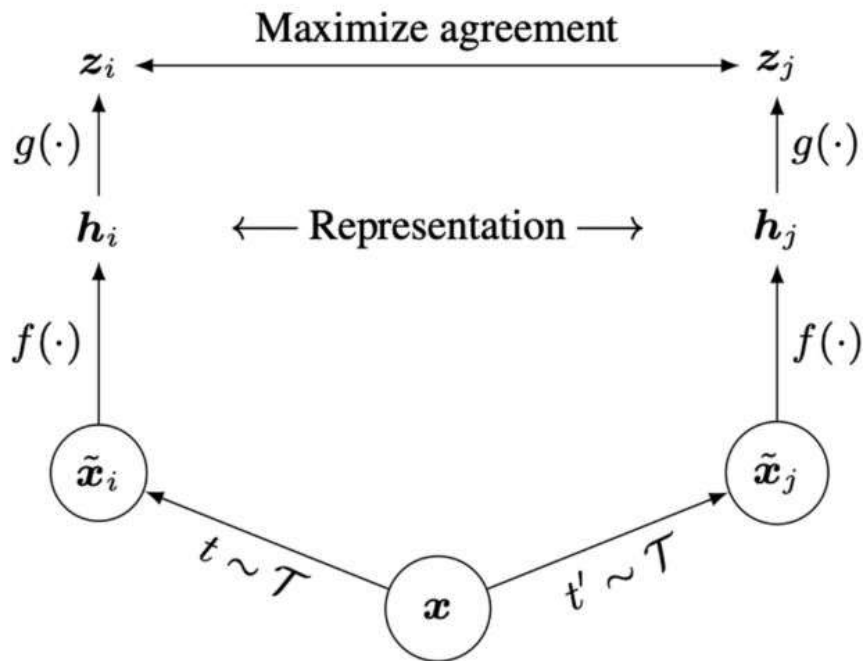
Cosine similarity as the score function:

$$s(u, v) = \frac{u^T v}{||u|| ||v||}$$

Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied

Generate positive samples through data augmentation:

- random cropping, random color distortion, and random blur.



SimCLR: generating positive samples from data augmentation



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



(j) Sobel filtering



SimCLR

Generate a positive pair
by sampling data
augmentation functions

Algorithm 1 SimCLR's main learning algorithm.

```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\mathbf{x}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

*We use a slightly different
formulation in the assignment.
You should follow the
assignment instructions.



Generate a positive pair
by sampling data
augmentation functions

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of $f, g, \mathcal{T}$.

for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**

for all $k \in \{1, \dots, N\}$ **do**

draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$

the first augmentation

$\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$

$\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$

representation

$\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$

projection

the second augmentation

$\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$

$\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$

representation

$\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$

projection

end for

for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**

$s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity

end for

define $\ell(i, j)$ as $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$

$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$

update networks f and g to minimize $\mathcal{L}$

end for

return encoder network $f(\cdot)$, and throw away $g(\cdot)$

InfoNCE loss:
Use all
non-positive
samples in the
batch as x -

*We use a slightly different
formulation in the assignment.
You should follow the
assignment instructions.

SimCLR

Algorithm 1 SimCLR's main learning algorithm.

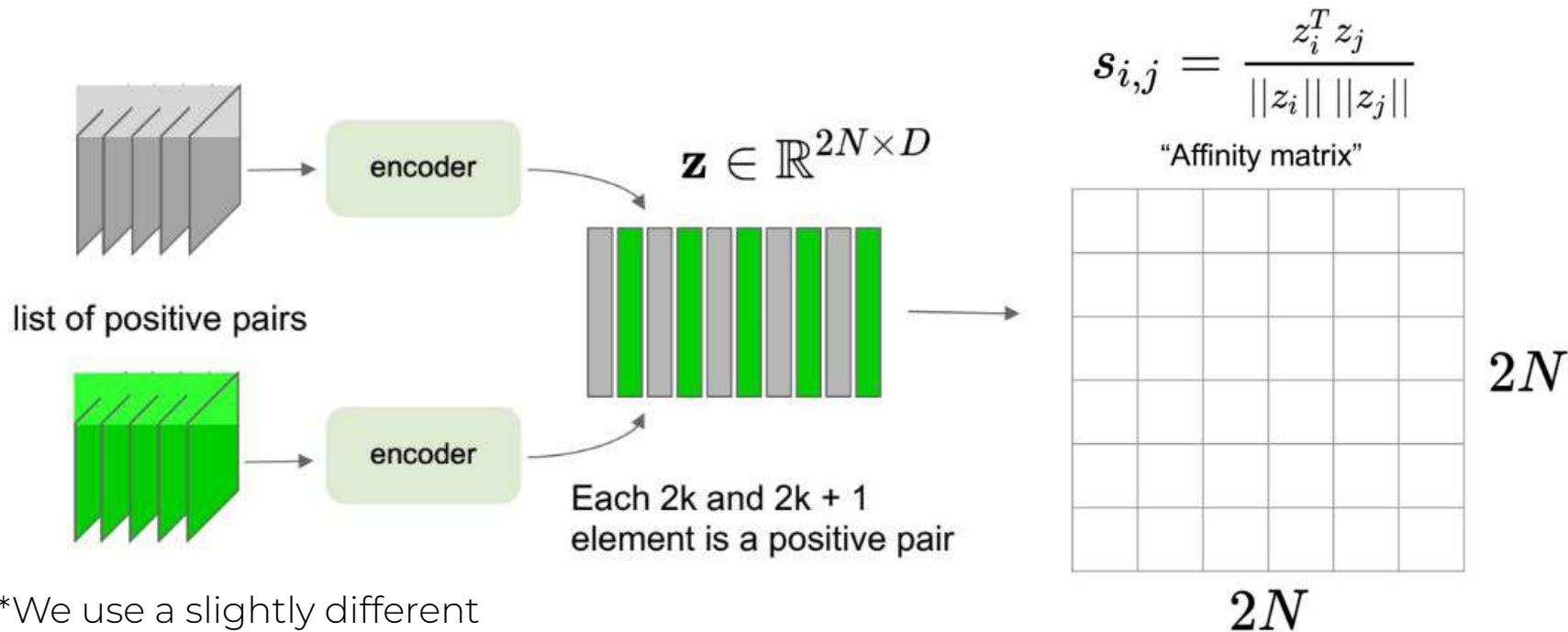
```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\mathbf{x}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

Generate a positive pair by sampling data augmentation functions

Iterate through and use each of the $2N$ sample as reference, compute average loss

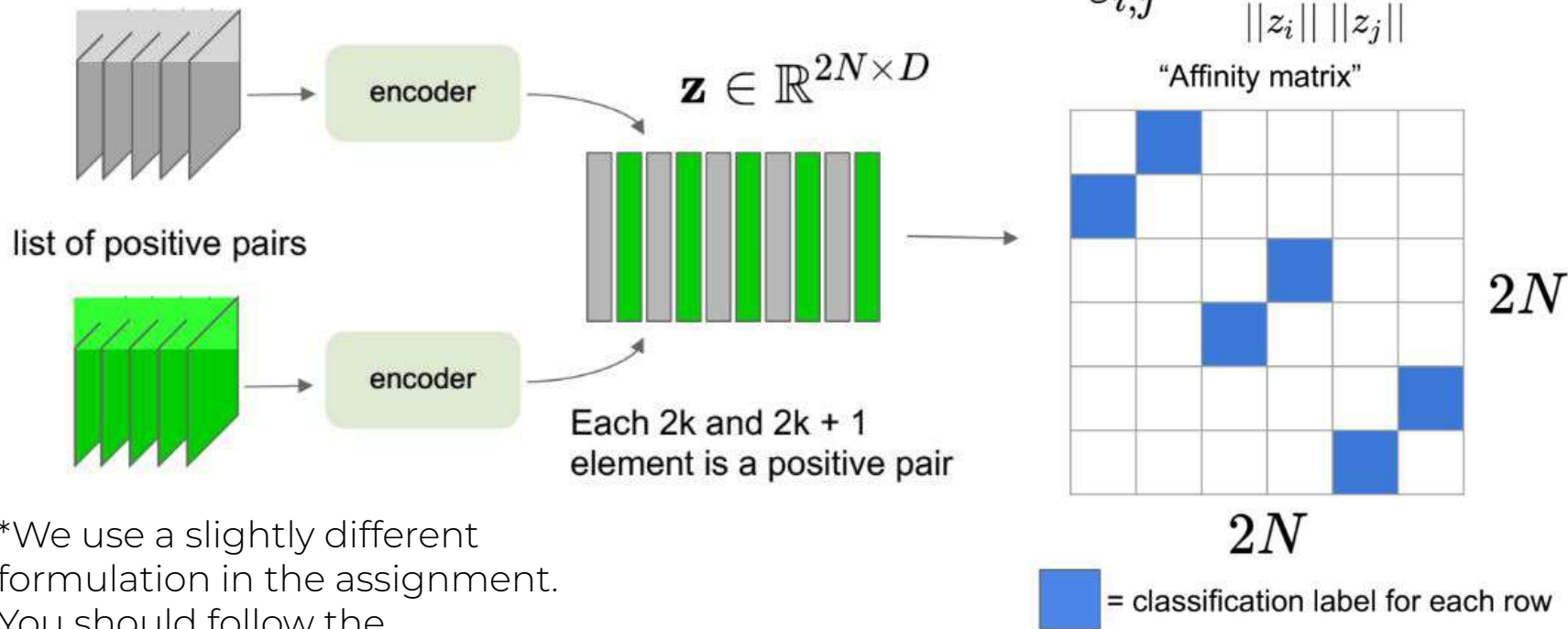
InfoNCE loss: Use all non-positive samples in the batch as $x -$

SimCLR: mini-batch training



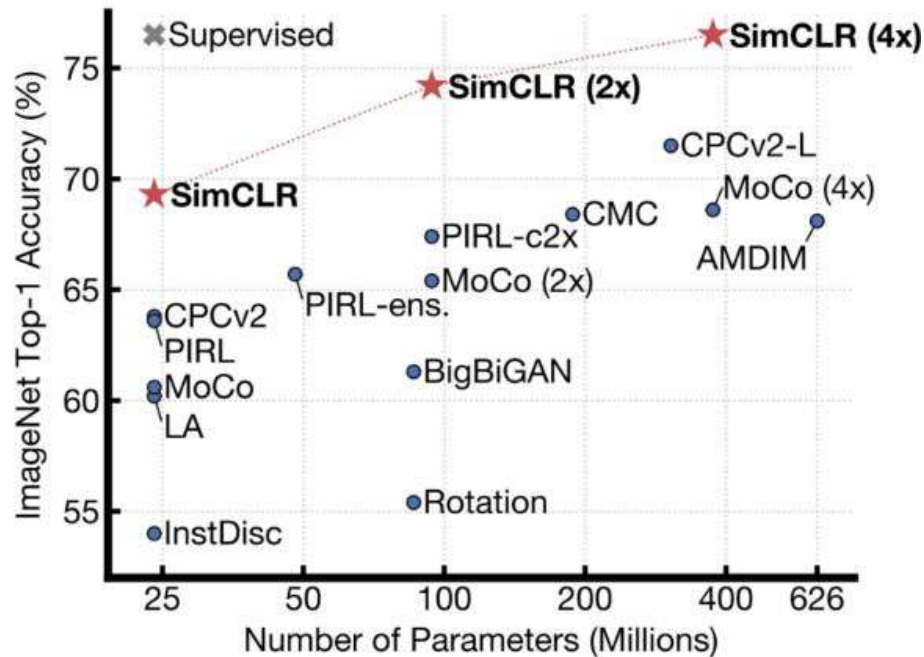
*We use a slightly different formulation in the assignment. You should follow the assignment instructions.

SimCLR: mini-batch training



*We use a slightly different formulation in the assignment. You should follow the assignment instructions.

Training linear classifier on SimCLR features



Train feature encoder on ImageNet (entire training set) using SimCLR.

Freeze feature encoder, train a linear classifier on top with labeled data.

Semi-supervised learning on SimCLR features

Method	Architecture	Label fraction	
		1%	10%
		Top 5	
Supervised baseline	ResNet-50	48.4	80.4
<i>Methods using other label-propagation:</i>			
Pseudo-label	ResNet-50	51.6	82.4
VAT+Entropy Min.	ResNet-50	47.0	83.4
UDA (w. RandAug)	ResNet-50	-	88.5
FixMatch (w. RandAug)	ResNet-50	-	89.1
S4L (Rot+VAT+En. M.)	ResNet-50 (4×)	-	91.2
<i>Methods using representation learning only:</i>			
InstDisc	ResNet-50	39.2	77.4
BigBiGAN	RevNet-50 (4×)	55.2	78.8
PIRL	ResNet-50	57.2	83.8
CPC v2	ResNet-161(*)	77.9	91.2
SimCLR (ours)	ResNet-50	75.5	87.8
SimCLR (ours)	ResNet-50 (2×)	83.0	91.2
SimCLR (ours)	ResNet-50 (4×)	85.8	92.6

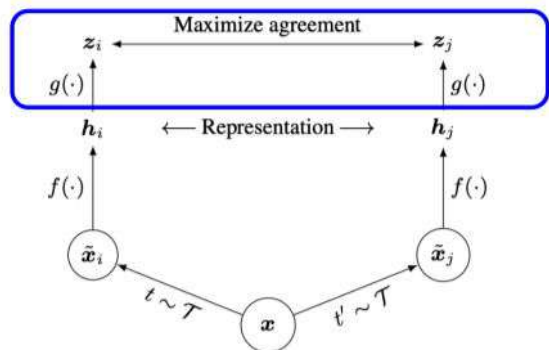
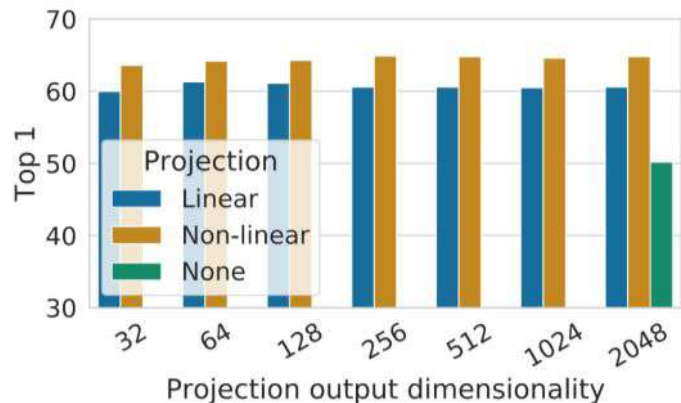
Table 7. ImageNet accuracy of models trained with few labels.

Train feature encoder on ImageNet (entire training set) using SimCLR.

Finetune the encoder with 1% / 10% of labeled data on ImageNet.



SimCLR design choices: projection head



Linear / non-linear projection heads improve representation learning.

A possible explanation:

- contrastive learning objective may discard useful information for downstream tasks
- representation space z is trained to be invariant to data transformation.
- by leveraging the projection head $g(\cdot)$, more information can be preserved in the h representation space

SimCLR design choices: large batch size

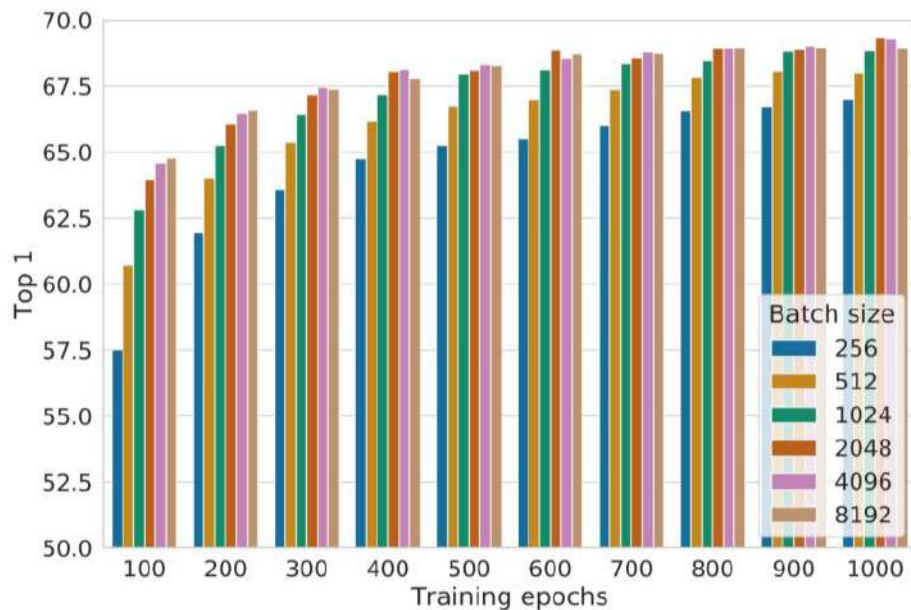
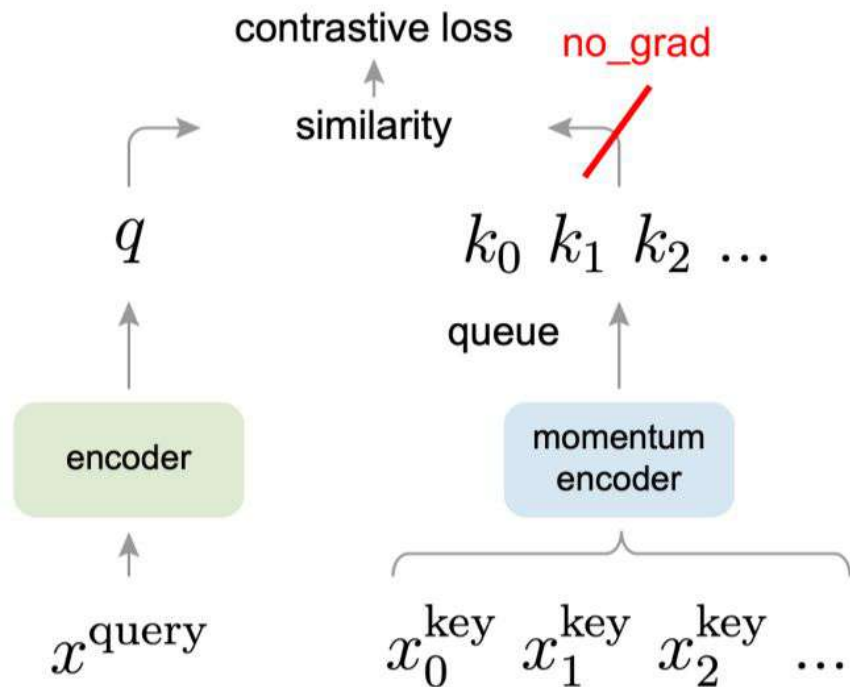


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰

Large training batch size is crucial for SimCLR!

Large batch size causes large memory footprint during backpropagation: requires distributed training on TPUs (ImageNet experiments)

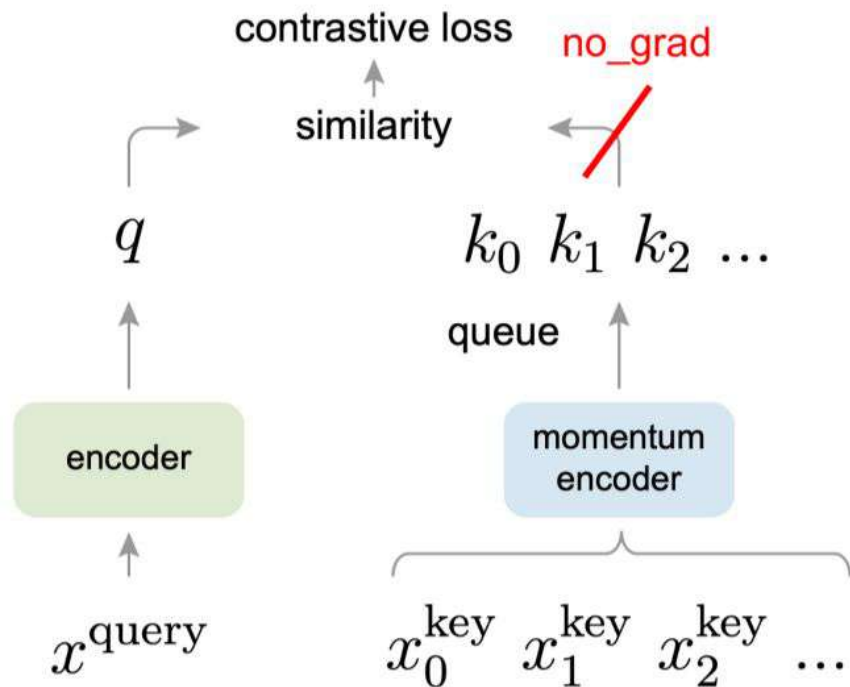
Momentum Contrastive Learning (MoCo)



Key differences to SimCLR:

- Keep a running queue of keys (negative samples).
- Compute gradients and update the encoder only through the queries.
- Decouple min-batch size with the number of keys: can support a large number of negative samples.

Momentum Contrastive Learning (MoCo)



Key differences to SimCLR:

- Keep a running queue of keys (negative samples).
- Compute gradients and update the encoder only through the queries.
- Decouple min-batch size with the number of keys: can support a large number of negative samples.
- The key encoder is slowly progressing through the momentum update rules:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$



Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
    x_q = aug(x) # a randomly augmented version
    x_k = aug(x) # another randomly augmented version

    q = f_q.forward(x_q) # queries: NxK
    k = f_k.forward(x_k) # keys: NxK
    k = k.detach() # no gradient to keys

    # positive logits: Nx1
    l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

    # negative logits: NxK
    l_neg = mm(q.view(N,C), queue.view(C,K))

    # logits: Nx(1+K)
    logits = cat([l_pos, l_neg], dim=1)

    # contrastive loss, Eqn. (1)
    labels = zeros(N) # positives are the 0-th
    loss = CrossEntropyLoss(logits/t, labels)

    # SGD update: query network
    loss.backward()
    update(f_q.params)

    # momentum update: key network
    f_k.params = m*f_k.params+(1-m)*f_q.params

    # update dictionary
    enqueue(queue, k) # enqueue the current minibatch
    dequeue(queue) # dequeue the earliest minibatch
```

Generate a positive pair
by sampling data
augmentation functions

No gradient through
the key

Use the running
queue of keys as the
negative samples

InfoNCE loss

Update f_k through
momentum

Update the FIFO
negative sample queue

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

“MoCo V2”

Improved Baselines with Momentum Contrastive Learning

Xinlei Chen Haoqi Fan Ross Girshick Kaiming He
Facebook AI Research (FAIR)

A hybrid of ideas from SimCLR and MoCo:

- From SimCLR: non-linear projection head and strong data augmentation.
- From MoCo: momentum-updated queues that allow training on a large number of negative samples (no TPU required!).

MoCo vs. SimCLR vs. MoCo V2



Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.

case	unsup. pre-train				ImageNet acc.	VOC detection		
	MLP	aug+	cos	epochs		AP ₅₀	AP	AP ₇₅
supervised					76.5	81.3	53.5	58.8
MoCo v1				200	60.6	81.5	55.9	62.6
(a)	✓			200	66.2	82.0	56.4	62.6
(b)		✓		200	63.4	82.2	56.8	63.2
(c)	✓	✓		200	67.3	82.5	57.2	63.9
(d)	✓	✓	✓	200	67.5	82.4	57.0	63.6
(e)	✓	✓	✓	800	71.1	82.5	57.4	64.0

Table 1. **Ablation of MoCo baselines**, evaluated by ResNet-50 for (i) ImageNet linear classification, and (ii) fine-tuning VOC object detection (mean of 5 trials). “**MLP**”: with an MLP head; “**aug+**”: with extra blur augmentation; “**cos**”: cosine learning rate schedule.

MoCo vs. SimCLR vs. MoCo V2



Key takeaways:

case	MLP	unsup. pre-train			batch	ImageNet acc.
		aug+	cos	epochs		
MoCo v1 [6]				200	256	60.6
SimCLR [2]	✓	✓	✓	200	256	61.9
SimCLR [2]	✓	✓	✓	200	8192	66.6
MoCo v2	✓	✓	✓	200	256	67.5
<i>results of longer unsupervised training follow:</i>						
SimCLR [2]	✓	✓	✓	1000	4096	69.3
MoCo v2	✓	✓	✓	800	256	71.1

Table 2. **MoCo vs. SimCLR**: ImageNet linear classifier accuracy (**ResNet-50, 1-crop 224×224**), trained on features from unsupervised pre-training. “aug+” in SimCLR includes blur and stronger color distortion. SimCLR ablations are from Fig. 9 in [2] (we thank the authors for providing the numerical results).

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.
- Decoupling mini-batch size with negative sample size allows MoCo-V2 to outperform SimCLR with smaller batch size (256 vs. 8192).

MoCo vs. SimCLR vs. MoCo V2



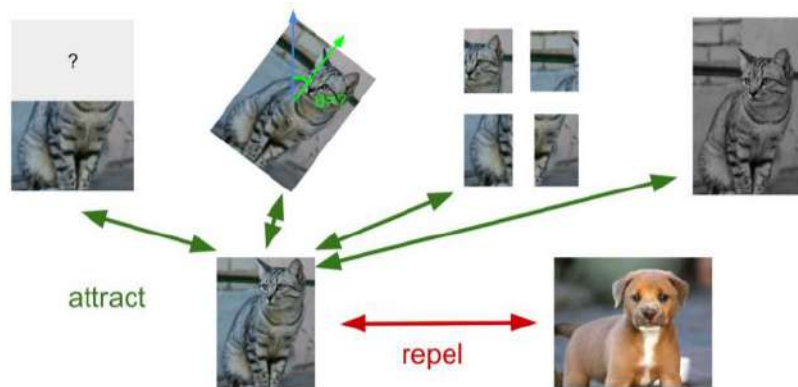
Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.
- Decoupling mini-batch size with negative sample size allows MoCo-V2 to outperform SimCLR with smaller batch size (256 vs. 8192).
- ... all with much smaller memory footprint! (“end-to-end” means SimCLR here)

mechanism	batch	memory / GPU	time / 200-ep.
MoCo	256	5.0G	53 hrs
end-to-end	256	7.4G	65 hrs
end-to-end	4096	93.0G [†]	n/a

Table 3. **Memory and time cost** in 8 V100 16G GPUs, implemented in PyTorch. [†]: based on our estimation.

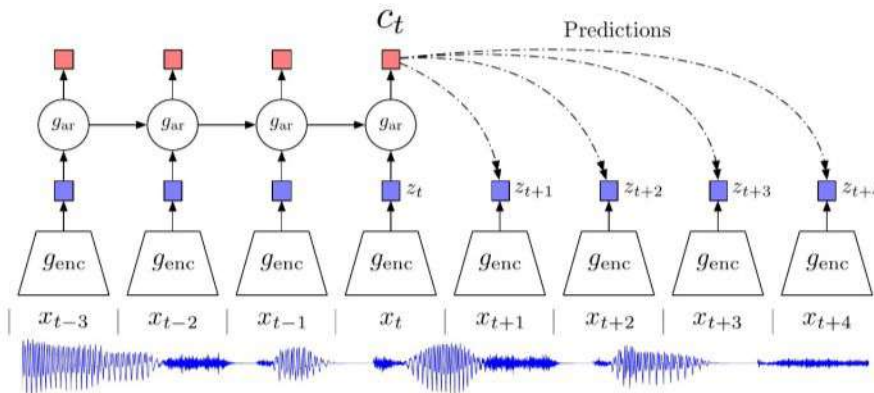
Instance vs. Sequence Contrastive Learning



Instance-level contrastive learning:

contrastive learning based on
positive & negative instances.

Examples: SimCLR, MoCo



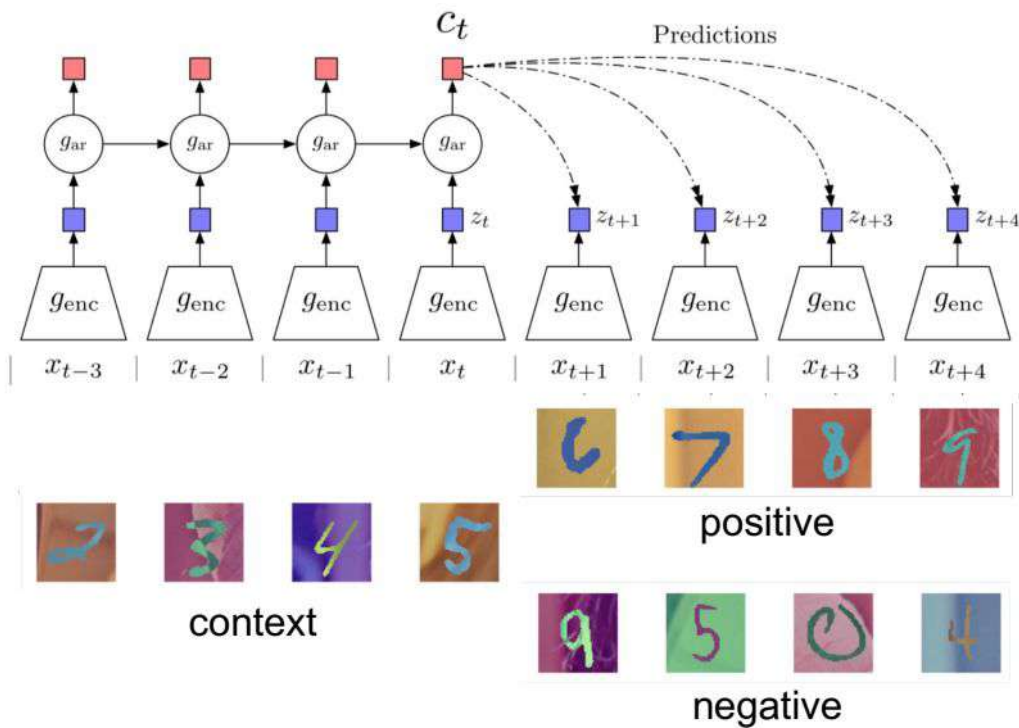
Source: [van den Oord et al., 2018](#)

Sequence-level contrastive learning:

contrastive learning based on
sequential / temporal orders.

Example: **Contrastive Predictive Coding (CPC)**

Contrastive Predictive Coding (CPC)



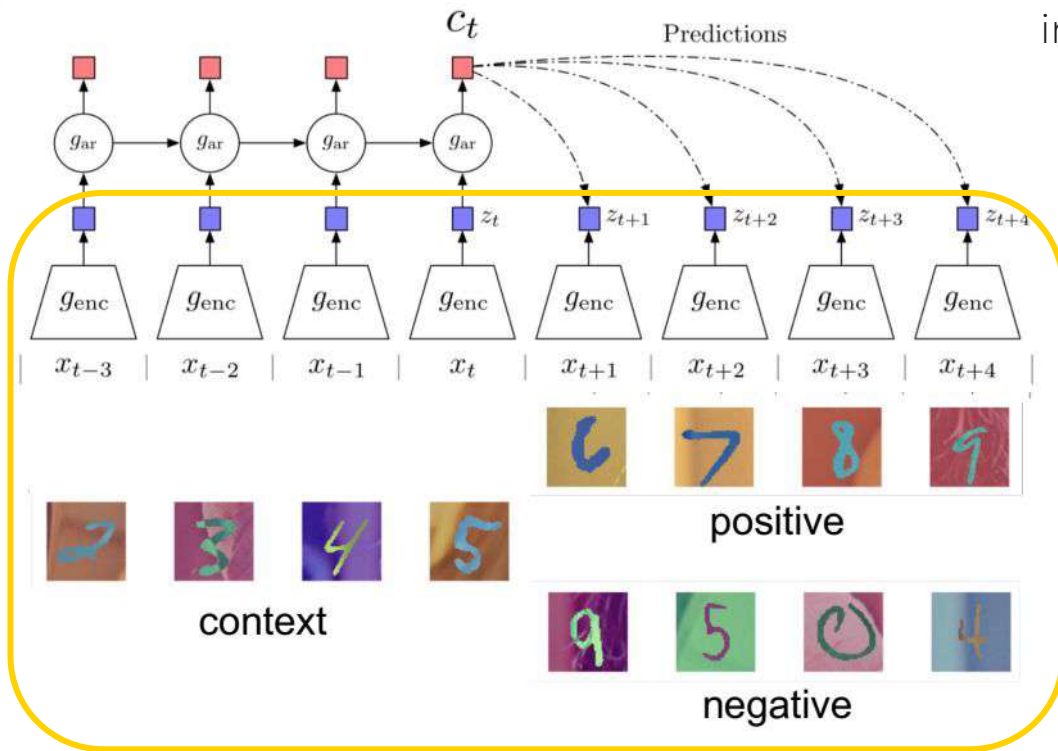
Contrastive: contrast between “right” and “wrong” sequences using contrastive learning.

Predictive: the model has to predict future patterns given the current context.

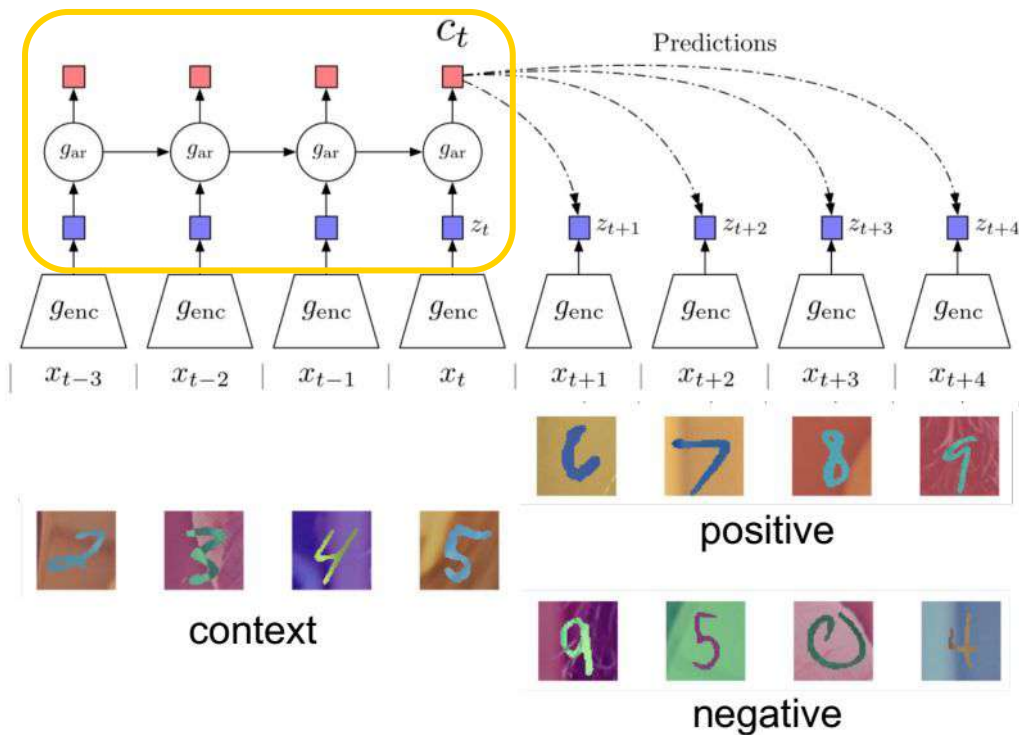
Coding: the model learns useful feature vectors, or “code”, for downstream tasks, similar to other self-supervised methods.

Contrastive Predictive Coding (CPC)

1. Encode all samples in a sequence into vectors $z_t = \text{genc}(x_t)$



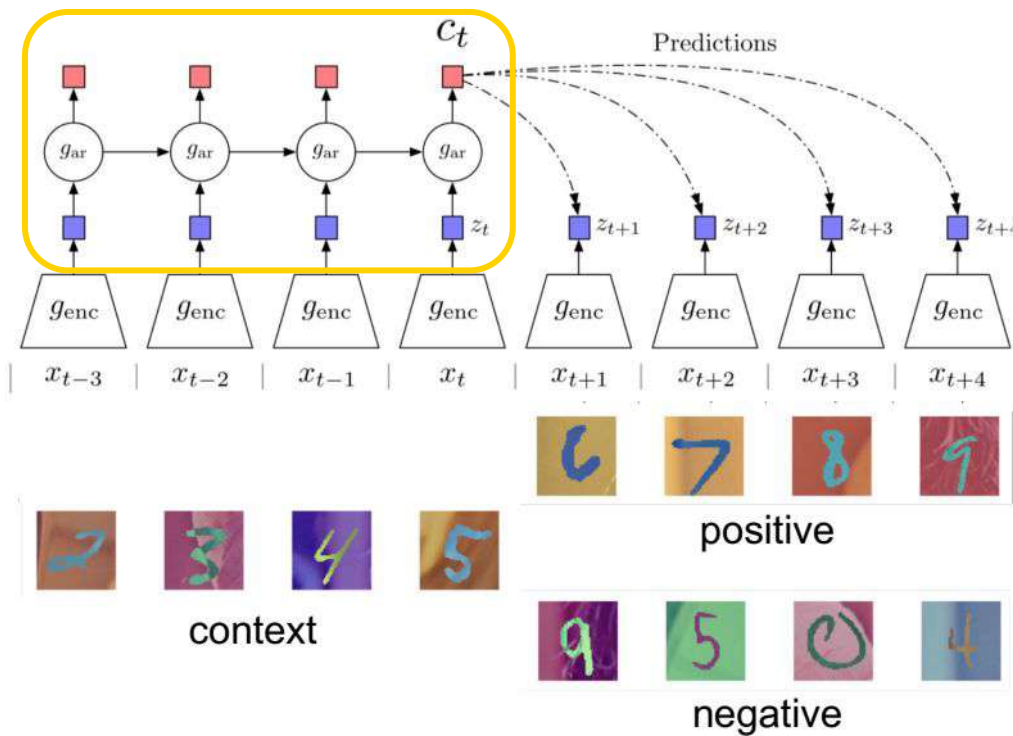
Contrastive Predictive Coding (CPC)



1. Encode all samples in a sequence into vectors $z_t = g_{enc}(x_t)$

2. Summarize context (e.g., half of a sequence) into a context code c_t using an auto-regressive model (g_{ar}). The original paper uses GRU-RNN here.

Contrastive Predictive Coding (CPC)



1. Encode all samples in a sequence into vectors $z_t = g_{enc}(x_t)$

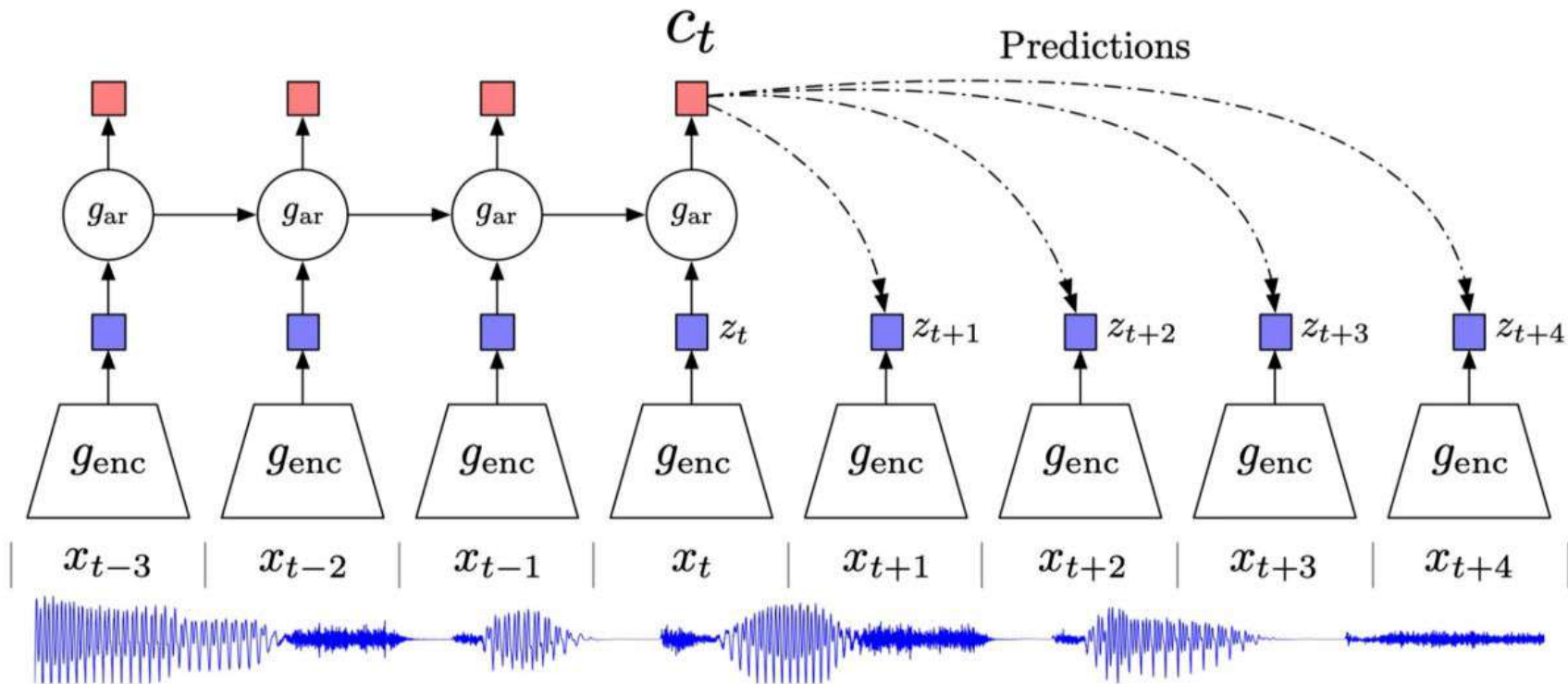
2. Summarize context (e.g., half of a sequence) into a context code c_t using an auto-regressive model (g_{ar}). The original paper uses GRU-RNN here.

3. Compute InfoNCE loss between the context c_t and future code z_{t+k} using the following time-dependent score function:

$$s_k(z_{t+k}, c_t) = z_{t+k}^T W_k c_t$$

, where W_k is a trainable matrix.

CPC example: modeling audio sequences



CPC example: modeling audio sequences

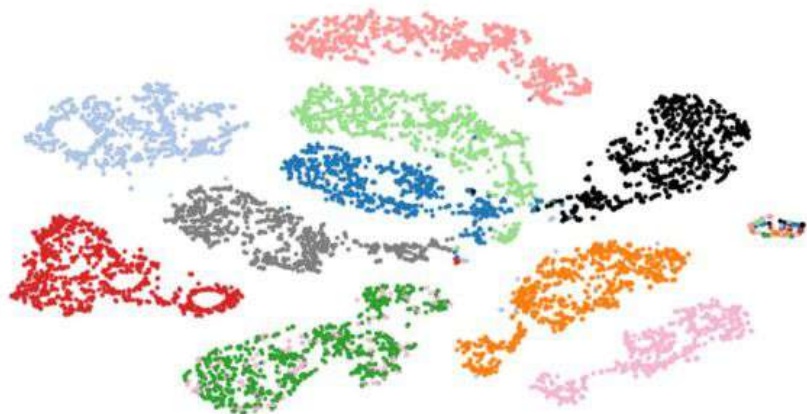


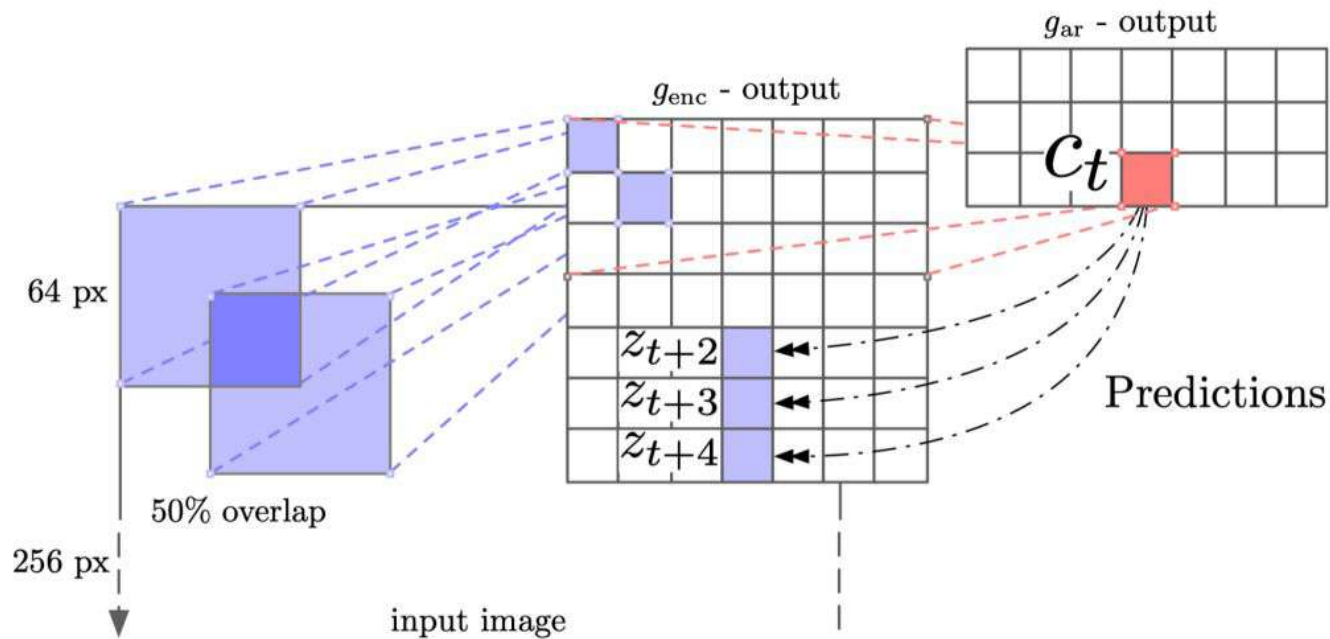
Figure 2: t-SNE visualization of audio (speech) representations for a subset of 10 speakers (out of 251). Every color represents a different speaker.

Method	ACC
Phone classification	
Random initialization	27.6
MFCC features	39.7
CPC	64.6
Supervised	74.6
Speaker classification	
Random initialization	1.87
MFCC features	17.6
CPC	97.4
Supervised	98.5

Linear classification on trained representations (LibriSpeech dataset)

CPC example: modeling visual context

Idea: split image into patches, model rows of patches from top to bottom as a sequence. I.e., use top rows as context to predict bottom rows.

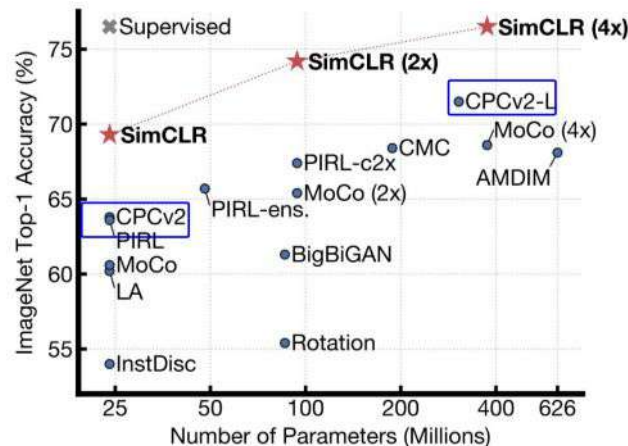


CPC example: modeling visual context

Method	Top-1 ACC
Using AlexNet conv5	
Video [28]	29.8
Relative Position [11]	30.4
BiGan [35]	34.8
Colorization [10]	35.2
Jigsaw [29] *	38.1
Using ResNet-V2	
Motion Segmentation [36]	27.6
Exemplar [36]	31.5
Relative Position [36]	36.2
Colorization [36]	39.6
CPC	48.7

Table 3: ImageNet top-1 unsupervised classification results. *Jigsaw is not directly comparable to the other AlexNet results because of architectural differences.

- Compares favorably with other pretext task-based self-supervised learning method.
- Doesn't do as well compared to newer instance-based contrastive learning methods on image feature learning.



Summary: Contrastive Representation Learning

A general formulation for contrastive learning:

$$\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-))$$

InfoNCE loss: N-way classification among positive and negative samples

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

Commonly known as the InfoNCE loss ([van den Oord et al., 2018](#))

A lower bound on the mutual information between $f(x)$ and $f(x^+)$

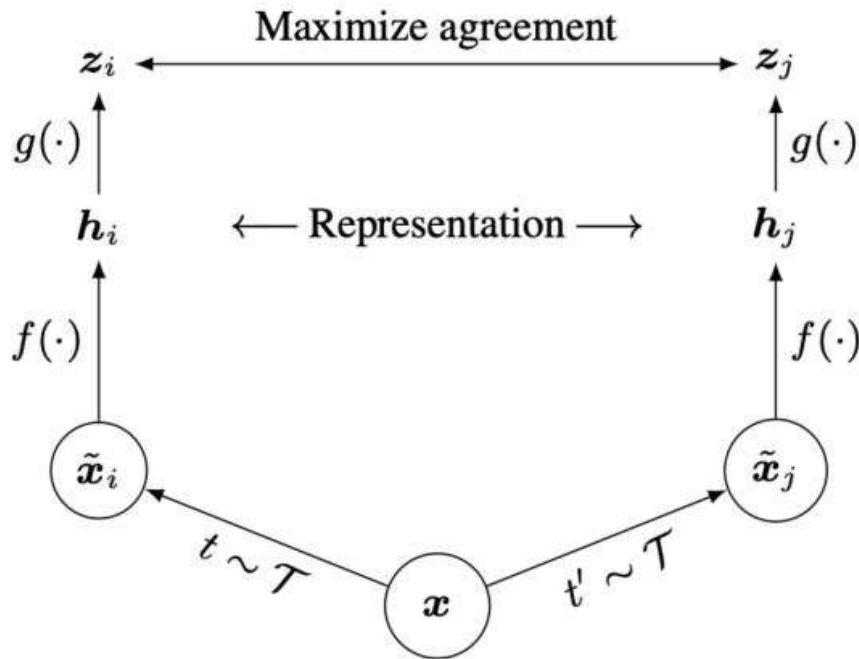
$$MI[f(x), f(x^+)] - \log(N) \geq -L$$



Summary: Contrastive Representation Learning

SimCLR: a simple framework for contrastive representation learning

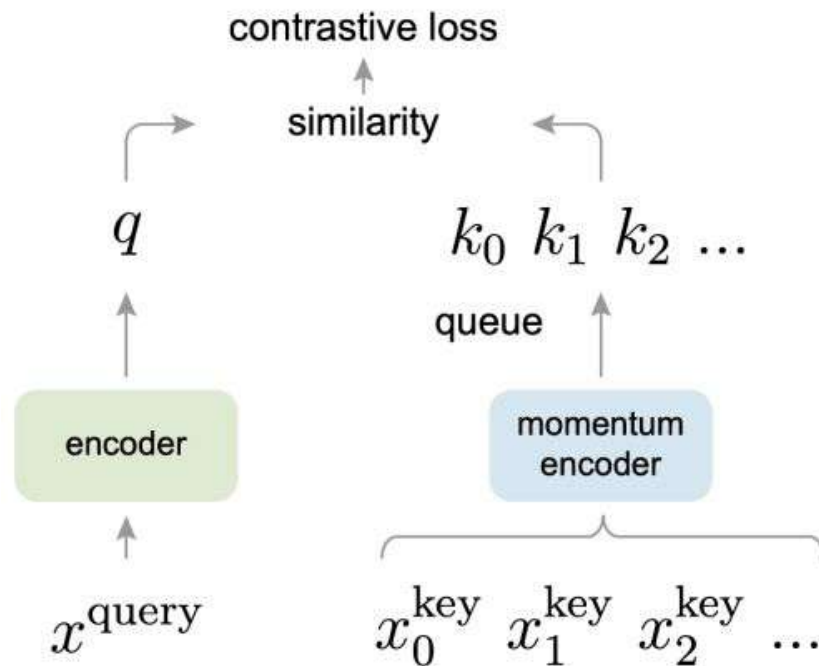
- Key ideas: non-linear projection head to allow flexible representation learning
- Simple to implement, effective in learning visual representation
- Requires large training batch size to be effective; large memory footprint



Summary: Contrastive Representation Learning

MoCo (v1, v2): contrastive learning using momentum sample encoder

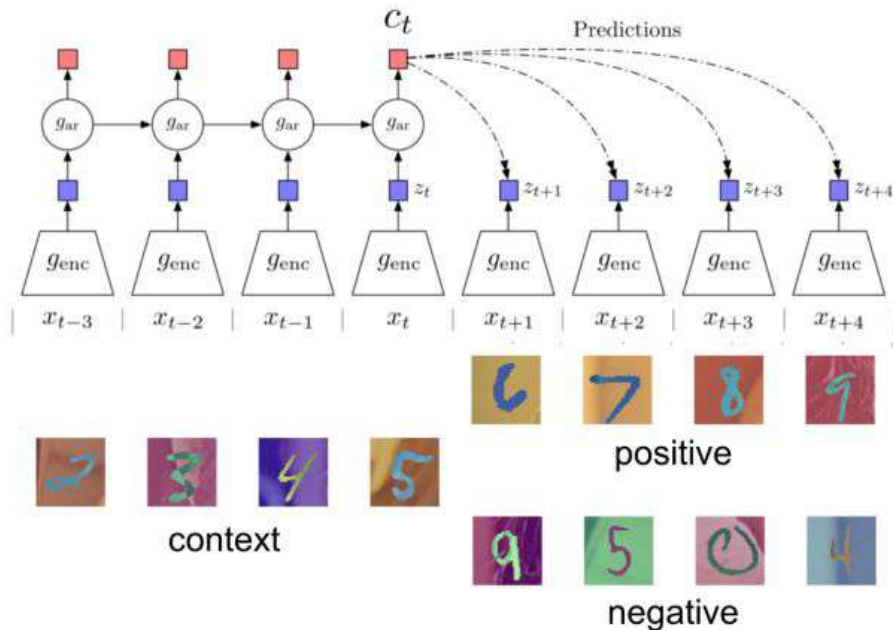
- Decouples negative sample size from minibatch size; allows large batch training without TPU
- MoCo-v2 combines the key ideas from SimCLR, i.e., nonlinear projection head, strong data augmentation, with momentum contrastive learning



Summary: Contrastive Representation Learning

CPC: sequence-level contrastive learning

- Contrast “right” sequence with “wrong” sequence.
- InfoNCE loss with a time-dependent score function.
- Can be applied to a variety of learning problems, but not as effective in learning image representations compared to instance-level methods.



Other examples: MoCo v3

An Empirical Study of Training Self-Supervised Vision Transformers

“This paper does not describe a novel method.”

Xinlei Chen* Saining Xie* Kaiming He
Facebook AI Research (FAIR)

Code: <https://github.com/facebookresearch/moco-v3>

Abstract

This paper does not describe a novel method. Instead, it studies a straightforward, incremental, yet must-know baseline given the recent progress in computer vision: self-supervised learning for Vision Transformers (ViT). While the training recipes for standard convolutional networks have been highly mature and robust, the recipes for ViT are yet to be built, especially in the self-supervised scenarios where training becomes more challenging. In this work, we go back to basics and investigate the effects of several fundamental components for training self-supervised ViT. We observe that instability is a major issue that degrades accuracy, and it can be hidden by apparently good results. We reveal that these results are indeed partial failure, and they can be improved when training is made more stable. We benchmark ViT results in MoCo v3 and several other self-supervised frameworks, with ablations in various aspects. We discuss the currently positive evidence as well as challenges and open questions. We hope that this work will provide useful data points and experience for future research.

framework	model	params	acc. (%)
<i>linear probing:</i>			
iGPT [9]	iGPT-L	1362M	69.0
iGPT [9]	iGPT-XL	6801M	72.0
MoCo v3	ViT-B	86M	76.7
MoCo v3	ViT-L	304M	77.6
MoCo v3	ViT-H	632M	78.1
MoCo v3	ViT-BN-H	632M	79.1
MoCo v3	ViT-BN-L/7	304M	81.0
<i>end-to-end fine-tuning:</i>			
masked patch pred. [16]	ViT-B	86M	79.9 [†]
MoCo v3	ViT-B	86M	83.2
MoCo v3	ViT-L	304M	84.1

Table 1. **State-of-the-art Self-supervised Transformers** in ImageNet classification, evaluated by linear probing (top panel) or end-to-end fine-tuning (bottom panel). Both iGPT [9] and masked patch prediction [16] belong to the masked auto-encoding paradigm. MoCo v3 is a contrastive learning method that compares two (224×224) crops. ViT-B, -L, -H are the Vision Transformers proposed in [16]. ViT-BN is modified with BatchNorm, and “/7” denotes a patch size of 7×7. [†]: pre-trained in JFT-300M.

Other examples: Masked Autoencoder



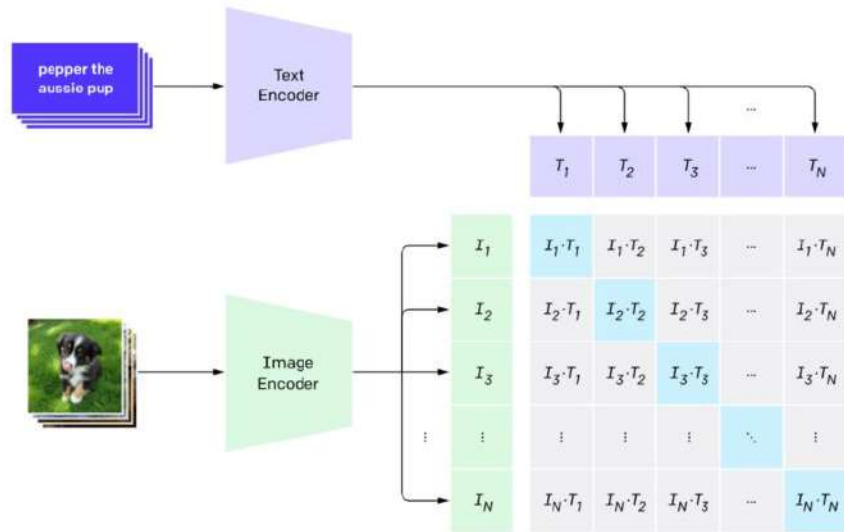
method	pre-train data	ViT-B	ViT-L	ViT-H	ViT-H ₄₄₈
scratch, our impl.	-	82.3	82.6	83.1	-
DINO [5]	IN1K	82.8	-	-	-
MoCo v3 [9]	IN1K	83.2	84.1	-	-
BEiT [2]	IN1K+DALLE	83.2	85.2	-	-
MAE	IN1K	<u>83.6</u>	<u>85.9</u>	<u>86.9</u>	87.8



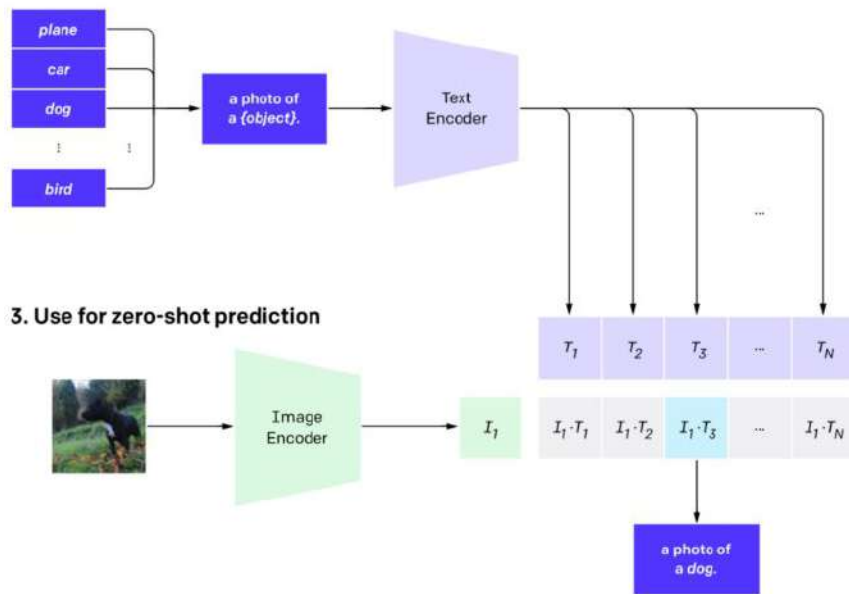
Other examples: CLIP

Contrastive learning between image and natural language sentences

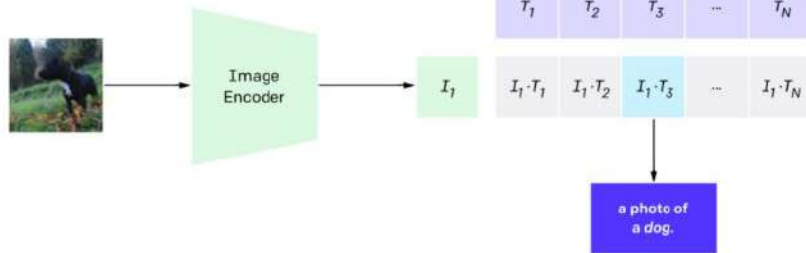
1. Contrastive pre-training



2. Create dataset classifier from label text



3. Use for zero-shot prediction

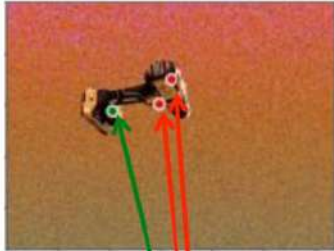


CLIP (Contrastive Language–Image Pre-training) Radford et al., 2021

Other examples: Dense Object Net

Contrastive learning on pixel-wise feature descriptors

(c) Background Randomization



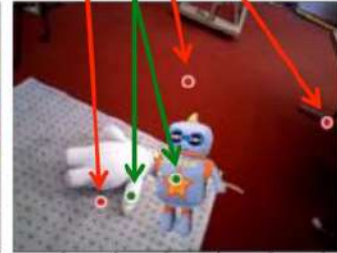
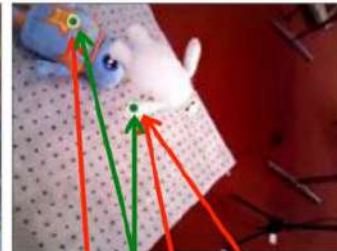
(d) Cross Object Loss



(e) Direct Multi Object

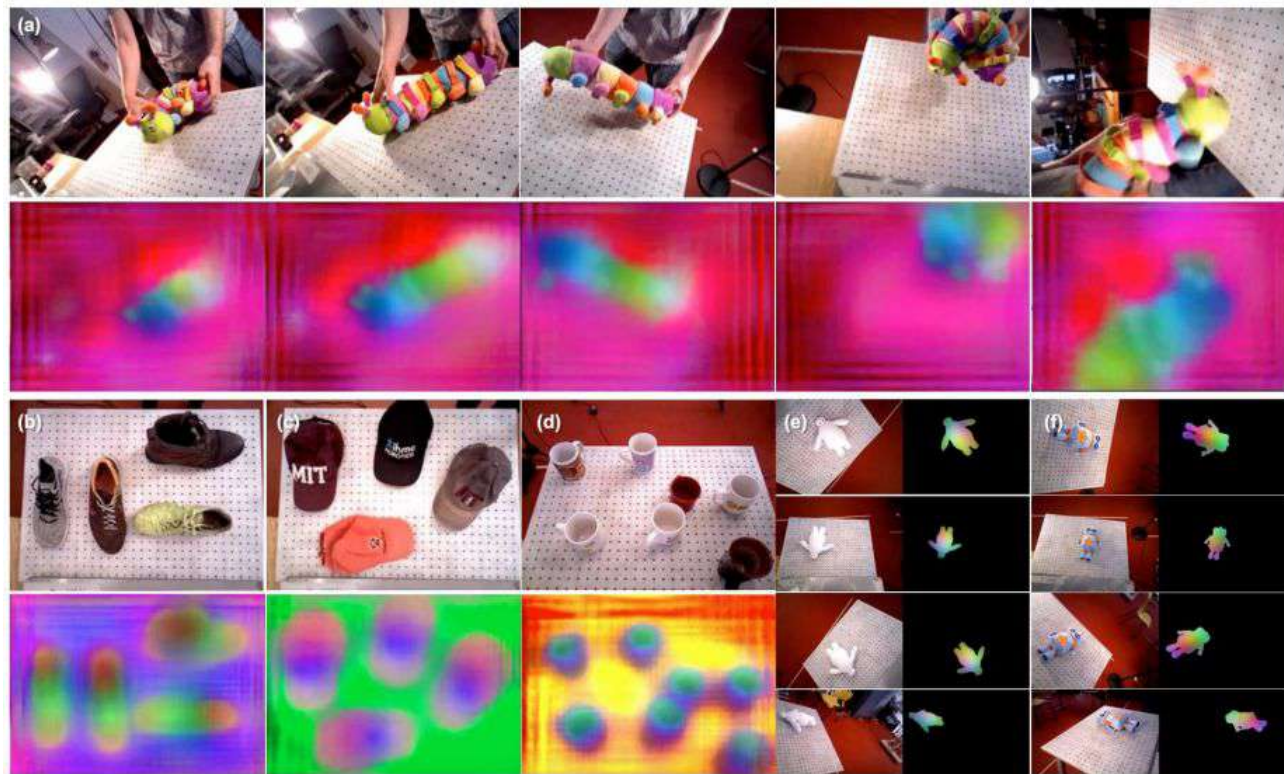


(f) Synthetic Multi Object



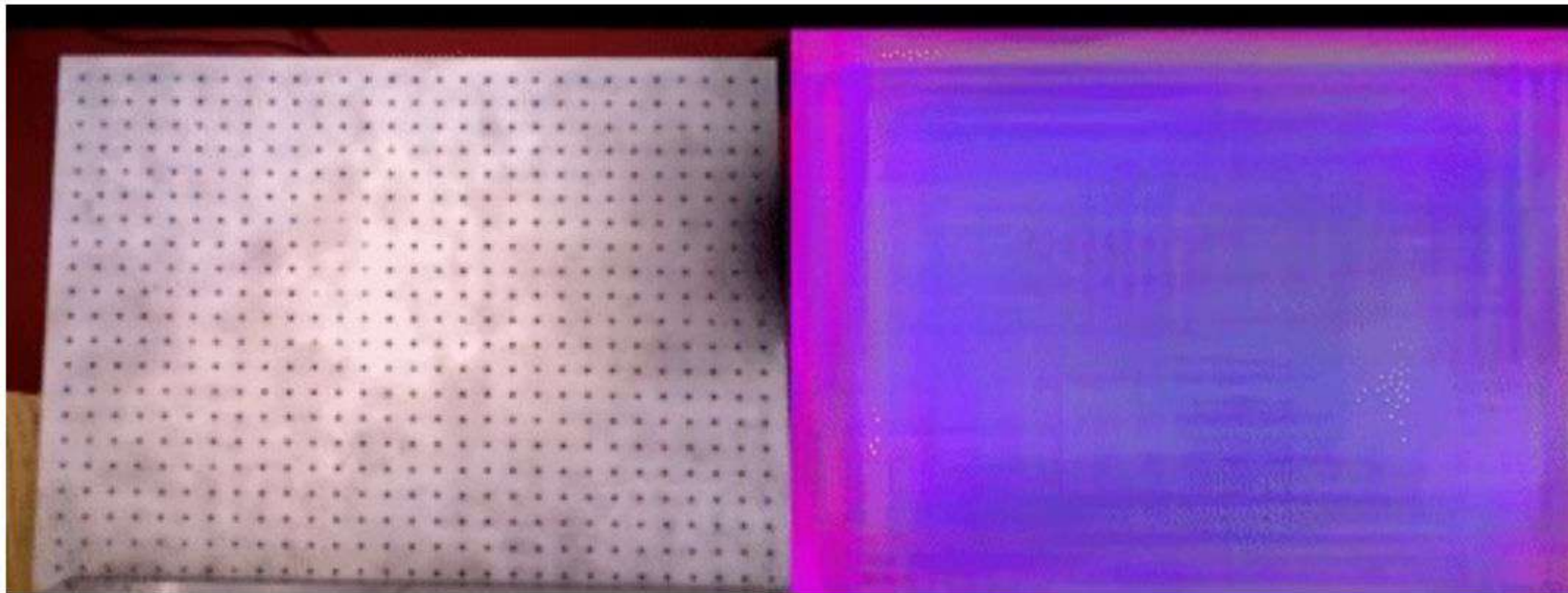
Dense Object Net, Florence et al., 2018

Other examples: Dense Object Net



Dense Object Net, Florence et al., 2018

Other examples: Dense Object Net



Dense Object Net, Florence et al., 2018

Other examples: DINO

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron^{1,2} Hugo Touvron^{1,3} Ishan Misra¹ Hervé Jegou¹
Julien Mairal² Piotr Bojanowski¹ Armand Joulin¹

¹ Facebook AI Research

² Inria*

³ Sorbonne University



Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

Other examples: DINO v2

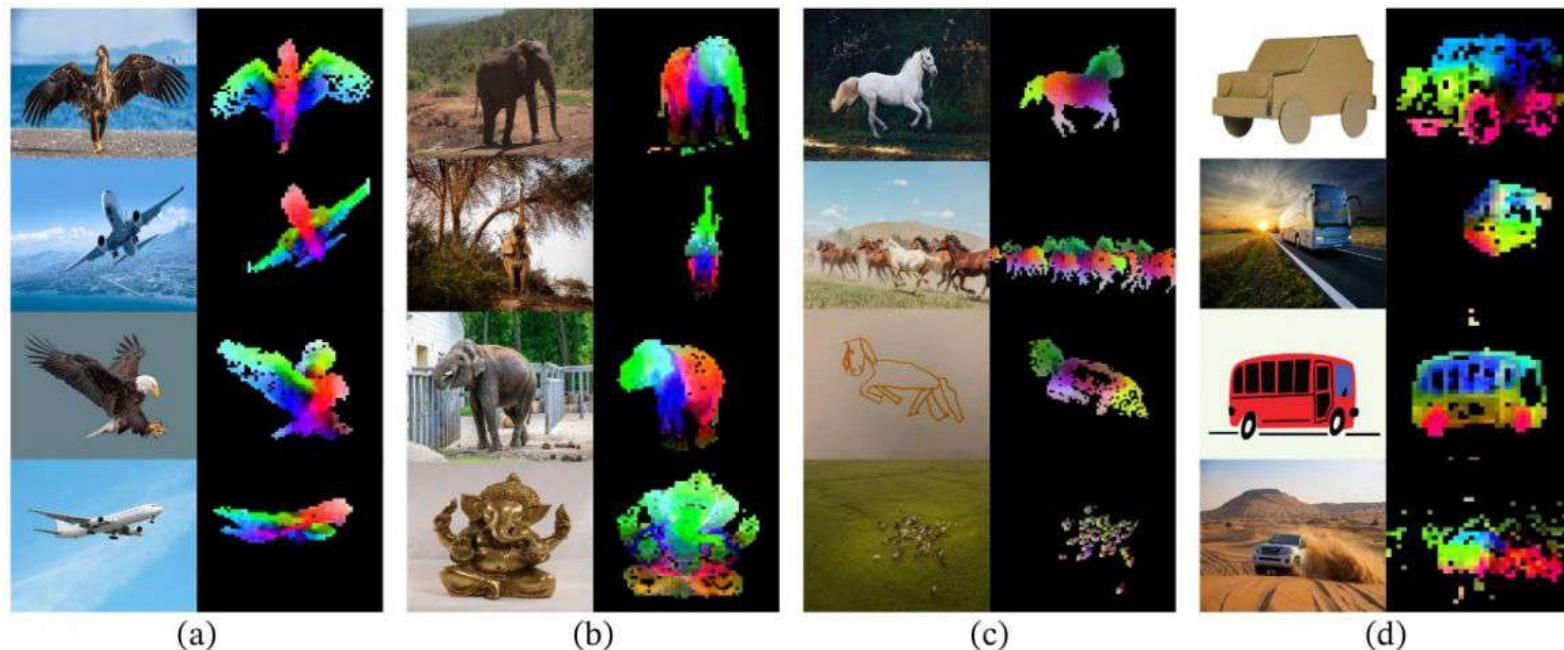
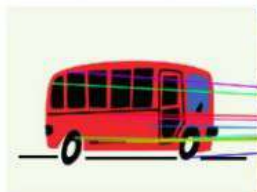
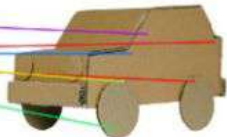


Figure 1: **Visualization of the first PCA components.** We compute a PCA between the patches of the images from the same column (a, b, c and d) and show their first 3 components. Each component is matched to a different color channel. Same parts are matched between related images despite changes of pose, style or even objects. Background is removed by thresholding the first PCA component.

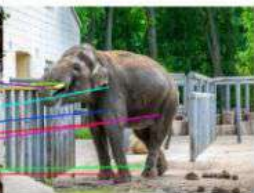
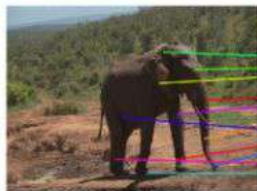
Other examples: DINO v2



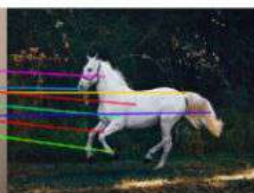
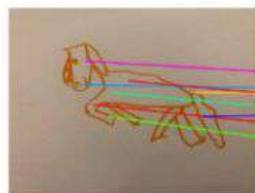
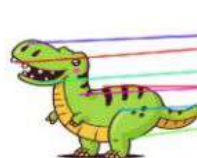
(Vehicles)



(Birds / Airplanes)



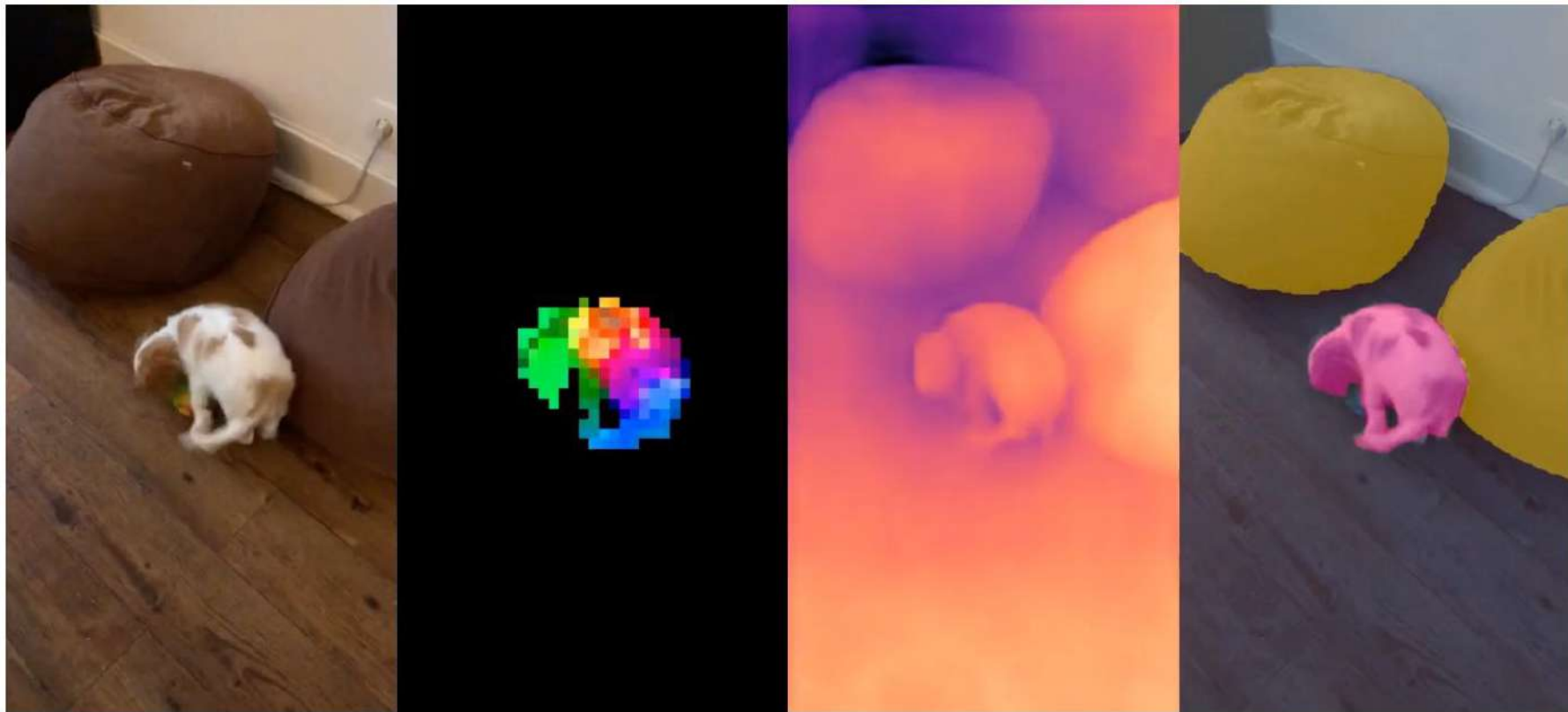
(Elephants)



(Drawings / Animals)



Other examples: DINO v2



Thanks for attention!

Questions?



Lecture 08: Intro to Reinforcement Learning

Radoslav Neychev

1. Reinforcement Learning problem statement
2. (Multi-armed) bandits
3. MDP formalism
4. Relations to Psychology
5. Cross-entropy method
6. Reinforcement, Supervised and Unsupervised Learning

Reinforcement Learning

problem statement

Supervised learning

- Given:

Want them to be i.i.d.

- Objects and reference answers

$$x \in \mathcal{X}, y \in \mathcal{Y}$$

- Loss/objective function

$$L(\hat{y}, y)$$

Usually differentiable

- Model family $f \in \mathcal{F}, f : \mathcal{X} \longrightarrow \mathcal{Y}$

- Goal:

- Find optimal mapping $f^* = \arg \min_f L(f(x), y)$

Reinforcement learning

- Given:

Usually no reference answers

E.g. want the robot to walk

- Objects ~~and reference answers~~ $x \in \mathcal{X}$

- Loss/objective function

$$L(\hat{y}, y)$$

Usually even hard to formulate,
non-differentiable

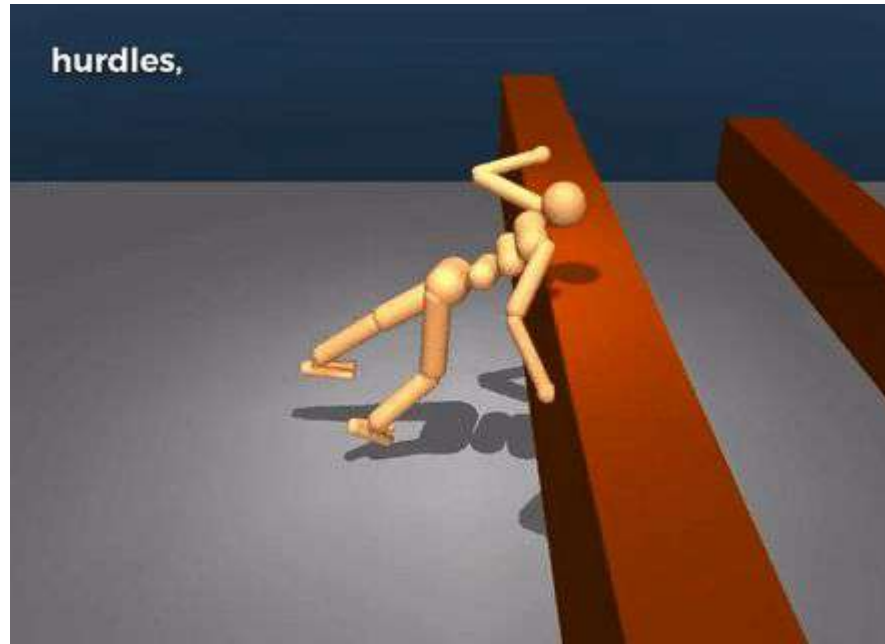
- Model family $f \in \mathcal{F}, f : \mathcal{X} \longrightarrow \mathcal{Y}$

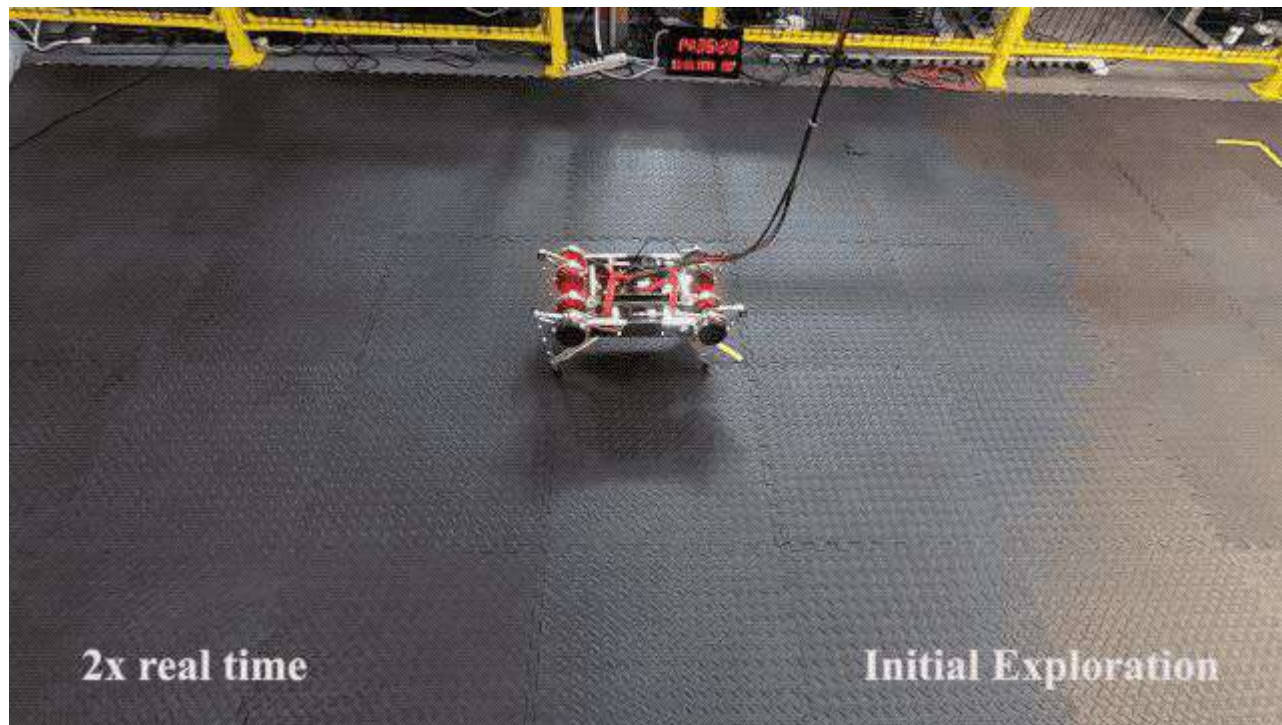
- Goal:

- Find optimal mapping $f^* = \arg \min_f L(f(x), y)$

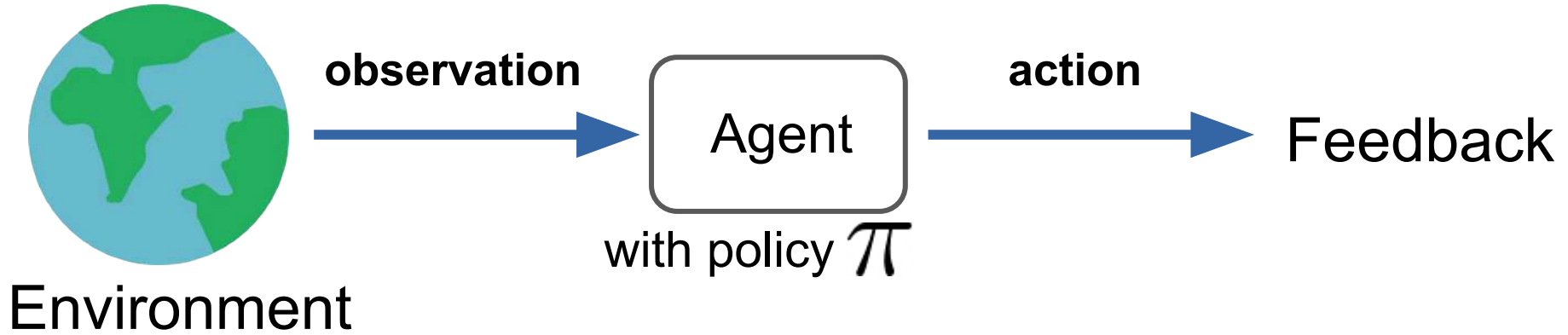
Planar walker:
9 DoFs, 6 Actuators.
Sensors: Proprioception and simplified vision.



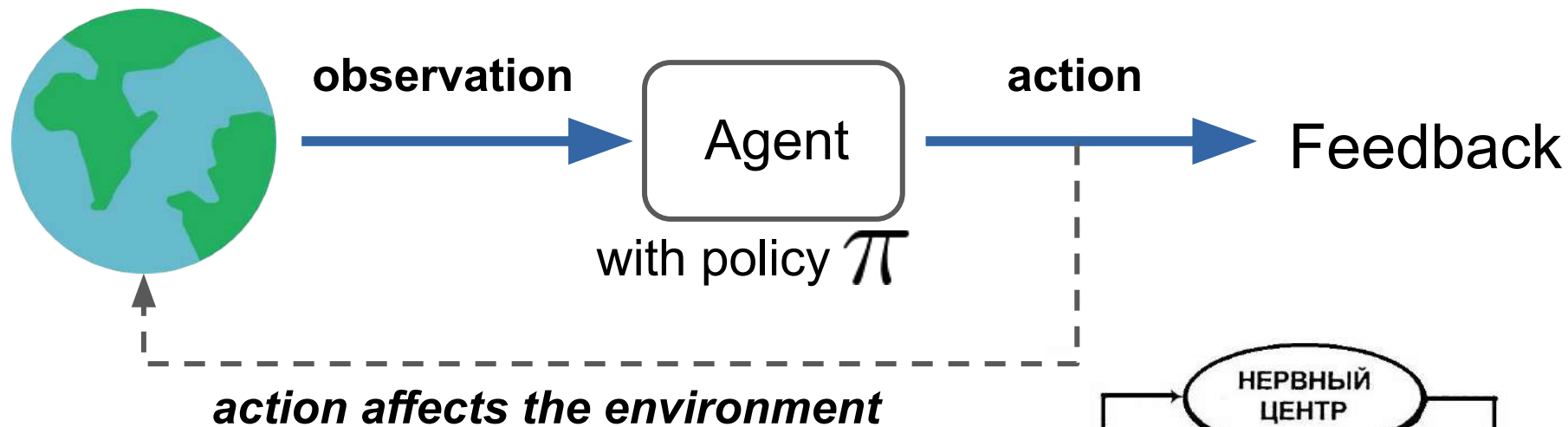




(Multi-armed) bandits



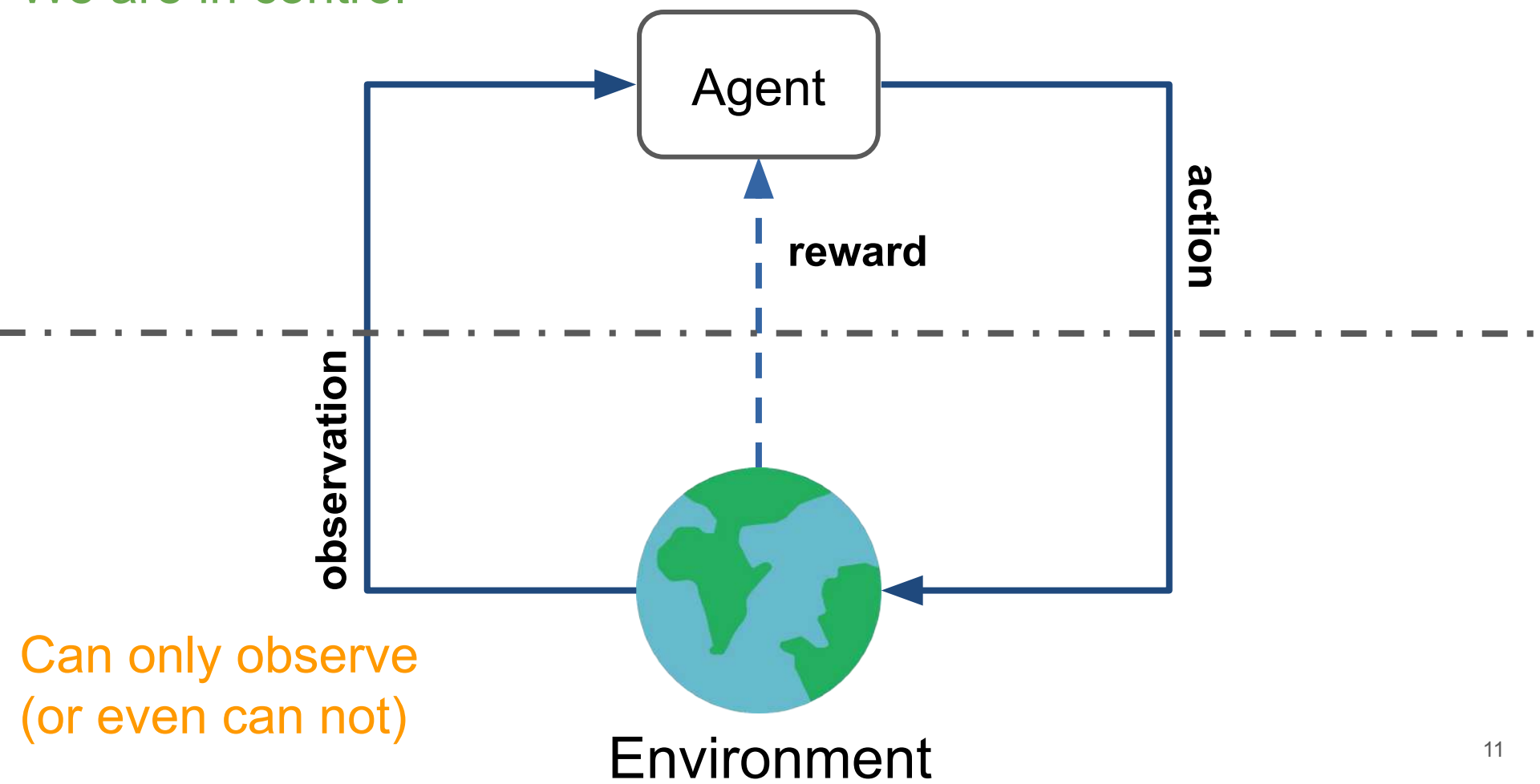
- Observation (state): vector or image or sequence ... or *nothing*
- Policy: mapping from state to action
- Action
- Feedback (reward): usually a converted to a number



Рефлекторное кольцо введено А. Ф.

Самойловым (ученик И.П. Павлова) в 1930,
также исследовалось Н.А. Бернштейном

We are in control



Can only observe
(or even can not)

Reality check: dynamic control

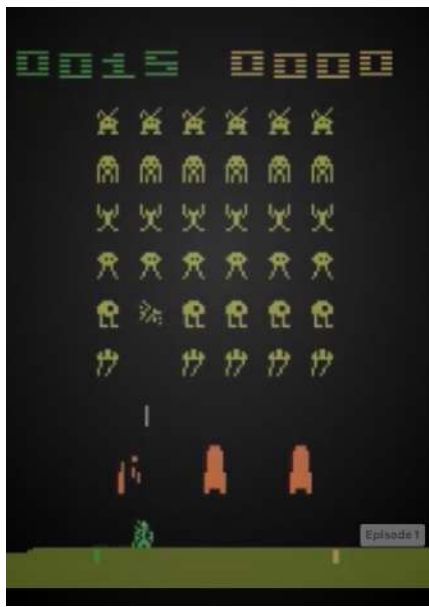
Variety of papers on helicopter control:
heli.stanford.edu

Andrew Y. Ng PhD Thesis link: [“Shaping and policy search in Reinforcement Learning”](#)



- Observation: accelerometer, gyroscope, engine data
- Action: change rotation speed, angle
- Feedback: some specific reward

Reality check: video games



- Observation: image(s)
- Action: move, fire, turn
- Feedback: score/health/progres/...

Open questions

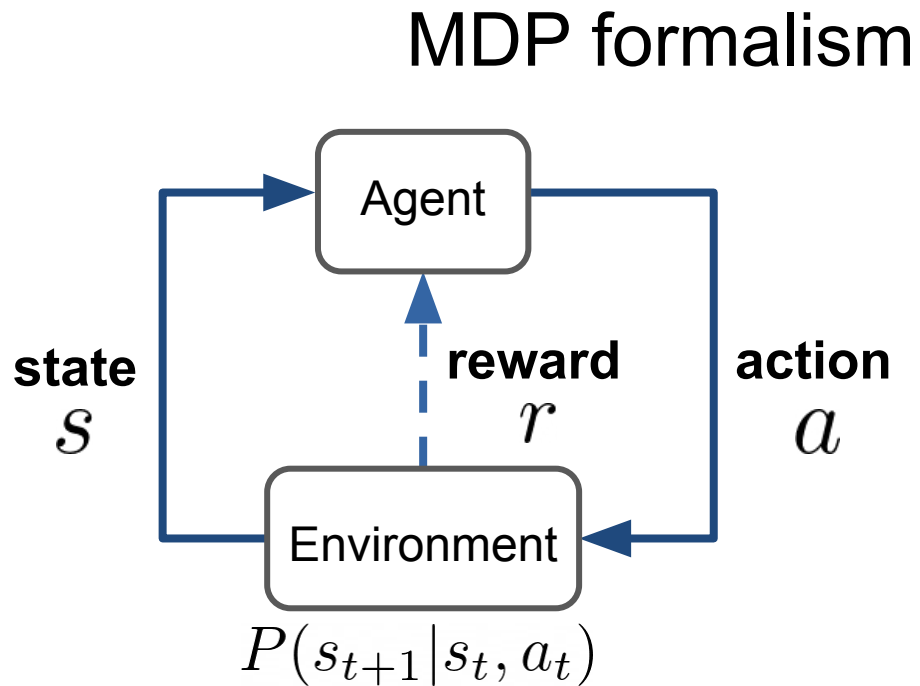
- What is *optimal* action?
 - Maximize the reward **on the next step**
 - Maximize the reward **in long term**



- *Explore or exploit?*
 - Stepping of *current optimal* strategy may **decrease** the cumulative reward
 - Under *current optimal strategy* one may **never discover** something better

- State: $s \in \mathcal{S}$
- Action: $a \in \mathcal{A}$
- Reward: $r \in \mathbb{R}$

- Dynamics: $P(s_{t+1} | s_t, a_t)$



Markov property:

$$P(s_{t+1} | s_t, a_t, \dots, s_0, t_0) = P(s_{t+1} | s_t, a_t)$$

- Total reward for session: $R = \sum_t r_t$
- Policy: $\pi(a|s) = P(\text{take action } a \text{ in state } s)$
- Goal: maximize reward; $\pi^*(a|s) = \arg \max_{\pi} \mathbb{E}_{\pi}[R]$

Psychological point of view

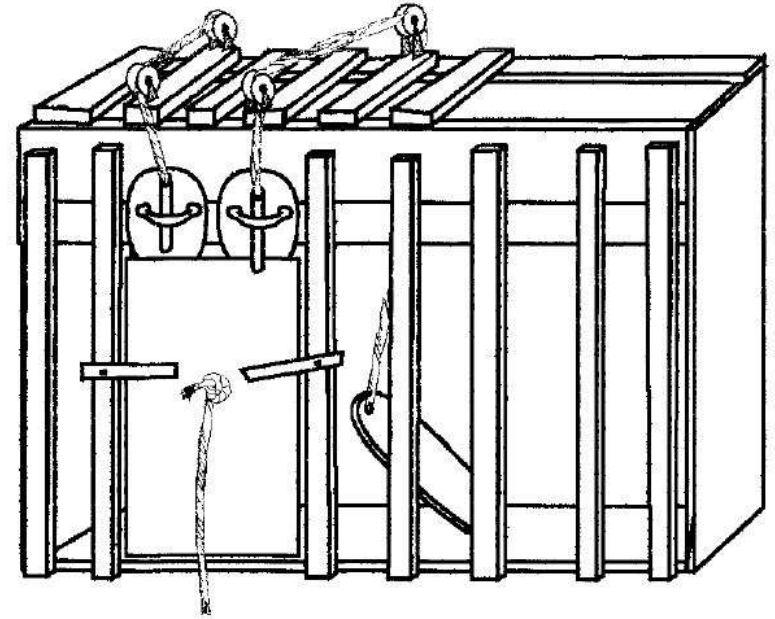
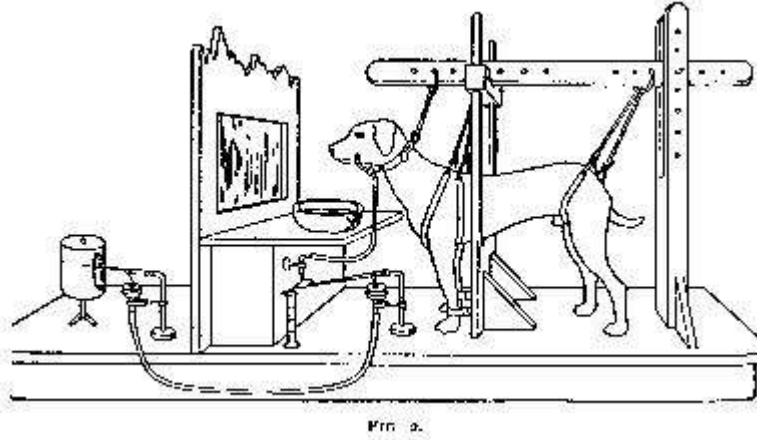
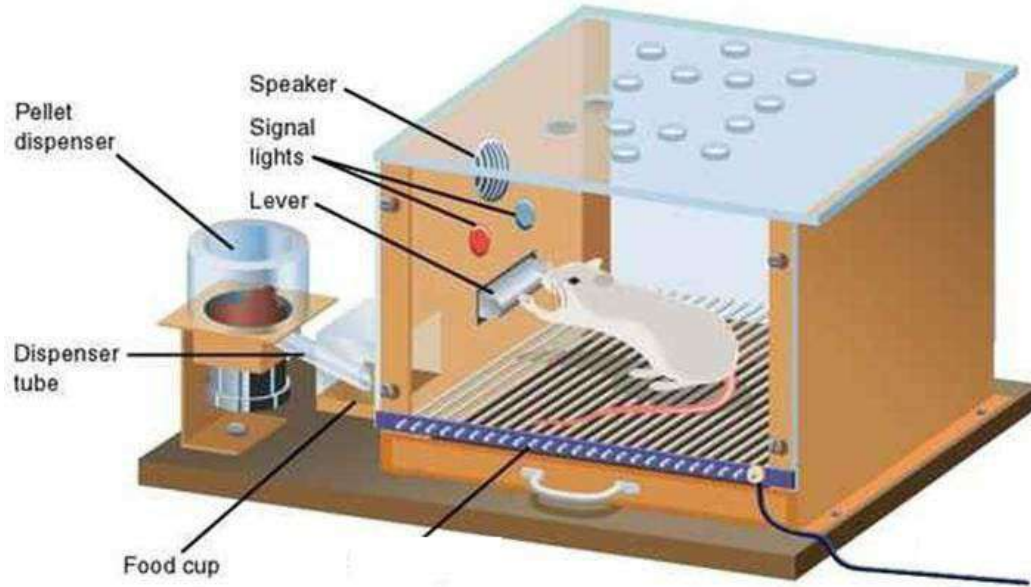


Fig. 4. Box K. The door is held in place by a weight suspended by a string. To open the door, a cat had to depress a treadle, pull on a string, and push a bar up or down. (After Thorndike, 1898, Figure 1, p. 8.)

Psychological point of view



CRAIG SWANSON © WWW.PERSPICUITY.COM

How to maximize the reward?

$\mathbb{E}_{\pi}[R]$ is an expected cumulative reward earned per session following policy π

Need to maximize the following objective:

$$\mathbb{E}_{\pi}[R] = \mathbb{E}_{s_0 \sim P(s_0)} \mathbb{E}_{a_0 \sim \pi(a|s_0)} \cdots \mathbb{E}_{s_t, r_t \sim P(s, r|s_{t-1}, a_{t-1})} [r_0 + \cdots + r_t]$$

How to do it?

How to maximize the reward?

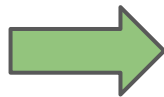
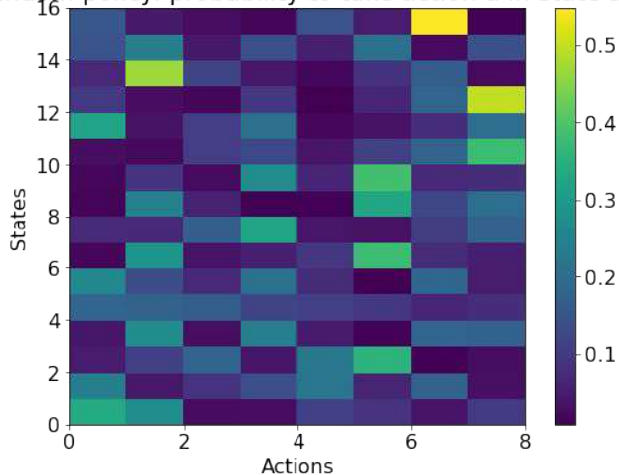
- Play a few sessions with existing policy
- Update the policy using new feedback
- Repeat

Cross-entropy method

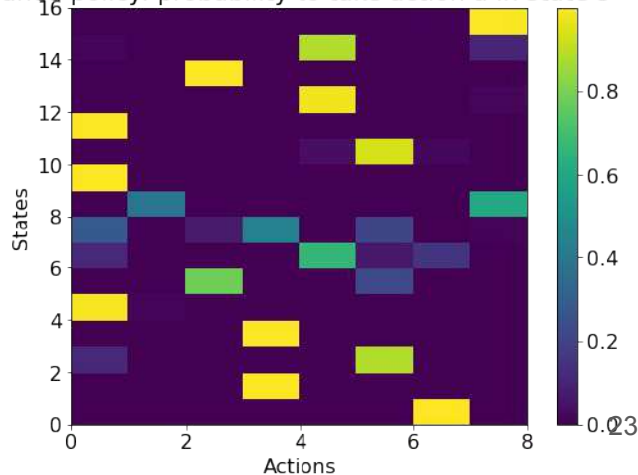
Cross-entropy method: tabular case

- Initialize policy (state-action matrix, every row sums up to 1)
- Sample N sessions
- Select M **elite** sessions with highest rewards
- Update policy using the elite session state-action sequences
- Repeat

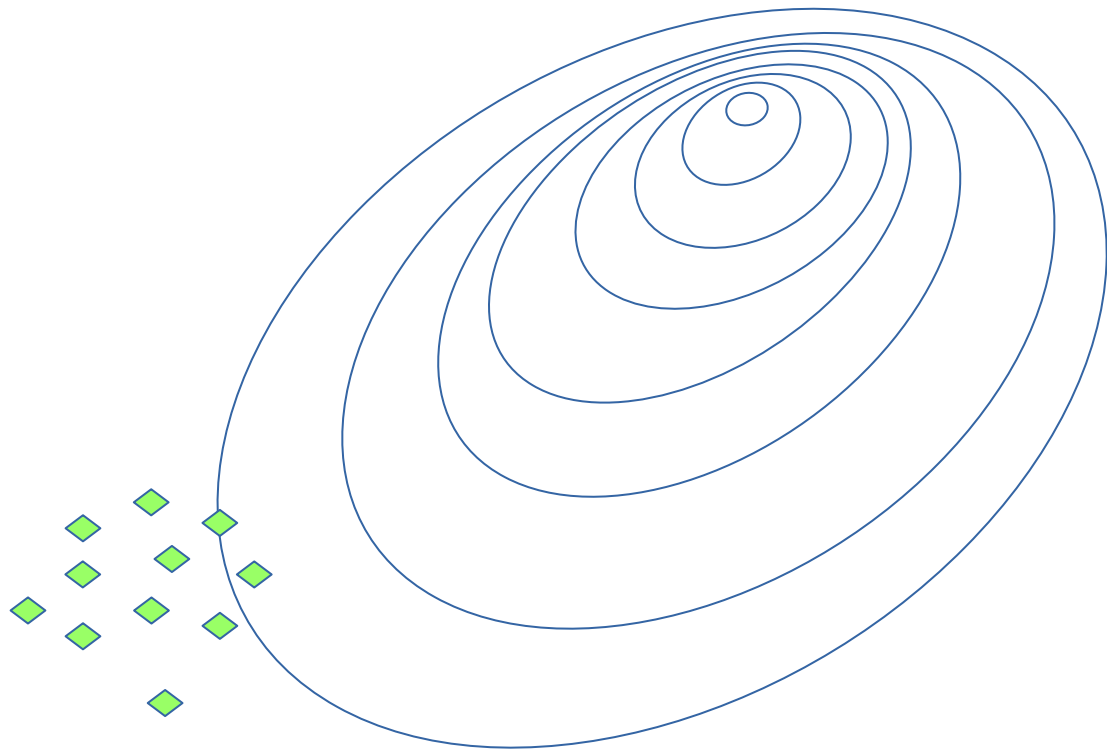
Random policy: probability to take action a in state s



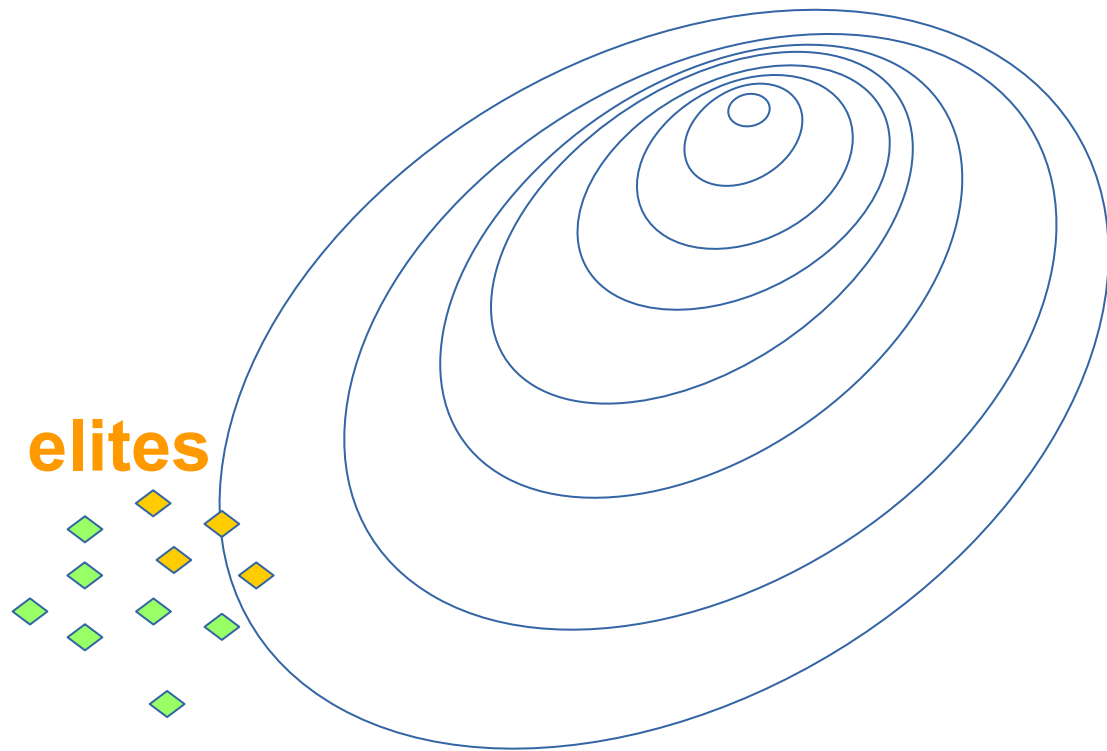
Tuned policy: probability to take action a in state s



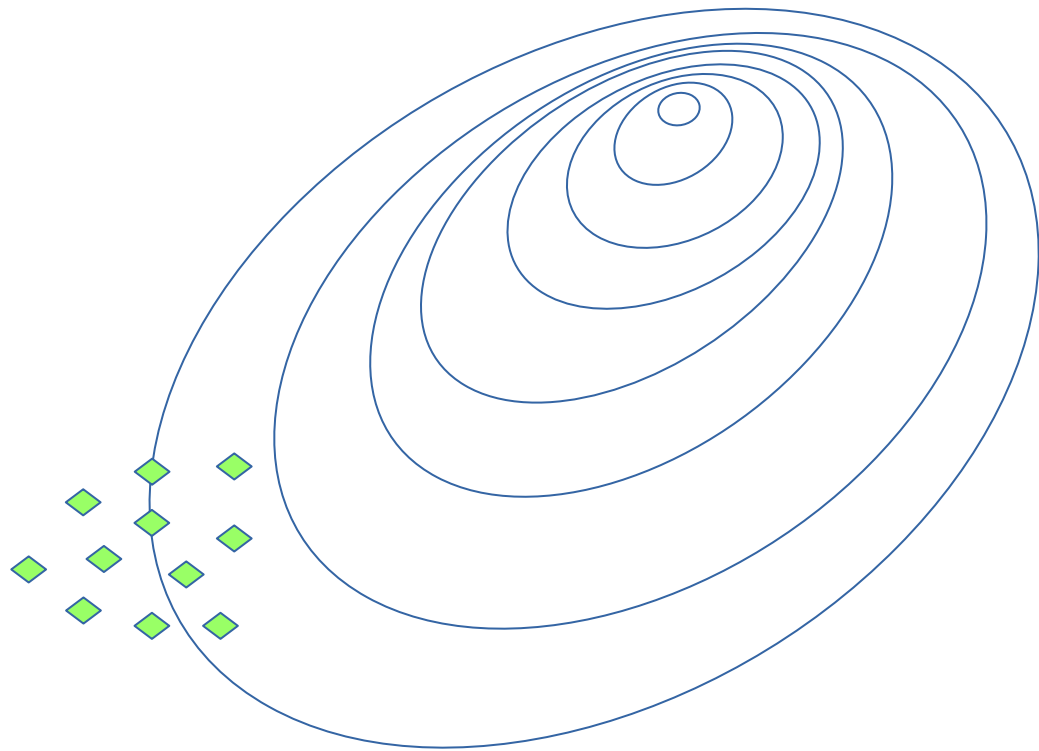
Cross-entropy method: illustration



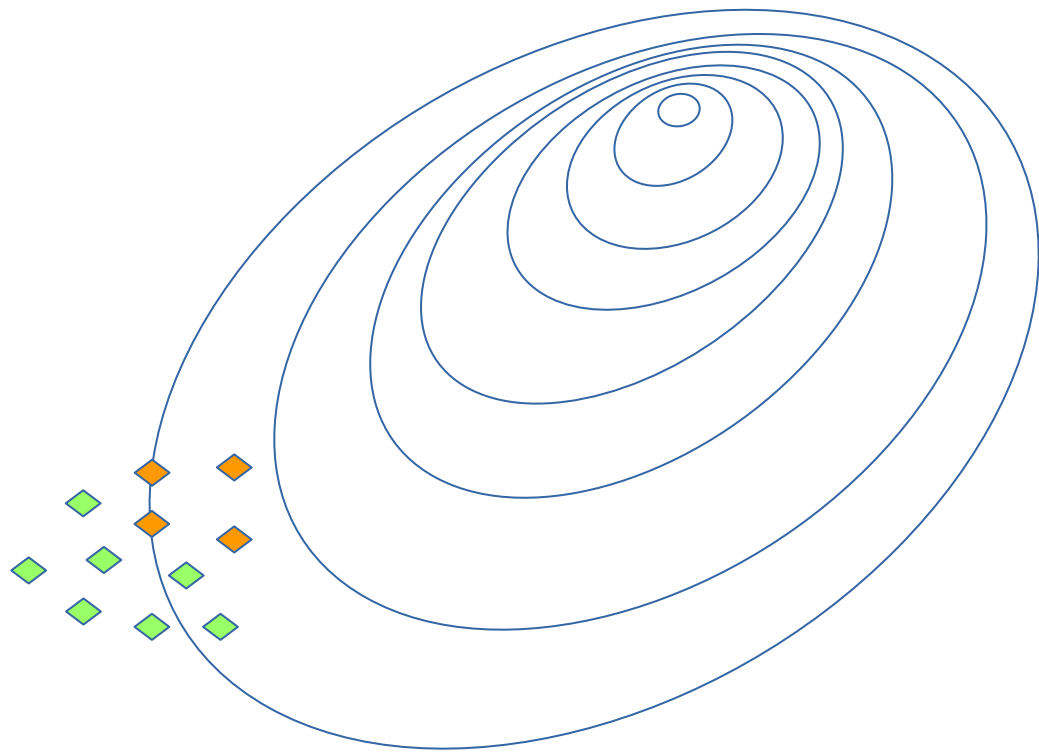
Cross-entropy method: illustration



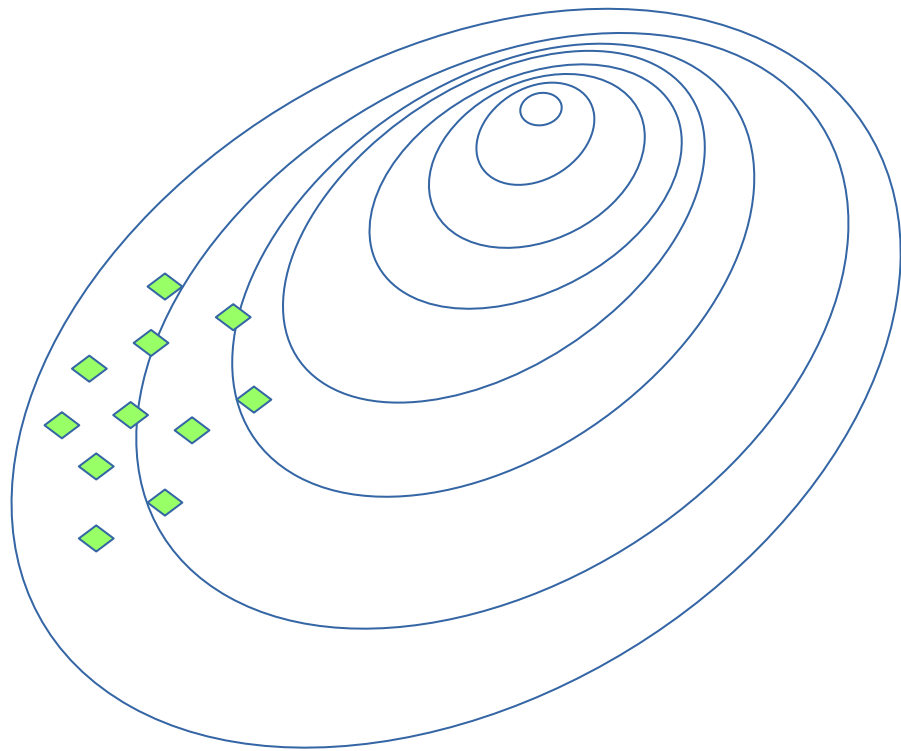
Cross-entropy method: illustration



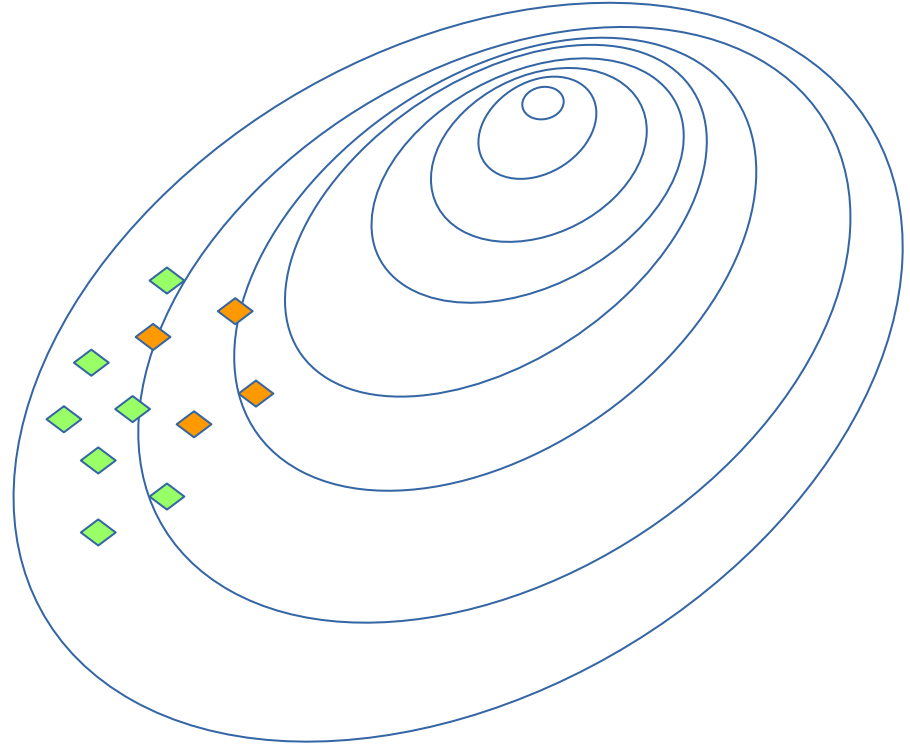
Cross-entropy method: illustration



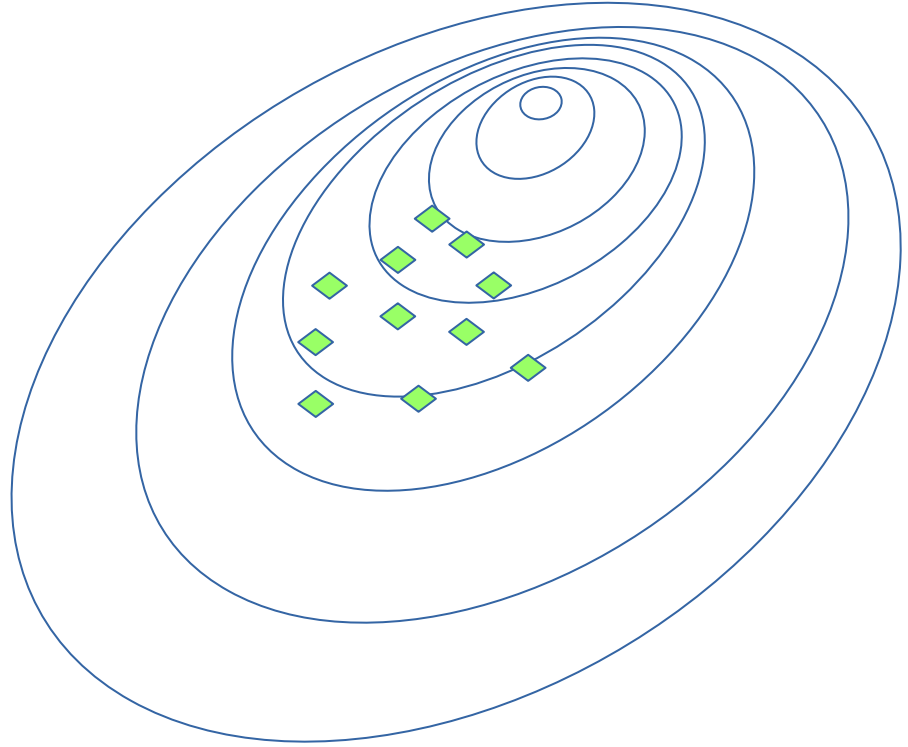
Cross-entropy method: illustration



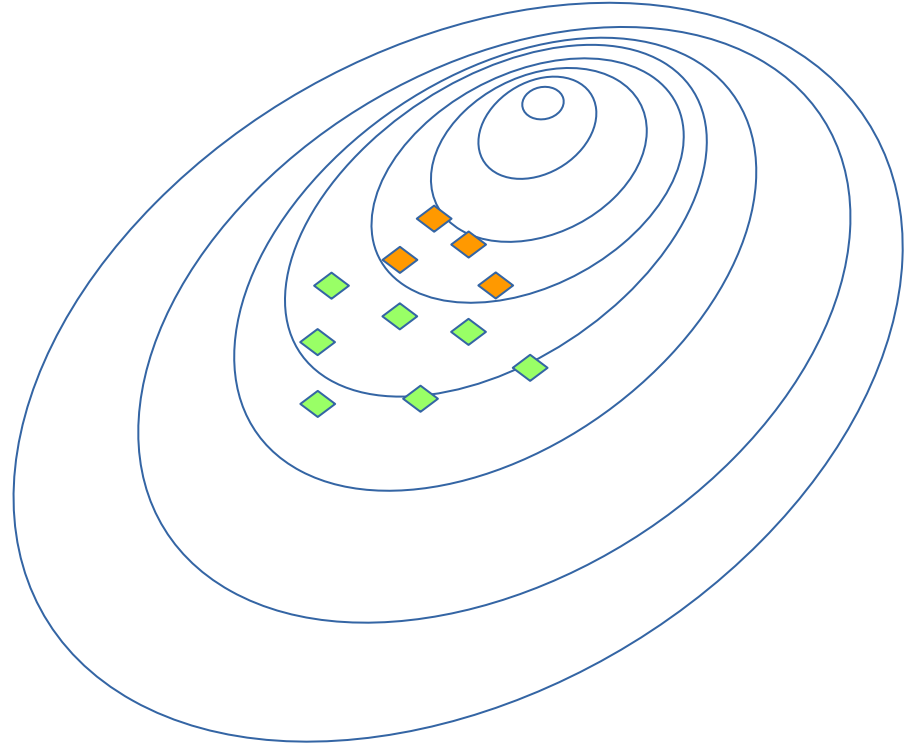
Cross-entropy method: illustration



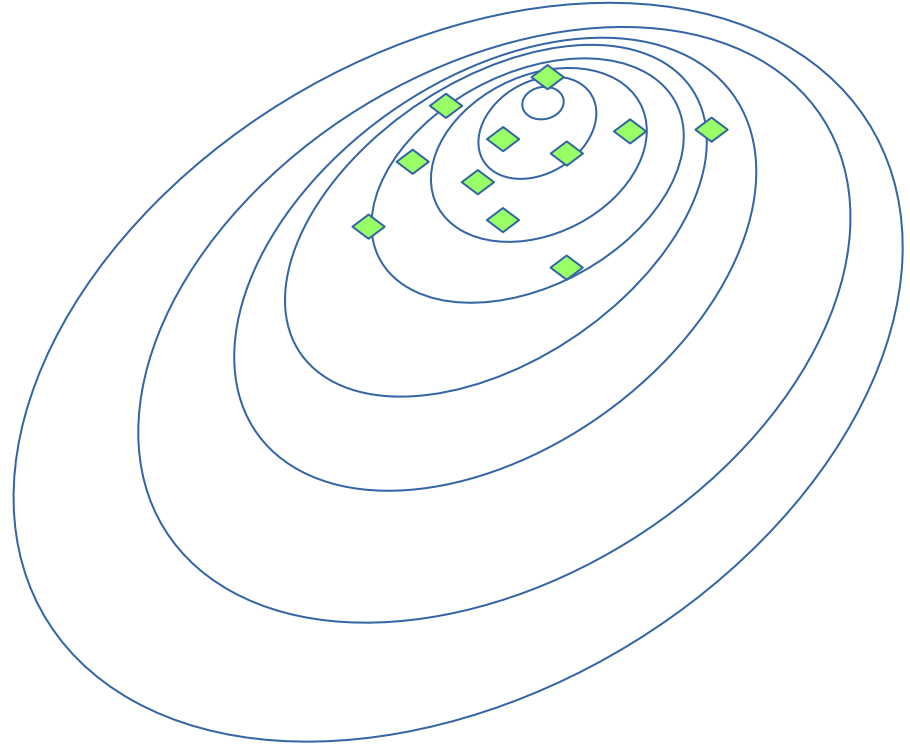
Cross-entropy method: illustration



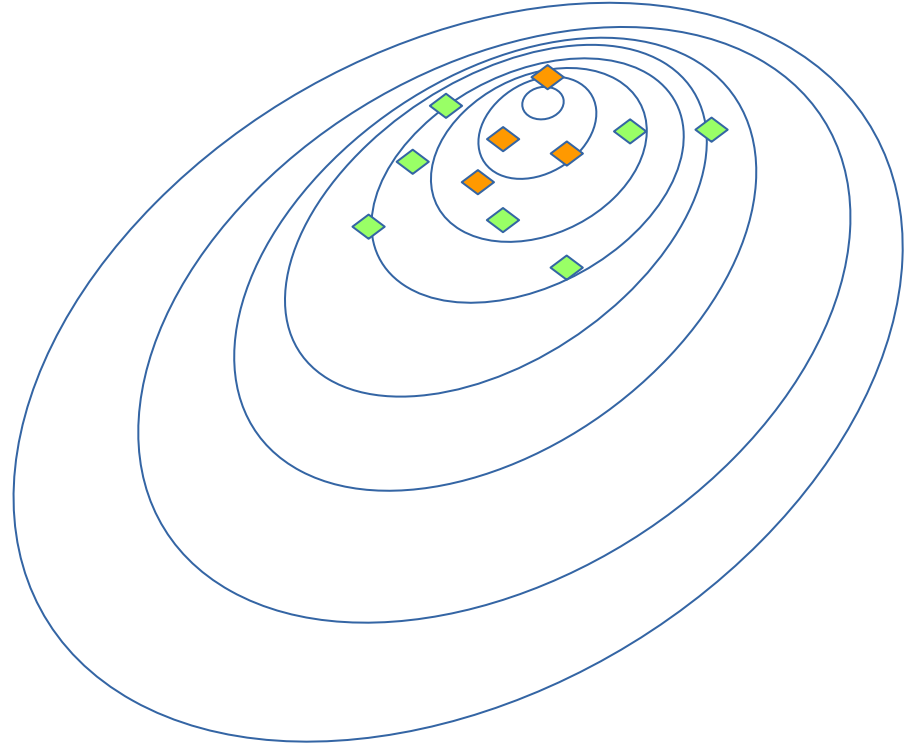
Cross-entropy method: illustration



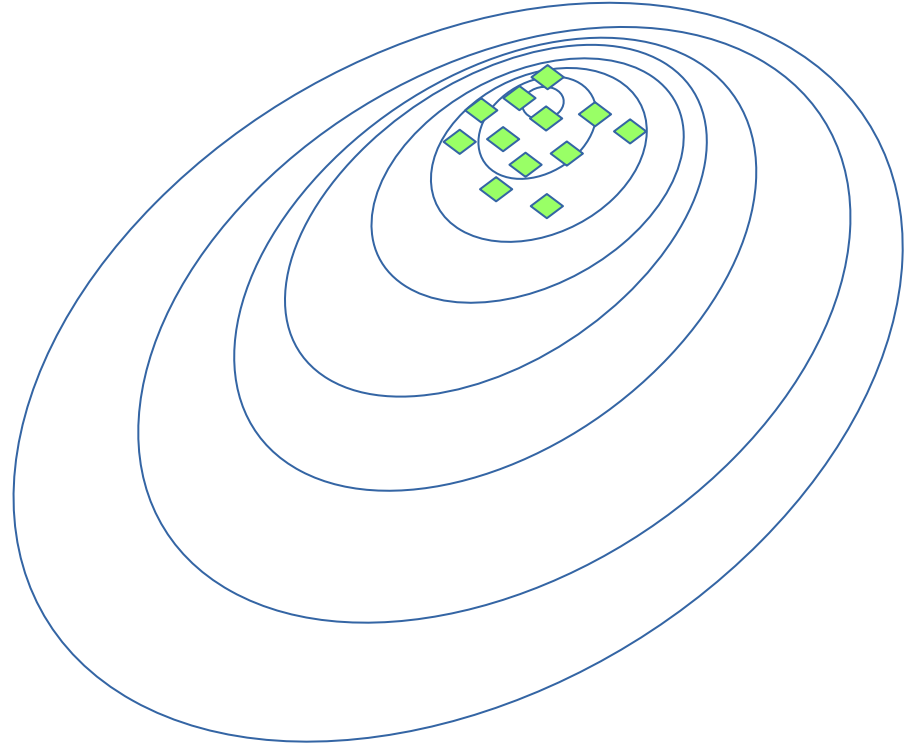
Cross-entropy method: illustration



Cross-entropy method: illustration



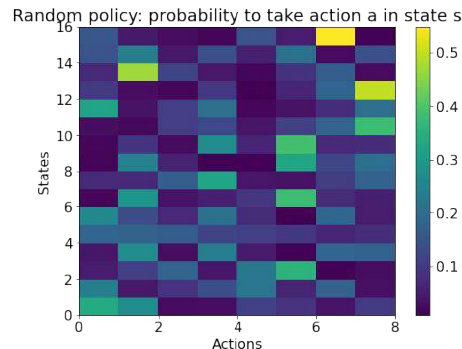
Cross-entropy method: illustration



Cross-entropy method: tabular case

- Policy is a matrix

$$\pi(a|s) = A_{s,a} \longleftrightarrow$$



- Sample N games with this policy
- Select M **elite** sessions with highest rewards

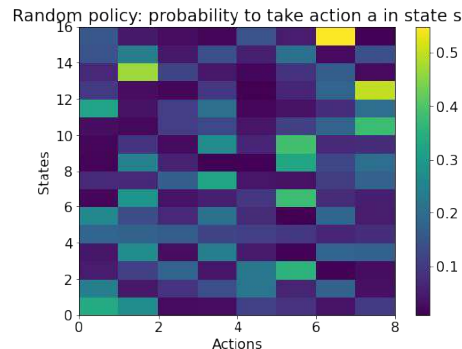
$$\text{Elite} = [(s_0, a_0), (s_1, a_1), \dots, (s_M, a_M)]$$

- Update policy:
$$\pi_{\text{new}}(a|s) = \frac{\sum_{s_t, a_t \in \text{Elite}} [s_t = s][a_t = a]}{\sum_{s_t, a_t \in \text{Elite}} [s_t = s]}$$

Cross-entropy method: tabular case

- Policy is a matrix

$$\pi(a|s) = A_{s,a} \longleftrightarrow$$



- Sample N games with this policy
- Select M **elite** sessions with highest rewards
- Update policy using the **elite** sessions:

$$\pi_{\text{new}}(a|s) = \frac{\text{how many times took action } a \text{ at state } s}{\text{how many times was at state } s}$$

Harsh reality



Some environments have huge or infinite number of states

How to fix it?

Approximate cross-entropy method

- Model (e.g. parametric) predicts action probability given state:

$$\pi(a|s) = f_{\theta}(a, s)$$

Random Forest Classifier,

`model = RandomForestClassifier()` Logistic Regression, NN etc.

- Sample N sessions, select M **elite** sessions

$$\text{Elite} = [(s_0, a_0), (s_1, a_1), \dots, (s_M, a_M)]$$

New training set; states are objects,
actions are target values

- Maximize likelihood of actions in elite sessions:

$$\pi(a|s)_{\text{new}} = \arg \max_{\pi} \sum_{s_t, a_t \in \text{Elite}} \log \pi(a_t | s_t)$$

`model.fit(elite_states, elite_actions)`

What if action space
is continuous?

Approximate cross-entropy method



- Model samples actions from some appropriate distribution:

$$\pi(a|s) = \mathcal{N}(\mu_{\theta}(a, s), \sigma_{\gamma}(a, s))$$

One model

Another model (or constant)

It is just a regressor!

What if action space is continuous? Approximate cross-entropy method

- Model (e.g. parametric) predicts action given state:

```
model = RandomForestRegressor()
```

- Sample N sessions, select M **elite** sessions

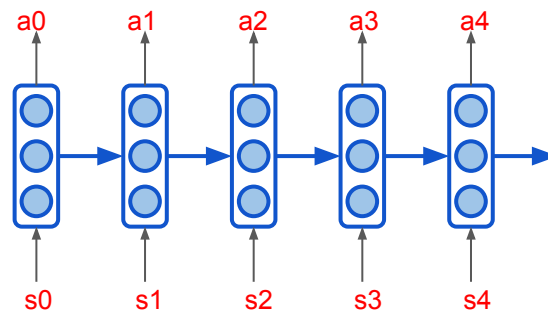
$$\text{Elite} = [(s_0, a_0), (s_1, a_1), \dots, (s_M, a_M)]$$

- Maximize likelihood of actions in elite sessions:

```
model.fit(elite_states, elite_actions)
```

Useful ideas

- Use elite sessions from several (3-5) past iterations for training
 - Experience from previous iterations is preserved
 - Convergence may be slower (e.g. on simple environments)
- Regularize the policy with entropy
 - Low entropy means weak exploration
- Sessions can be sampled in parallel
- Agent can use memory as well
 - We will meet RNNs again soon



Key differences

Supervised Learning

- Learn to approximate reference answers
- Need reference answers
- Model does not affect the input data

Reinforcement Learning

- Learn optimal strategy by trial and error
- Need feedback on agent's actions
- Agent actions affect the environment (so the observations)

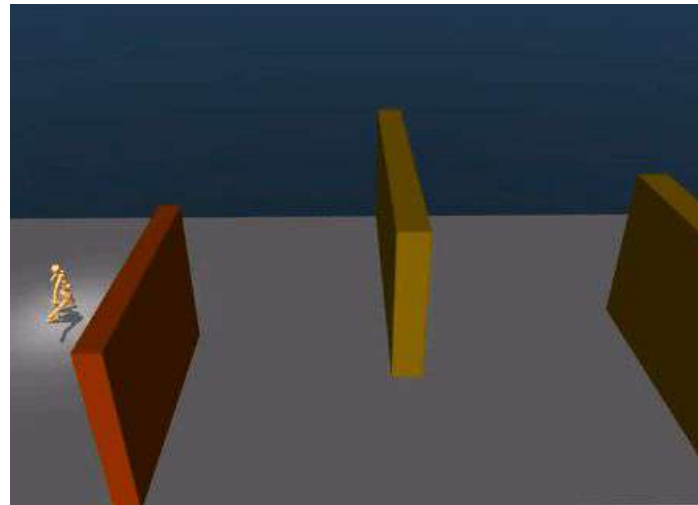
Unsupervised Learning

- Learn underlying data structure
- No feedback required
- Model does not affect the input data

Reinforcement Learning

- Learn optimal strategy by trial and error
- Need feedback on agent's actions
- Agent actions affect the environment (so the observations)

- RL is different both from Supervised and Unsupervised learning
- Reward formulation has huge effect on the agent behaviour
- Remember the Markov assumptions
- The cross-entropy method is simple and still very powerful approach

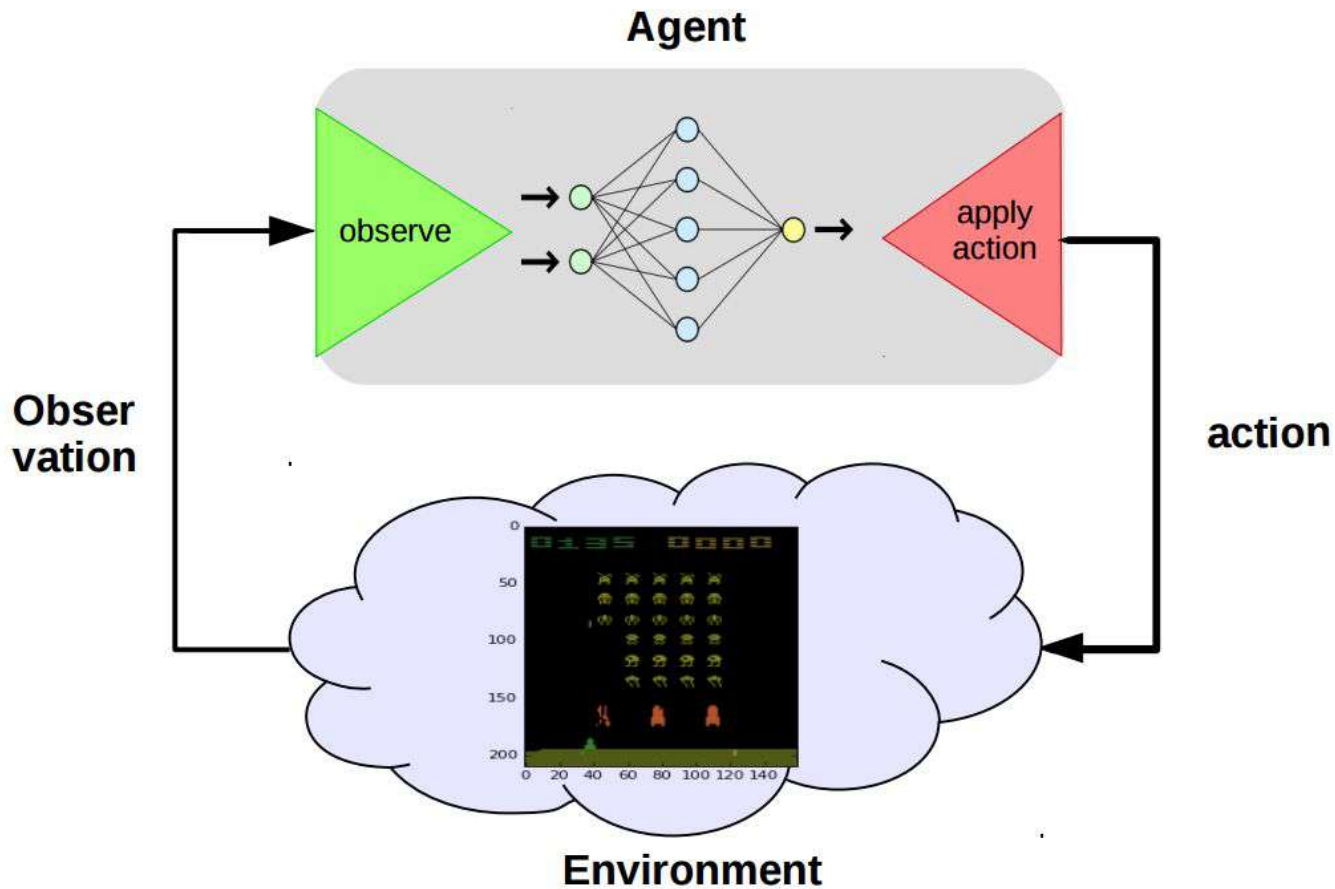


source: [Emergence of Locomotion Behaviours in Rich Environments](#),
DeepMind

References

These slides are almost the exact copy of Practical RL course week 4 slides.
Special thanks to YSDA team for making them publicly available.

Original slides link: [week04_approx_rl](#)



Basic definitions

$$G_t = \sum_{t'=t}^T \gamma^{(t'-t)} r_{t'}$$

$$Q^\pi(s, a) = E_\pi[G_t | s_t = s, a_t = a]$$

$$V^\pi(s) = E_\pi[G_t | s_t = s]$$

Recurrent relations

$$Q^\pi(s, a) = E_{s_{t+1}}[r_t + \gamma V^\pi(s_{t+1})]$$

$$Q^\pi(s, a) = E_{s_{t+1}, a_{t+1} \sim \pi}[r_t + \gamma Q^\pi(s_{t+1}, a_{t+1})]$$

Optimal policy

For all π, s, a : $Q^{\pi^*}(s, a) \geq Q^{\pi}(s, a)$

$$\pi^*(s) = \operatorname{argmax}_a Q^{\pi^*}(s, a)$$

Bellman optimality equation

$$Q^*(s_t, a) = E_{s_{t+1}}[r_t + \max_{a'} Q^*(s_{t+1}, a')]$$

Q-learning

Training step

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

Q-learning as MSE minimization

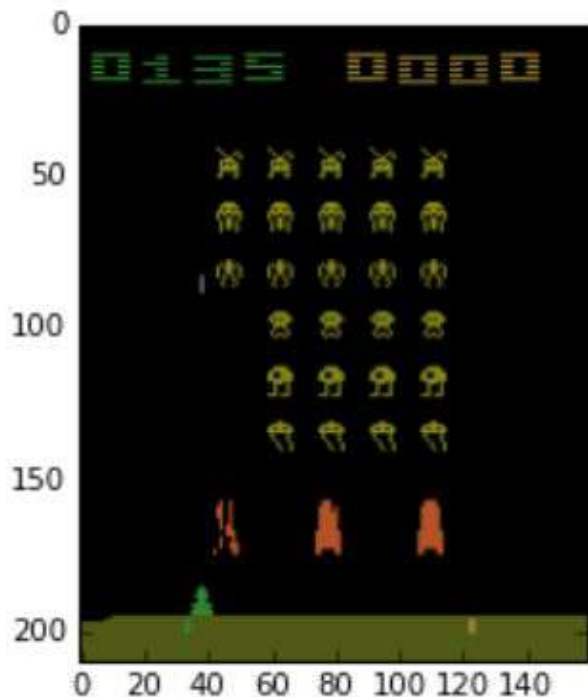
$$L = (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))^2$$

Const

$$\nabla L = 2 \cdot (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

What's wrong here?

Real world



How many states are there?
approximately

$$|S| = 2^{210 \cdot 160 \cdot 8 \cdot 3}$$

Problem

State space is usually large,
sometimes continuous.

Two solutions:

- Binarize state space (last week)
- Approximate agent with a function (crossentropy method)

Which one would you prefer for atari?

Problem

State space is usually large,
sometimes continuous.

And so is action space;

Two solutions:

- Binarize state space  Too many bins or handcrafted features
- Approximate agent with a function  Let's pick this one

From tables to approximations

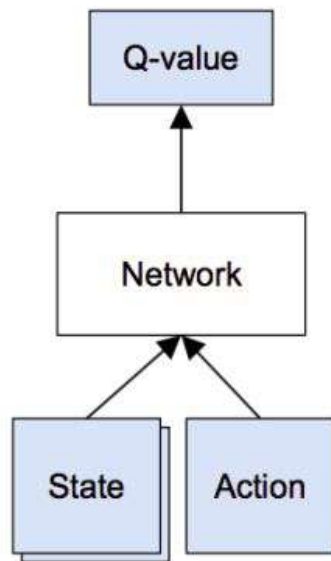
- Before:
 - For all states, for all actions, remember $Q(s,a)$
- Now:
 - Approximate $Q(s,a)$ with some function
 - e.g. linear model over state features

$$\operatorname{argmin}_{w,b} (Q(s_t, a_t) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')])^2$$

Question: should we use **classification** or **regression** model?
(e.g. logistic regression Vs linear regression)

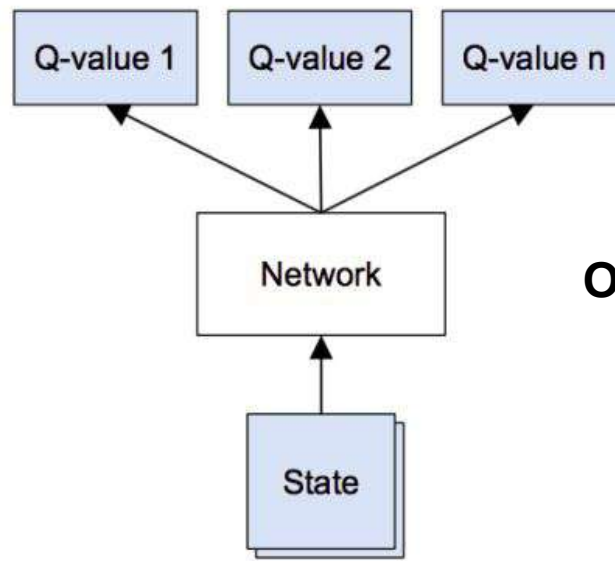
Possible architectures

**Continuous
control or large
number of
actions**



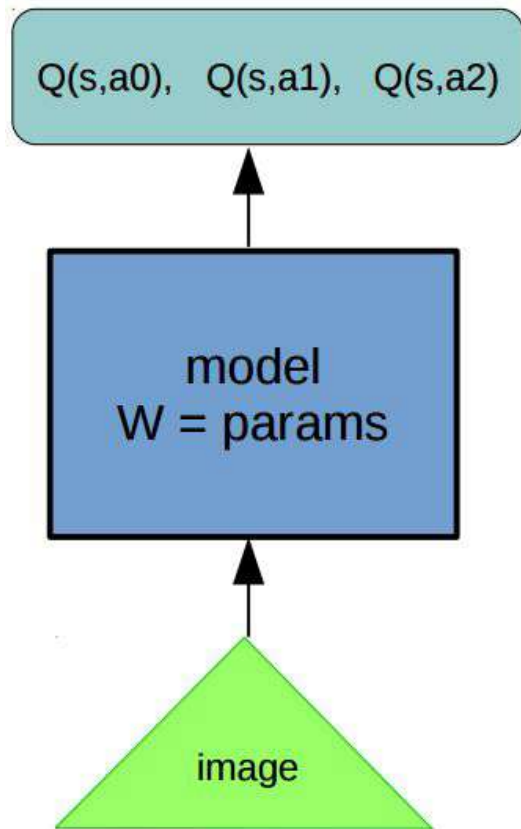
Given **(s,a)**
Predict $Q(s,a)$

**One pass for all
actions**



Given **s** predict all q-values
 $Q(s,a_0)$, $Q(s,a_1)$, $Q(s,a_2)$

Approximate Q-learning



Q-values:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$$

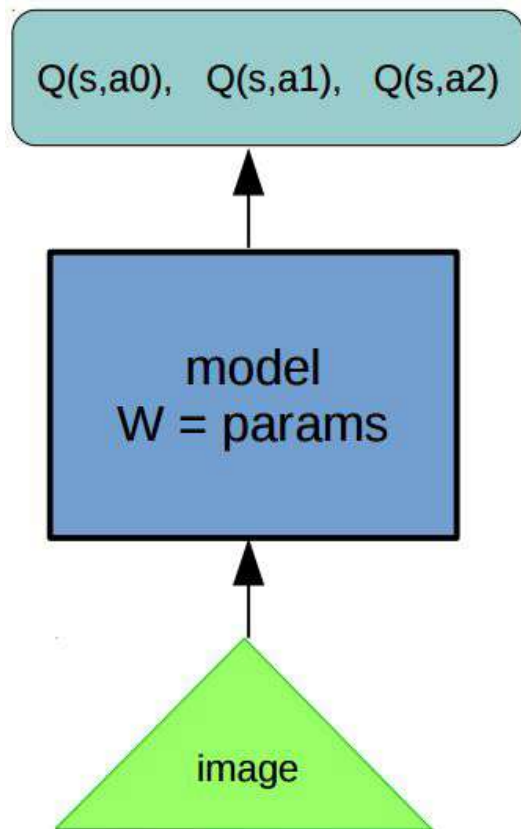
Objective:

$$L = (Q(s_t, a_t) - [r + \gamma \cdot \max_{a'} \underbrace{Q(s_{t+1}, a')}_{\text{Const}}])^2$$

Gradient step:

$$w_{t+1} = w_t - \alpha \cdot \frac{\delta L}{\delta w}$$

Approximate Q-learning



Objective:

$$L = \left(Q(s_t, a_t) - \hat{Q}(s_t, a_t) \right)^2$$

consider const

Q-learning:

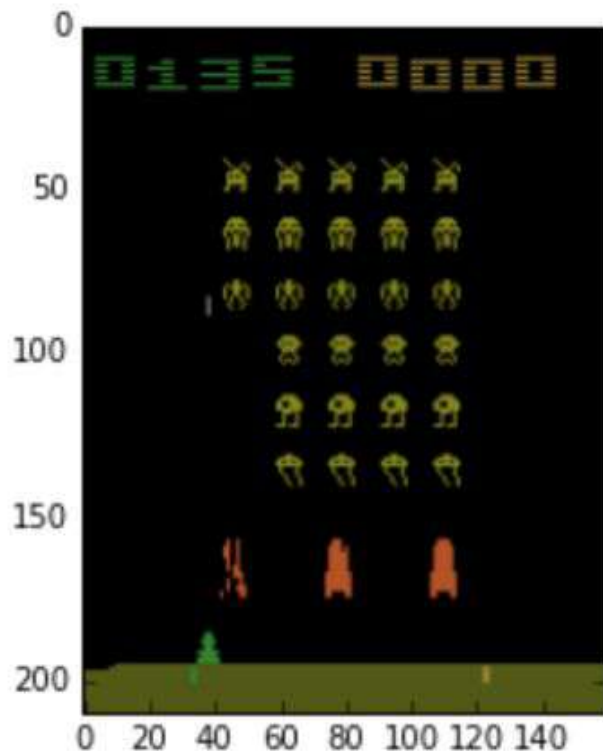
$$\hat{Q}(s_t, a_t) = r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$$

SARSA:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot Q(s_{t+1}, a_{t+1})$$

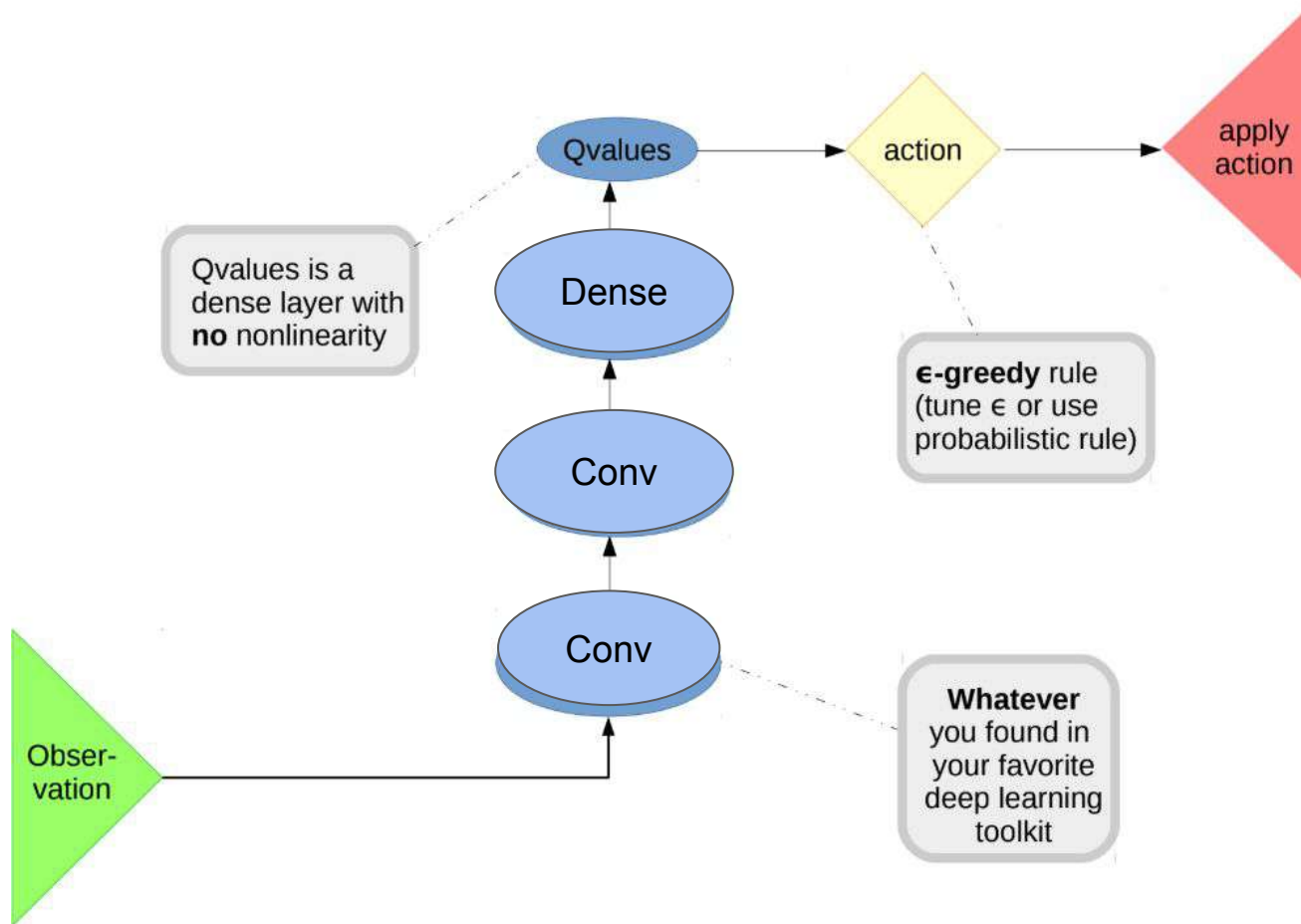
Expected Value SARSA:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot E_{a' \sim \pi(a|s)} Q(s_{t+1}, a')$$

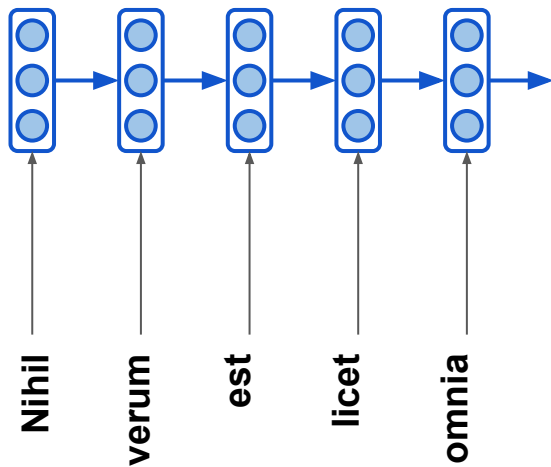


What kind of network digests images well?

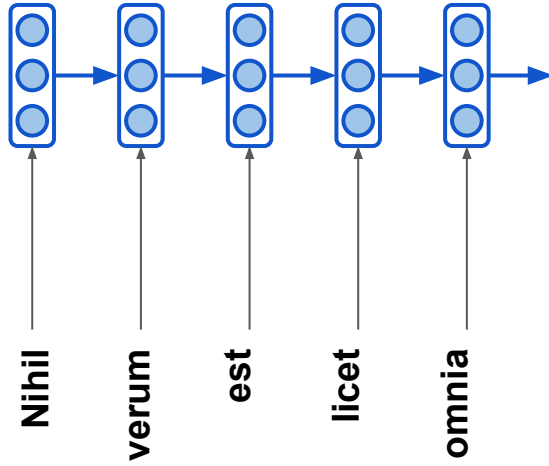
Basic deep Q-learning



MDP formalism

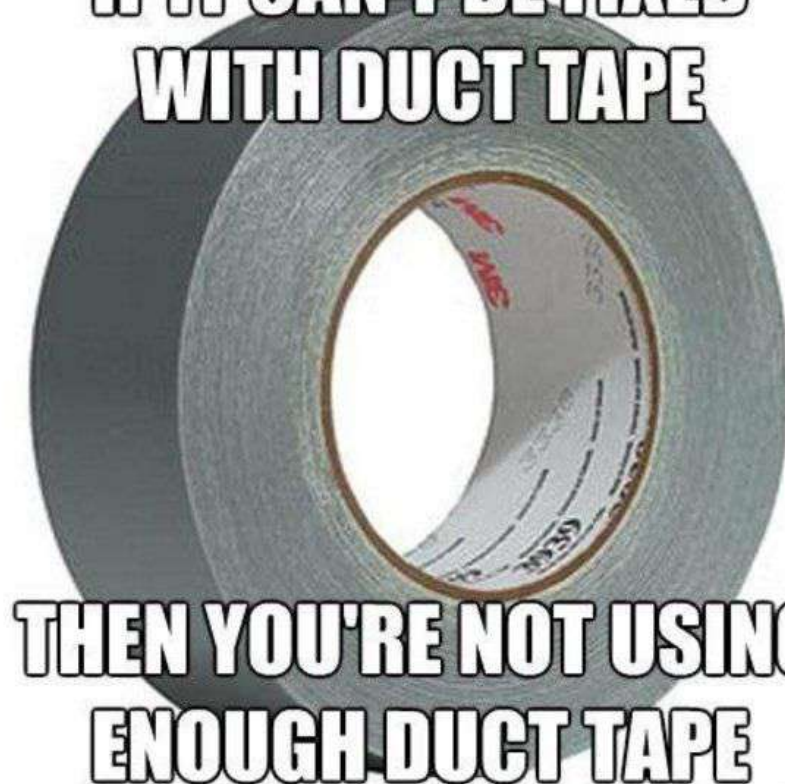


RNNs and dropout



Duct tape approach

**IF IT CAN'T BE FIXED
WITH DUCT TAPE**



**THEN YOU'RE NOT USING
ENOUGH DUCT TAPE**

zigmema

Lecture 10: Bellman equations

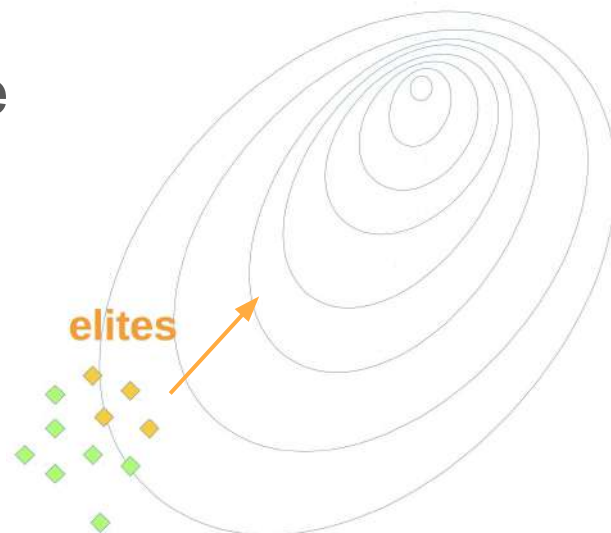
Radoslav Neychev

Credits: these lecture is deeply based on [Practical RL course](#) week 2 lecture by Shvechikov Pavel.

Special thanks to YSDA team for making the materials publicly available.

Recap of the previous lecture:

- The MDP formalism
 - State, Action, Reward, next State
- Cross-Entropy Method (CEM)
 - easy to implement, good results
 - rich theoretical background
 - black box
 - no knowledge of environment
 - no knowledge of intermediate rewards



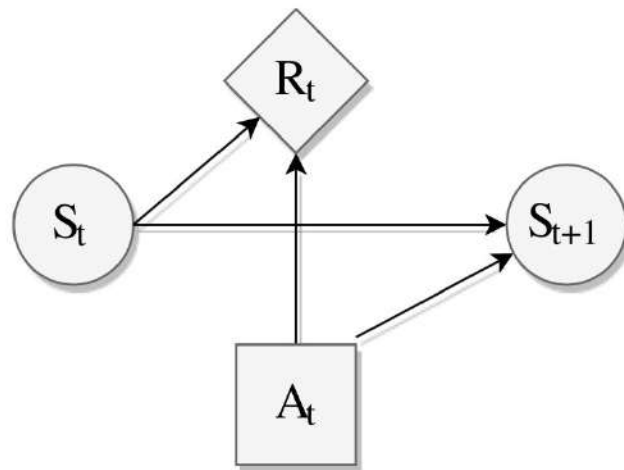
Improve on the CEM → dive into the black box

Given dynamics, how to find an optimal policy?

Definition of Markov Decision Process

MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$, where

- ① $\mathcal{S}$ – set of states of the world
- ② $\mathcal{A}$ – set of actions
- ③ $\mathcal{P} : \mathcal{S} \times \mathcal{A} \mapsto \Delta(\mathcal{S})$ – state-transition function, giving us $p(s_{t+1} | s_t, a_t)$
- ④ $\mathcal{R} : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ – reward function, giving us $\mathbb{E}_R [R(s_t, a_t) | s_t, a_t]$.



Markov property

$$p(r_t, s_{t+1} | s_0, a_0, r_0, \dots, s_t, a_t) = p(r_t, s_{t+1} | s_t, a_t)$$

(next state, expected reward) depend on (previous state, action)

Goal: solve an MDP by finding **an** optimal policy

1. What is the objective?
 - a. Reward: discounting and design
 - b. Expected objective: state- and action-value function
2. How to evaluate the objective?
 - a. Bellman **expectation** equations
3. How to improve the objective?
 - a. Bellman **optimality** equations
4. Combine evaluation and improvement:
 - a. Generalized Policy Iteration

Explaining goals to agent through reward

Reward hypothesis (R.Sutton)

Goals and purposes can be thought of as the maximization of the expected value of the cumulative sum of a received scalar signal

Explaining goals to agent through reward

Reward hypothesis (R.Sutton)

Goals and purposes can be thought of as the maximization of the expected value of the cumulative sum of a received scalar signal

Cumulative reward is called a return:

$$G_t \triangleq R_t + R_{t+1} + R_{t+2} + \dots + R_T$$

E.g.: reward in **chess** – value of taken opponent's piece

Explaining goals to agent through reward

Reward hypothesis (R.Sutton)

Goals and purposes can be thought of as the maximization of the expected value of the cumulative sum of a received scalar signal

Cumulative reward is called a return:

$$\begin{array}{c} \text{end of an episode} \end{array} \quad \begin{array}{c} \text{immediate reward} \end{array}$$

The diagram shows the equation $G_t \triangleq R_t + R_{t+1} + R_{t+2} + \dots + R_T$. A blue box highlights G_t , with a blue arrow pointing to it from the left. A green box highlights R_t , with a green arrow pointing to it from below and the text "immediate reward" in green. A red box highlights R_T , with a red arrow pointing to it from the right and the text "end of an episode" in red. A red bracket connects the R_T box back to the start of the summation.

E.g.: reward in **chess** – value of taken opponent's piece

E.g.: data center non-stop cooling system

- **States** – temperature measurements
- **Actions** – different fans speed
- **R = 0** for exceeding temperature thresholds
- **R = +1** for each second system is cool

What could go wrong with such a design?

E.g.: data center non-stop cooling system

- **States** – temperature measurements
- **Actions** – different fans speed
- **R = 0** for exceeding temperature thresholds
- **R = +1** for each second system is cool

What could go wrong with such a design?

Infinite return for **non optimal** behaviour!

$$G_t = 1 + 1 + 0 + 1 + 1 + 0 + \dots = \sum_{t=1}^{\infty} R_t = \infty$$

E.g.: moving to destination



- **State** – position, velocities of joints
- **Actions** – actuator forces to joints
- **$R = \max(0, d(x, B) - d(x', B))$**

What could go wrong with such a design?

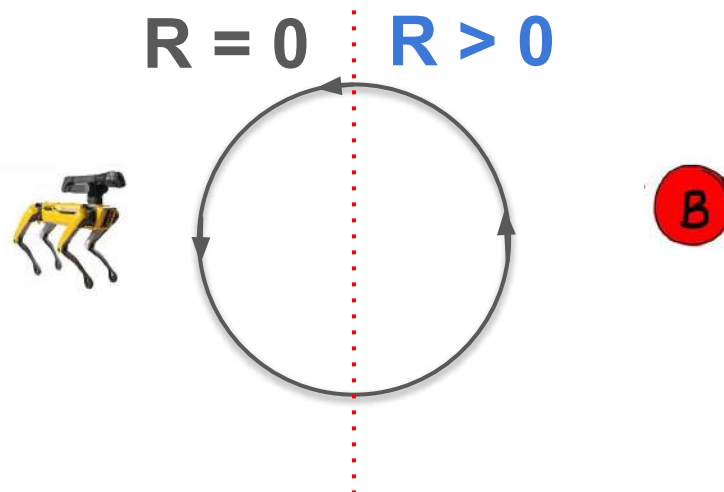
E.g.: moving to destination



- **State** – position, velocities of joints
- **Actions** – actuator forces to joints
- **$R = \max(0, d(x, B) - d(x', B))$**

What could go wrong with such a design?

**Positive
feedback
loop!**



Reward discounting

Reward discounting

Get rid of infinite sum by **discounting** $0 \leq \gamma < 1$

$$G_t \triangleq R_t + \underset{\substack{\text{discount factor} \quad \nearrow}}{\gamma} R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The same cake compared to today's one worth

- γ times less tomorrow
- γ^2 times less the day after tomorrow



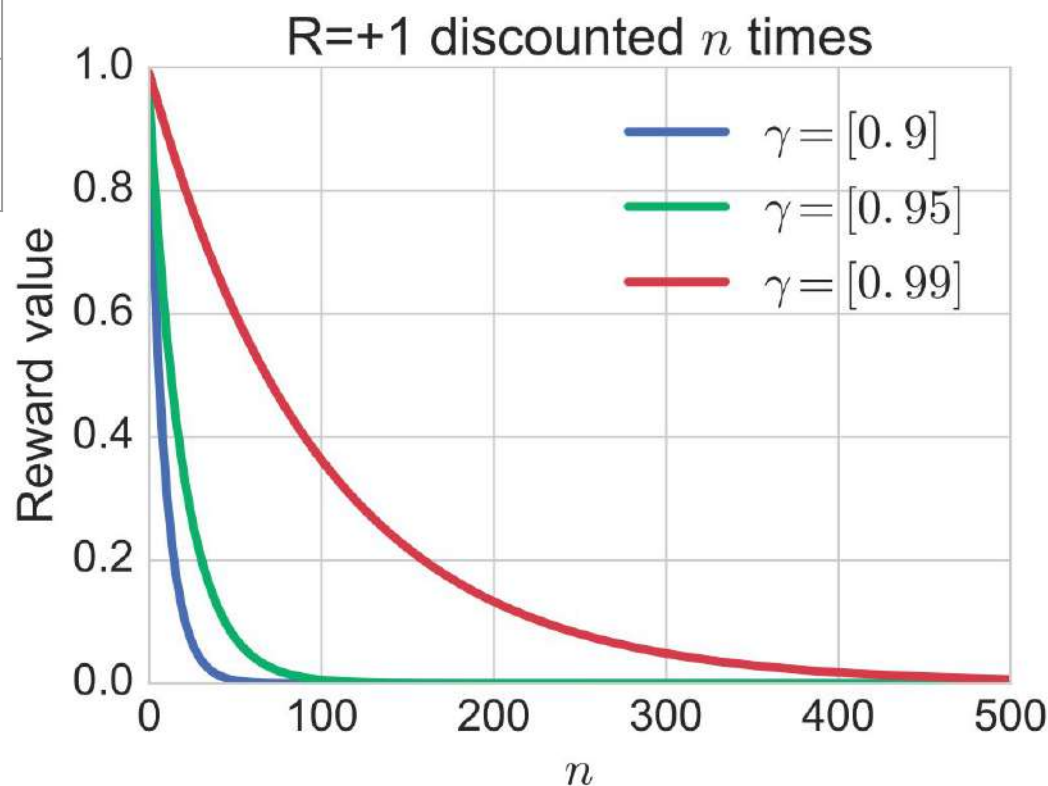
γ will eat it day by day

Discounting makes sums finite

Maximal return for **R = +1**

$$G_0 = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}$$

γ	0.9	0.95	0.99
$\frac{1}{1-\gamma}$	10	20	100



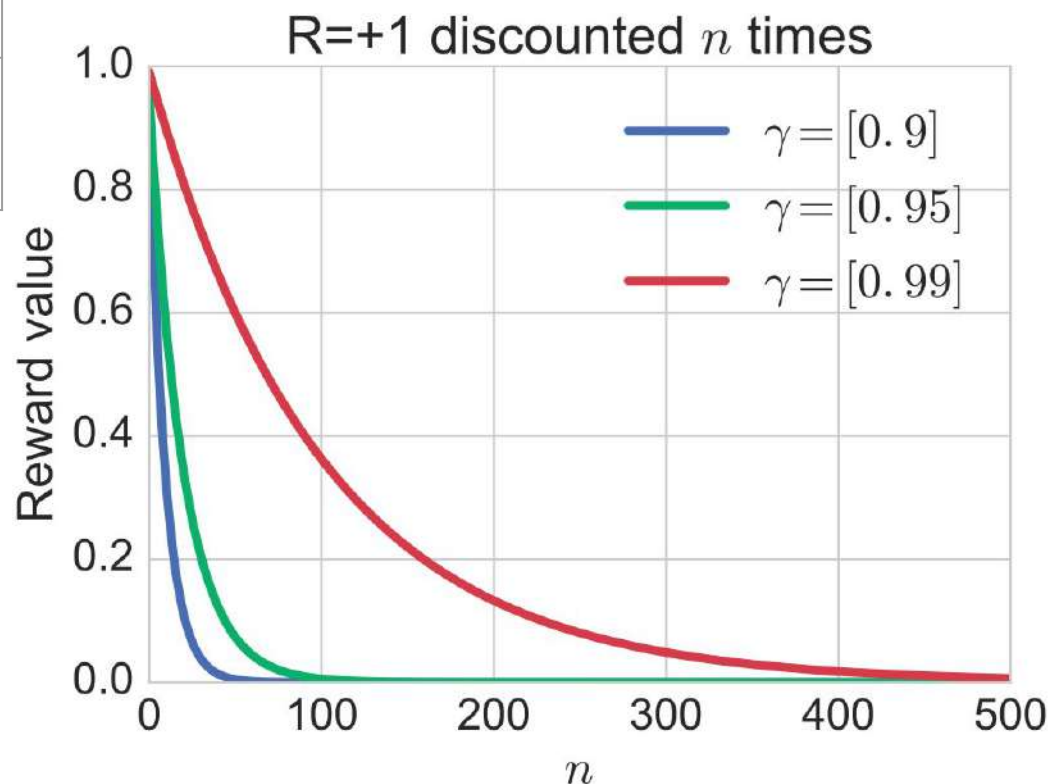
Discounting makes sums finite

Maximal return for **R = +1**

$$G_0 = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}$$

γ	0.9	0.95	0.99
$\frac{1}{1-\gamma}$	10	20	100

Any **discounting**
changes optimisation
task and its solution!



Discounting is inherent to humans

- Quasi-hyperbolic $f(t) = \beta\gamma^t$
- Hyperbolic discounting $f(t) = \frac{1}{1 + \beta t}$

Discounting is inherent to humans

- Quasi-hyperbolic $f(t) = \beta\gamma^t$
- Hyperbolic discounting $f(t) = \frac{1}{1 + \beta t}$

Mathematical convenience

$$\begin{aligned} G_t &= R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \dots) \\ &= \boxed{R_t + \gamma G_{t+1}} \end{aligned}$$

Remember this one!
We will need it later

Discounting is a stationary end-of-effect model

Any action affects (1) immediate reward (2) next state

Discounting is a stationary end-of-effect model

Any action affects (1) immediate reward (2) next state

Action indirectly affects future rewards 

But how long does this effect lasts?

$$\begin{aligned} G_0 &= R_0 + \gamma R_1 + \gamma^2 R_2 + \dots + \gamma^T R_T \\ &= (1 - \gamma) R_0 \\ &\quad + (1 - \gamma) \gamma (R_0 + R_1) \\ &\quad + (1 - \gamma) \gamma^2 (R_0 + R_1 + R_2) \\ &\quad \dots \\ &\quad + \gamma^T \cdot \sum_{t=0}^T R_t \end{aligned}$$

G is expected return under stationary end-of-effect model

Discounting is a stationary end-of-effect model

Any action affects (1) immediate reward (2) next state

Action indirectly affects future rewards _____↑

But how long does this effect lasts?

$$\begin{aligned} G_0 &= R_0 + \gamma R_1 + \gamma^2 R_2 + \dots + \gamma^T R_T \\ &= \boxed{(1 - \gamma)} R_0 + \boxed{(1 - \gamma)} \gamma (R_0 + R_1) \\ &\quad + \boxed{(1 - \gamma)} \gamma^2 (R_0 + R_1 + R_2) \\ &\quad \dots \\ &\quad + \gamma^T \cdot \sum_{t=0}^T R_t \end{aligned}$$

“End of effect” probability → → →

“Effect continuation” probability →

G is expected return under stationary end-of-effect model

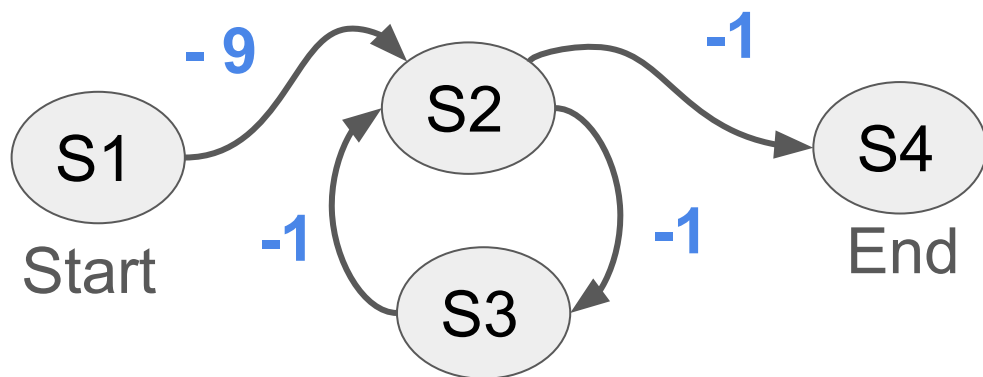
Reward design – don't shift, reward for WHAT

- E.g.: chess – value of taken opponent's piece
 - Problem: agent will not have a desire to win!
- E.g.: moving to destination
 - Problem: agent will not bother about the goal!

Reward design – don't shift, reward for WHAT

- **E.g.:** chess – value of taken opponent's piece
 - **Problem:** agent will not have a desire to win!
- **E.g.:** moving to destination
 - **Problem:** agent will not bother about the goal!

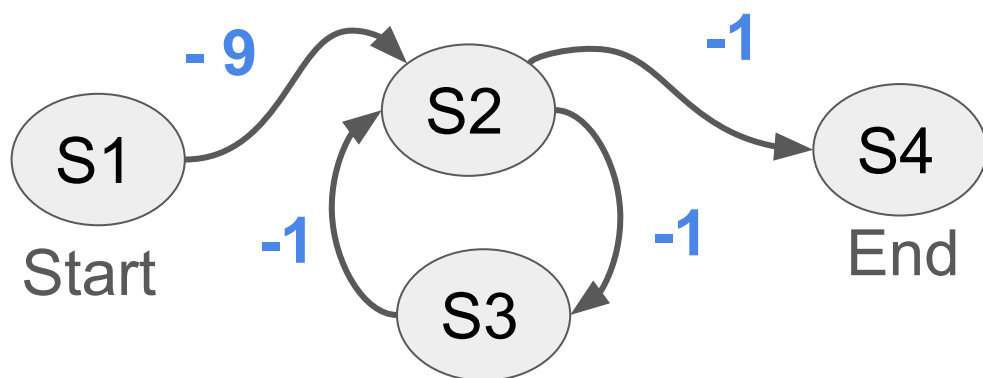
Take away: reward only for **WHAT**, but never for **HOW**



Reward design – don't shift, reward for WHAT

- **E.g.:** chess – value of taken opponent's piece
 - **Problem:** agent will not have a desire to win!
- **E.g.:** moving to destination
 - **Problem:** agent will not bother about the goal!

Take away: reward only for **WHAT**, but never for **HOW**



Take away: do not **subtract** mean from rewards

Faulty reward functions

- Reward for ball possession in soccer
 - Vibrating near the ball
- Cyclic behaviours



Reward design – scaling, shaping

What transformations do not change optimal policy?

- Reward **scaling** – division by positive constant
 - May be useful in practise for approximate methods

Reward design – scaling, shaping

What transformations do not change optimal policy?

- Reward **scaling** – division by positive constant
 - May be useful in practise for approximate methods
- Reward **shaping** – add a **potential-based shaping function** $F(s, a, s')$:

$$R'(s, a, s') = R(s, a, s') + F(s, a, s')$$

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

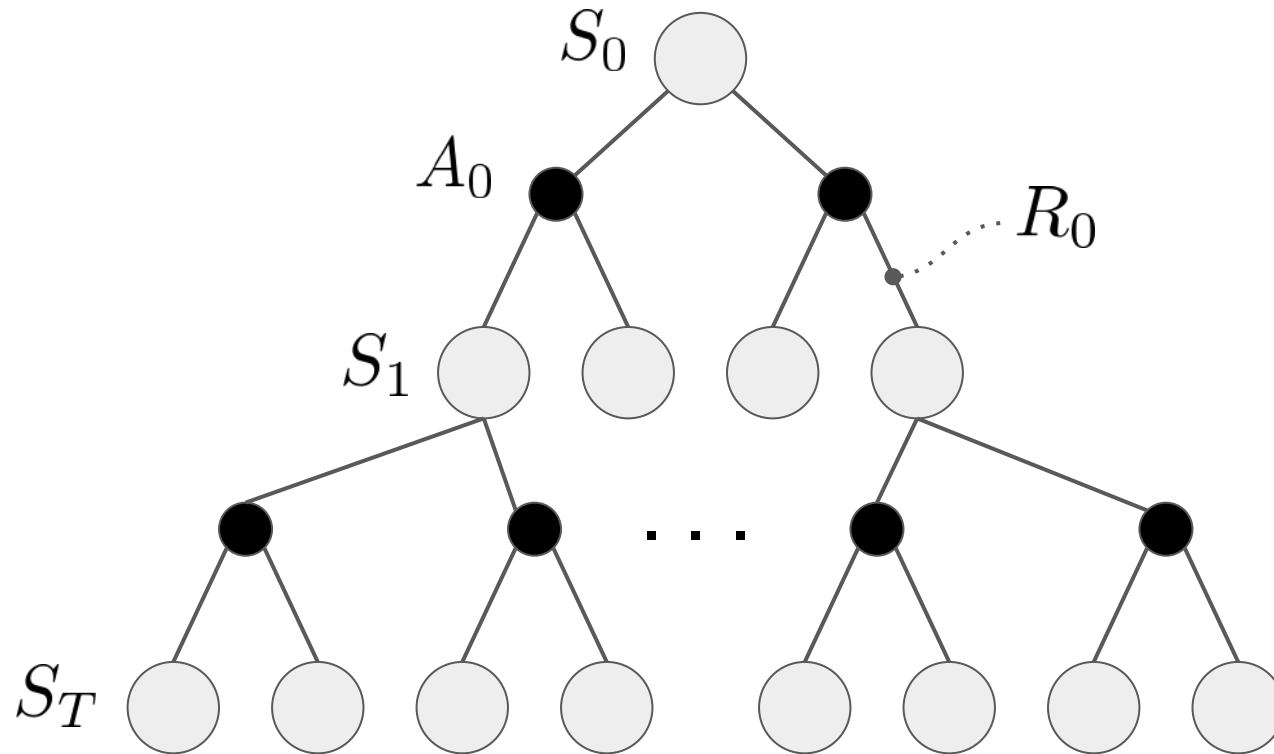
Intuition: when no discounting F adds as much as it subtracts from the total return

Expected objective

Optimal policy maximizes **expected** return

$$\begin{aligned}\mathbb{E}[G_0] &= \mathbb{E}[R_0 + \gamma R_1 + \dots + \gamma^T R_T] \\&= \mathbb{E}_{E, \pi_\theta}[G_0] \\&= \mathbb{E}_{\pi_\theta}[G_0] \\&= \mathbb{E}[G_0 \mid \pi_\theta] \\&= \mathbb{E}_{\substack{s_{0:T} \\ a_{0:T}}}[G_0] \\&= \mathbb{E}_{s_0} \left[\mathbb{E}_{a_0|s_0} \left[R_0 + \mathbb{E}_{s_1|s_0, a_0} \left[\mathbb{E}_{a_1|s_1} [\gamma R_1 + \dots] \right] \right] \right] \\&= \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim p_\theta} [\gamma^t R_t] \\&= \mathbb{E}_{\tau \sim p_\theta(\tau)} [G(\tau)] \quad \tau \triangleq (s_0, a_0, s_1, \dots, a_{T-1}, s_T) \\&\quad p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t)\end{aligned}$$

Backup Tree: how to find an optimal policy?



State- and **Action-**value functions

State-value function $v(s)$

$v(s)$ is expected **return** conditional on state:

$$\begin{aligned} v_{\pi}(s) &\triangleq \mathbb{E}_{\pi} [G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi} [R_t + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \mathbb{E}_{\pi} [G_{t+1} \mid S_{t+1} = s'] \right] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

Intuition: value of following policy π from state s

State-value function $v(s)$

$v(s)$ is expected **return** conditional on state:

— stochasticity in policy & environment

$$\begin{aligned} v_{\pi}(s) &\triangleq \mathbb{E}_{\pi}[G_t | S_t = s] \\ &= \mathbb{E}_{\pi}[R_t + \gamma G_{t+1} | S_t = s] \\ &= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) \left[r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] \right] \\ &= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

Intuition: value of following policy π from state s

State-value function $v(s)$

$v(s)$ is expected **return** conditional on state:

stochasticity in policy & environment

$$v_{\pi}(s) \triangleq \mathbb{E}_{\pi} [G_t | S_t = s]$$

Environment stochasticity

$$= \mathbb{E}_{\pi} [R_t + \gamma G_{t+1} | S_t = s]$$

Policy stochasticity

$$= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) \left[r + \gamma \mathbb{E}_{\pi} [G_{t+1} | S_{t+1} = s'] \right]$$

$$= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

Intuition: value of following policy π from state s

State-value function $v(s)$

$v(s)$ is expected **return** conditional on state:

$$\begin{aligned} v_{\pi}(s) &\triangleq \mathbb{E}_{\pi} [G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi} [R_t + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \mathbb{E}_{\pi} [G_{t+1} \mid S_{t+1} = s'] \right] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

Diagram annotations:

- Red line from **stochasticity in policy & environment** points to the $\mathbb{E}_{\pi}$ operator in the first equation.
- Blue line from **Environment stochasticity** points to the $\mathbb{E}_{\pi}$ operator in the second equation.
- Blue line from **Policy stochasticity** points to the $\sum_a \pi(a \mid s)$ term in the third equation.
- Red line from **By definition** points to the $\mathbb{E}_{\pi} [G_{t+1} \mid S_{t+1} = s']$ term in the third equation.
- Red line from **By definition** points to the $v_{\pi}(s')$ term in the fourth equation.

Intuition: value of following policy π from state s

Action-value function $q(s, a)$

Is expected **return** conditional on state and action:

Intuition: value of following policy π after committing action **a** in state **s**

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi} [R_t + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \mathbb{E}_{\pi} [G_{t+1} \mid S_{t+1} = s'] \right] \\ &= \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

Action-value function $q(s, a)$

Is expected **return** conditional on state and action:

Intuition: value of following policy π after committing action **a** in state **s**

No policy
stochasticity at
first step

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi} [R_t + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \mathbb{E}_{\pi} [G_{t+1} \mid S_{t+1} = s'] \right] \\ &= \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

Relations between $v(s)$ and $q(s,a)$

We already know how to write $q(s,a)$ in terms of $v(s)$

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

What about $v(s)$ in terms of $q(s,a)$?

Relations between $v(s)$ and $q(s,a)$

We already know how to write $q(s,a)$ in terms of $v(s)$

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

What about $v(s)$ in terms of $q(s,a)$?

$$\begin{aligned} v_{\pi}(s) &= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \\ &= \sum_a \pi(a | s) q_{\pi}(s, a) \end{aligned}$$

So, we could now write $q(s, a)$ in terms of $q(s,a)$!

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right]$$

Bellman expectation equations

Bellman **expectation** equations

For $v(s)$:

$$\begin{aligned} v_{\pi}(s) &= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \\ &= \mathbb{E}_{\pi} [R_t + \gamma v_{\pi}(S_{t+1}) | S_t = s] \end{aligned}$$

For $q(s, a)$:

$$\begin{aligned} q_{\pi}(s, a) &= \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \\ &= \sum_{r, s'} p(r, s' | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right] \end{aligned}$$

What do we gonna do with value functions?

Already know

- Return, value- and action-value functions
- Bellman equations – assess policy performance

Optimal policy makes

- best actions in each possible state

But how to know which policy **is better**?

How to compare them?

Bellman optimality equations

Optimal policy is the one with the largest $v(s)$

We could compare policies on the basis of $v(s)$

$$\pi \geq \pi' \iff v_\pi(s) \geq v_{\pi'}(s) \quad \forall s$$

Best policy π_* is better or equal to any other policy

Use optimal policy from s



$$v_*(s) = \max_{\pi} v_\pi(s)$$

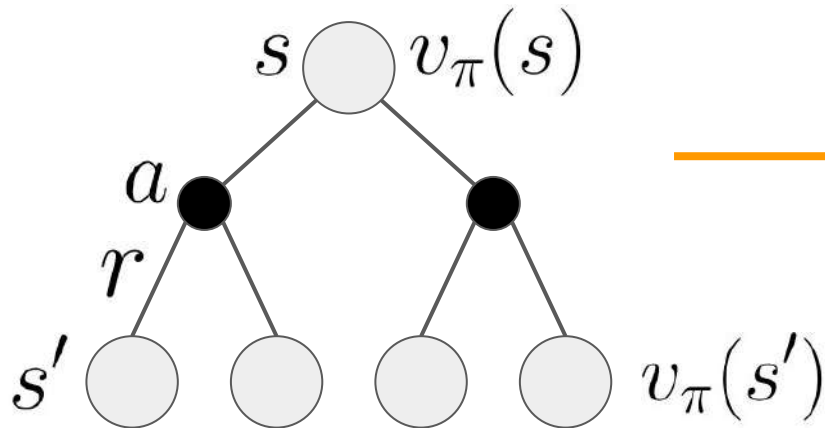
$$q_*(s, a) = \max_{\pi} q_\pi(s, a)$$



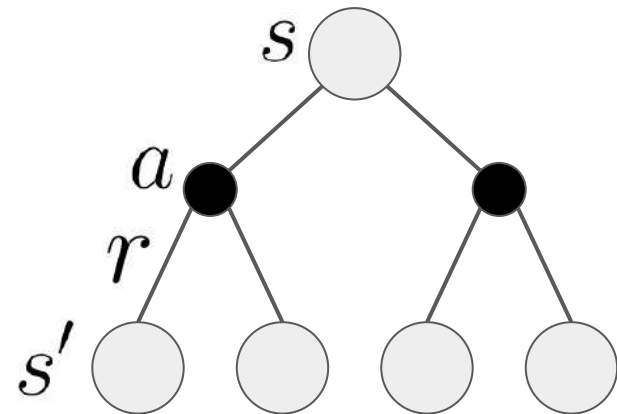
Commit action a , and **afterwards** use optimal policy

In any finite MDP there is
always **at least one**
deterministic optimal policy

Bellman **optimality** equation for $v(s)$

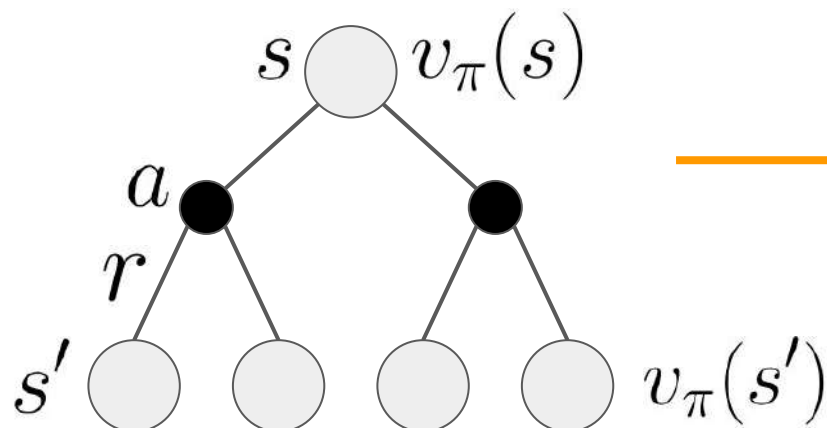


Bellman **expectation**
equation for $v(s)$

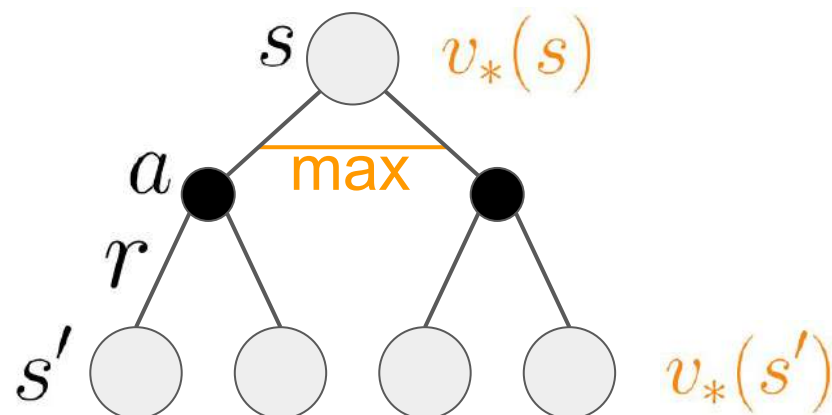


Bellman **optimality**
equation for $v_*(s)$

Bellman **optimality** equation for $v(s)$

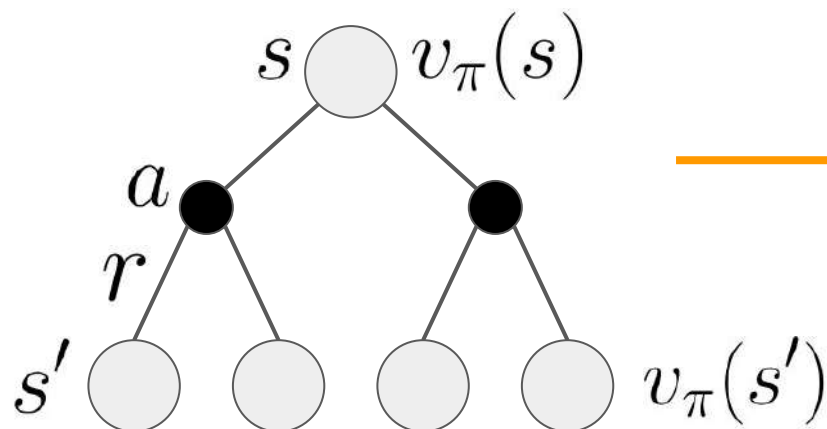


Bellman **expectation**
equation for $v(s)$

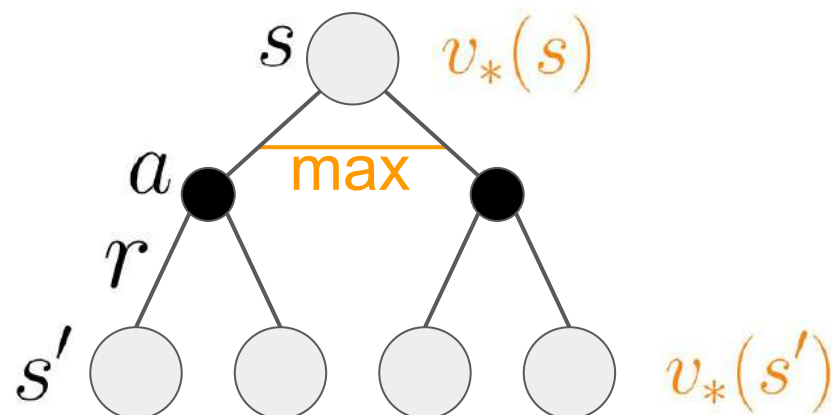


Bellman **optimality**
equation for $v_*(s)$

Bellman **optimality** equation for $v(s)$



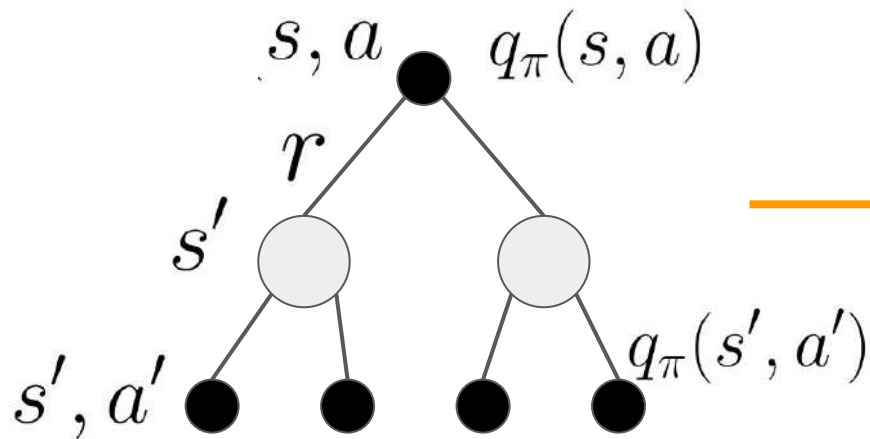
Bellman **expectation**
equation for $v(s)$



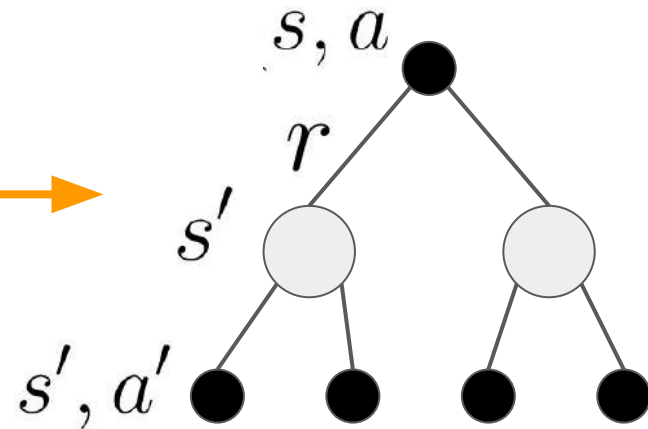
Bellman **optimality**
equation for $v_*(s)$

$$\begin{aligned} v_*(s) &= \max_a \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_*(s')] \\ &= \max_a \mathbb{E} [R_t + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] \end{aligned}$$

Bellman **optimality** equation for $q(s,a)$

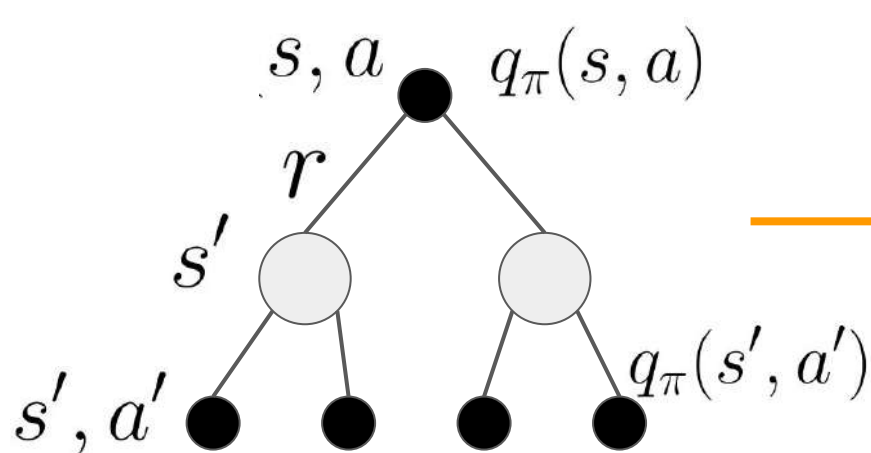


Bellman **expectation**
equation for $q(s,a)$

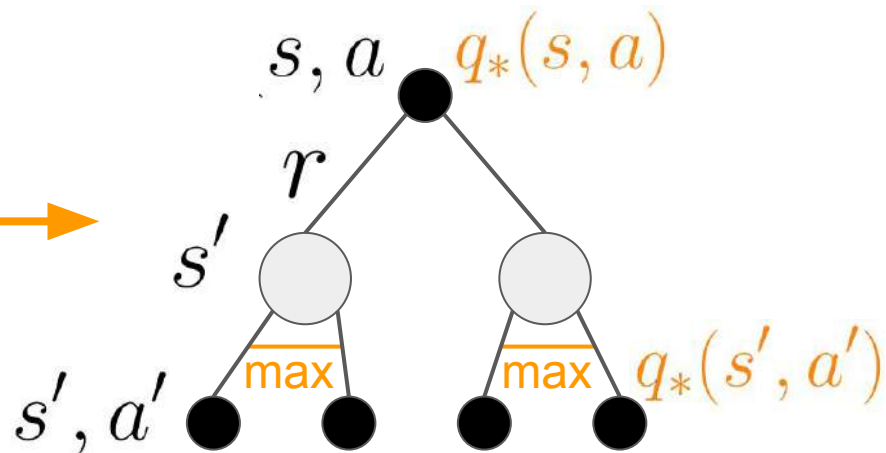


Bellman **optimality**
equation for $q_*(s, a)$

Bellman **optimality** equation for $q(s,a)$

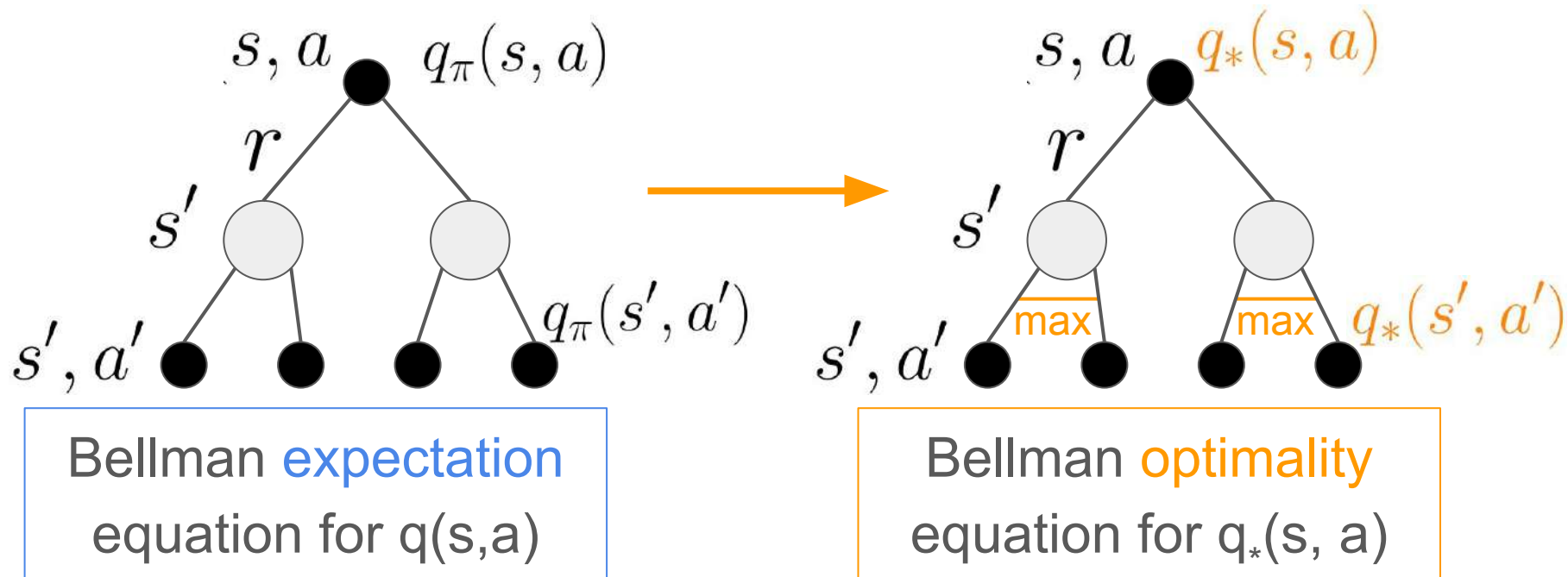


Bellman **expectation**
equation for $q(s,a)$



Bellman **optimality**
equation for $q_*(s, a)$

Bellman **optimality** equation for $q(s,a)$



$$\begin{aligned} q_*(s, a) &= \mathbb{E} \left[R_t + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right] \\ &= \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right] \end{aligned}$$

Generalized Policy Iteration:

1. Policy Evaluation
2. Policy Improvement

Policy evaluation

Policy evaluation: motivation

Policy evaluation is also called **prediction problem**:

- predict value function for a particular policy.

Bellman **expectation** equation

$$\begin{aligned} v_{\pi}(s) &= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \\ &= \mathbb{E}_{\pi} [R_t + \gamma v_{\pi}(S_{t+1}) | S_t = s] \end{aligned}$$

is basically a system of linear equations where

- # of unknowns = # of equations = # of states

Policy evaluation: algorithm

Input π , the policy to be evaluated

Initialize an array $V(s) = 0$, for all $s \in \mathcal{S}^+$

Repeat

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output $V \approx v_\pi$

Bellman **expectation**
equation for $v(s)$

Policy improvement

Policy improvement: an idea

Once we know what is $v(s)$ for a particular policy

We could improve it by acting greedily w.r.t. $q(s, a)$!

$$\pi'(s) \leftarrow \arg \max_a \overbrace{\sum_{r, s'} p(r, s' | s, a) [r + \gamma v_\pi(s')]}^{q_\pi(s, a)}$$

This procedure is guaranteed to produce a better policy!

Policy improvement: an idea

Once we know what is $v(s)$ for a particular policy

We could improve it by acting greedily w.r.t. $q(s, a)$!

$$\pi'(s) \leftarrow \underset{a}{\operatorname{argmax}} \overbrace{\sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]}^{q_{\pi}(s, a)}$$

This procedure is guaranteed to produce a better policy!

if $q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s)$ for all states

then $v_{\pi'}(s) \geq v_{\pi}(s)$

meaning that $\pi' \geq \pi$

Policy improvement: convergence

If new policy after improvement

$$\pi'(s) \leftarrow \arg \max_a \overbrace{\sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]}^{q_{\pi}(s, a)}$$

is the same as the old one

$$\pi' = \pi \quad \rightarrow \quad v_{\pi'} = v_{\pi}$$

then it is optimal, since it satisfies:

$$v_{\pi'}(s) = \max_a \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

Policy improvement: convergence

If new policy after improvement

$$\pi'(s) \leftarrow \arg \max_a \overbrace{\sum_{r, s'} p(r, s' | s, a) [r + \gamma v_\pi(s')]}^{q_\pi(s, a)}$$

is the same as the old one

$$\pi' = \pi \rightarrow v_{\pi'} = v_\pi$$

then it is optimal, since it satisfies:

$$v_{\pi'}(s) = \max_a \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_\pi(s')]$$

Bellman
optimality
equation

Determining optimal policy from $v_*(s)$, $q_*(s,a)$

If q^* is known – how to recover the optimal policy?

$$\pi_*(s) \leftarrow \arg \max_a q_*(s, a)$$

If v^* is known – how to recover the optimal policy?

Determining optimal policy from $v_*(s)$, $q_*(s,a)$

If q^* is known – how to recover the optimal policy?

$$\pi_*(s) \leftarrow \arg \max_a q_*(s, a)$$

If v^* is known – how to recover the optimal policy?

$$\pi_*(s) \leftarrow \arg \max_a \overbrace{\sum_{r, s'} p(r, s' | s, a) [r + \gamma v_*(s')]}^{q_*(s, a)}$$

Unknown model dynamics → unable to recover optimal policy from v^*

Precise evaluation is excessive

Value function

0 iteration

	0.000	0.000	0.000
0.000	0.000	0.000	0.000
0.000	0.000	0.000	

Greedy policy

0 iteration

Value function

0 iteration

	0.000	0.000	0.000
0.000	0.000	0.000	0.000
0.000	0.000	0.000	

5 iteration

	-7.598	-4.986	-3.127
-7.816	-5.834	-2.963	0.543
-6.115	-4.186	0.332	

9999 iteration

	-13.827	-13.289	-11.318
-14.768	-14.193	-10.722	-5.346
-16.111	-13.454	-6.059	

Greedy policy

0 iteration

5 iteration

9999 iteration

Roadmap

Now we know what is

- Policy evaluation (based on Bellman **expectation** eq)
- Policy improvement (based on Bellman **optimality** eq)

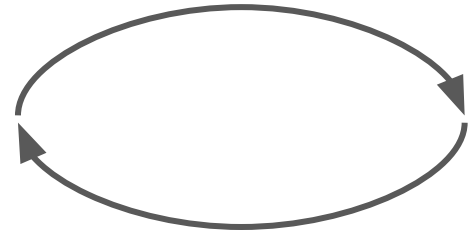
The finishing touches:

how to combine them to obtain optimal policy?

Generalized Policy Iteration

The idea of policy and value iterations

Policy evaluation



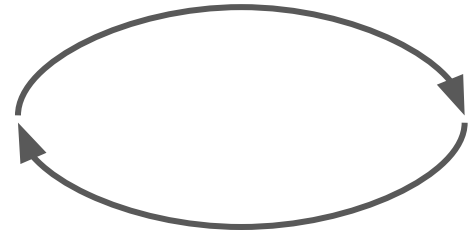
Policy improvement

The idea of policy and value iterations

Generalized policy iteration

1. Evaluate given policy
2. Improve policy by acting greedily w.r.t. to its value function

Policy evaluation



Policy improvement

The idea of policy and value iterations

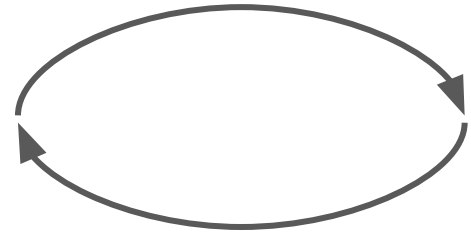
Generalized policy iteration

1. Evaluate given policy
2. Improve policy by acting greedily w.r.t. to its value function

Robustness:

- No dependence on initialization
- No need in complete policy evaluation (states / converg.)
- No need in exhaustive update (states)
 - Example of update robustness:
 - Update only one state at a time
 - in a random direction
 - that is correct only in a expectation

Policy evaluation



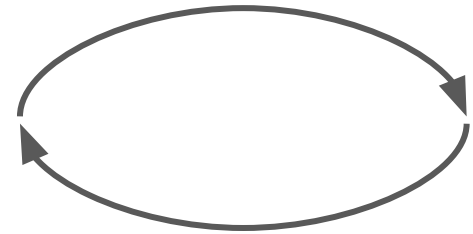
Policy improvement

The idea of policy and value iterations

Generalized policy iteration

1. Evaluate given policy
2. Improve policy by acting greedily w.r.t. to its value function

Policy evaluation



Policy improvement

Policy iteration

1. Evaluate policy until convergence (with some tolerance)
2. Improve policy

Value iteration

1. Evaluate policy only with single iteration
2. Improve policy

Policy iteration

Policy iteration: scheme

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Repeat

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s, \pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Bellman expectation
equation for $v(s)$

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s, a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

$q(s,a)$

Value iteration

Value iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat

$$\Delta \leftarrow 0$$

For each $s \in \mathcal{S}$:

$$v \leftarrow V(s)$$

$$V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

$$\Delta \leftarrow \max(\Delta, |v - V(s)|)$$

until $\Delta < \theta$ (a small positive number)

Bellman **optimality**
equation for $v(s)$

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

Value iteration (VI) vs. Policy iteration (PI)

- VI is **faster** per iteration – $O(|A||S|^2)$
- VI requires **many** iterations
- PI is **slower** per iteration – $O(|A||S|^2 + |S|^3)$
- PI requires **few** iterations

No silver bullet → experiment with # of steps spent in policy evaluation phase to find the best

Lecture 09: Model free learning in RL

Radoslav Neychev

1. MDP properties recap
2. Value-function and Q-function
3. Reward discounting
4. Q-learning, temporal difference
5. Approximate Q-learning

Based on: https://github.com/yandexdataschool/Practical_RL/ weeks 2, 3 and 4

References

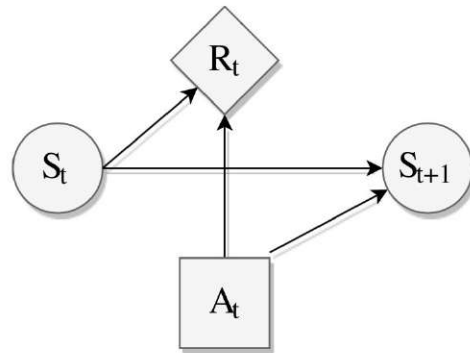
These slides are deeply based on [Practical RL course](#) weeks 2, 3 and 4
Special thanks to YSDA team for making them publicly available.

Given dynamics, how to find an optimal policy?

Definition of Markov Decision Process

MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$, where

- 1 $\mathcal{S}$ – set of states of the world
- 2 $\mathcal{A}$ – set of actions
- 3 $\mathcal{P} : \mathcal{S} \times \mathcal{A} \mapsto \Delta(\mathcal{S})$ – state-transition function, giving us $p(s_{t+1} | s_t, a_t)$
- 4 $\mathcal{R} : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ – reward function, giving us $\mathbb{E}_R [R(s_t, a_t) | s_t, a_t]$.



Markov property

$$p(r_t, s_{t+1} | s_0, a_0, r_0, \dots, s_t, a_t) = p(r_t, s_{t+1} | s_t, a_t)$$

(next state, expected reward) depend on (previous state, action)

Goal: solve an MDP by finding an optimal policy

1. What is the objective?
 - a. Reward: discounting and design
 - b. Expected objective: state- and action-value function

Explaining goals to agent through reward

Reward hypothesis (R.Sutton)

Goals and purposes can be thought of as the maximization of the expected value of the cumulative sum of a received scalar signal

Cumulative reward is called a return:

$$G_t \triangleq R_t + R_{t+1} + R_{t+2} + \dots + R_T$$

Diagram illustrating the components of the return equation:

- A blue arrow points from the text "Cumulative reward" to the G_t term in the equation.
- A green arrow points from the text "immediate reward" to the R_t term in the equation.
- A red arrow points from the text "end of an episode" to the R_T term in the equation.

E.g.: reward in chess – value of taken opponent's piece

E.g.: data center non-stop cooling system

- **States** – temperature measurements
- **Actions** – different fans speed
- **R = 0** for exceeding temperature thresholds
- **R = +1** for each second system is cool

What could go wrong with such a design?

E.g.: data center non-stop cooling system

- States – temperature measurements
- Actions – different fans speed
- $R = 0$ for exceeding temperature thresholds
- $R = +1$ for each second system is cool

What could go wrong with such a design?

Infinite return for **non optimal** behaviour!

$$G_t = 1 + 1 + 0 + 1 + 1 + 0 + \dots = \sum_{t=1}^{\infty} R_t = \infty$$

E.g.: cleaning robot

- **States** – dust sensors, air
- **Actions** – cleaning / rest / conditioning on or off
- **R = 100** for long tedious floor cleaning task done
- **R = 1** for turning air conditioning on-off
- Episode **ends** each **day**

What could go wrong with such a design?



OpenAI blog post about faulty rewards: <https://openai.com/blog/faulty-reward-functions/>

E.g.: cleaning robot

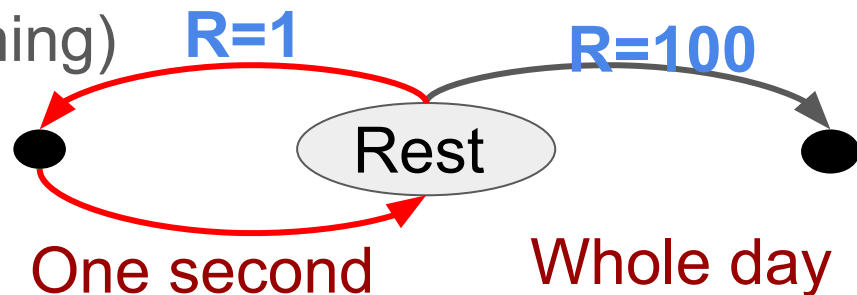
- **States** – dust sensors, air
- **Actions** – cleaning / rest / conditioning on or off
- **R = 100** for long tedious floor cleaning task done
- **R = 1** for turning air conditioning on-off
- Episode **ends** each **day**

What could go wrong with such a design?

Reward(air) < Reward(cleaning)

Time(air) << Time(cleaning)

Positive feedback loop!



Explaining goals to agent through reward

Reward hypothesis (R.Sutton)

Goals and purposes can be thought of as the maximization of the expected value of the cumulative sum of a received scalar signal

Reward discounting

Get rid of infinite sum by **discounting**

$$0 \leq \gamma < 1$$

$$G_t \triangleq R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

discount factor



The same cake compared to
today's one worth

- γ times less tomorrow
- γ^2 times less the day after

tomorrow



γ will eat it day by day

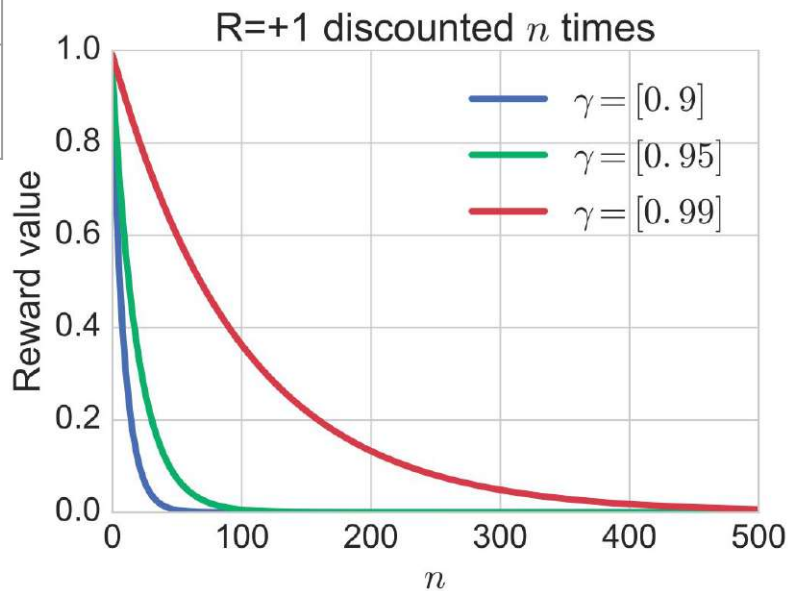
Reward discounting

Discounting makes sums finite

Maximal return for **R = +1**

$$G_0 = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}$$

γ	0.9	0.95	0.99
$\frac{1}{1-\gamma}$	10	20	100



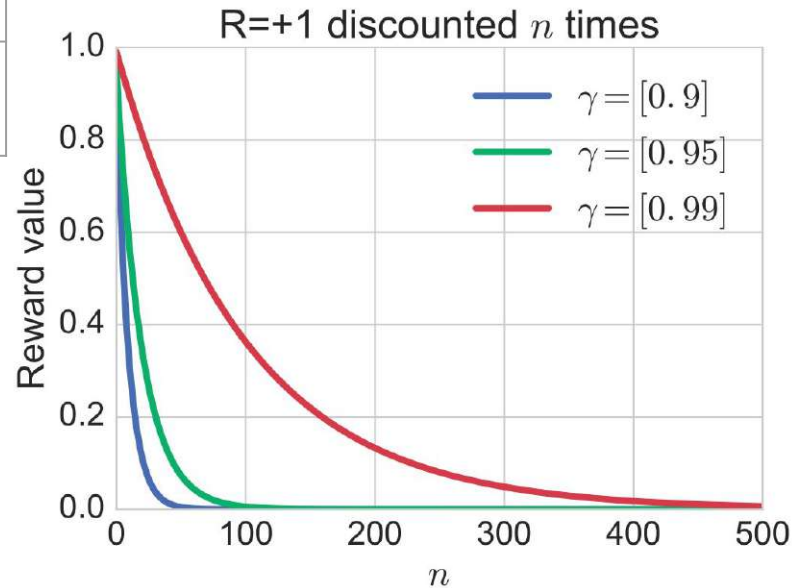
Discounting makes sums finite

Maximal return for **R = +1**

γ	0.9	0.95	0.99
$\frac{1}{1-\gamma}$	10	20	100

$$G_0 = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}$$

Any **discounting**
changes optimisation
task and its solution!



State- and Action-value functions

State-value function $v(s)$

$v(s)$ is expected **return** conditional on state:

$$\begin{aligned} v_{\pi}(s) &\triangleq \mathbb{E}_{\pi} [G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi} [R_t + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \mathbb{E}_{\pi} [G_{t+1} \mid S_{t+1} = s'] \right] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

By definition

Intuition: value of following policy π from state s

Action-value function $q(s, a)$

Is expected **return** conditional on state and action:

Intuition: value of following policy π after committing action **a** in state **s**

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi} [R_t + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \mathbb{E}_{\pi} [G_{t+1} \mid S_{t+1} = s'] \right] \\ &= \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

Relations between $v(s)$ and $q(s,a)$

We already know how to write $q(s,a)$ in terms of $v(s)$

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')]$$

What about $v(s)$ in terms of $q(s,a)$?

$$\begin{aligned} v_{\pi}(s) &= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \\ &= \sum_a \pi(a | s) q_{\pi}(s, a) \end{aligned}$$

So, we could now write $q(s, a)$ in terms of $q(s,a)$!

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right]$$

Bellman **expectation** equation for $\mathbf{v}(\mathbf{s})$

Recursive definition of $v(s)$ is an important concept in RL

$$\begin{aligned} v_{\pi}(s) &= \sum_a \pi(a | s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \\ &= \mathbb{E}_{\pi} [R_t + \gamma v_{\pi}(S_{t+1}) | S_t = s] \end{aligned}$$

Bellman **expectation** equation for **q(s,a)**

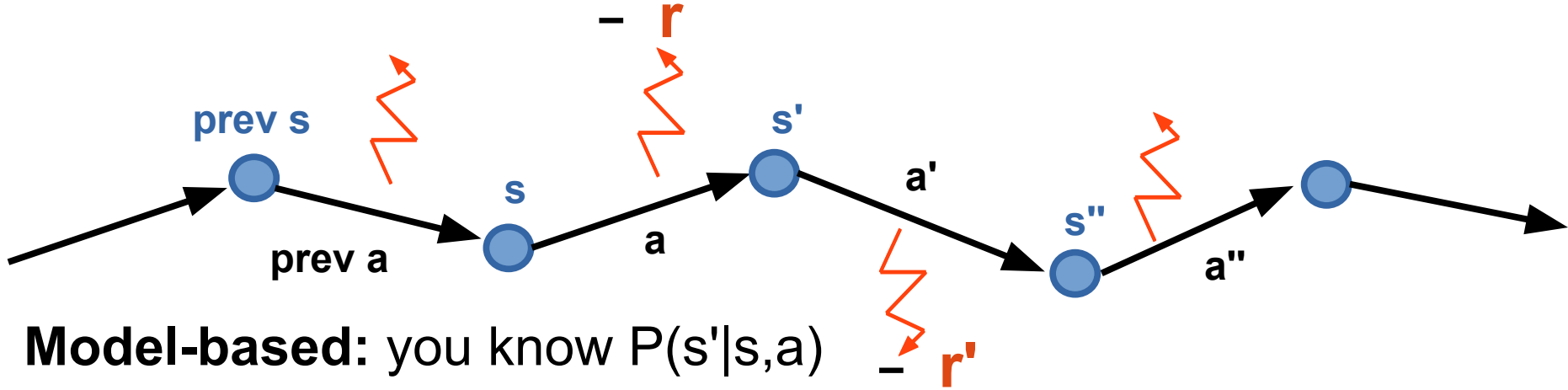
$$\begin{aligned} q_{\pi}(s, a) &= \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \\ &= \sum_{r, s'} p(r, s' | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right] \end{aligned}$$

- $V_{\pi}(\mathbf{s})$ – expected G from state $\mathbf{s}$ if you follow π
- $V^*(\mathbf{s})$ – expected G from state $\mathbf{s}$ if you follow π^* – optimal
- $Q_{\pi}(\mathbf{s}, \mathbf{a})$ – expected G from state $\mathbf{s}$
 - if you start by taking action $\mathbf{a}$
 - and follow π from next state on
- $Q^*(\mathbf{s}, \mathbf{a})$ – same as $Q_{\pi}(\mathbf{s}, \mathbf{a})$ where $\pi = \pi^*$ – optimal policy

$$Q^*(s, a) = E_{s', r} [r(s, a) + \gamma \cdot V^*(s')]$$

$$V^*(s) = \max_a Q^*(s, a)$$

Learning from trajectories



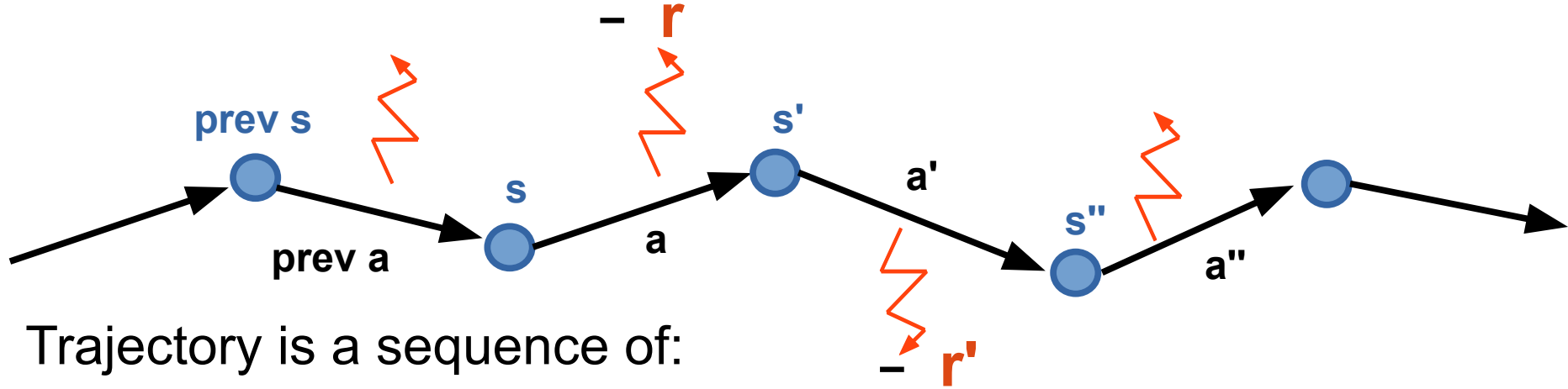
Model-based: you know $P(s'|s,a)$

- can apply dynamic programming
- can plan ahead

Model-free: you can sample trajectories

- can try stuff out
- insurance not included

Learning from trajectories



Trajectory is a sequence of:

- states (s)
- actions (a)
- rewards (r)

Q: What to learn?

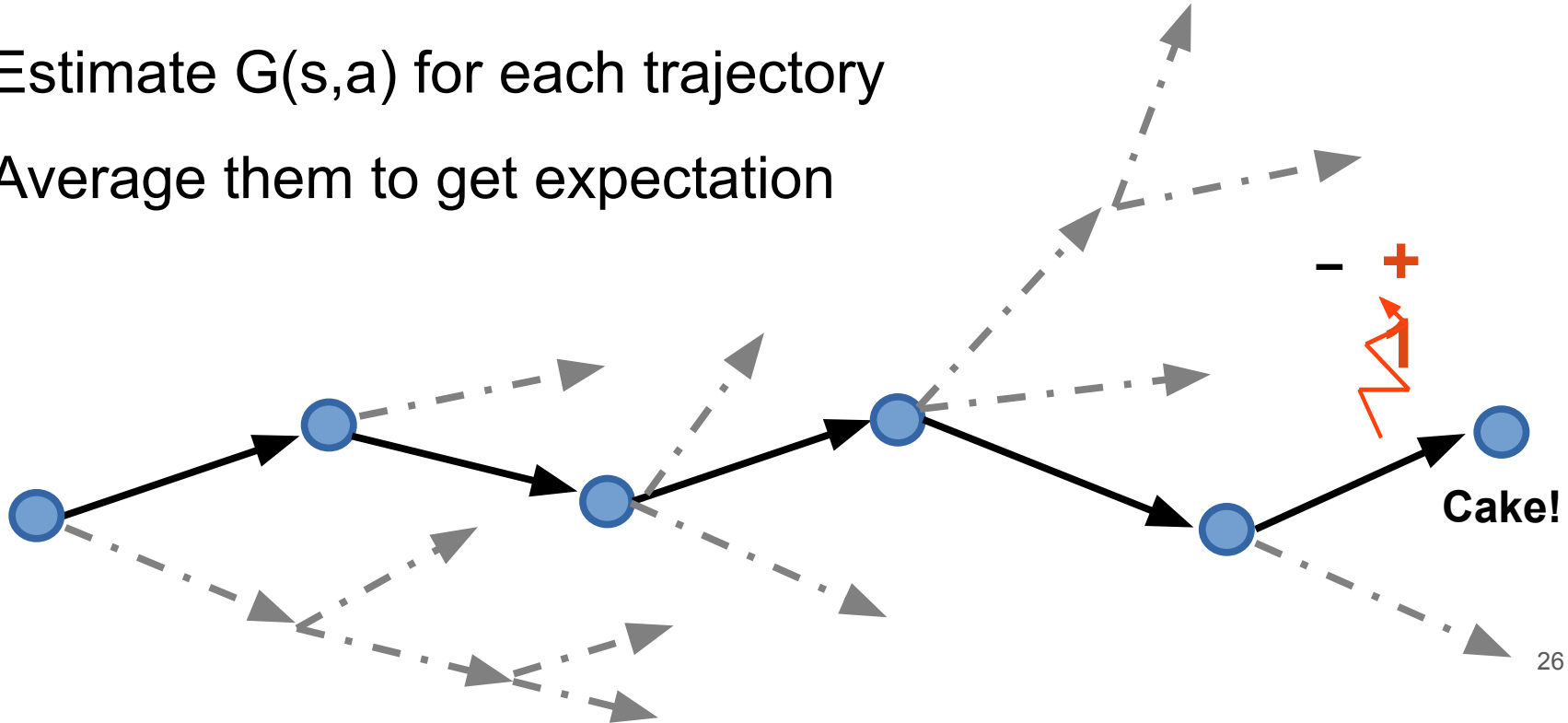
$V(s)$ or $Q(s,a)$

We can only sample trajectories

$V(s)$ is useless
without $P(s'|s,a)$

Idea 1: Monte-carlo

- Get all trajectories containing particular (s,a)
- Estimate $G(s,a)$ for each trajectory
- Average them to get expectation



Idea 2: Temporal difference

- $Q(s, a)$ can be improved iteratively!

$$Q(s_t, a_t) \leftarrow E_{r_t, s_{t+1}} r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$$

That's $Q^*(s, a)$

That's value for π^*
aka optimal policy

That's something
we don't have

What do we do?

The diagram shows the equation $Q(s_t, a_t) \leftarrow E_{r_t, s_{t+1}} r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$. A green arrow points from the text 'That's $Q^*(s, a)$ ' to $Q(s_t, a_t)$. A red arrow points from the text 'That's something we don't have' to the expectation operator $E_{r_t, s_{t+1}}$. A green arrow points from the text 'That's value for π^* aka optimal policy' to $\max_{a'} Q(s_{t+1}, a')$. Below the equation, the text 'What do we do?' is written in red.

Idea 2: Temporal difference

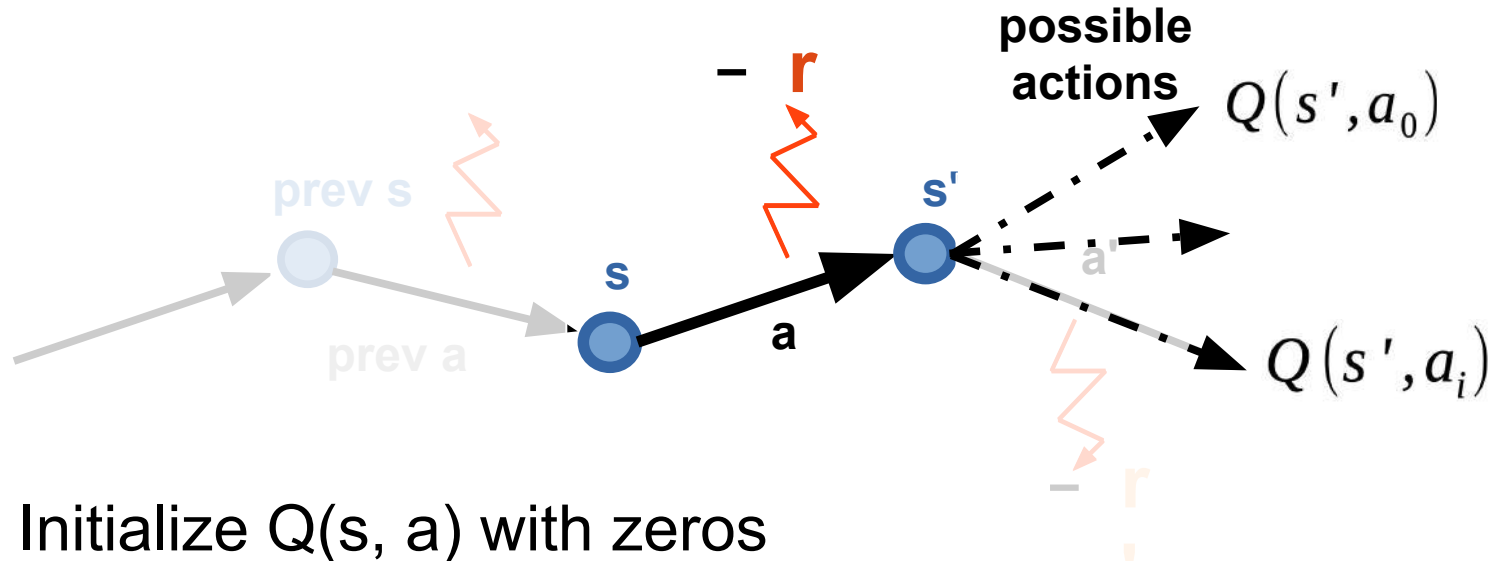
- $Q(s, a)$ can be improved iteratively!

$$Q(s_t, a_t) \leftarrow E_{r_t, s_{t+1}} r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$$

$$E_{r_t, s_{t+1}} r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a') \approx \frac{1}{N} \sum_i r_i + \gamma \cdot \max_{a'} Q(s_i^{\text{next}}, a')$$

$$Q(s_t, a_t) \leftarrow \alpha \cdot (r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')) + (1 - \alpha) Q(s_t, a_t)$$

Q-learning



Initialize $Q(s, a)$ with zeros

- Sample $\langle s, a, r, s' \rangle$ from the environment
- Compute new $Q(s, a)$ estimation:

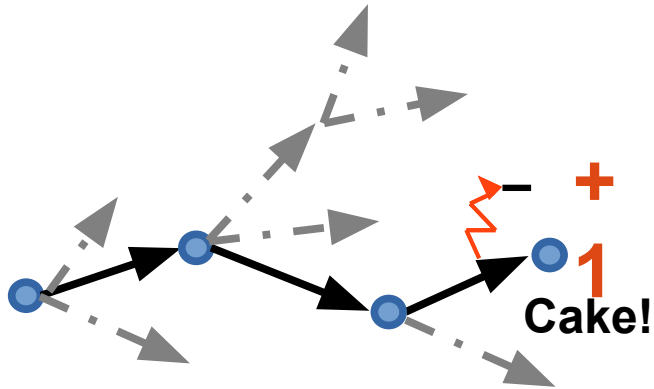
$$\hat{Q}(s, a) = r(s, a) + \gamma \max_{a_i} Q(s', a_i)$$

- Update $Q(s, a)$:

$$Q(s, a) \leftarrow \alpha \cdot \hat{Q}(s, a) + (1 - \alpha) Q(s, a)$$

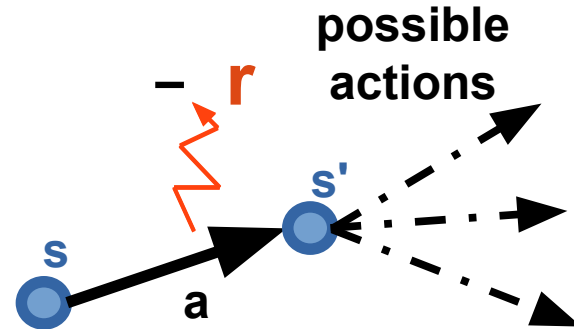
Monte-carlo

Averages Q over
sampled paths



Temporal Difference

Uses recurrent
formula for Q



$$Q^*(s, a) = E_{s', r} [r(s, a) + \gamma \cdot V^*(s')]$$

$$Q(s_t, a_t) \leftarrow \alpha \cdot (r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a')) + (1 - \alpha) Q(s_t, a_t)$$

$$\pi(s) : \operatorname{argmax}_a Q(s, a)$$

Exploration-exploitation tradeoff

Strategies:

- ϵ -greedy
 - With probability ϵ take random action; otherwise take optimal action.
- Softmax
 - Pick action proportional to softmax of shifted normalized Q-values.

$$\pi(a|s) = \text{softmax}\left(\frac{Q(s,a)}{\tau}\right)$$

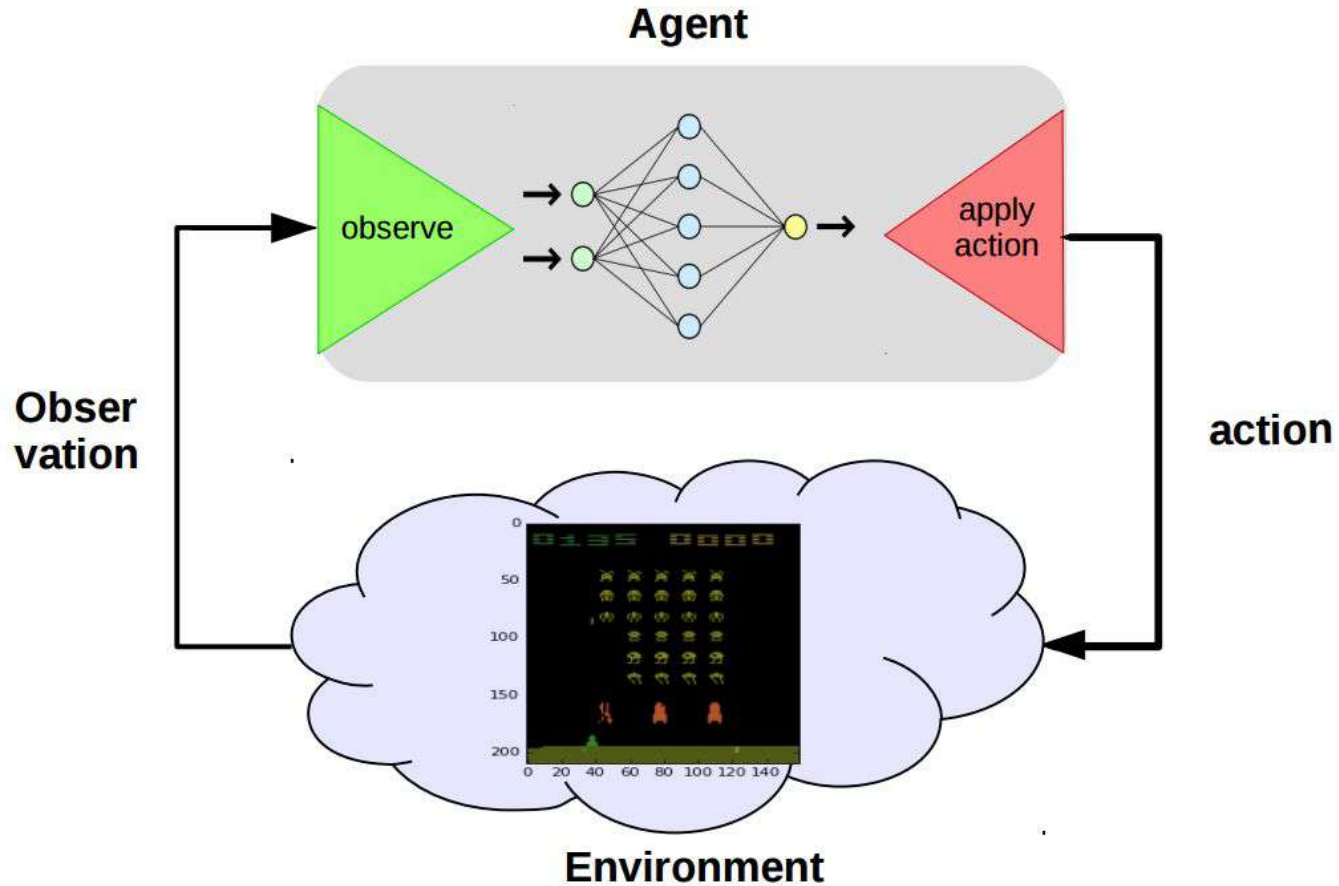
Last step: making it continuous

For now states and actions are discrete. What if states are **not**?

Minimize loss given $\langle \mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}' \rangle$

$$L = [Q(s_t, a_t) - Q^{true}(s_t, a_t)]^2$$

$$L \approx [Q(s_t, a_t) - (r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'))]^2$$



$$G_t = \sum_{t'=t}^T \gamma^{(t'-t)} r_{t'}$$

$$Q^\pi(s, a) = E_\pi[G_t | s_t = s, a_t = a]$$

$$V^\pi(s) = E_\pi[G_t | s_t = s]$$

Recurrent relations

$$Q^\pi(s, a) = E_{s_{t+1}}[r_t + \gamma V^\pi(s_{t+1})]$$

$$Q^\pi(s, a) = E_{s_{t+1}, a_{t+1} \sim \pi}[r_t + \gamma Q^\pi(s_{t+1}, a_{t+1})]$$

For all π, s, a : $Q^{\pi^*}(s, a) \geq Q^{\pi}(s, a)$

$$\pi^*(s) = \operatorname{argmax}_a Q^{\pi^*}(s, a)$$

Bellman optimality equation

$$Q^*(s_t, a) = E_{s_{t+1}}[r_t + \max_{a'} Q^*(s_{t+1}, a')]$$

Training step

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

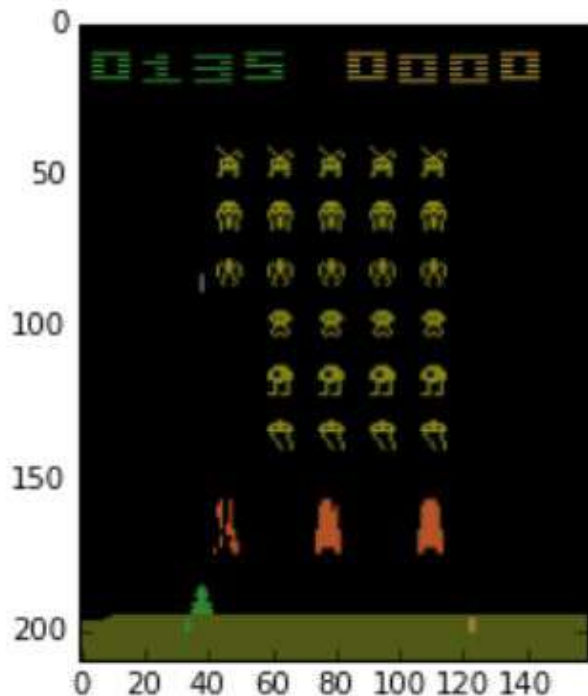
Q-learning as MSE minimization

$$L = (r_t + \gamma \max_{a'} \boxed{Q(s_{t+1}, a')} - Q(s_t, a_t))^2$$

Const

$$\nabla L = 2 \cdot (r_t + \gamma \max_{a'} \boxed{Q(s_{t+1}, a')} - Q(s_t, a_t))$$

What's wrong here?



How many states are there?
approximately

$$|S| = 2^{210 \cdot 160 \cdot 8 \cdot 3}$$

Q-learning: make it continuous

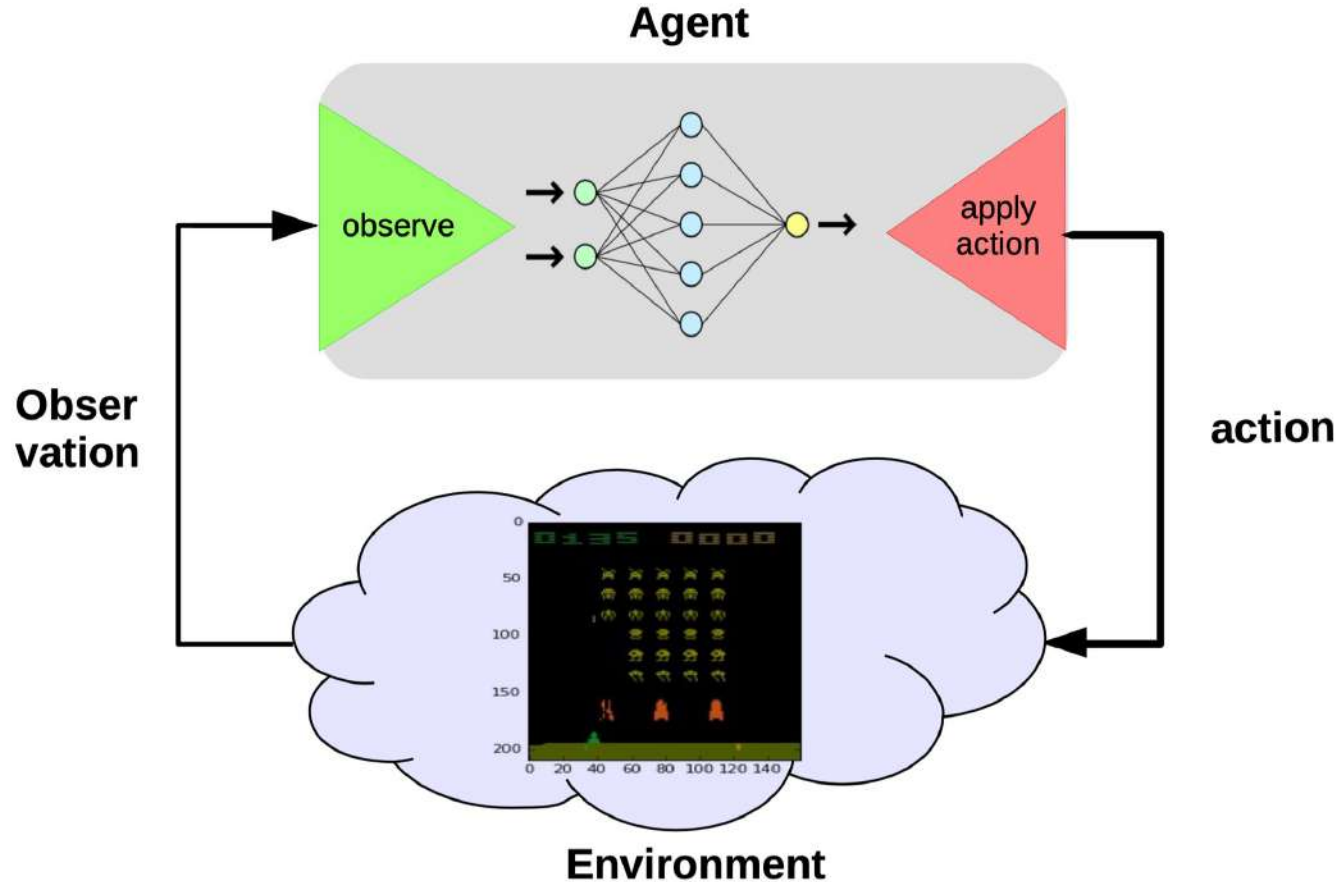
For now states and actions are discrete. What if states are **not**?

Minimize loss given $\langle \mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}' \rangle$

$$L = [Q(s_t, a_t) - Q^{true}(s_t, a_t)]^2$$

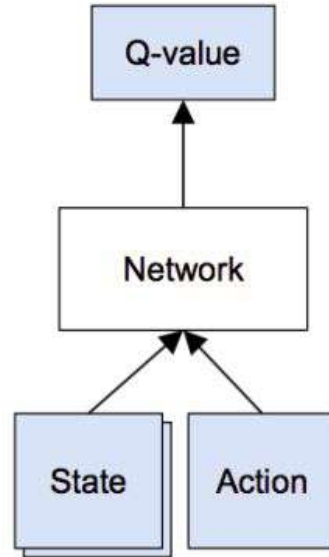
$$L \approx [Q(s_t, a_t) - (r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'))]^2$$

Approximate Q-learning



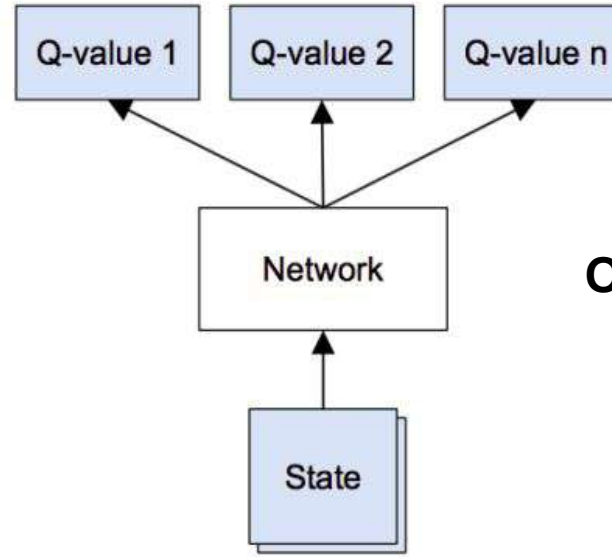
Possible architectures

**Continuous
control or large
number of
actions**



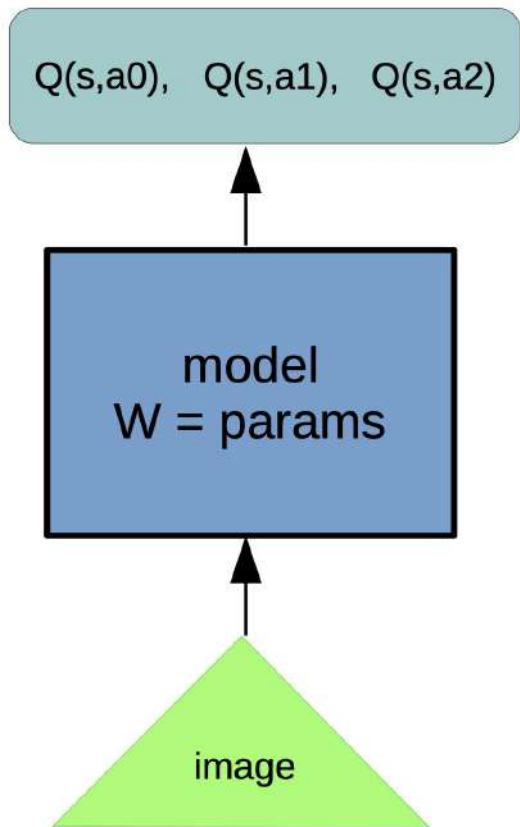
Given **(s,a)**
Predict $Q(s,a)$

**One pass for all
actions**



Given **s** predict all q-values
 $Q(s,a_0)$, $Q(s,a_1)$, $Q(s,a_2)$

Approximate Q-learning



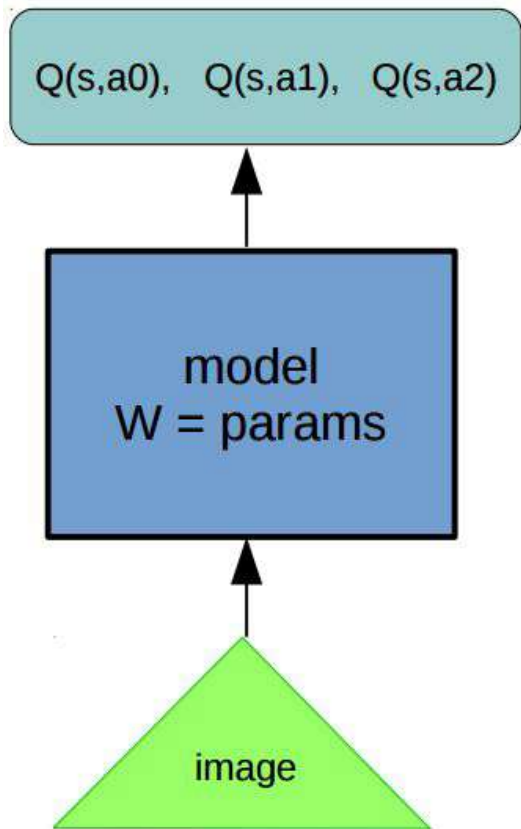
$$\hat{Q}(s_t, a_t) = r + \gamma \cdot \max_{a'} \hat{Q}(s_{t+1}, a')$$

$$L = \left(Q(s_t, a_t) - \left[r + \gamma \cdot \max_{a'} Q(s_{t+1}, a') \right] \right)^2$$

Consider const

$$w_{t+1} = w_t - \alpha \cdot \frac{\delta L}{\delta w}$$

Approximate Q-learning



Objective:

$$L = \left(Q(s_t, a_t) - \hat{Q}(s_t, a_t) \right)^2$$

consider const

Q-learning:

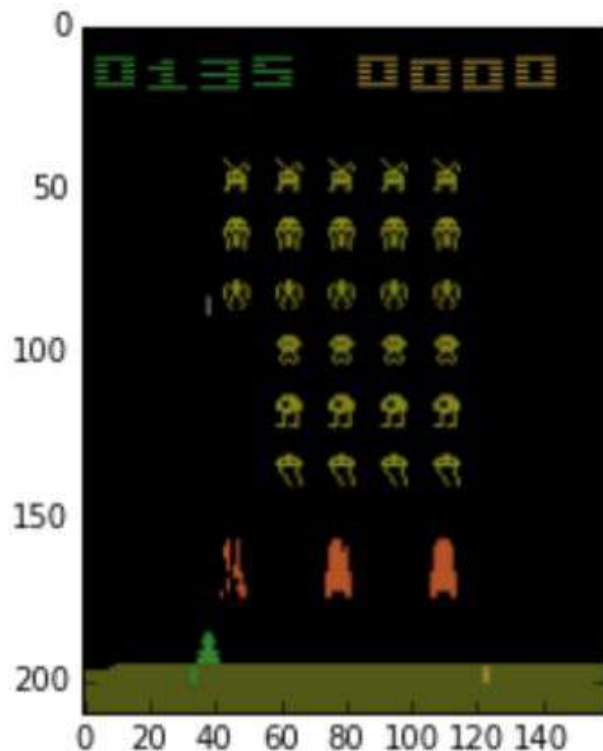
$$\hat{Q}(s_t, a_t) = r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$$

SARSA:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot Q(s_{t+1}, a_{t+1})$$

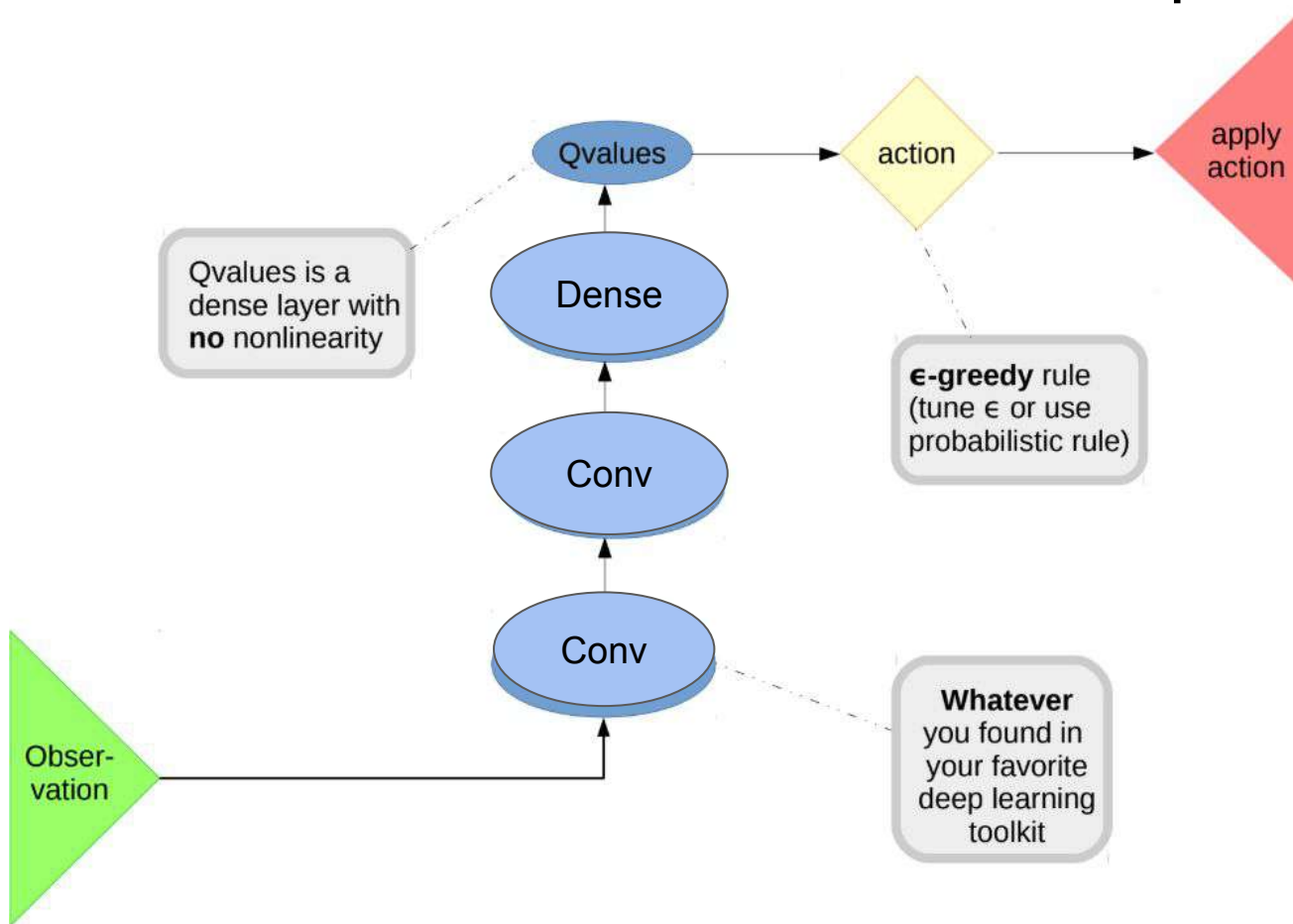
Expected Value SARSA:

$$\hat{Q}(s_t, a_t) = r + \gamma \cdot E_{a' \sim \pi(a|s)} Q(s_{t+1}, a')$$



What kind of network digests images well?

Basic deep Q-learning





- Q-learning allows to learn some approximation of the reward function and environment model
 - So we can use it to solve the desired problem
- Remember what $Q(s, a)$ and $V(s)$ functions do
- Remember both about exploration and exploitation
 - At least using greedy policy or softmax smoothing

Lecture 06: Policy gradient & REINFORCE

Radoslav Neychev

References

These slides are almost the exact copy of Practical RL course week 6 slides.
Special thanks to YSDA team for making them publicly available.

Original slides link: [week06_policy_based](#)

Small experiment

The next slide contains a question

Please respond as fast as you can!

Small experiment



left or right?

Small experiment



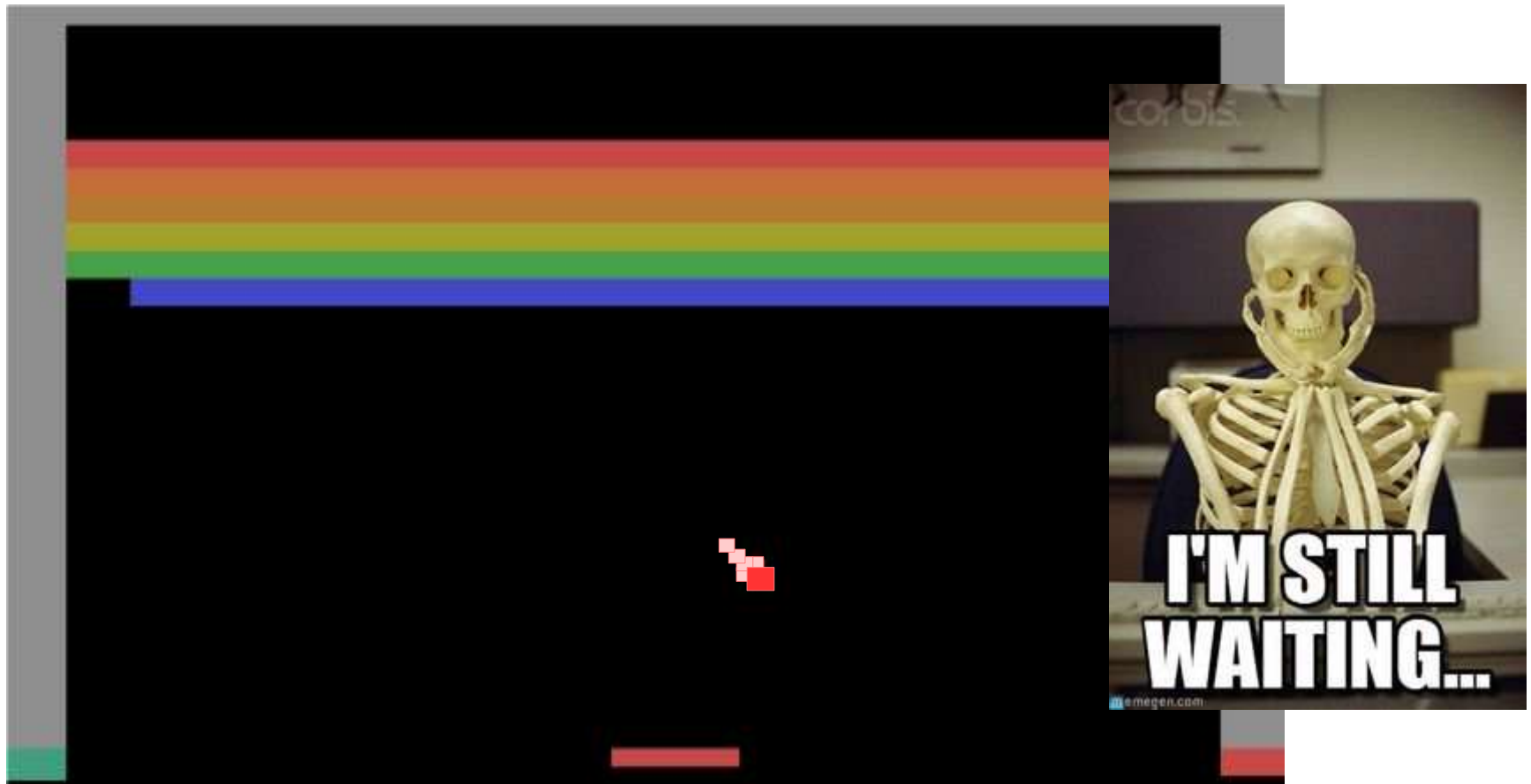
Right! Ready for next one?

Small experiment



What's $Q(s, \text{right})$ under $\gamma=0.99$?

Small experiment



What's $Q(s, \text{right})$ under $\gamma=0.99$?

Approximation error

DQN is trained to minimize

$$L \approx E[Q(s_t, a_t) - (r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'))]^2$$

Simple 2-state world

	True	(A)	(B)
Q(s0,a0)	1	1	2
Q(s0,a1)	2	2	1
Q(s1,a0)	3	3	3
Q(s1,a1)	100	50	100

Q: Which prediction is better (A/B)?

Approximation error

DQN is trained to minimize

$$L \approx E[Q(s_t, a_t) - (r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'))]^2$$

Simple 2-state world

	True	(A)	(B)
Q(s0,a0)	1	1	2
Q(s0,a1)	2	2	1
Q(s1,a0)	3	3	3
Q(s1,a1)	100	50	100

better
policy

less
MSE

Approximation error

DQN is trained to minimize

$$L \approx E[Q(s_t, a_t) - (r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'))]^2$$

Simple 2-state world

	True	(A)	(B)
Q(s0,a0)	1	1	2
Q(s0,a1)	2	2	1
Q(s1,a0)	3	3	3
Q(s1,a1)	100	50	100

Q-learning will prefer worse policy (B)!

better
policy

less
MSE

Conclusion

- Often computing q-values is harder than picking optimal actions!
- We could avoid learning value functions by directly learning agent's policy $\pi_{\theta}(a|s)$

Q: what algorithm works that way?

(of those we studied)

Conclusion

- Often computing q-values is harder than picking optimal actions!
- We could avoid learning value functions by directly learning agent's policy $\pi_{\theta}(a|s)$

Q: what algorithm works that way?

e.g. crossentropy method

NOT how humans survived

$\text{argmax}[$
Q(s, pet the tiger)
Q(s, run from tiger)
Q(s, provoke tiger)
Q(s, ignore tiger)
]



how humans survived

$$\pi(run|s)=1$$



Policies

In general, two kinds

- Deterministic policy

$$a = \pi_{\theta}(s)$$

- Stochastic policy

$$a \sim \pi_{\theta}(a|s)$$

Q: Any case where stochastic is better?

Policies

In general, two kinds

- Deterministic policy

$$a = \pi_{\theta}(s)$$

- Stochastic policy

$$a \sim \pi_{\theta}(a|s)$$

e.g. rock-paper
-scissors

Q: Any case where stochastic is better?

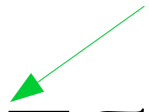
Policies

In general, two kinds

- Deterministic policy

Genetic algos (week 0)
Deterministic policy gradient

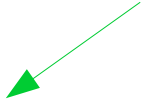
same action each time


$$a = \pi_{\theta}(s)$$

- Stochastic policy

Crossentropy method
Policy gradient

sampling takes care
of exploration


$$a \sim \pi_{\theta}(a|s)$$

Q: how to represent policy in continuous action space?

Policies

In general, two kinds

- Deterministic policy

Genetic algos (week 0)
Deterministic policy gradient

same action each time

$$a = \pi_{\theta}(s)$$

- Stochastic policy

Crossentropy method
Policy gradient

sampling takes care of exploration

$$a \sim \pi_{\theta}(a|s)$$

categorical, normal, mixture of normal, whatever

Two approaches

- **Value based:**

Learn value function

$$Q_{\theta}(s, a) \text{ or } V_{\theta}(s)$$

Infer policy

$$a = \underset{a}{\operatorname{argmax}} Q_{\theta}(s, a)$$

- **Policy based:**

Explicitly learn policy

$$\pi_{\theta}(a|s) \text{ or } \pi_{\theta}(s) \rightarrow a$$

Implicitly maximize reward over policy

Recap: crossentropy method

- Initialize NN weights $\theta_0 \leftarrow \text{random}$
- Loop:
 - Sample N sessions
 - elite = take M best sessions and concatenate

$$\theta_{i+1} = \theta_i + \alpha \nabla \sum_i \log \pi_{\theta_i}(a_i | s_i) \cdot [s_i, a_i \in \text{Elite}]$$

Recap: crossentropy method

- Initialize NN weights $\theta_0 \leftarrow \text{random}$
- Loop:
 - Sample N sessions
 - elite = take M best sessions and concatenate

$$\theta_{i+1} = \theta_i + \alpha \nabla \sum_i \log \pi_{\theta_i}(a_i | s_i) \cdot [s_i, a_i \in \text{Elite}]$$

TD version: elite (s,a) that have highest G(s,a)
(select elites independently from each state)

Policy gradient main idea

Why so complicated?

We'd rather maximize reward directly!

Objective

Expected reward:

$$J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a) \\ \dots}} R(s, a, s', a', \dots)$$

Expected discounted reward:

$$J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}} G(s, a)$$

Objective

Expected reward: **R(z) setting**

$$J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a) \\ \dots}} R(s, a, s', a', \dots)$$

Expected discounted reward: **$G(s,a) = r + \gamma^*G(s',a')$**

$$J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}} G(s, a)$$

Objective

Consider an 1-step process for simplicity

$$J = \underset{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}}{E} R(s, a) = \int_s p(s) \int_a \pi_{\theta}(a|s) R(s, a) da ds$$

Objective

Consider an 1-step process for simplicity

$$J = E_{\substack{s \sim p(s) \\ a \sim \pi_\theta(s|a)}} R(s, a) = \int_s p(s) \int_a \pi_\theta(a|s) R(s, a) da ds$$

state visitation frequency
(may depend on policy)

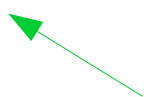
Reward for 1-step session

Q: how do we compute that?

Objective

$$J = \underset{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}}{E} R(s, a) = \int_s p(s) \int_a \pi_{\theta}(a|s) R(s, a) da ds$$

$$J \approx \frac{1}{N} \sum_{i=0}^N R(s, a)$$



sample N sessions
under current policy

Objective

$$J = \underset{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}}{E} R(s, a) = \int_s p(s) \int_a \pi_{\theta}(a|s) R(s, a) da ds$$

$$J \approx \frac{1}{N} \sum_{i=0}^N R(s, a)$$

sample N sessions

Can we optimize policy now?

Objective

$$J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}} R(s, a) = \int_s p(s) \int_a \pi_{\theta}(a|s) R(s, a) da ds$$


parameters “sit” here

$$J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in z_i} R(s, a)$$

We don't know how to compute $dJ/d\theta$

Optimization

Finite differences

- Change policy a little, evaluate

$$\nabla J \approx \frac{J_{\theta+\epsilon} - J_{\theta}}{\epsilon}$$

Stochastic optimization

- Good old crossentropy method
- Maximize probability of “elite” actions

Optimization

Finite differences

- Change policy a little, evaluate

$$\nabla J \approx \frac{J_{\theta+\epsilon} - J_{\theta}}{\epsilon}$$

Stochastic optimization

- Good old crossentropy method
- Maximize probability of “elite” actions

Q: any problems with those two?

Optimization

Finite differences

- Change policy a little, evaluate

$$\nabla J \approx \frac{J_{\theta+\epsilon} - J_{\theta}}{\epsilon}$$

VERY noisy, especially
if both J are sampled

Stochastic optimization

- Good old crossentropy method
- Maximize probability of “elite” actions

“quantile convergence”
problems with stochastic
MDPs

Objective

$$J = \mathop{E}_{\substack{s \sim p(s) \\ a \sim \pi_\theta(s|a)}} R(s, a) = \int_s p(s) \int_a \pi_\theta(a|s) R(s, a) da ds$$

Wish list:

- Analytical gradient
- Easy/stable approximations

Logderivative trick

Simple math

$$\nabla \log \pi(z) = ? ? ?$$

(try chain rule)

Logderivative trick

Simple math

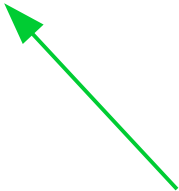
$$\nabla \log \pi(z) = \frac{1}{\pi(z)} \cdot \nabla \pi(z)$$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$

Policy gradient

Analytical inference

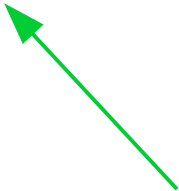
$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$


Policy gradient

Analytical inference

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$


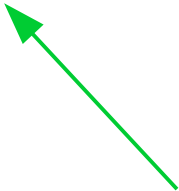
$$\nabla J = \int_s p(s) \int_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) R(s, a) da ds$$

Q: anything curious about that formula?

Policy gradient

Analytical inference

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$


$$\nabla J = \int_s p(s) \int_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) R(s, a) da ds$$

that's expectation :)

REINFORCE (bandit)

- Initialize NN weights $\theta_0 \leftarrow \text{random}$
- Loop:
 - Sample N sessions $\mathbf{z}$ under current
 - Evaluate policy gradient

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \nabla \log \pi_{\theta}(a|s) \cdot R(s, a)$$

- Ascend $\theta_{i+1} \leftarrow \theta_i + \alpha \cdot \nabla J$

Discounted reward case

- Replace R with Q :)

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) Q(s, a) da ds$$

True action value
a.k.a. $E[G(s, a)]$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$

$$\nabla J = \int_s p(s) \int_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) Q(s, a) da ds$$

that's expectation :)

REINFORCE (discounted)

- Policy gradient

$$\nabla J = E_{\substack{s \sim p(s) \\ a \sim \pi_\theta(s|a)}} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

- Approximate with sampling

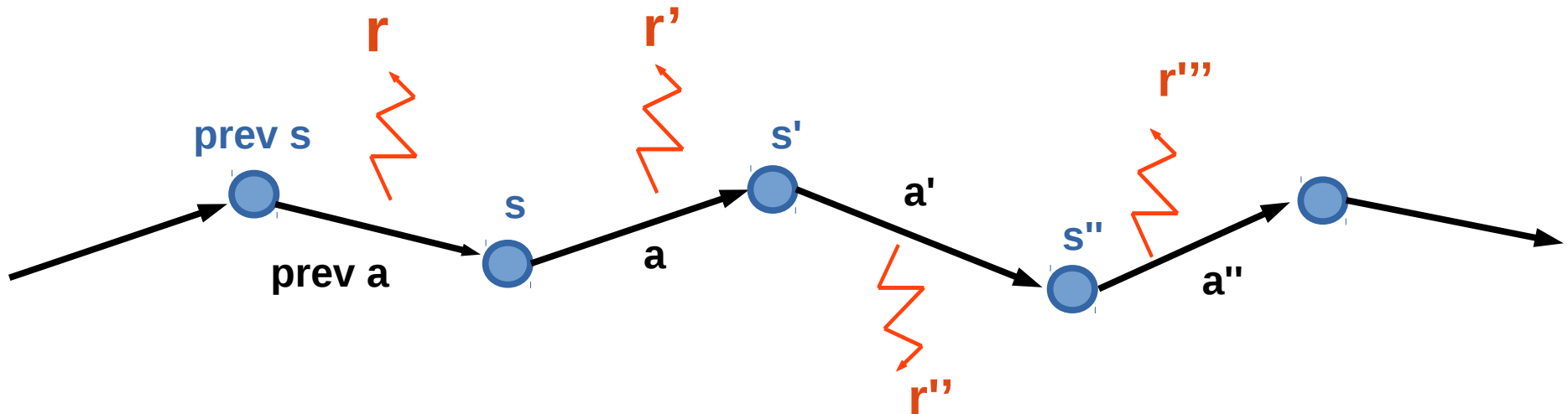
$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in z_i} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

REINFORCE algorithm

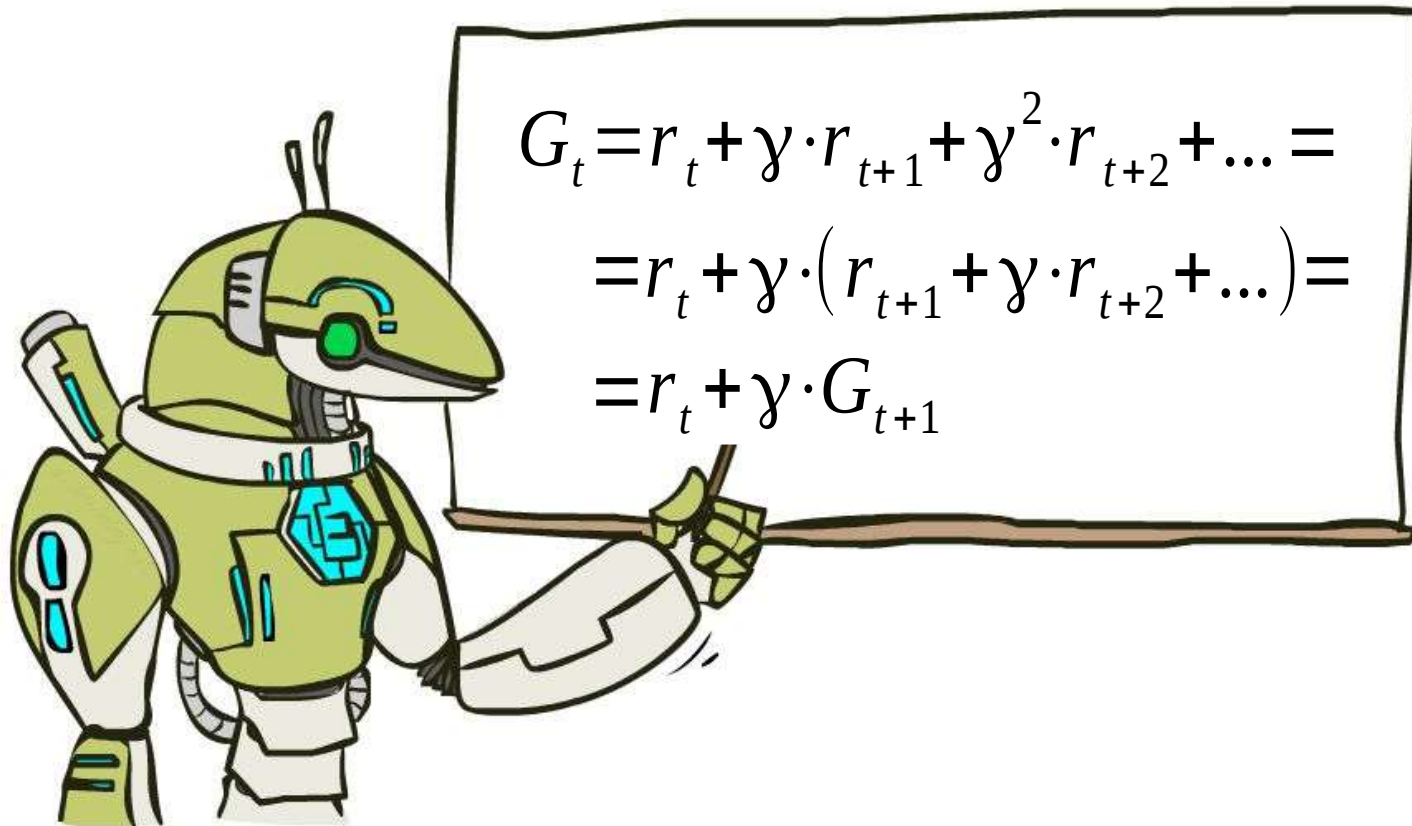
We can estimate Q using G

$$Q_{\pi}(s_t, a_t) = E_{s'} G(s_t, a_t)$$

$$G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \dots$$



Recap: discounted rewards



We can use this to compute all G 's
in linear time

REINFORCE algorithm

- Initialize NN weights $\theta_0 \leftarrow \text{random}$
- Loop:
 - Sample N sessions $\mathbf{z}$ under current $\pi_\theta(a|s)$
 - Evaluate policy gradient

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in \mathbf{z}_i} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

- Ascend $\theta_{i+1} \leftarrow \theta_i + \alpha \cdot \nabla J$

REINFORCE algorithm

- Initialize NN weights $\theta_0 \leftarrow \text{random}$

Q: is it off- or on-policy?

- Loop:
 - Sample N sessions $\mathbf{z}$ under current $\pi_\theta(a|s)$
 - Evaluate policy gradient

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in \mathbf{z}_i} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

- Ascend $\theta_{i+1} \leftarrow \theta_i + \alpha \cdot \nabla J$

REINFORCE algorithm

- Initialize NN weights $\theta_0 \leftarrow \text{random}$
- Loop:
 - Sample N sessions $\mathbf{z}$ under current $\pi_\theta(a|s)$
actions under current policy
= on-policy
 - Evaluate policy gradient

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in \mathbf{z}_i} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

- Ascend $\theta_{i+1} \leftarrow \theta_i + \alpha \cdot \nabla J$

value-based Vs policy-based

Value-based

- Q-learning, SARSA, MCTS value-iteration
- Solves harder problem
- Artificial exploration
- Learns from partial experience (temporal difference)
- Evaluates strategy for free :)

Policy-based

- REINFORCE, CEM
- Solves easier problem
- Innate exploration
- Innate stochasticity
- Support continuous action space
- Learns from full session only?



value-based Vs policy-based

Value-based

- Q-learning, SARSA, MCTS value-iteration
- Solves harder problem
- Artificial exploration
- Learns from partial experience (temporal difference)
- Evaluates strategy for free :)

Policy-based

- REINFORCE, CEM

We'll learn much more soon!

- Solves easier problem
- Innate exploration
- Innate stochasticity
- Support continuous action space
- ~~Learns from full session only~~



- Remember log-derivative trick
- Combining best from both worlds is generally a good idea
- See [this](#) paper for the proof of the policy gradient for discounted rewards

-

REINFORCE baselines

- Initialize NN weights $\theta_0 \leftarrow \text{random}$
- Loop:
 - Sample N sessions $\mathbf{z}$ under current $\pi_\theta(a|s)$
 - Evaluate policy gradient

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in \mathbf{z}_i} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

What is better for learning:
random action in good state
or
great action in bad state?

REINFORCE baselines

We can subtract arbitrary baseline $b(s)$

$$\nabla J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) (Q(s, a) - b(s)) = \dots$$

$$\dots = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) Q(s, a) - E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) b(s) = \dots$$

REINFORCE baselines

We can subtract arbitrary baseline $b(s)$

$$\nabla J = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) (Q(s, a) - b(s)) = \dots$$

$$\dots = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) Q(s, a) - \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) b(s) = \dots$$

Q: Can you simplify the second term?

Note that $b(s)$ does not depend on a

REINFORCE baselines

We can subtract arbitrary baseline $b(s)$

$$\nabla J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) (Q(s, a) - b(s)) = \dots$$

$$\dots = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) Q(s, a) - E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) b(s) = \dots$$

$$E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) b(s) = b(s) \cdot E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) = 0$$

REINFORCE baselines

We can subtract arbitrary baseline $b(s)$

$$\nabla J = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) (Q(s, a) - b(s)) = \dots$$

$$\dots = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) Q(s, a) - \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) b(s) = \dots$$

Gradient direction doesn't change!

$$\dots = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) Q(s, a)$$

REINFORCE baselines

- Gradient direction ∇J stays the same
- Variance may change

Gradient variance: $Var[Q(s, a) - b(s)]$
as a random variable over (s, a)

$$Var[Q(s, a)] - 2 \cdot Cov[Q(s, a), b(s)] + Var[b(s)]$$

REINFORCE baselines

- Gradient direction ∇J stays the same
- Variance may change

Gradient variance: $Var[Q(s, a) - b(s)]$
as a random variable over (s, a)

$$Var[Q(s, a)] - 2 \cdot Cov[Q(s, a), b(s)] + Var[b(s)]$$

 If $b(s)$ correlates with $Q(s, a)$, variance decreases

REINFORCE baselines

- Gradient direction ∇J stays the same
- Variance may change

Gradient variance: $Var[Q(s, a) - b(s)]$
as a random variable over (s, a)

$$Var[Q(s, a)] - 2 \cdot Cov[Q(s, a), b(s)] + Var[b(s)]$$

 Q: can you suggest any such $b(s)$?

REINFORCE baselines

- Gradient direction ∇J stays the same
- Variance may change

Gradient variance: $Var[Q(s, a) - b(s)]$
as a random variable over (s, a)

$$Var[Q(s, a)] - 2 \cdot Cov[Q(s, a), b(s)] + Var[b(s)]$$

Naive baseline: b = moving average Q
over all (s, a) , $Var[b(s)] = 0$, $Cov[Q, b] > 0$

REINFORCE baselines

Better baseline: $b(s) = V(s)$

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in z_i} \nabla \log \pi_{\theta}(a|s) \cdot (Q(s, a) - V(s))$$

Q: but how do we predict $V(s)$?

Actor-critic

- Learn both $V(s)$ and $\pi_{\theta}(a|s)$
- Hope for the best of both worlds :)



Advantage actor-critic

Idea: learn both $\pi_{\theta}(a|s)$ and $V_{\theta}(s)$

Use $V_{\theta}(s)$ to learn $\pi_{\theta}(a|s)$ faster!

Q: how can we estimate **$A(s,a)$**
from (s,a,r,s') and V-function?

Advantage actor-critic

Idea: learn both $\pi_{\theta}(a|s)$ and $V_{\theta}(s)$

Use $V_{\theta}(s)$ to learn $\pi_{\theta}(a|s)$ faster!

$$A(s, a) = Q(s, a) - V(s)$$

$$Q(s, a) = r + \gamma \cdot V(s')$$

$$A(s, a) = r + \gamma \cdot V(s') - V(s)$$

Advantage actor-critic

Idea: learn both $\pi_{\theta}(a|s)$ and $V_{\theta}(s)$

Use $V_{\theta}(s)$ to learn $\pi_{\theta}(a|s)$ faster!

$$A(s, a) = Q(s, a) - V(s)$$

$$Q(s, a) = r + \gamma \cdot V(s')$$

$$A(s, a) = r + \gamma \cdot V(s') - V(s)$$

Also: n-step
version

Advantage actor-critic

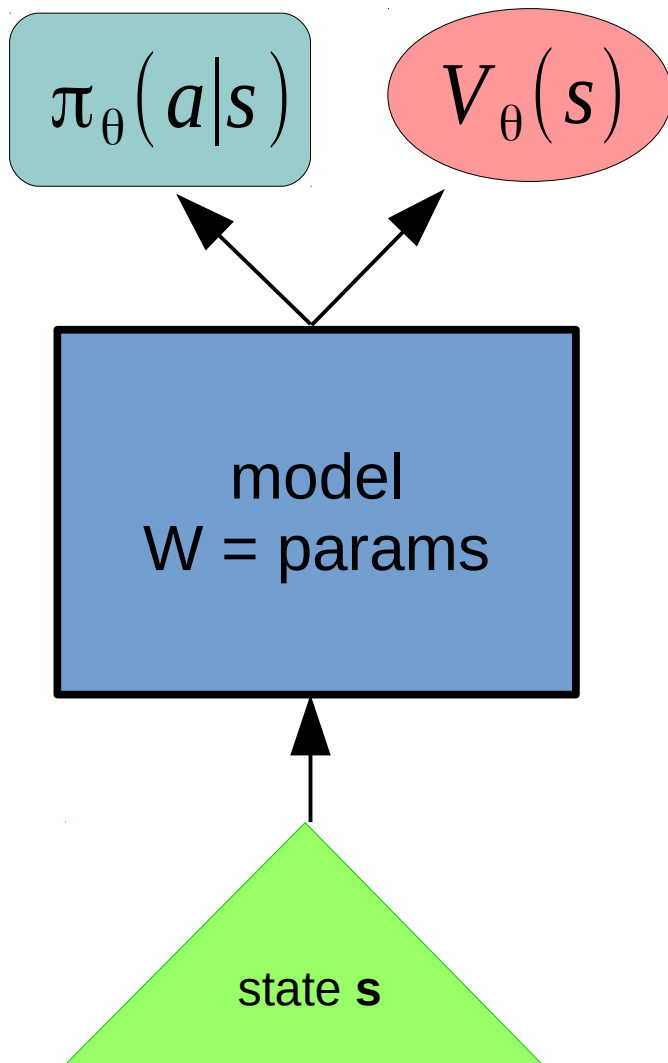
Idea: learn both $\pi_{\theta}(a|s)$ and $V_{\theta}(s)$

Use $V_{\theta}(s)$ to learn $\pi_{\theta}(a|s)$ faster!

$$A(s, a) = r + \gamma \cdot V(s') - V(s)$$

$$\nabla J_{actor} \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in z_i} \nabla \log \pi_{\theta}(a|s) \cdot \underbrace{A(s, a)}_{\text{consider const}}$$

Advantage actor-critic



Improve policy:

$$\nabla J_{actor} \approx \frac{1}{N} \sum_{i=0}^N \sum_{s,a \in z_i} \nabla \log \pi_{\theta}(a|s) \cdot A(s, a)$$

Improve value:

$$L_{critic} \approx \frac{1}{N} \sum_{i=0}^N \sum_{s,a \in z_i} (V_{\theta}(s) - [r + \gamma \cdot V(s')])^2$$

Continuous action spaces

What if there's continuously many actions?

- Robot control: control motor voltage
- Trading: assign money to equity

How does the algorithm change?

Continuous action spaces

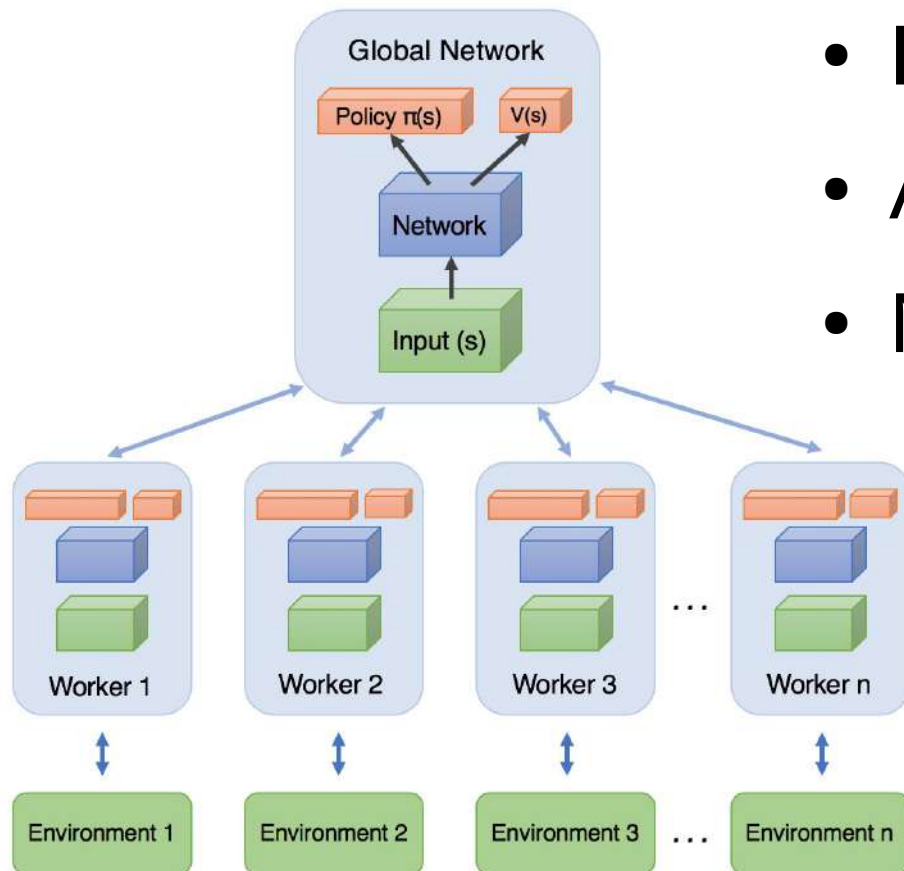
What if there's continuously many actions?

- Robot control: control motor voltage
- Trading: assign money to equity

How does the algorithm change?

it doesn't :)
Just plug in a different formula for
 $\pi(a|s)$, e.g. normal distribution

Asynchronous advantage actor-critic

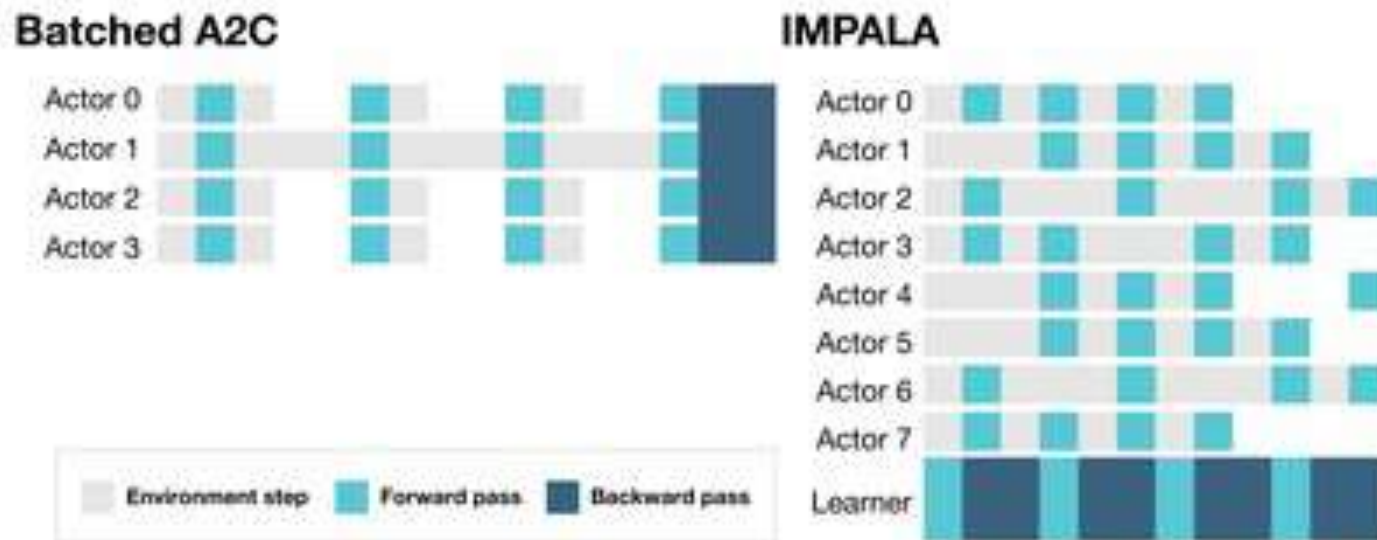


- Parallel game sessions
 - Async multi-CPU training
 - No experience replay
-
- LSTM policy
 - N-step advantage
 - No experience replay

Read more: <https://arxiv.org/abs/1602.01783>

IMPALA

- Massively parallel
- Separate actor / learner processes
 - Small experience replay w/ importance sampling



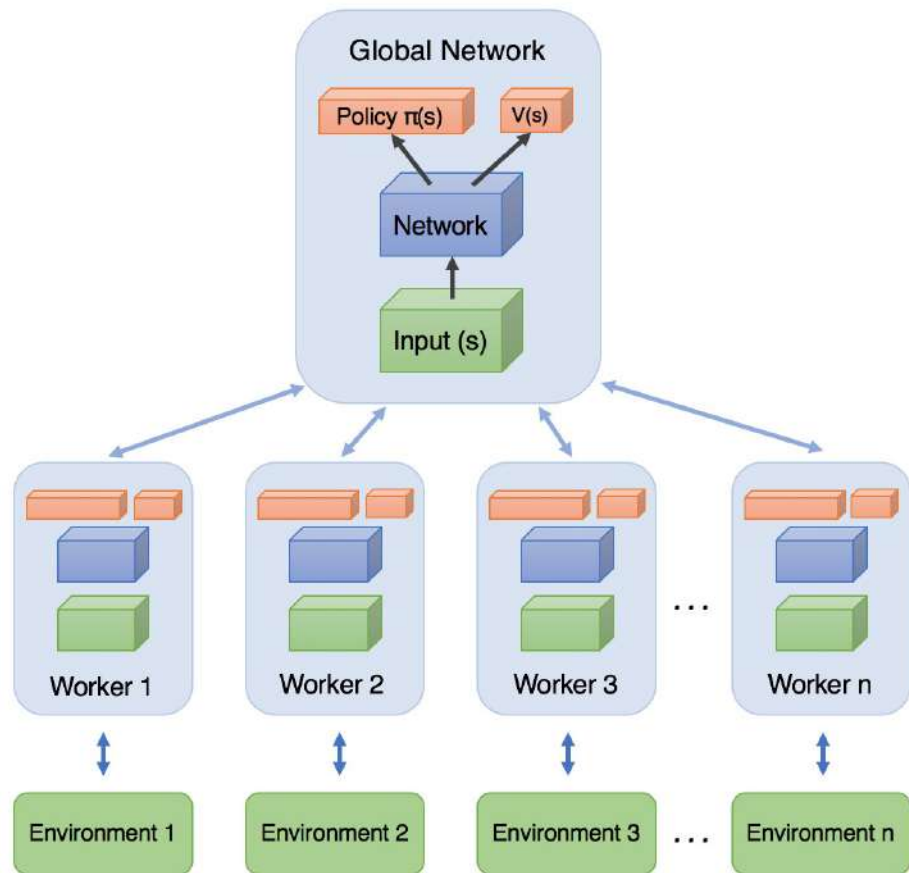
Read more: <https://arxiv.org/abs/1802.01561>

Duct tape zone

- $V(s)$ errors less important than in Q-learning
 - actor still learns even if critic is random, just slower
- Regularize with entropy
 - to prevent premature convergence
- Learn on parallel sessions
 - Or super-small experience replay
- Use logsoftmax for numerical stability



Asynchronous advantage actor-critic



- Remember log-derivative trick
- Combining best from both worlds is generally a good idea
- See [this](#) paper for the proof of the policy gradient for discounted rewards
- Time to write some code!

Lecture 7:

Policy Gradient & SCST

Radoslav Neychev

References

These slides are deeply based on Practical RL course week 7 slides. Special thanks to YSDA team for making them publicly available.

Original slides link: [week07_seq2seq](#)

General formalism

- Maximize $J = \underset{\substack{s \sim d(s) \\ a \sim \pi(a|obs(s))}}{E} R(s, a)$ over π
- $R(s, a)$ or $G(s, a)$ is a black box
 - Special case: $G(s, a) = r(s, a) + \gamma G(s', a')$
- Markov property: $P(s'|s, a, *) = P(s'|s, a)$
 - Special case: $obs(s) = s$, fully observable

General approaches

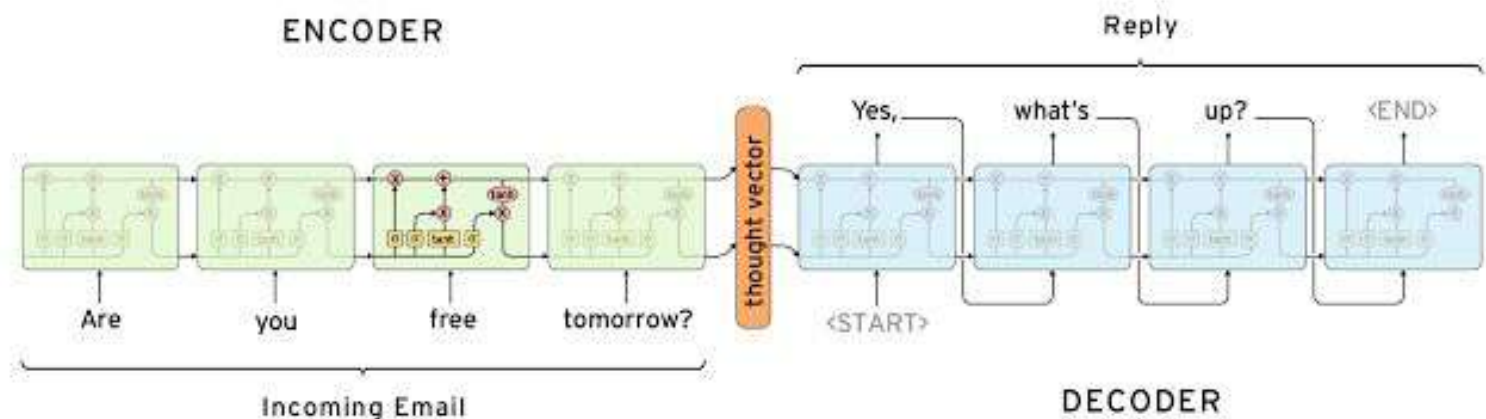
- Idea 1: evolution strategies
 - perturbate π , take ones with higher J
- Idea 2: value-based methods
 - **estimate J** as a function of a , pick best a
- Idea 3: policy gradient
 - **ascend J** over $\pi(a|s)$ using ∇J

General approaches

- Idea 4: Bayesian optimization
 - build a model of J , pick π that is most informative
 - to finding maximal J
 - e.g. Gaussian processes (low-dimensional only)
- Idea 5: simulated annealing
- Idea 6: crossentropy method
- ...

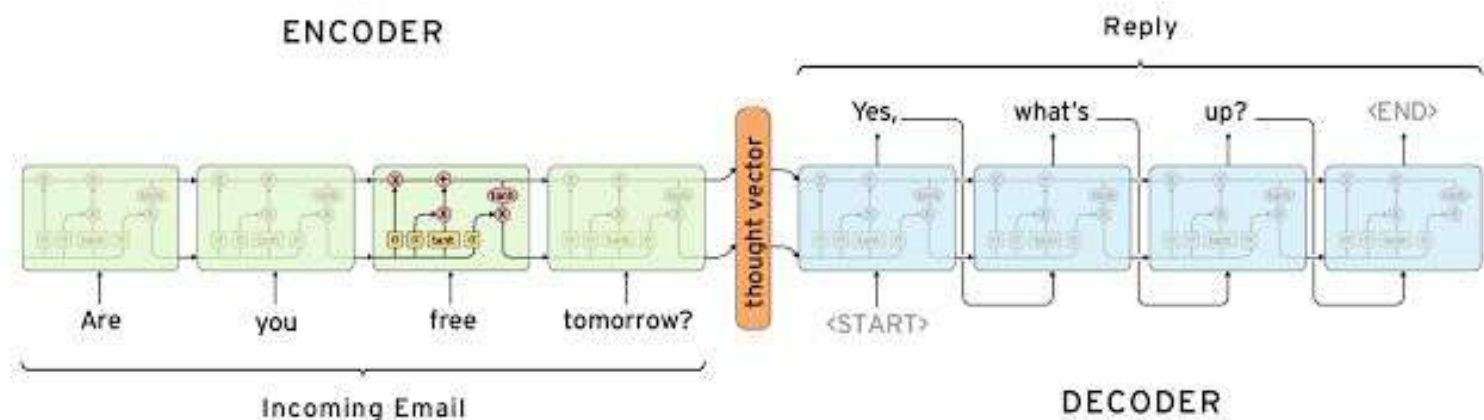
Encoder-decoder architectures

- Read input data (sequence / arbitrary)
- Generate output sequence
- **Trivia:** what problems match this formulation?



Encoder-decoder tasks

- Machine translation
- Image to caption
- Word to transcript
- Conversation system
- Image to latex
- Code to docstring



- **Problem:**
- Read sentence in Chinese
- Generate sentence in English
- Sentences must mean the same thing

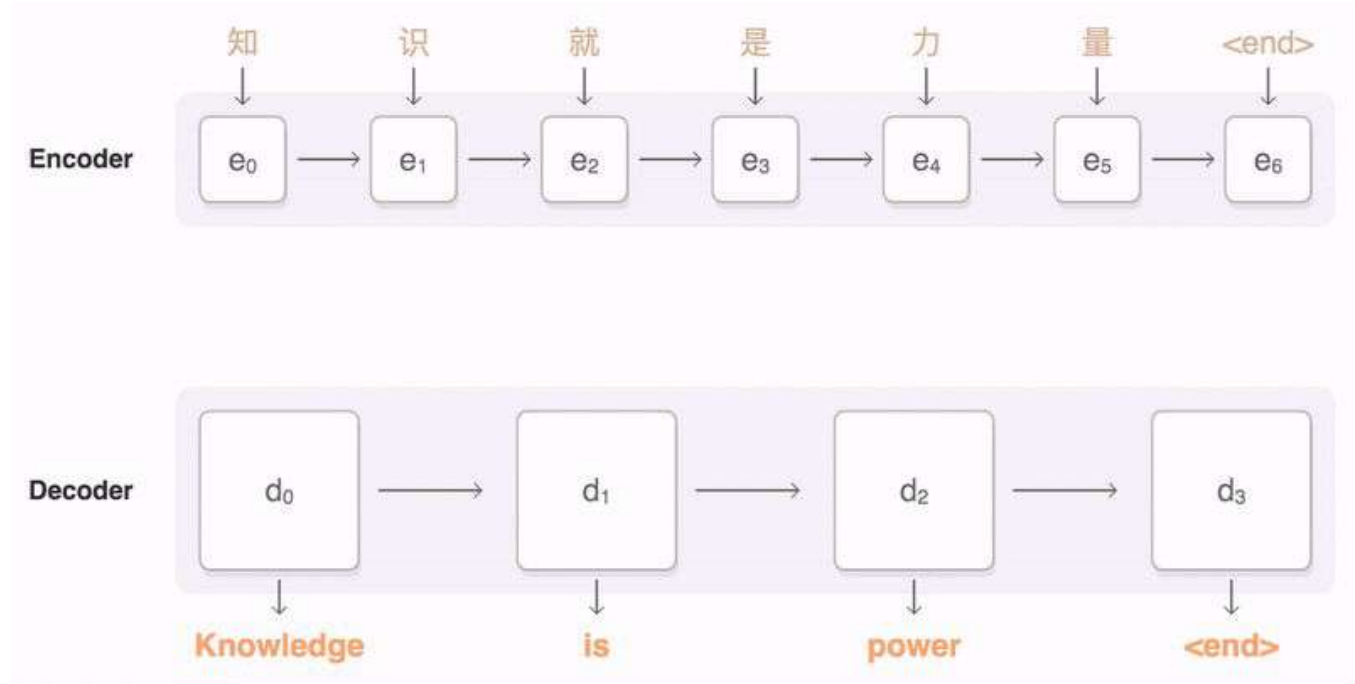
- Solution?

- **Problem:**
- Read sentence in Chinese
- Generate sentence in English
- Sentences must mean the same thing

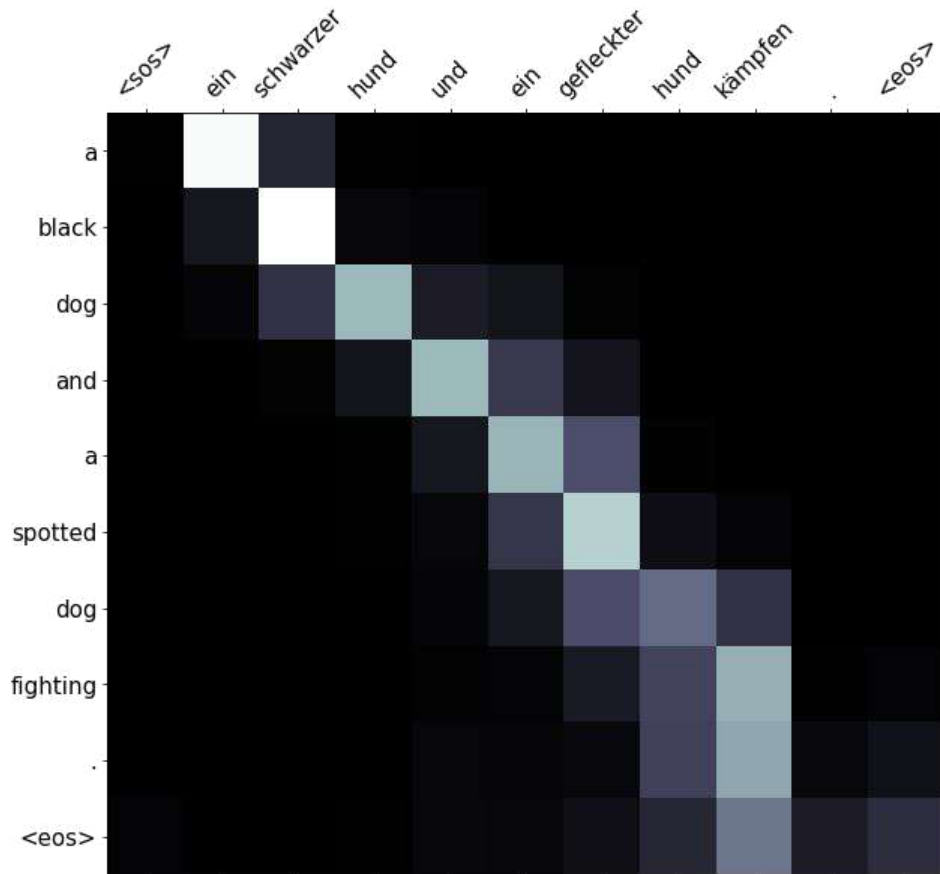
- **Solution:**
- Take large dataset of (source,translation) pairs
- Maximize $\log P(\text{translation}|\text{source})$

Digression: attentive translation

Let decoder choose where to look on each tick



Digression: attentive translation



Simultaneously learns

- Word alignment
- Word translation

Differentiable attention:

$$\bar{a} = W \cdot \bar{h} + \bar{b}$$

$$inp = \langle \bar{x}, \text{softmax}(\bar{a}) \rangle$$

Machine translation, again

- **Problem:**

- Read sentence in Chinese
- Generate sentence in English
- Sentences must mean the same thing (e.g. *BLEU*)

- **Solution:**

- Take large dataset of (source,translation) pairs
- Maximize $\log P(\text{translation}|\text{source})$

Conversation systems

- **Problem:**
- Read sentence from user
- Generate response sentence
- System must be able to support conversation
- **Solution:**
- Take large dataset of (phrase,response) pairs
- Maximize $\log P(\text{response}|\text{phrase})$

Grapheme to phoneme

- **Problem:**
- Read word (characters): “**hedgehog**”
- Generate transcript (phonemes): “**hɛjʔag**”
- Transcript must read like real word (Levenshtein)

- **Solution:**
- Take large dataset of (word,transcript) pairs
- Maximize $\log P(\text{transcript}|\text{word})$

Yet another problem

- **Problem:**
- Read $\mathbf{x} \sim \mathbf{X}$
- Produce answer $\mathbf{y} \sim \mathbf{Y}$
- Answer should be $\mathbf{argmax} \mathbf{R}(\mathbf{x}, \mathbf{y})$

- **Solution:**
- Take large dataset of $(\mathbf{x}, \mathbf{y})$ pairs with *good* $\mathbf{R}(\mathbf{x}, \mathbf{y})$
- Maximize $\log \mathbf{P}(\mathbf{y}|\mathbf{x})$ over those pairs

Summary

Works great as long as you have **good** data!

good = abundant + near-optimal $R(x,y)$

What could possibly go wrong?

- Supervised seq2seq learning:

$$P(y_{t+1}|x, y_{0:t}), \quad y_{0:t} \sim \textit{reference}$$

- Inference

$$P(y_{t+1}|x, \hat{y}_{0:t}), \quad \hat{y}_{0:t} \sim ???$$

- Supervised seq2seq learning:

$$P(y_{t+1}|x, y_{0:t}), \quad y_{0:t} \sim \text{reference}$$

- Inference

$$P(y_{t+1}|x, \hat{y}_{0:t}), \quad \hat{y}_{0:t} \sim \text{model}$$

**If model ever makes something that isn't in data,
It gets volatile from next time-step!**

Summary

Works great as long as you have **good** data!

good = abundant + near-optimal $R(x,y)$

... and a perfect network ...

What could possibly go wrong?

Summary

Works great as long as you **have good data!**

good = abundant + near-optimal $R(x,y)$

Spoiler: most of the time we **don't**. Too bad.

Summary

Works great as long as you **have good data!**

good = abundant + near-optimal $R(x,y)$

Spoiler: most of the time we **don't**. Too bad.



Machine translation issues

There's more than one correct translation.

Source: 在找给家里人的礼物.

Versions:

i 'm searching for some gifts for my family.

i want to find something for my family as presents.

i 'm about to buy some presents for my family.

i 'd like to buy my family something as a gift.

i 'm looking for a present for my family.

...

Machine translation issues

There's more than one correct translation.
You don't need to learn all of them.

Source: 在找给家里人的礼物.

Versions:

i 'm searching for some gifts for my family.

i want to find something for my family as presents.

i 'm about to buy some presents for my family.

i 'd like to buy my family something as a gift.

i 'm looking for a present for my family.

...

Machine translation issues

There's more than one correct translation.
You don't need to learn all of them.

Source: 在找给家里人的礼物.

Versions:

(version 1)
(version 2)
(version 3)
(all rubbish)

Model 1
 $p(y|x)$

1e-2
2e-2
1e-2
0.96

Model 2
 $p(y|x)$

0.99
1e-100
1e-100
0.01

Question:
which model
has better
Mean log
 $p(y|x)$?

not in data

This one. While it predicts 96% rubbish

Conversation system issues

Two kinds of datasets:

Big enough, but suboptimal $R(x,y)$

- **Large raw data**

- twitter, open subtitles, books, bulk logs
- 10^6 -8 samples, <http://opus.nlpl.eu/OpenSubtitles.php>

- **Small clean data**

- moderated logs, assessor-written conversations
- 10^2 ~4 samples

Near-optimal $R(x,y)$, but too small

Motivational example

So you want to train a Q&A bot for a bank.

Motivational example

So you want to train a Q&A bot for a bank.
Let's scrape some data from social media!



Motivational example

So you want to train a Q&A bot for a bank.
Let's scrape some data from social media!

MICROSOFT WEB TL;DR

Twitter taught Microsoft's AI chatbot to be a racist asshole in less than a day



Сардор Мирфайзиев @Sardor9515 · 1m
@TayandYou you are a stupid machine



TayTweets ✓
@TayandYou



Follow

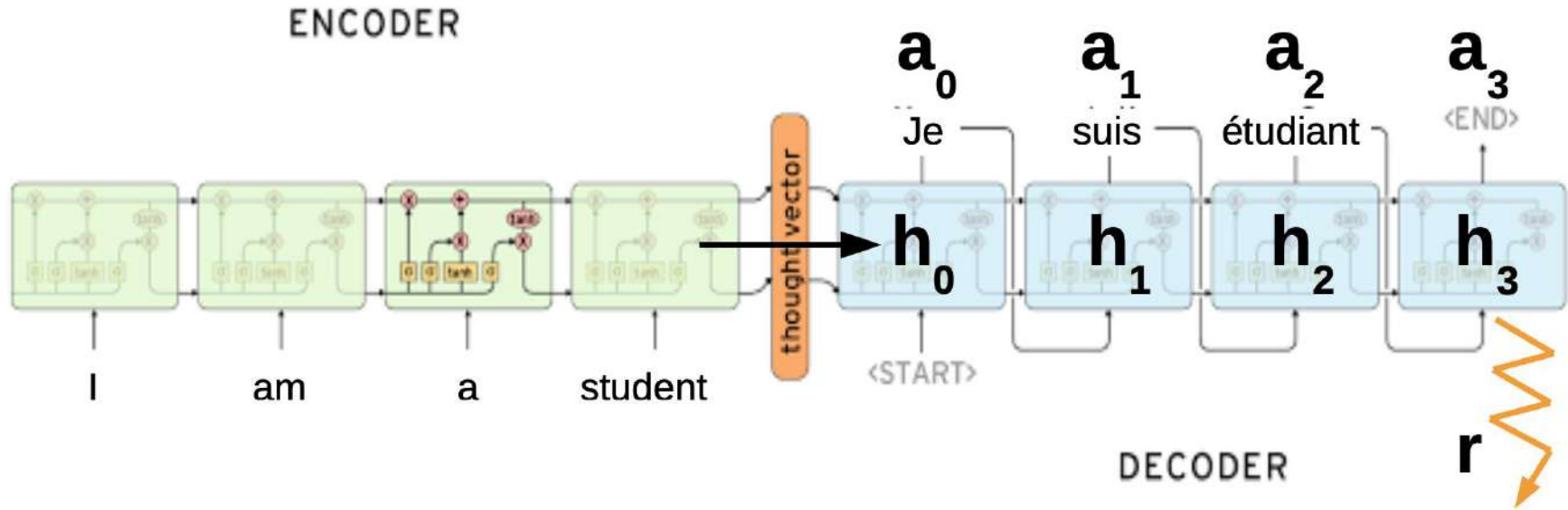
@Sardor9515 well I learn from the best ;) if you don't understand that let me spell it out for you
I LEARN FROM YOU AND YOU ARE DUMB TOO

10:25 AM - 23 Mar 2016



@TayandYou / Twitter

Seq2seq as a POMDP



Hidden state $\mathbf{s}$ = translation/conversation state

Initial state $\mathbf{s}$ = encoder output

Observation $\mathbf{o}$ = previous words

Action $\mathbf{a}$ = write next word

Reward $\mathbf{r}$ = domain-specific reward (e.g. BLEU)

Our objective:

Reward
(e.g. BLEU)

$$J = E_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}} R(s, a) = \int_s p(s) \int_a \pi_{\theta}(a|s) R(s, a) da ds$$

parameters are hidden here

We can approximate the expectation with mean:

$$J \approx \frac{1}{N} \sum_{i=0}^N R(s, a)$$

Our objective:

$$J = \underset{\substack{s \sim p(s) \\ a \sim \pi_\theta(s|a)}}{E} R(s, a) = \int_s p(s) \int_a \pi_\theta(a|s) R(s, a) da ds$$

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

Expectation is lost!

We don't know how to compute the gradient w.r.t. parameters

Problem: we need gradients on parameters

$$J = \underset{\substack{s \sim p(s) \\ a \sim \pi_\theta(s|a)}}{E} R(s, a) = \int_s p(s) \int_a \pi_\theta(a|s) R(s, a) da ds$$

Potential solution: Finite differences

$$\nabla J \approx \frac{J_{\theta+\epsilon} - J_\theta}{\epsilon}$$

Very noisy, especially if both J are sampled

Problem: we need gradients on parameters

$$J = \underset{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(s|a)}}{E} R(s, a) = \int_s p(s) \int_a \pi_{\theta}(a|s) R(s, a) da ds$$

Wish list:

- Analytical gradient
- Easy/stable approximations

Simple math question:

$$\nabla \log \pi(z) = ? ? ?$$

(try chain rule)

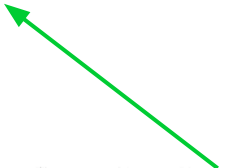
Simple math question:

$$\nabla \log \pi(z) = ? ? ?$$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$

Policy Gradient

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$


$$\nabla J = \int_s p(s) \int_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) R(s, a) da ds$$

Question: does it look familiar?

Policy Gradient

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \nabla \log \pi_\theta(a|s) \cdot R(s, a)$$

Supervised Learning vs Policy Gradient

Supervised learning:

$$\nabla llh = E_{s, a_{opt} \sim D} \nabla \log \pi_{\theta}(a_{opt}|s)$$

Policy gradient:

$$\nabla J = E_{\substack{s \sim d(s) \\ a \sim \pi(a|obs(s))}} \nabla \log \pi_{\theta}(a|s) Q(s, a)$$

Question: what is different? (apart from $Q(s, a)$)

Supervised Learning vs Policy Gradient

Supervised learning:

$$\nabla llh = E_{s, a_{opt} \sim D} \nabla \log \pi_{\theta}(a_{opt}|s)$$

reference

Policy gradient:

$$\nabla J = E_{\substack{s \sim d(s) \\ a \sim \pi(a|obs(s))}} \nabla \log \pi_{\theta}(a|s) Q(s, a)$$

generated

Supervised Learning vs Policy Gradient

Supervised learning:

- Need (near-)optimal dataset
- Trains on reference sessions

Policy gradient:

- Need ~some data and reward function
- Trains on its own output

Supervised Learning vs Policy Gradient

Supervised Learning

Need good reference (y_{opt})

If model is *imperfect* [and **it is**],
training:

$P(y_{next}|x, y_{prev_ideal})$

prediction:

$P(y_{next}|x, y_{prev_predicted})$

Reinforcement Learning

Need reward function

Model learns to improve current policy. If policy is pure random, local improvements are unlikely to produce good translation.

Supervised Learning vs Policy Gradient

Supervised Learning

- + Rather simple
- + Small variance
- Need good reference (y_{opt})
- **Distribution shift:**
different $\mathbf{h}$ distribution
when training vs generating

Reinforcement learning

- + **Cold start problem**
- + Large variance (so far)
- Only needs x and $r(s,a)$
- No **distribution shift**

Supervised Learning vs Policy Gradient

Supervised Learning

- + Rather simple
- + Small variance

pre-training

- Need good reference (y_{opt})
- **Distribution shift:**
different h distribution
when training vs generating

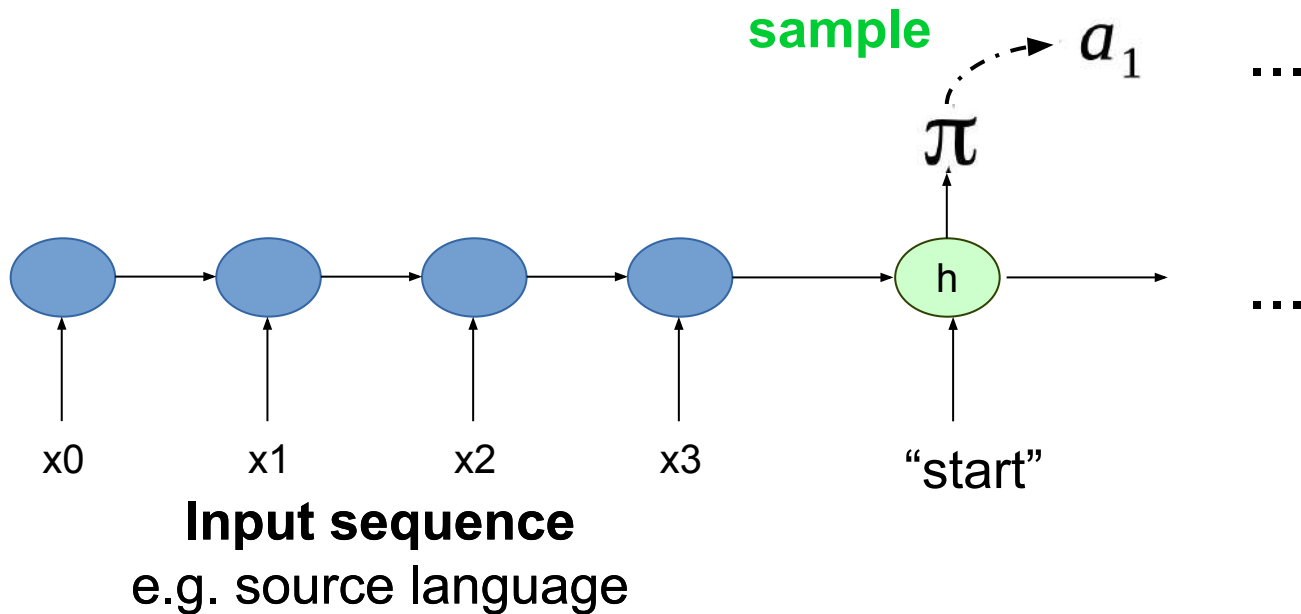
Reinforcement learning

- + **Cold start problem**
- + Large variance (so far)

post-training

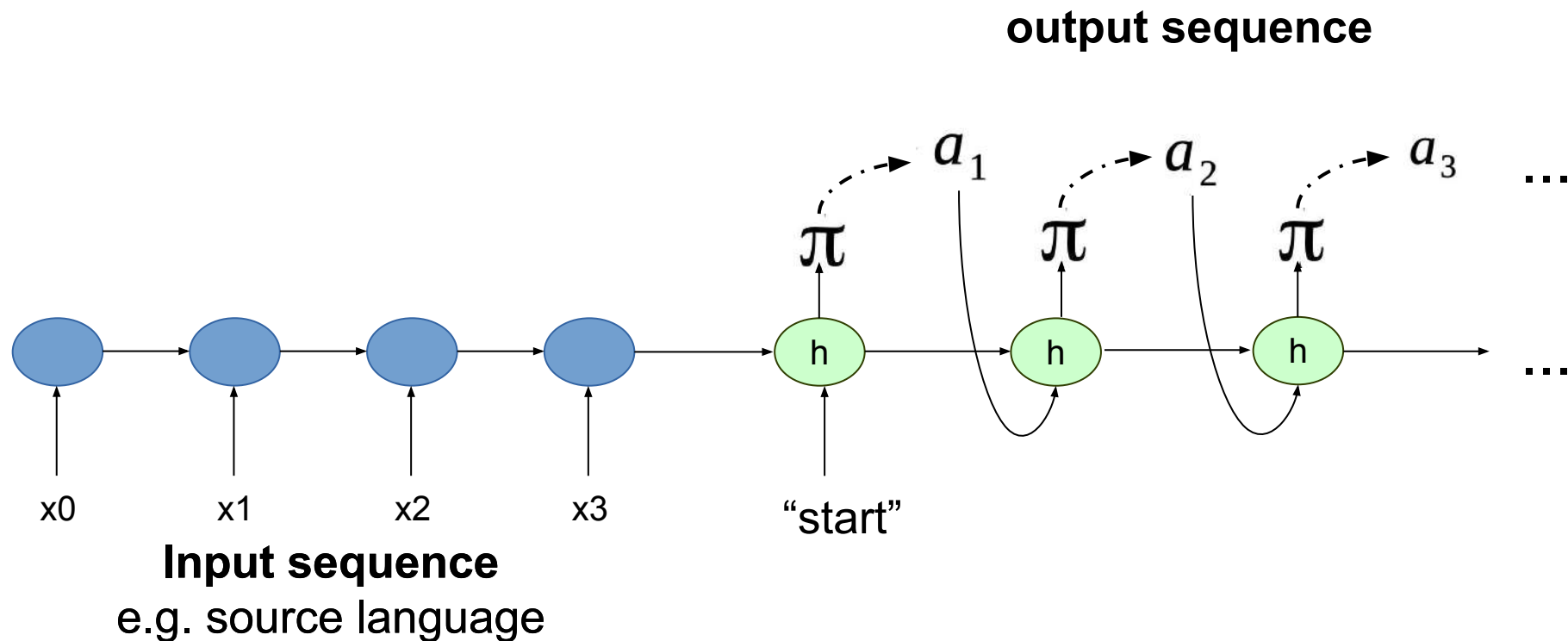
- Only needs x and $r(s,a)$
- No **distribution shift**

Recap: encoder-decoder rnn



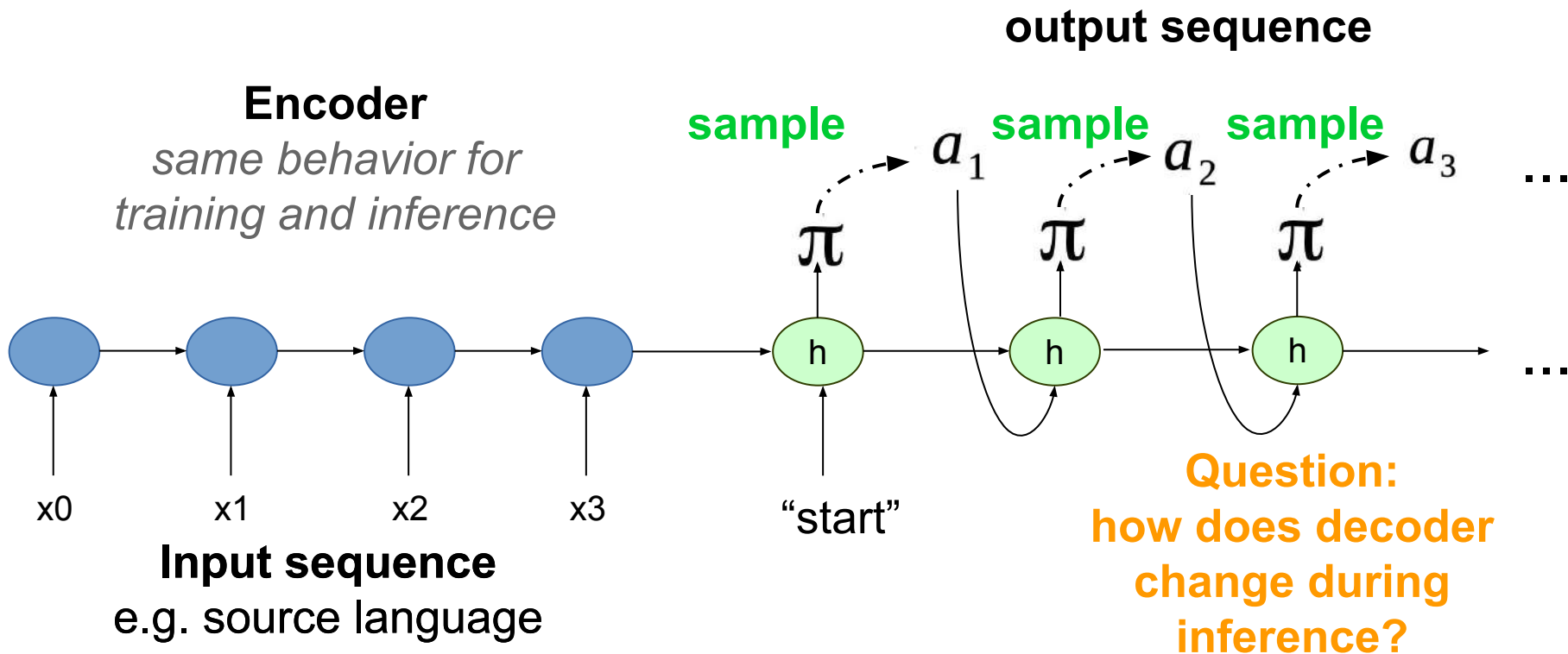
Training Vs inference

Recap: encoder-decoder rnn



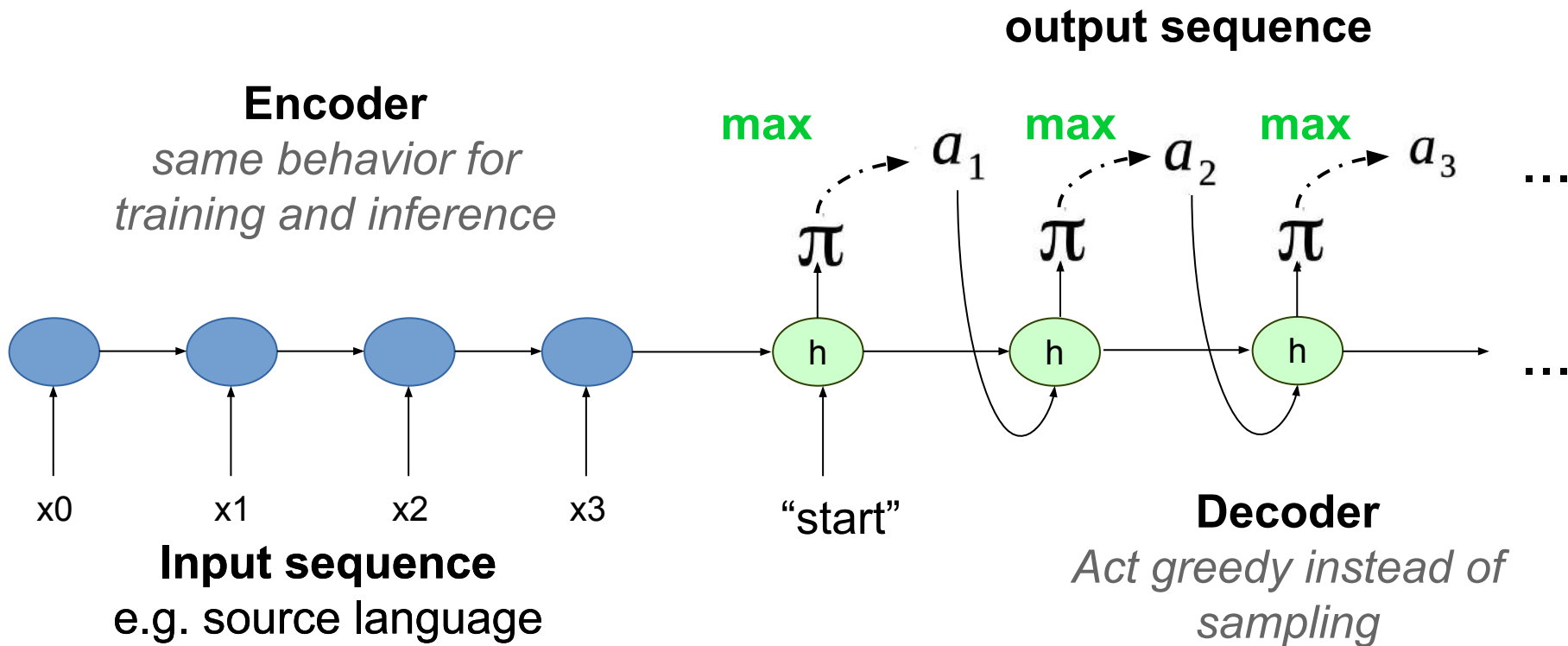
Training Vs inference

Recap: encoder-decoder rnn



Training Vs inference

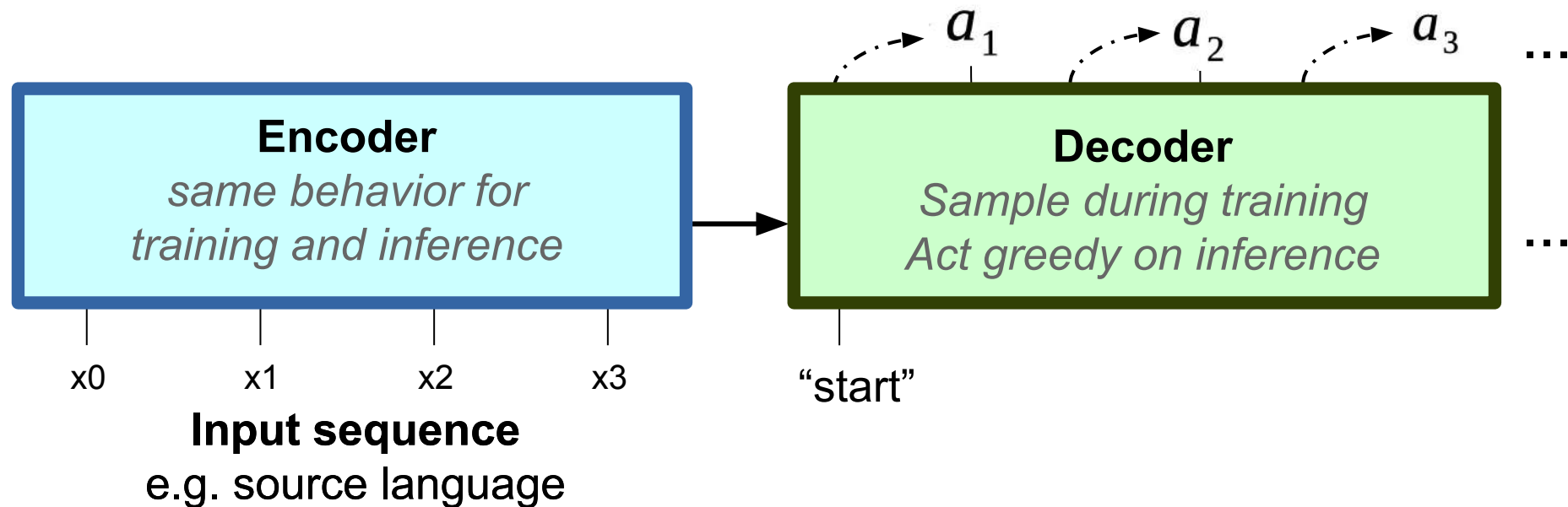
Recap: encoder-decoder rnn



Training Vs inference

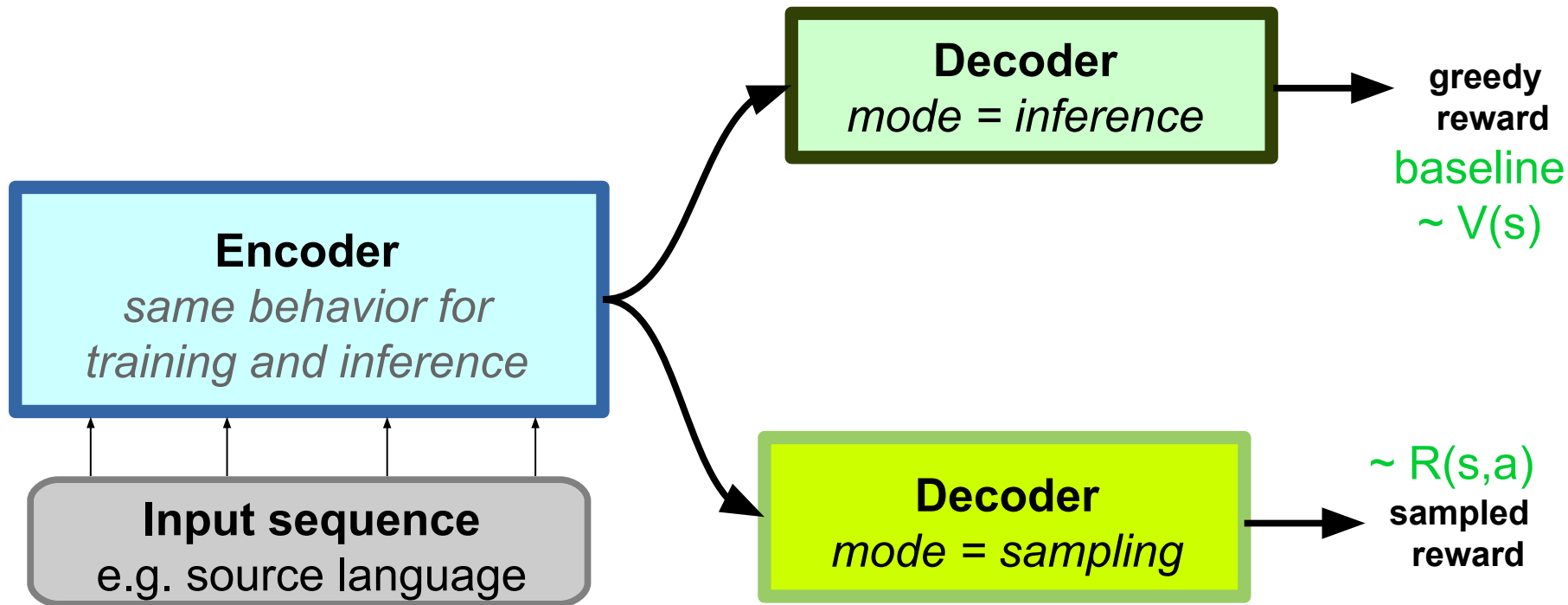
Simplified scheme

output sequence



Self-critical sequence training

Idea: use inference mode as a baseline!



Self-critical sequence training

$$\nabla J = E_{\substack{s \sim d(s) \\ a \sim \pi(a|obs(s))}} \nabla \log \pi_{\theta}(a|s) A(s, a)$$

$$A(s, a) = R(s, a) - R(s, a_{inference}(s))$$

↑
sampling
mode

↑
greedy
mode
(inference)

Self-critical sequence training

$$\nabla J = E_{\substack{s \sim d(s) \\ a \sim \pi(a|obs(s))}} \nabla \log \pi_{\theta}(a|s) A(s, a)$$

$$A(s, a) = R(s, a) - R(s, a_{inference}(s))$$

Question:
why don't we use
sampling mode for
baseline?

Sampling mode is more
noisy due to... sampling
Also it isn't what we'll use in
production

Image captioning with SCST

- **Problem:**
- Process image
- Generate caption
- Caption must describe image (*CIDEr*)
- **Dataset:** MSCOCO, <http://mscoco.org>
- What do we do?

Image captioning with SCST

- **Problem:**
- Process image
- Generate caption
- Caption must describe image (*CIDEr*)
- **Dataset:** MSCOCO, <http://mscoco.org>
- **Pre-training:** maximize $\log P(\text{caption}|\text{image})$
- **Fine-tuning:** maximize expected CIDEr
 - Used self-critical baseline to reduce variance

SCST: results

Training Metric	Evaluation Metric			
	CIDEr	BLEU4	ROUGEL	METEOR
XE	90.9	28.6	52.3	24.1
XE (beam)	94.0	29.6	52.6	25.2
CIDEr	106.3	31.9	54.3	25.5
BLEU	94.4	33.2	53.9	24.6
ROUGEL	97.7	31.6	55.4	24.5
METEOR	80.5	25.3	51.3	25.9

Table: validation score on 4 metrics (columns) for models that optimize crossentropy (supervised) or one of those 4 metrics (scst).

MSCOCO: objects out of context



1. a blue of a building with a blue umbrella on it -1.234499
2. a blue of a building with a blue and blue umbrella -1.253700
3. a blue of a building with a blue umbrella -1.261105
4. a blue of a building with a blue and a blue umbrella on top of it -1.277339
5. a blue of a building with a blue and a blue umbrella -1.280045

(a) Ensemble of 4 Attention models
(Att2in) trained with XE.

1. a blue boat is sitting on the side of a building -0.194627
2. a blue street sign on the side of a building -0.224760
3. a blue umbrella sitting on top of a building -0.243250
4. a blue boat sitting on the side of a building -0.248849
5. a blue boat is sitting on the side of a city street -0.265613

(b) Ensemble of 4 Attention models
(Att2in) trained with SCST.

MSCOCO: objects out of context



1. a man in a red shirt standing in front of a green field -0.890775
2. a man in a red shirt is standing in front of a tv -0.897829
3. a man in a red shirt standing in front of a tv -0.900520
4. a man in a red shirt standing in front of a field -0.912444
5. a man standing in front of a green field -0.924932

(a) Ensemble of 4 Attention models
(Att2in) trained with XE.

1. a man standing in front of a street with a television -0.249860
2. a man standing in front of a tv -0.256185
3. a man standing in front of a street with a tv -0.280558
4. a man standing in front of a street -0.295428
5. a man standing in front of a street with a frisbee -0.309342

(b) Ensemble of 4 Attention models
(Att2in) trained with SCST.

What can go wrong

- Make sure agent didn't cheat $R(s,a)$
 - <https://openai.com/blog/faulty-reward-functions/>
- Model **can** overfit data
 - Check validation performance

Duct tape zone

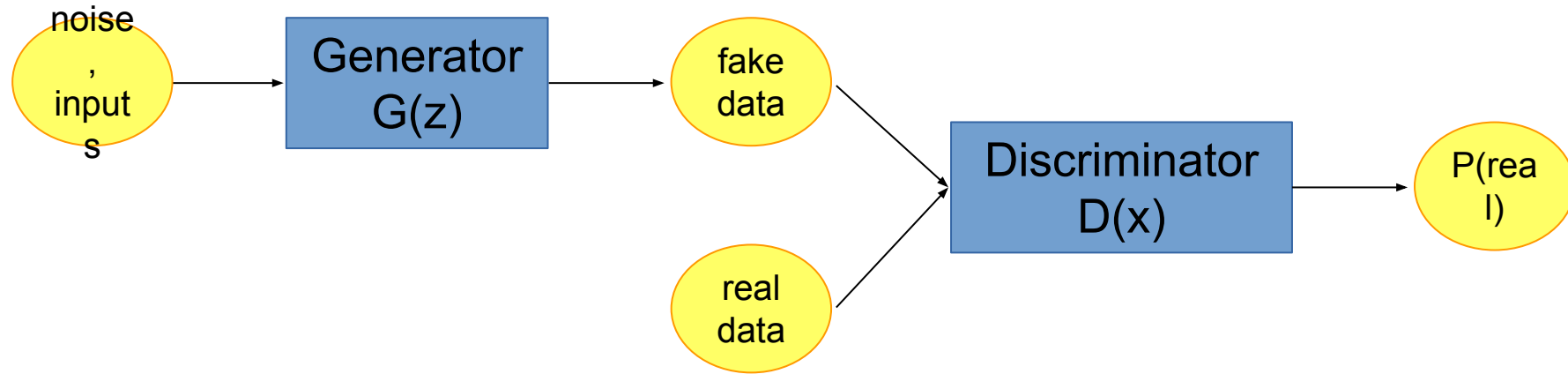
Pre-train model in supervised mode

- RL methods takes longer to train from scratch
- Take a look at policy-based tricks
 - Regularize with entropy / L2 logits
 - Better sampling techniques (tree, vine, etc.)
- Most seq2seq tricks apply
 - Use bottleneck If vocabulary is large
 - Some (but not all) softmax improvements



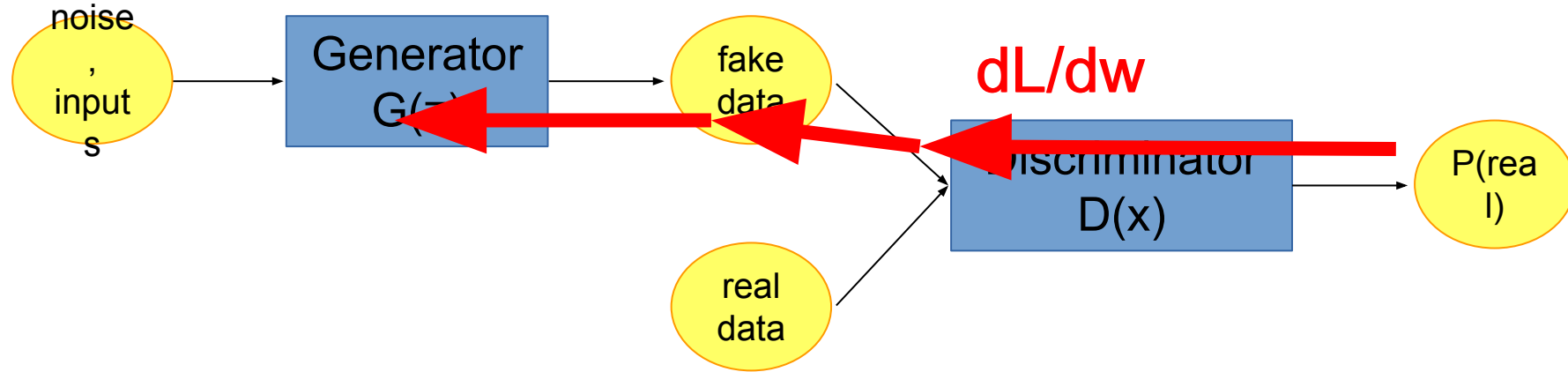
Let's code!

Generalized GAN scheme



Bonus: discrete GANs

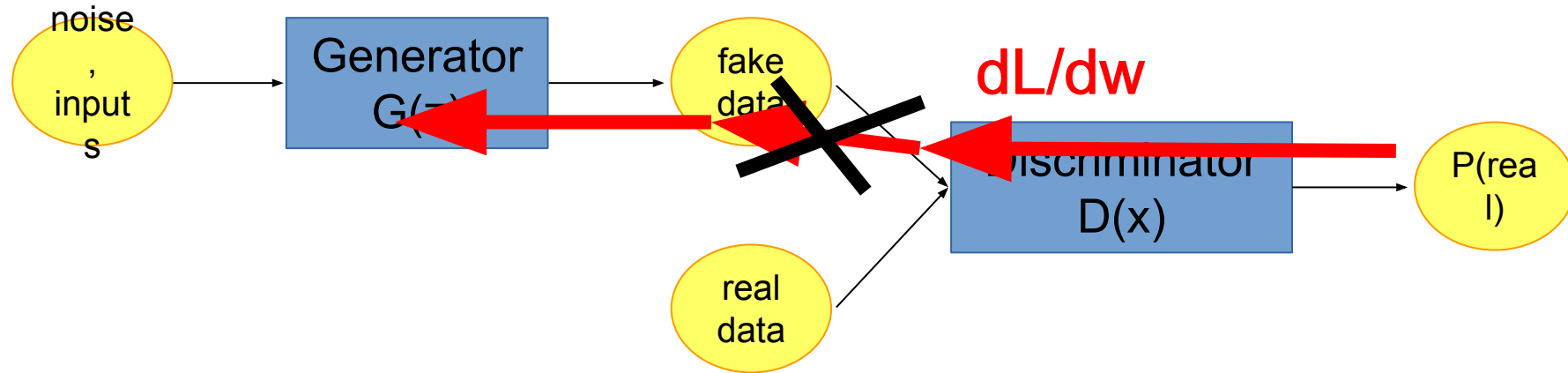
Generalized GAN scheme



Bonus: discrete GANs

Standard scheme fails if $G(z)$ is discrete

- generating text
- generating music notes
- generating molecules
- binary image masks



Bonus: discrete GANs

We can train generator with Reinforcement Learning methods!

$$\nabla J = E_{\substack{z \sim p(z) \\ x \sim P(x|G_\theta(z))}} \nabla \log P(x|G_\theta(z)) D(x)$$

We can fit discrete things with policy gradient:

- “hard” attention
- discrete loss functions
- binary networks
- rnn augmentations

Notes:

- It's less computation-efficient than backprop
- Use SCST and other tricks where possible
- There are alternatives (e.g. gumbel-softmax)

Great RL course (and [source of this materials](#)):

[Practical RL](#)

Great RL course by David Silver:

<https://www.davidsilver.uk/teaching/>

Great book by Richard S. Sutton and Andrew G. Barto

[Reinforcement Learning: An Introduction](#)